LNCS

LNC

Lecture Notes in Computer Science

Edited by G. Goos and J. Hartmanis

304

David Chaum Wyn L. Price (Eds.)

Advances in Cryptology – EUROCRYPT '87

Workshop on the Theory and Application of Cryptographic Techniques
Amsterdam, The Netherlands, April 13–15, 1987
Proceedings



Springer-Verlag
Berlin Heidelberg New York London Paris Tokyo

Editorial Board

D. Barstow W. Brauer P. Brinch Hansen D. Gries D. Luckham
C. Moler A. Pnueli G. Seegmüller J. Stoer N. Wirth

Editors

David Chaum
Centre for Mathematics and Computer Science (CWI)
Kruislaan 413, 1098 SJ Amsterdam, The Netherlands

Wyn L. Price
National Physical Laboratory
Teddington, Middlesex TW11 OLW, U.K.

Workshop Organizers:

General Chairman:	D. Chaum (CWI)	
Programme Committee:	W.L. Price (NPL), Chairman	T. Herlestam (U. Lund)
	T. Beth (U. Karlsruhe)	F. Piper (U. London)
	J.-H. Evertse (CWI)	J.J. Quisquater (Philips)
	L. Guillou (CCETT)	
Organising Committee:	J. van de Graaf (CWI)	G. Roelofsen (PTT-DNL)
	C.J. Jansen (Philips USFA)	J. van Tilburg (PTT-DNL)

The workshop was sponsored by the
International Association for Cryptologic Research
with support from the
Centre for Mathematics and Computer Science (CWI), Amsterdam

CR Subject Classification (1987): D.4.6, E.3, F.1.3, F.2.1, H.2.0, H.2.4

ISBN 3-540-19102-X Springer-Verlag Berlin Heidelberg New York
ISBN 0-387-19102-X Springer-Verlag New York Berlin Heidelberg

Library of Congress Cataloging-in-Publication Data. EUROCRYPT '87 (1987: Amsterdam, Netherlands) Advances in cryptology–EUROCRYPT '87. (Lecture notes in computer science; 304) Sponsored by the International Association for Cryptologic Research. Includes indexes. 1. Computers–Access control–Congresses. 2. Cryptography–Congresses. I. Chaum, David. II. Price, Wyn L., 1930-. III. International Association for Cryptologic Research. IV. Title. V. Series. QA76.9.A25E963 1987 005.8 88-6664
ISBN 0-387-19102-X (U.S.)

This work is subject to copyright. All rights are reserved, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, re-use of illustrations, recitation, broadcasting, reproduction on microfilms or in other ways, and storage in data banks. Duplication of this publication or parts thereof is only permitted under the provisions of the German Copyright Law of September 9, 1965, in its version of June 24, 1985, and a copyright fee must always be paid. Violations fall under the prosecution act of the German Copyright Law.

© Springer-Verlag Berlin Heidelberg 1988
Printed in Germany

Printing and binding: Druckhaus Beltz, Hemsbach/Bergstr.
2145/3140-543210

Preface

1987 marked a major upswing in attendance and contributions for this fifth in the series of Eurocrypt meetings. Response was so great that, to our regret, we were only able to accommodate less than half the submitted papers. Attendance was also up by a healthy margin.

The first two open meetings devoted to modern cryptography were organised independently: one by Allen Gersho during late Summer 1981 in Santa Barbara,[1] and the other by Thomas Beth and Rudiger Dierstein in Germany the following Spring.[2] David Chaum organised a successor to the Santa Barbara meeting the next year,[3] which launched the International Association for Cryptologic Research. The sponsorship of the association has enabled the series of annual Summer CRYPTO meetings in the U.S.[4-7] and annual Spring EUROCRYPT meetings in Europe to be continued unbroken.[8-11]

It is our pleasure to thank all those who contributed to making these proceedings possible: the authors, programme committee, organising committee, IACR officers and directors, and all the attendees.

We were all deeply saddened when we learned that Tore Herlestam, a member of the programme committee, had died unexpectedly. This volume is dedicated to him.

Amsterdam, the Netherlands D.C.
London, England W.L.P.
January 1988

---

1. Advances in Cryptology: A Report on CRYPTO 81, Allen Gersho, Ed., UCSB ECE Report no. 82-04, Department of Electrical and Computer Engineering, Santa Barbara CA 93106.
2. Cryptography: Proceedings, Burg Feuerstein 1982 (Lecture Notes in Computer Science; 149), Thomas Beth, Ed., Springer-Verlag, 1983.
3. Advances in Cryptology: Proceedings of CRYPTO 82, David Chaum, Ronald L. Rivest, and Alan T. Sherman, Eds., Plenum NY, 1983.
4. Advances in Cryptology: Proceedings of CRYPTO 83, David Chaum, Ed., Plenum NY, 1984.

5. Advances in Cryptology: Proceedings of CRYPTO 84 (Lecture Notes in Computer Science; 196), G.R. Blakley and David Chaum, Eds., Springer-Verlag, 1985.
6. Advances in Cryptology: Proceedings of CRYPTO 85 (Lecture Notes in Computer Science; 218), Hugh C. Williams, Ed., Springer-Verlag, 1986.
7. Advances in Cryptology: Proceedings of CRYPTO 86 (Lecture Notes in Computer Science; 263), A.M Odlyzko, Ed., Springer-Verlag, 1987.
8. No proceedings were published for EUROCRYPT 83, which was held in Udine Italy.
9. Advances in Cryptology: Proceedings of EUROCRYPT 84 (Lecture Notes in Computer Science; 209), T. Beth, N. Cot, and I. Ingemarsson, Eds., Springer-Verlag, 1985.
10. Advances in Cryptology: Proceedings of EUROCRYPT 85 (Lecture Notes in Computer Science; 219), Franz Pichler, Ed., Springer-Verlag, 1986.
11. No proceedings were published for EUROCRYPT 86, which was held in Linköping, Sweden.

CONTENTS

SECTION I: SEQUENCES AND LINEAR COMPLEXITY

In Memoriam Tore Herlestam (1929-1986) 3
James L. Massey

Alternating Step Generators
Controlled by De Bruijn Sequences 5
C.G. Günther

Generation of Binary Sequences with Controllable
Complexity and Ideal *r*-Tupel Distribution 15
Thomas Siegenthaler, Amstein Walthert Kleiner, and Réjane Forré

Some Remarks on the Cross Correlation Analysis
of Pseudo Random Generators 25
Sibylle Mund, Dieter Gollmann, and Thomas Beth

Sequences with Almost Perfect Linear Complexity Profile 37
Harald Niederreiter

When Shift Registers Clock Themselves 53
Rainer A. Rueppel

Finite State Machine Modelling
of Cryptographic Systems in Loops 65
Franz Pichler

SECTION II: HARDWARE TOPICS

Random Sources for Cryptographic Systems 77
G.B. Agnew

Physical Protection of Cryptographic Devices 83
Andrew J. Clark

The RSA Cryptography Processor 95
Holger Sedlak

SECTION III: PUBLIC KEY TOPICS

Extension of Brickell's Algorithm
for Breaking High Density Knapsacks 109
F. Jorissen, J. Vandewalle, and R. Govaerts

On Privacy Homomorphisms
(Extended Abstract) 117
Ernest F. Brickell and Yacov Yacobi

An Improved Protocol for Demonstrating Possession
of Discrete Logarithms and Some Generalizations 127
David Chaum, Jan-Hendrik Evertse, and Jeroen van de Graaf

A Public Key Analog Cryptosystem 143
George I. Davida and Gilbert G. Walter

SECTION IV: AUTHENTICATION AND SECURE TRANSACTIONS

Message Authentication with Arbitration
of Transmitter/Receiver Disputes 151
Gustavus J. Simmons

Perfect and Essentially Perfect Authentication Schemes
(Extended Abstract) 167
Albrecht Beutelspacher

Message Authentication and Dynamic Passwords 171
H.J. Beker and G.M. Cole

IC-Cards in High-Security Applications 177
I. Schaumüller-Bichl

SECTION V: HASH FUNCTIONS AND SIGNATURES

Collision Free Hash Functions and Public Key Signature Schemes 203
Ivan Bjerre Damgård

Hash-Functions Using Modulo-N Operations 217
Marc Girault

Blinding for Unanticipated Signatures 227
David Chaum

SECTION VI: SYMMETRIC CIPHERS: THEORY

Non-Expanding, Key-Minimal, Robustly-Perfect, Linear and Bilinear Ciphers 237
James L. Massey, Ueli Maurer, and Muzhong Wang

Linear Structures in Blockciphers 249
Jan-Hendrik Evertse

Fast Data Encipherment Algorithm FEAL 267
Akihiro Shimizu and Shoji Miyaguchi

SECTION VII: SYMMETRIC CIPHERS: APPLICATION

Modes of Blockcipher Algorithms and their Protection Against Active Eavesdropping 281
Cees J.A. Jansen and Dick E. Boekee

Security Considerations in the Design and Implementation of a New DES Chip 287
Ingrid Verbauwhede, Frank Hoornaert, Joos Vandewalle, and Hugo De Man

High-Performance Interface Architectures for Cryptographic Hardware 301
David P. Anderson and P. Venkat Rangan

Author Index 311

Keyword Index 313

IN MEMORIAM

TORE HERLESTAM (1929-1986)

The cryptologic community was deeply saddened by the sudden death on 11 November 1986 from a cerebral stroke of one of its most prominent members, Tore Herlestam.

Tore, who received his doctorate in mathematics from the University of Göteborg, Sweden, in 1959 had joined the newly-formed Department of Information Theory at the University of Lund, Sweden, as a Research Professor just three months prior to his death. For more than twenty years previously, he served in the Department of Signal Security of the General Staff of Defense in Sweden. It is a measure of his stature in the field of cryptology that he was permitted to perform his defense work at home in his beloved Skåne (southern Sweden) rather than at the Department's Stockholm facility. Tore's untimely death not only shortened cruelly the academic career that he had anticipated with much pleasure, but it also prevented him entirely from taking up his planned duties on the Editorial Board of the IACR's forthcoming *Journal of Cryptology*.

Much of Tore's research was in the classified sector of cryptology where his influence extended well beyond the Swedish borders. He was a frequent and welcome guest at many cryptologic workshops sponsored by other west European signal security agencies. Tore's published research in cryptology spans a broad spectrum, but his greatest contributions were probably those he made to the theory of stream ciphers. I remember especially Tore's work to establish under what conditions the linear complexity of the output sequence from a boolean combining function, $f(x_1, x_2, \cdots, x_n)$, fed by binary linear feedback shift registers with lengths L_1, L_2, $\cdots$, L_n, would be given by $f(L_1, L_2, \cdots, L_n)$, the formal evaluation of the algebraic normal form of the boolean function with the boolean variable x_i replaced by the integer L_i and with the boolean EXCLUSIVE-OR and AND operations replaced by integer addition and multiplication. Tore's lucid presentation on this problem at the 1982 International Symposium on Information Theory captured the imagination of my then doctoral student, Rainer Rueppel, and led eventually to the paper by Rueppel and Othmar Staffelbach in the January 1987 *IEEE Transactions on Information Theory*, which gives the most general formulation yet of "Herlestam's theorem".

Like many other members of the classified research community, Tore was deeply mistrustful of public-key cryptography. Unlike most, he gave some solid reasons for his mistrust. Together with his friend and colleague, Rolf Johannesson, Tore presented a paper ("On Computing Logarithms over $GF(2^p)$") at the 1981 International Symposium on Information Theory suggesting that taking logarithms in an extension field of $GF(2)$ was much easier than in a prime field $GF(p)$ of about the same size—many researchers (including this writer) were unconvinced. Ian Blake and his colleagues at the University of Waterloo, Canada, however, took the suggestion in earnest and made important contributions to the problem. The *coup de grace* was applied by Don Coppersmith in his July 1984 *IEEE Transactions on Information Theory* paper that fully vindicated Tore's intuition and prevented further work on what would have been an insecure public-key technique.

Tore's 1978 paper, "Critical Remarks on Some Public-Key Cryptosystems", which appeared in the Scandinavian journal *BIT*, suggested on the basis of small examples some weaknesses in the Merkle-Hellman trapdoor knapsack public-key cryptosystem and in the Rivest-Shamir-Adleman (RSA) public-

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 3-4, 1988.
© Springer-Verlag Berlin Heidelberg 1988

key cryptosystem. We all recall Adi Shamir's later brilliant demonstration of the insecurity of the trapdoor knapsack scheme. Ron Rivest, however, was able to show that the purported "weaknesses" in RSA were of no danger in a full-scale RSA cryptosystem. But Tore's intuition of a weakness in RSA continues to haunt some cryptographers and will probably continue to do so until either the RSA cryptosystem is broken or it is proved in entirety to be secure.

I remember Tore's technical writing with especial pleasure because of its clarity. His paper, "On Functions of Shift Register Sequences", which apears in *Advances in Cryptology EUROCRYPT 85*, is a beautiful exposition on the fundamental theory of stream ciphers. I know I will miss the useful mathematical advice that Tore was always generous in supplying from his encyclopedic store. And like many other cryptologists who were personal friends of Tore, I miss and will miss his quick and hearty laughter.

James L. Massey
President, IACR

ALTERNATING STEP GENERATORS CONTROLLED BY DE BRUIJN SEQUENCES

C.G. Günther
Brown Boveri Research Center
5405 Baden, Switzerland

ABSTRACT

The alternating step generator (ASG) is a new generator of pseudorandom sequences which is closely related to the stop-and-go generator. It shares all the good properties of this latter generator without possessing its weaknesses. The ASG consists of three subgenerators K, M, and $\overline{M}$. The main characteristic of its structure is that the output of one of the subgenerators, K, controls the clock of the two others, M and $\overline{M}$. In the present contribution, we determine the period, the distribution of short patterns and a lower bound for the linear complexity of the sequences generated by an ASG. The proof of the lower bound is greatly simplified by assuming that K generates a de Bruijn sequence. Under this and other not very restrictive assumptions the period and the linear complexity are found to be proportional to the period of the de Bruijn sequence. Furthermore the frequency of all short patterns as well as the autocorrelations turn out to be ideal. This means that the sequences generated by the ASG are provably secure against the standard attacks.

1. INTRODUCTION

In stream cipher cryptography messages are usualy combined with pseudorandom sequences by modular addition. Therefore, schemes for the generation of such sequences are important. They are generally based on finite state machines and most frequently on linear feedback shift registers (LFSR's). To avoid certain classes of attacks, these sequences are required to have a large period, a high linear complexity and good statistical properties.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 5-14, 1988.
© Springer-Verlag Berlin Heidelberg 1988

In one approach to the generation of these sequences, the clock of an LFSR is controlled by the output of another LFSR. Examples of generators based on this principle are various kinds of stop-and-go generators [1]-[5] and binary rate multipliers [6]. Both types of generators easily produce sequences of large period and high linear complexity (exponential in the length of the register which controls the clock). The binary rate multipliers furthermore generate sequences with good statistical properties. One disadvantage of these generators is, however, that they need several clock cycles for the generation of one single pseudorandom bit.

Amongst the various kinds of stop-and-go generators we consider the following one:

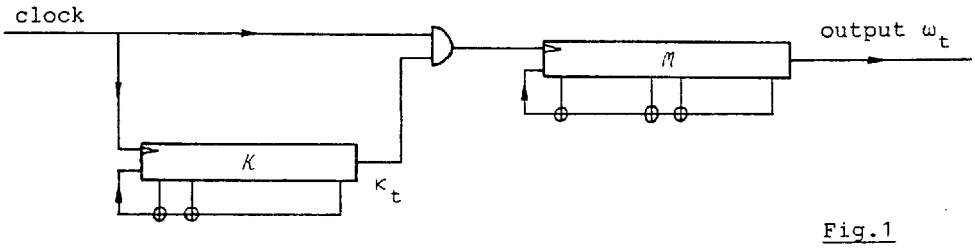


Fig.1

In this generator the output of M is repeated each time the K register produces a "0". On the one side, this leads, under suitable conditions, to a large period and a high linear complexity, on the other side, this always implies bad statistics (eg. $p(00) \cong p(11) \cong \frac{3}{8}$, $p(01) = p(10) \cong \frac{1}{8}$). Furthermore, the fact that the output ω_t can only change if $\kappa_t = 1$, determines one half of all the "1"'s present in the sequence κ. This can strongly reduce the effort needed to reconstruct κ. Similar weaknesses exist in all known stop-and-go generators.

II. THE ALTERNATING STEP GENERATOR

The alternating step generator (ASG) is closely related to the stop-and-go generator. Noteworthy is that it has all the good properties of the latter generator but does not share its weaknesses.

The ASG consists of three subgenerators K, M and $\overline{M}$, which are interconnected such that M and $\overline{M}$ are clocked when the output of K equals "1" and "0", respectively (Fig. 2).

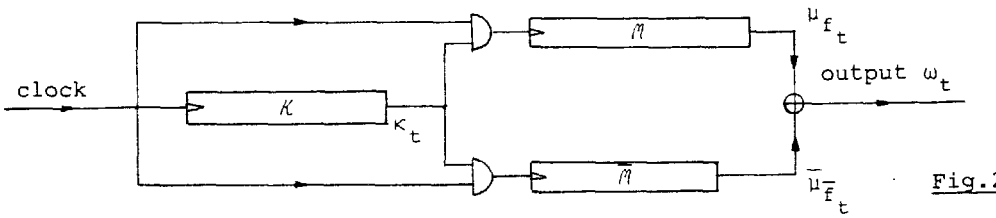


Fig.2

Mathematically, this generator can be described as follows: let κ, μ and $\bar{\mu}$ be the sequences generated by the subgenerators K, M and $\overline{M}$, when they are independently clocked. In addition, let $f_t := \sum_{s=0}^{t-1} \kappa_s$ and $\bar{f}_t = t - f_t$, then the output ω_t is described by

$$\omega_t = \mu_{f_t} \oplus \bar{\mu}_{\bar{f}_t} \quad . \tag{1}$$

In practice the sequences κ, μ amd $\bar{\mu}$ will typically be either maximum length linear recurring sequences (m-sequences) or linear recurring sequences. In the present paper, however, κ will be assumed to be a de Bruijn sequence [7]. Such a sequence can easily be obtained from an m-sequence. In the case of a de Bruijn sequence, the proof for a lower bound on the linear complexity becomes particularly simple. A treatment of the case in which κ, μ and $\bar{\mu}$ are all linear recurring sequences as well as some clues on the cascading of the structure can be found in [8].

The only attack on the ASG we could find so far is a correlation attack on κ [9]. In the present case, however, it does not substan-

tially reduce the effort to break the system. In this correlation attack (Fig. 3) a trial sequence $\tilde{\kappa}$ is correlated with κ using the relation

$$\omega_t \oplus \omega_{t-1} = \begin{cases} \mu_{f_t} \oplus \mu_{f_t-1} \\ \bar{\mu}_{\bar{f}_t} \oplus \bar{\mu}_{\bar{f}_t-1} \end{cases} \quad \text{if} \quad \kappa_{t-1} = \begin{cases} 1 \\ 0 \end{cases} \tag{1}$$

and the fact that the sum of two linear recurring sequences is again such a sequence. The signature for $\tilde{\kappa} = \kappa$ is that the linear complexity of the sequences ν and/or $\bar{\nu}$ (Fig.3) does not increase beyond the length of M and/or $\bar{M}$, respectively. This is determined by the Massey-Berlekamp algorithm.

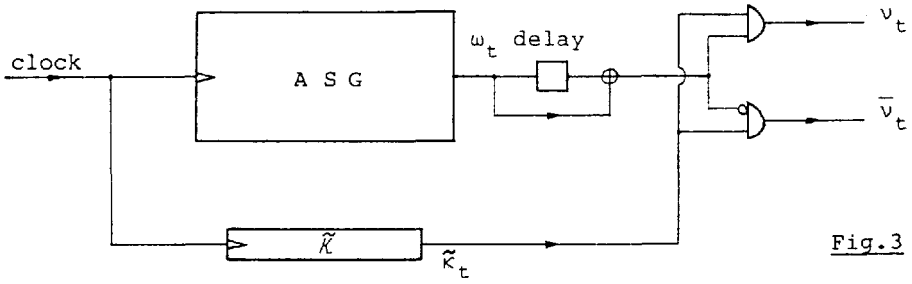


Fig.3

Fortunately, this attack only reduces the effort to break the system to essentially the third root of the effort needed for an exhaustive search. For typical parameters $K = T(\kappa) \sim 2^{127}$ it would need 10^{18} years to search through all phases if 10^{12} phases could be tested per second. In the following section the results on the period, linear complexity and frequencies of short patterns are presented.

III. THE MAIN RESULTS

Theorem 1: (period and linear complexity)

Assume: a) κ is a de Bruijn sequence of period $K = 2^k$,
b) the characteristic polynomials $p(x)$ and $\bar{p}(x)$ of μ and $\bar{\mu}$ are irreducible and different and have the degrees m and $\bar{m}$ and the periods M and $\bar{M}$, respectively,
c) $M, \bar{M} > 1$; $\gcd(M,\bar{M}) = 1$.

Under these assumptions the period T and the linear complexity L of ω satisfy the following relations:

$$T = 2^k M\bar{M} \tag{3}$$

$$(m+\bar{m})2^{k-1} < L \leq (m+\bar{m})2^k \tag{4}$$

Proof: Using that $p(x)$ and $\bar{p}(x)$ are relatively prime, the proof follows immediatly ([10]-[11]) from

i) $S := T(\mu_f) = 2^k M$

ii) the characteristic polynomial of μ_f has the form $p(x)^\ell$ with $2^{k-1} < \ell \leq 2^k$,

and corresponding assertions for $\bar{\mu}_{\bar{f}}$.

The proof of i) only requires $2 \nmid M$, which is implied by the irreducibility of $p(x)$ [10]. It reads as follows: The defining equation of S, i.e. $\mu_{f_{t+S}} = \mu_{f_t}$, $\forall\, t \in Z$, implies $f_{t+S} \equiv f_t \pmod M$, $\forall\, t \in Z$. With $M > 1$ the difference of this equation and of the corresponding equation for $t+1$, i.e. $\kappa_{t+S} \equiv \kappa_t \pmod M$, becomes $\kappa_{t+S} = \kappa_t$, i.e. $S = \gamma 2^k$. As a de Bruijn sequence is "1" with frequency one half, this implies $f_t + \gamma 2^{k-1} \equiv f_t \pmod M$ and as $2 \nmid M$: $\gamma = M$, i.e. $S = 2^k M$.

The proof of ii) is very similar to that of the lower bound for the linear complexity of a de Bruijn sequence [12]. Let D be the time shift operator $D\kappa_t = \kappa_{t-1}$ and let $p(x) = \bigoplus_{i=0}^{m} \pi_i x^i$, then

$$
\begin{aligned}
p(D^{2^k})\mu_{f_t} &= \bigoplus_{i=0}^{m} \pi_i \, \mu_{f_{t-i2^k}} \\
&= \bigoplus_{i=0}^{m} \pi_i \, \mu_{f_t - i2^{k-1}} \qquad (5) \\
&= p(\tilde{D}^{2^{k-1}})\mu_{\tilde{t}} \\
&= p(\tilde{D})^{2^{k-1}} \mu_{\tilde{t}} \\
&= 0 \quad ,
\end{aligned}
$$

where $\tilde{t}$ and $\tilde{D}$ were defined by $\tilde{t} = f_t$ and $\tilde{D}\mu_{\tilde{t}} = \mu_{\tilde{t}-1}$. This equation implies that the characteristic polynomial of μ_{f_t} must divide $p(x^{2^k}) = p(x)^{2^k}$, i.e. it must have the form $p(x)^{\ell}$, with $\ell \leq 2^k$. Now assume $\ell \leq 2^{k-1}$, then

$$
p(x)^{\ell} \mid (x^M-1)^{\ell} \mid (x^M-1)^{2^{k-1}} = x^{2^{k-1}M} - 1 \quad , \qquad (6)
$$

which contradicts $S = 2^k M$. This completes the proof. □

The results of theorem 1 are easily adapted to the case that no assumptions are made on $p(x)$ and $\bar{p}(x)$:

$$
T = 2^k M\bar{M} \qquad (7)
$$

$$
2^{k+1} < L \leq (m+\bar{m})2^k \quad . \qquad (8)
$$

The proof is based on the fact that $\gcd(M,\bar{M}) = 1$ implies $\gcd(p(x),\bar{p}(x)) \mid x-1$ and can easily be figured out.

The following theorem on the frequency of patterns holds for almost arbitrary κ. However, we will restrict ourselves to the case where κ is a de Bruijn sequence, since we would otherwise need a more general assertion on the period. For a more general statement we refer to [8]. In this theorem we use the notation $Z/(T) := \{0,1,\ldots,T-1\}$.

Theorem 2: (frequency of short patterns)

Assume: a) κ is a de Bruijn sequence of period $K = 2^k$,
b) μ and $\bar{\mu}$ are m-sequences with the periods $M = 2^m-1$ and $\bar{M} = 2^{\bar{m}}-1$, respectively,
c) gcd $(M,\bar{M}) = 1$.

Under these assumptions the frequency of any pattern σ of length $\ell \leq \min\{m,\bar{m}\}$ is $2^{-\ell}$ up to an error of order $O(\frac{1}{2^{m-\ell}}) + O(\frac{1}{2^{\bar{m}-\ell}})$, i.e.

$$\frac{1}{T}\,\mathrm{card}\{t\varepsilon Z/(T) \mid \omega_{t+i} = \sigma_i,\ \forall i\varepsilon Z/(\ell)\} = \frac{1}{2^\ell} + O(\frac{1}{2^{m-\ell}}) + O(\frac{1}{2^{\bar{m}-\ell}}) \tag{9}$$

for any $\sigma = (\sigma_0,\ldots,\sigma_{\ell-1}) \varepsilon \{0,1\}^\ell$.

Remark:

We note that the deviation of this distribution from an ideal one is very similar to the corresponding deviation for an m-sequence. In addition, this deviation is due to the corresponding deviation for m-sequences.

Proof of theorem 2: Let $t \varepsilon Z/(T)$ be represented in the form $t = r + (s+\bar{s}M)2^k$, $r \varepsilon Z/(2^k)$, $s \varepsilon Z/(M)$, $\bar{s} \varepsilon Z/(\bar{M})$ and let us first consider the frequency of patterns for a fixed $r \varepsilon Z/(2^k)$. Let $\rho = \rho(r)$ and $\bar{\rho} = \bar{\rho}(r)$ be defined by

$$\rho_0 := 0 \quad , \quad \bar{\rho}_0 := \sigma_0 \quad ,$$

$$\rho_{i+1} := \rho_i \oplus \kappa_{r+i}(\sigma_{i+1} \oplus \sigma_i) \quad , \tag{10}$$

$$\bar{\rho}_{i+1} := \bar{\rho}_i \oplus (1-\kappa_{r+i})(\sigma_{i+1} \oplus \sigma_i) \quad ,$$

for $i \varepsilon Z/(\ell-1)$. Then σ can be decomposed into $(i \varepsilon Z/(\ell))$

$$\sigma_i = \rho_i \oplus \bar{\rho}_i \quad . \tag{11}$$

For the matching condition at time t

$$w_{t+i} = \sigma_i \quad , \quad i \in Z/(\ell) \quad , \tag{12}$$

this implies

$$\mu_{f_{t+i}} \oplus \bar{\mu}_{\bar{f}_{t+i}} = \rho_i \oplus \bar{\rho}_i \quad , \quad i \in Z/(\ell) \quad . \tag{13}$$

Using the following relations

$$\begin{aligned} f_{r+i+1} &= f_{r+i} + \kappa_{r+i} \\ \bar{f}_{r+i+1} &= \bar{f}_{r+i} + 1-\kappa_{r+i} \end{aligned} \qquad i \in Z/(\ell-1) \quad , \tag{14}$$

the sum of equation (13) and of the corresponding equation for i+1 becomes: (i ε Z/(ℓ-1))

$$\begin{aligned} \mu_{f_{t+i+1}} \oplus \mu_{f_{t+i}} &= \rho_{i+1} \oplus \rho_i \\ \bar{\mu}_{\bar{f}_{t+i+1}} \oplus \bar{\mu}_{\bar{f}_{t+i}} &= \bar{\rho}_{i+1} \oplus \bar{\rho}_i \quad . \end{aligned} \tag{15}$$

This has two solutions: (i ε Z/(ℓ))

$$\mu_{f_{t+i}} = \rho_i \quad , \quad \bar{\mu}_{\bar{f}_{t+i}} = \bar{\rho}_i \tag{16a}$$

and

$$\mu_{f_{t+i}} = 1 \oplus \rho_i \quad , \quad \bar{\mu}_{\bar{f}_{t+i}} = 1 \oplus \bar{\rho}_i \quad . \tag{16b}$$

The number of solutions to this equation is equal to the number of occurences of the pattern σ in the sequence $w_{r+(s+\bar{s}M)2^k}$, s ε Z/(M), $\bar{s}$ ε Z/($\bar{M}$), i.e. to the quantity we want to determine.

Without restricting ourselves we consider the solutions of equation (16a). Making use of the fact that κ has the period $K = 2^k$ and that $\sum_{s=0}^{K-1} \kappa_s = 2^{k-1}$, this equation becomes: ($i \in Z/(\ell)$)

$$\mu_{f_{r+i}} + s2^{k-1} = \rho_i \quad , \tag{17}$$

$$\bar{\mu}_{\bar{f}_{r+i}} + (s+\bar{s}M)2^{k-1} = \bar{\rho}_i \quad . \tag{18}$$

Let $\phi_r := f_{r+\ell-1} - f_r$, then the assumptions $2 \nmid M$ and μ an m-sequence imply that equation (17) has $2^{m-\phi_r-1}$ solutions if $\rho \neq 0$. Let $\bar{\phi}_r = \ell-1-\phi_r$, then similarly $2 \nmid \bar{M}$, $\gcd(M,\bar{M}) = 1$ and $\bar{\mu}$ an m-sequence imply that equation (18) has $2^{\bar{m}-\bar{\phi}_r-1}$ solutions if $\bar{\rho} \neq 0$. This remains true for $\rho = 0$ and/or $\bar{\rho} = 0$ if we accept an error of at most $0(\frac{1}{2^{m-\ell}}) + 0(\frac{1}{2^{\bar{m}-\ell}})$. Clearly the same result also holds for equation (16b). Hence the total number of solutions to equation (12) is $2 \cdot 2^{m-\phi_r-1}\, 2^{\bar{m}-\bar{\phi}_r-1} = 2^{m+\bar{m}-\ell}$, which is independent of r. This finally implies that the frequency of the pattern σ is given by

$$\frac{2^{m+\bar{m}-\ell}}{M\bar{M}} + 0(\frac{1}{2^{m-\ell}}) + 0(\frac{1}{2^{\bar{m}-\ell}}) \quad , \tag{19}$$

and thereby yields the assertion. □

IV. CONCLUDING REMARKS

Under suitable assumptions the alternating step generator (ASG) is a simple and very efficient pseudorandom number generator. It is fast and provably satisfies the usual criteria.

The autocorrelations, which were not dealt with in the present paper, are also ideal for a large range of delays ($|\tau| \in Z/(K)$). [8].

The structure of the ASG is favorable to cascading, i.e. to have one or several of the subgenerators K, M and $\bar{M}$ being ASG's themselves. This is further discussed in [8].

SELECTED REFERENCES

[1] S.A. Tretter, "Properties of PN^2 sequences", IEEE Trans. Inform. Theory, vol. IT-20, pp. 295-297, March 1974.

[2] K. Kjeldsen and E. Andresen, "Some randomness properties of cascaded sequences", IEEE Trans. Inform. Theory, vol. IT-26, pp. 227-232, March 1980.

[3] T. Beth and F. Piper, "The stop-and-go-generator", in Proc. of EUROCRYPT 84, Springer Lect. Notes in Comp. Science, vol. 209, pp. 88-92.

[4] R. Vogel, "On the linear complexity of cascaded sequences", in Proc. of EUROCRYPT 84, Springer Lect. Notes in Comp. Science, vol. 209, pp. 99-109.

[5] D. Gollman, "Pseudo random properties of cascade connections of clock controlled shift registers", in Proc. of EUROCRYPT 84, Springer Lect. Notes in Comp. Science, vol. 209, pp. 93-98.

[6] W.G. Chambers and S.M. Jennings, "Linear equivalence of certain BRM shiftregister sequences", Electronics Letters, vol. 20, pp. 1018-1019, Nov. 1984.

[7] N.G. de Bruijn, "A combinatorial problem", Proc. K. Ned. Akad. Wet., vol. 49, pp 758-764, 1946.

[8] C.G. Günther, "Alternating step generators", submitted to IEEE Trans. on Inform. Theory.

[9] T. Siegenthaler, "Correlation-immunity of non-linear combining functions for cryptographic applications", IEEE Trans. on Inform. Theory, vol. IT-30, pp. 776-780, Sept. 1984.

[10] N. Zierler, "Linear recurring sequences", J. Soc. Indust. Appl. Math., vol. 7, pp. 31-48, March 1959.

[11] E.S. Selmer, Linear Recurrence Relations Over Finite Fields, Department of Mathematics, University of Bergen, Norway 1966.

[12] A.H. Chan, R.A. Games and E.L. Key, "On the complexities of de Bruijn sequences", J. of Comb. Theory, Series A, vol. 33, pp. 233-246, 1982.

Generation of Binary Sequences with Controllable Complexity and Ideal $r-$Tupel Distribution

Thomas Siegenthaler
Amstein Walthert Kleiner*

Réjane Forré
Inst. for Communications Technology †

Abstract

A key stream generator is analyzed which consists of a single linear feedback shift register (LFSR) with a primitive connection polynomial and a nonlinear feedforward logic. It is shown, how, for arbitrary integers n and r and a binary LFSR of length $L = n \cdot r$ the linear complexity of the generated keystream can be determined for a large class of nonlinear feedforward logics. Moreover, a simple condition imposed on these logics ensures an ideal $r-$tupel distribution for these keystreams. Practically useful solutions exist where the keystream has linear complexity $n \cdot r^{n-1}$ together with an ideal $r-$tupel distribution.

1 Introduction

A common type of keystream generator consists of a single binary linear feedback shift register (LFSR) and a feedforward logic (see Fig. 1). If the sequence produced by the LFSR has period p, all binary (key-stream-) sequences of length p are generated by suitable feedforward logics. This makes the keystream generator of Fig. 1 attractive from the theoretical point of view. The type shown in Fig. 1 is also of considerable practical interest because it needs only a single (instead of several) LFSR. However, in the general case the analysis of this type of keystream generator has shown to be rather difficult [1]. Groth [2] proposed a layered structure for the feedforward logic to control the linear complexity of the generated keystream. This arrangement generates keystreams of large linear complexities, however, the statistics of these keystreams are hard to control. Rueppel suggested [3] a simply realisable and therefore practically useful class of feedforward logics such that a lower bound for the keystream's linear complexity is guaranteed. A closely related structure had independently been proposed by Günther/Bernasconi [4] which is also simple realisable and also guarantees a minimal linear complexity of the keystream. The latter two methods are based on the existence of one or several

*Information Systems Engineering AG, Leutschenbachstr. 45, 8050 Zürich, Switzerland
†ETH Zentrum, Sternwartstr. 7, 8092 Zürich, Switzerland

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 15-23, 1988.
© Springer-Verlag Berlin Heidelberg 1988

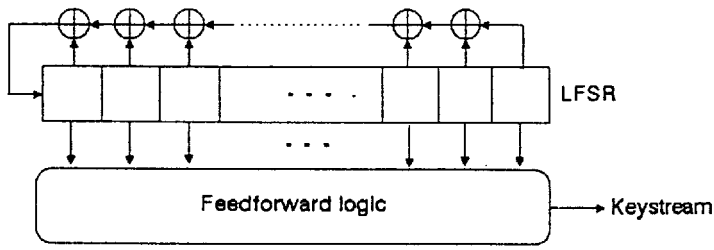


Figure 1: A common type of keystream generator

high order products in the corresponding algebraic normalform of the feedforward logic. A new approach [8] is proposed here. First, a number of "well chosen" delayed replicas (called "phases") of the sequence generated by the LFSR are picked, then every nonlinear feedforward logic is allowed. The analysis uses the theory of finite fields GF(2^n). The approach is strongly based on an interpretation of two results recently obtained by Brynielsson. It is assumed that the LFSR of Fig. 1 has a primitive connection polynomial.

2 Synthesis of keystream generators

In finite fields, every function $f : \mathrm{GF}(q) \longrightarrow \mathrm{GF}(q)$ with $x \longmapsto f(x)$ can be expressed as a polynomial [5]:

$$f(x) = \sum_{i=0}^{q-1} a_i x^i \tag{1}$$

with coefficients

$$a_i = \sum_{x \neq 0} [f(0) - f(x)] x^{-i}, \quad a_i \in \mathrm{GF}(q). \tag{2}$$

Definition 1 *If the symbol y_k of the sequence $\{y_k\}$ over* GF(q) *is obtained as $y_k = f(x_k)$ where f denotes the polynomial in (1) and $\{x_k\}$ is a sequence over* GF(q), *then $\{y_k\}$ is called a* **polynomial sequence**.

The following theorem is shown to be crucial for the computation of the linear complexity of the keystream produced by a generator as given in Fig. 1.

Theorem 1 (Brynielsson [6]) *Let $\{x_k\}$ be a maximum length sequence over* GF(2^n) *with a primitive characteristic polynomial of degree r and let $H(i)$ denote the Hamming weight of the integer i. The polynomial sequence $\{y_k\}$ with $y_k = f(x_k)$ has linear complexity* LK($\{y_k\}$):

$$\mathrm{LK}(\{y_k\}) = \sum_{a_i \neq 0} r^{H(i)}, \quad a_i \in \mathrm{GF}(2^n), \tag{3}$$

where the a_i's denote the coefficients in (1).

At a first glance, polynomial sequences together with Theorem 1 seem not to have any connection to the system of Fig. 1. Next, this connection is worked out with the help of Lemma 1 and Lemma 2. We consider a maximum length sequence $\{x_k\}$ over GF(2^n). Symbols x_k from GF(2^n) may be written as

$$x_k = x_{n-1,k}\alpha^{n-1} + x_{n-2,k}\alpha^{n-2} + \cdots + x_{1,k}\alpha + x_{0,k} \tag{4}$$

where the $x_{i,k}$'s belong to GF(2) and where α denotes a primitive element of GF(2^n). The n binary sequences $\{x_{i,k}\}$, $i = 0, 1, \ldots, n-1$, in (4) are called the **binary subsequences** of $\{x_k\}$.

Lemma 1 (Brynielsson [7]) *Let $\{x_k\}$ be a maximum length sequence over* GF(2^n) *with (primitive) characteristic polynomial $p(x)$ of degree r. The binary subsequences $\{x_{i,k}\}$, $i = 0, 1, \ldots, n-1$, of $\{x_k\}$ are linear independent and fulfil the* **same linear recursion** *with an associated (primitive) characteristic polynomial $q(x)$ of degree $L = r \cdot n$.*

Therefore, the subsequences $\{x_{1,k}\}$ differ only by delays of each other. The polynomial $q(x)$ can be determined [7,8]. The following Lemma 2 is well known.

Lemma 2 *Let $\{z_k\}$ be a binary maximum length sequence with (primitive) characteristic polynomial $q(x)$ of degree L. Every delayed version $\{z_{k-d}\}$, where d denotes an integer in the range $[0, \ldots, 2^L - 1]$ of $\{z_k\}$ can be obtained by some linear combination of the sequences $\{z_{k-1}\}$,$\{z_{k-2}\}$, $\ldots$, $\{z_{k-L}\}$.*

This means that every phase of the maximum length sequence generated by the LFSR of Fig. 1 can be obtained as a linear combination of the sequences from the L stages of this LFSR. We are now ready to establish the connection between Theorem 1, Lemma 1, Lemma 2 and a system as given in Fig. 1. Consider a maximum length sequence $\{x_k\}$ over GF(2^n). Choose any of the binary subsequences $\{x_{i,k}\}$ mentioned in Lemma 1, say $\{x_{0,k}\}$. This binary subsequence is generated by a binary LFSR of length L, its feedback connections are known from $q(x)$. The binary subsequences $\{x_{i,k}\}$, $i = 1, 2, \ldots, n-1$, are only phase shifts of $\{x_{0,k}\}$ (Lemma 1) and can be obtained as linear combinations of the sequences at the L stages of the LFSR that generates $\{x_{0,k}\}$ due to Lemma 2. (Instead of generating the maximum length sequence $\{x_k\}$ over GF(2^n) by a corresponding LFSR of length r with feedback connections due to $p(x)$, $\{x_k\}$ is generated by a (binary) LFSR of length L with feedback connections due to $q(x)$ and linear combinations of the sequences occuring at the L stages of this LFSR.) Every feedforward logic can now be applied to the n binary sequences $\{x_{0,k}\}$, $\{x_{1,k}\}$, $\ldots$, $\{x_{n-1,k}\}$ to produce the binary keystream $\{y_k\}$. This feedforward logic is then described as a polynomial f : GF(2^n) $\longrightarrow$ GF(2) with $y_k = f(x_k)$ as given in expression (1). The linear

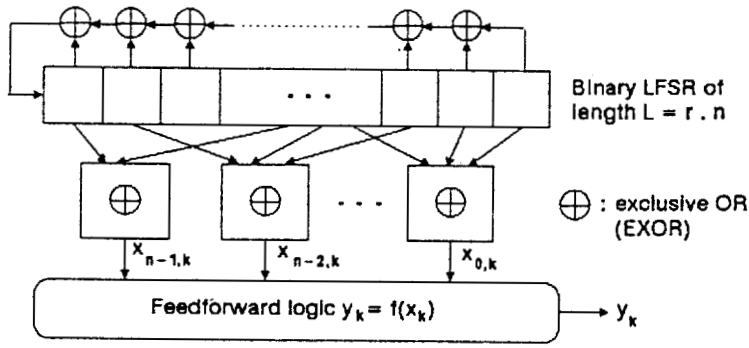


Figure 2: Synthesis of keystream generators

complexity of $\{y_k\}$ is computed by theorem 1. The corresponding system is shown in Fig. 2.

Because of the required EXOR blocks, the system of Fig. 2 is a slightly restricted version of that shown in Fig. 1. But the linear complexity can be exactly determined for arbitrary feedforward logics as given in Fig. 2.

So far we have not mentioned the $r-$tupel distribution of the keystream $\{y_k\}$. The $r-$tupel $\underline{y_i}$ is defined as a sequence $[y_i, y_{i+1}, \ldots, y_{i+r-1}]$ of successing symbols of $\{y_k\}$. The set $\underline{Y} = [\underline{y_0}, \underline{y_1}, \ldots, \underline{y_{p-1}}]$ contains all $r-$tupels of the sequence $\{y_k\}$ of period p. The following definition is useful:

Definition 2 *A binary sequence $\{y_k\}$ of period $p = 2^L - 1$ exhibits an* **ideal $r-$tupel distribution** *$\underline{Y}$, $1 \leq r \leq L$, if exactly one of the 2^r possible and disjoint binary $r-$tuples occurs $2^{L-r} - 1$ times in a period of $\{y_k\}$ and each of the others occurs 2^{L-r} times.*

Lemma 3 *An ideal $r-$tupel distribution of $\underline{Y}$ implies ideal $r'-$tupel distribution of $\underline{Y}$ for all r' with $1 \leq r' \leq r$.*

> **Proof:** From an ideal $r-$tupel distribution follows that exactly one of the 2^r possible and disjoint binary $r-$tupels occurs $2^{L-r} - 1$ times and each of the others 2^{L-r} times. Therefore, exactly one $r'-$tupel, $1 \leq r' \leq r$, occurs $(2^{L-r} - 1) + 2^{L-r} \cdot (2^{r-r'} - 1) = 2^{L-r'} - 1$ times and each of the others occurs $2^{L-r} \cdot 2^{r-r'} = 2^{L-r'}$ times, as was to be shown.

Theorem 2 *Let $\{x_k\}$ denote a maximum length sequence over $\mathrm{GF}(2^n)$ of period $2^{nr} - 1$ and f a polynomial $f : \mathrm{GF}(2^n) \longrightarrow \mathrm{GF}(2)$. A polynomial sequence $\{y_k\} = f(\{x_k\})$ exhibits an ideal $r'-$tupel distribution for all r' with $1 \leq r' \leq r$ for $x \in \mathrm{GF}(2^n)$ if and only if*

$$| \{x : f(x) = 1\} | = 2^{n-1}. \tag{5}$$

where $| \{.\} |$ denotes the cardinality of the enclosed set $\{.\}$.

Proof: Assume $| \{x : f(x) = 1\} | = b$ and $| \{x : f(x) = 0\} | = c$ with $b + c = 2^n$ and $f(0) = 0$. All $r-$tupels $\underline{x_i} = [x_i, x_{i+1}, \ldots, x_{i+r-1}]$ for $i = 0, 1, \ldots, 2^{nr} - 1$ in the maximum length sequence $\{x_k\}$ are disjoint and every possible 2^n-ary nonzero $r-$tupel occurs exactly once. Binary $r-$tupels in $\{y_k\}$ occur from $\underline{x_i} = [x_i, x_{i+1}, \ldots, x_{i+r-1}]$ as $\underline{y_i} = [y_i, y_{i+1}, \ldots, y_{i-r-1}]$ with $y_i = f(x_i)$. First, we note that the 1-tupel distribution of $\underline{Y}$ is ideal iff $b = c = 2^{n-1}$. Lemma 3 implies that none of the $r'-$tupel distributions for $1 \leq r'$ is ideal if the $1-$tupel distribution of $\underline{Y}$ is not. Therefore, (5) is a necessary condition for an ideal $r-$tupel distribution of $\underline{Y}$. This condition is also sufficient as is shown now. First, nonzero $r-$tupels $\underline{y_i}$ are considered. From the assumption $f(0) = 0$ follows that for nonzero $\underline{y_i}$'s the involved $\underline{x_i}$'s are nonzero too. From (5) follows that $(2^{n-1})^r - 1$ or $2^{L-r} - 1$ (for $L = n \cdot r$) $r-$tupels $\underline{x_i}$ are mapped into $\underline{y_1} = \underline{0}$, where the -1 accounts for the missing $r-$tupel $\underline{x_i} = \underline{0}$ in the maximum length sequence $\{x_k\}$. This completes the proof. If $f(0) = 1$ is assumed, a similar proof exists.

3 Nonlinear feedforward logic

From theorem 2 follows that a system as given in Fig. 2 generates a keystream $\{y_k\}$ with an ideal $r-$tupel distribution iff the polynomial $f : \mathrm{GF}(2^n) \longrightarrow \mathrm{GF}(2)$ which describes the feedforward logic of Fig. 2 fulfils condition (5). The designer of such a system prefers polynomials f as given in (1) such that the following properties hold

(a) $f : \mathrm{GF}(2^n) \longrightarrow \mathrm{GF}(2)$ (produces a binary sequence)

(b) f such that $| \{x : f(x) = 1\} | = | \{x : f(x) = 0\} |$ (ideal $r-$tupel distribution)

(c) f produces a keystream of large linear complexity

(d) f is easy to implement.

Solutions which fulfil all of the above requirements are described in [9] and will be discussed hereafter. As the polynomial f has to map $\mathrm{GF}(2^n)$ onto $\mathrm{GF}(2)$, it makes sense to use the so-called "trace" function.

Definition 3 *For* $\alpha \in \mathrm{GF}(q^m)$, *the* **trace** *of* α *over* $\mathrm{GF}(q)$ *is defined by*

$$\mathrm{Tr}_{\mathrm{GF}(q^m)/\mathrm{GF}(q)}(\alpha) = \alpha + \alpha^q + \alpha^{q^2} + \cdots + \alpha^{q^{m-1}}. \tag{6}$$

It can be shown [10, Theorem 2.23] that $\mathrm{Tr}_{\mathrm{GF}(q^m)/\mathrm{GF}(q)}$ is a linear transformation from $\mathrm{GF}(q^m)$ onto the subfield $\mathrm{GF}(q)$. Moreover, considering the special case $q = 2$, one can prove [9] that the function $\mathrm{Tr}_{\mathrm{GF}(q^m)/\mathrm{GF}(q)}(\alpha)$ computed for all the α's in $\mathrm{GF}(2^n)$ takes on the value 0 exactly 2^{n-1} times, and the value 1 consequently

2^{n-1} times. Therefore, the conditions (a) and (b) will both be satisfied by a function

$$f(x) = \mathrm{Tr}_{\mathrm{GF}(2^n)/\mathrm{GF}(2)}(g(x)), \quad x \in \mathrm{GF}(2^n) \tag{7}$$

where $g(x)$ permutes the finite field $\mathrm{GF}(2^n)$. In [10, Theorem 7.8] it is shown that the monomial x^b is a permutation polynomial of $\mathrm{GF}(q)$ if and only if b and $q-1$ are relatively prime. Consider the function

$$f(x) = \mathrm{Tr}_{\mathrm{GF}(2^n)/\mathrm{GF}(2)}(x^b) \tag{8}$$

where

$$\gcd(b, 2^n - 1) = 1. \tag{9}$$

From definition (2) we get

$$f(x) = x^b + x^{b\cdot 2} + x^{b\cdot 2^2} + \cdots + x^{b\cdot 2^{n-1}}. \tag{10}$$

Theorem 1 can be employed to compute the linear complexity of the polynomial sequence $\{y_k\}$ with $y_k = f(x_k)$:

$$\mathrm{LK}(\{y_k\}) = \sum_{i:a_i \neq 0} r^{H(i)} \tag{11}$$

Nonzero a_i's only occur for indices $i = b, b\cdot 2, b\cdot 2^2, \ldots, b\cdot 2^{n-1}$. All of these indices are simply obtained by shifting the binary representation of the integer b. Thus

$$H(i) = H(b), \quad \text{for } i = b, b\cdot 2, b\cdot 2^2, \ldots, b\cdot 2^{n-1} \tag{12}$$

In order to obtain a keystream-sequence of large linear complexity, one should choose an integer b of large Hamming weight $H(b)$. On the other hand, b and $2^n - 1$ must be relatively prime, according to (9). Thus, the choice $b = 2^n - 1$ (which would provide the maximal Hamming weight $H(b) = n$ is excluded, and therefore

$$b = 2^n - 2 = -1 \bmod (2^n - 1), \quad \text{with } H(b) = n - 1 \tag{13}$$

is optimal. Let

$$f(x) = \mathrm{Tr}_{\mathrm{GF}(2^n)/\mathrm{GF}(2)}(x^{-1}) \tag{14}$$

be the filtering polynomial applied to the symbols x_k of the maximum length sequence $\{x_k\}$ over $\mathrm{GF}(2^n)$. According to Theorem 1, the polynomial sequence $\{y_k\} = f(\{x_k\})$ has linear complexity

$$\mathrm{LK}(\{y_k\}) = \sum_{i:a_i \neq 0} r^{n-1} = r^{n-1} \cdot n. \tag{15}$$

The maximal linear complexity reachable for given integers r and n is easily computed by considering the case where all the a_i's in (1) are different from zero [9]:

$$\mathrm{LK}_{max}(\{y_k\}) = \sum_{a_i} r^{H(i)} = \sum_{i=0}^{n} \binom{n}{i} r^i = (r+1)^n \tag{16}$$

From (2) follows that all coefficients a_i of the polynomial

$$f_{max}(x) = \begin{cases} 0, & \text{when } x = 1 \\ 1, & \text{else} \end{cases} \tag{17}$$

are nonzero and therefore this polynomial reaches the maximal linear complexity as given in (16). However, note that this polynomial does not fulfil Theorem 2 and that the statistical properties of the generated polynomial sequence are quite disastrous.

The ratio

$$\frac{\mathrm{LK}(\{y_k\})}{\mathrm{LK}_{max}(\{y_k\})} = \frac{nr^{n-1}}{(r+1)^n} = \rho$$

can be optimized with respect to r for any given integer n by means of a simple derivation. The value $r = n - 1$ turns out to be optimal, and we obtain

$$\rho_{max}(n) = \left(\frac{n-1}{n}\right)^{n-1} = \left(1 - \frac{1}{n}\right)^{n-1} \tag{18}$$

For large values of n, and consequently of r, this ratio converges to e^{-1}. This means that the proposed structure can provide a pseudo-random sequence with a linear complexity of about 1/3 of the reachable maximal linear complexity together with an ideal $(n-1)-$tupel distribution.

3.1 Connection to GMW-Sequences

After finishing this work our attention was drawn to the so called GMW-sequences (Gordon, Mills and Welch [11]). These binary sequences have correlation properties identical to those of maximum length sequences but possess a larger linear complexity. Some of these sequences $\{y_k\}$ can be specified as

$$y_k = \mathrm{Tr}_{GF(2^n)/GF(2)}\left([\mathrm{Tr}_{GF(2^{nr})/GF(2^n)}(\alpha^k)]^b\right) \tag{19}$$

where α is a primitive element of $\mathrm{GF}(2^{nr})$ and b is any integer relatively prime to $2^n - 1$, r in the range $0 < r < 2^n - 1$. The interior Trace-function corresponds to a maximum length sequence over $\mathrm{GF}(2^n)$. This has been discussed in [12] together with an analysis of the tuple distribution, periodic autocorrelation and linear complexity of GMW-sequences as defined in (19). The results of our analysis with respect to the ideal tuple distribution and the linear complexity coincide with the results in [12]. However, the following difference concerning the derivation should be mentioned: Our analysis is based on Brynielsson's powerful Theorem 1 from which the linear complexity for every polynomial f applied to a maximum length sequence can be computed even if we use it only for a function as specified in (8). This function belongs to the same class of functions used in the GMW-construction according to expression (19). Moreover, Theorem 2 gives the necessary and sufficient condition for a polynomial $f(x)$ such that the corresponding polynomial sequence exhibits an ideal $r-$tuple distribution. The function $f(x)$

as defined in expressions (8) and (9) is a special case only which fulfils the requirements of Theorem 2. Finally, we proposed a practical implementation of these keystream generators which is completely different to the mechanization shown in [12].

Acknowledgement

The authors are grateful to L. Brynielsson for many stimulating discussions and private correspondence and also to I. Vajda and Prof. Dr. J. Massey for drawing our attention to reference [12].

References

[1] T. Herlestam, **On Functions of Linear Shift Register Sequences**, Advances in Cryptology, Eurocrypt '85, Lecture Notes in Computer Science, No. 219, Springer Verlag, 1985, p. 119-129.

[2] E.J. Groth, **Generation of Binary Sequences with Controllable Complexity**, IEEE Tr. on Inf. Theory, Vol. IT-17, No. 3, May 1971, p. 288-296.

[3] R.A. Rueppel **New Approaches to Stream Ciphers**, Diss. ETH No. 7714, Zürich, 1984.

[4] J. Bernasconi, C.G. Günther, **Analysis of a Nonlinear Feedforward Logic for Binary Sequence Generators**, Advances in Cryptology, Eurocrypt '85, Lecture Notes in Computer Science, No. 219, Springer Verlag, 1985, p. 161-168.

[5] B. Benjauthrit, I.S. Reed, **Galois Switching Functions and their Applications**, IEEE Tr. on Comp., Vol. C-25, No. 1, Jan. 1976, p. 78-86.

[6] L. Brynielsson, **On the Linear Complexity of Combined Shift Register Sequences**, Advances in Cryptology, Eurocrypt '85, Lecture Notes on Computer Science, No. 219, Springer Verlag, 1985, p. 156-160.

[7] -, **Entwurf und Analyse eines Kryptosystems über** GF(16), Kryptologie Aufbauseminar, J. Kepler Universität, Linz, 1985.

[8] Th. Siegenthaler, **Methoden für den Entwurf von Stream Cipher-Systemen**, Diss. ETH No. 8185, Dec. 1986.

[9] R. Forré, **Analyse eines Chiffriergenerators**, Diploma Project, Inst. for Communications Technology, ETH Zürich, Dec. 1986.

[10] R. Lidl, H. Niederreiter, **Finite Fields**, Addison-Wesley Publishing Company, 1983.

[11] B. Gordon, W. H. Mills, and L. R. Welch, **Some new difference sets**, Canad. J. Math., vol. 14, pp. 614-625, 1962.

[12] M. K. Simon, J. K. Omura, R. A. Scholtz, and B. K. Levitt, **Spread Spectrum Communications**, Vol. 1, Computer Science Press, 1985

SOME REMARKS ON THE CROSS CORRELATION ANALYSIS OF PSEUDO RANDOM GENERATORS

Sibylle Mund, Dieter Gollmann, Thomas Beth

Fakultät für Informatik
Universität Karlsruhe
7500 Karlsruhe, West Germany

ABSTRACT

Siegenthaler has shown how cross-correlation techniques can be applied to identify pseudo random generators consisting of linear feedback shift registers and a scrambling function [7]. These techniques may allow to attack one register in such a generator at a time. The original algorithm needs $O(R2^rN)$ operations to identify one register. (r denotes the length of the register examined, R the number of primitive polynomials of degree r, and N the minimal number of bits one has to observe). Employing Walsh-Hadamard transform this analysis can be done in $O(R(r2^r+N))$ operations [8].
We show that there exists a trade-off between the dimension of the Hadamard matrix and the number of bits required to compute the cross correlation coefficients. The complexity of this attack is $O(R(r2^{r-\delta}+2^\delta N))$. The integer δ can be selected so that the cost of the attack is minimized. The MSR-generator will serve as an example to demonstrate our algorithm.
Furthermore we examine the correlation immunity of the S-boxes used in the DES.

1. INTRODUCTION : CROSS CORRELATION ANALYSIS OF A CERTAIN CLASS OF PSEUDO RANDOM GENERATORS

We will give a short outline of the cross correlation techniques developed by Siegenthaler and of the improvements due to Xiao and Massey. Most of the technical details have been omitted and the reader is referred to [7] and [8] for a full description.

We consider pseudo random generators (Fig.1) consisting of a scrambling function f and s linear feedback shift registers (LFSR). We use (x_i^n) as a shorthand for $(x_i^n)_n$, the sequence generated by register i. The generator produces the sequence (z^n),

$$z^n := f(x_1^n,\ldots,x_s^n).$$

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 25-35, 1988.
© Springer-Verlag Berlin Heidelberg 1988

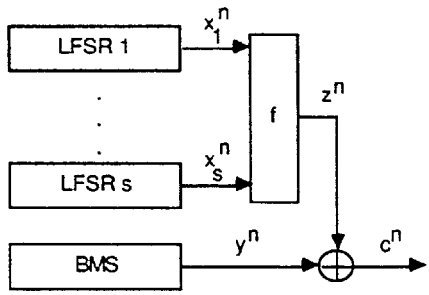


Fig.1 A Pseudo Random Generator consisting of s LFSRs and a scrambling function f

In our experiments we will encipher (y^n) , the output of a binary memory- less source (BMS) with $P(y^n=0)\neq 0.5$. The ciphertext (c^n) is thus defined by $c^n := y^n+z^n$ (addition mod 2). Siegenthaler has shown that cross correlation techniques can be used to identify initial state and feedback polynomial of register i independently of the other registers when f "leaks" some information from (x_i^n) to (z^n).

We will concentrate on the analysis of a single register. Let r denote the length of the register examined, R the number of primitve polynomials of degree r and N the minimal number of output bits necessary for the correlation analysis as established by Siegenthaler.

The correlation test works as follows. A feedback polynomial is selected. Let q_0 be a specially designated initial state of the register and let q_n be the state reached after n time steps. When we denote the state transition matrix of the register by A we have $q_n = A^n q_0$. Let C denote the output matrix of the register.

Let α_k denote the cross correlation between (c^n) and $(CA^n q_k)$, the output generated by the initial state q_k . If α_k exceeds the bound of the cross correlation test we assume that the correct feedback polynomial has been found and the initial state q_k can be computed. However, this inital state is not necessarily identified uniquely. If no α_k exceeds the bound the test is repeated for another feedback polynomial.

In the original version of the correlation attack $O(R\cdot N\cdot 2^r)$ operations are required to compute the feedback polynomial and initial state [7].

It is possible to speed up this attack by using the Walsh-Hadamard Transform to compute simultaneously the cross correlation between (c^n) and the outputs for all possible initial states of the given shift register [8].

The Walsh-Coefficients of a function $g:\{0,1\}^n \to \{-1,0,1\}$ are defined as

$$C(t) := 2^{-n} \sum_{v\in\{0,1\}^n} g(v)(-1)^{<t,v>} \qquad \text{for all } t\in\{0,1\}^n \ ,$$

where <t,v> is the real scalar product of the vectors t and v.

For our purposes define g: $\{0,1\}^n \to \{-1,0,1\}$ by

$$g(q_n) := \begin{cases} (-1)^{c^n} & \text{for } 1 \le n \le N \\ 0 & \text{else} \end{cases}$$

I.e. we reorder the ciphertext (c^n) so that the output at time n is assigned to the position q_n. With $s_k := CA^k$ we then have

$$CA^n q_k = CA^{n+k} q_0 = CA^k A^n q_0 = \langle s_k, q_n \rangle$$

and furthermore

$$\alpha_k = \sum_{n=1}^{N} (-1)^{c^n \oplus CA^n q_k} = \sum_{n=1}^{N} (-1)^{c^n \oplus \langle s_k, q_n \rangle} = \sum_{n=0}^{2^r-1} g(q_n)(-1)^{\langle s_k, q_n \rangle} = C(s_k) .$$

Thus the cross correlation coefficients $(\alpha_0,\ldots,\alpha_{2^r-1})$ are the Walsh- Coefficients of our function g. These Walsh-Coefficients can be computed by multiplying the vector

$$g := (g(0), g(1), \ldots, g(2^r-1))$$

with the Hadamard matrix H_r. (The arguments of g are identified with binary strings of length n). The Hadamard-Matrices are defined recursivly by

$$H_1 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} , \qquad H_m = \begin{bmatrix} H_{m-1} & H_{m-1} \\ H_{m-1} & -H_{m-1} \end{bmatrix} , \quad m \ge 2 .$$

The complexity of this algorithm is $O(R(r \cdot 2^r + r \cdot N))$. For each feedback polynomial we need $O(r \cdot 2^r)$ operations to compute $g \cdot H_r$ recursively [1], [3] and $O(r \cdot N)$ operations to initialize g.

2. A TRADE-OFF BETWEEN THE DIMENSION OF THE HADAMARD-MATRIX AND THE LENGTH OF CIPHERTEXT

If r grows N will lag behind 2^r. Therefore a substantial "portion" of g will be equal to 0 and thus not contribute to the computation of the correlation coefficients. Simply cutting down the dimension of the Hadamard-Matrix does not work. However, if the length of the ciphertext is increased one will again get a sufficient number of ciphertext bits in a shortened vector g . Thus a lower dimension Walsh-Transform will compute meaningful correlation coefficients.

This statement will now be explained in closer detail. We cut down the vector g by the factor 2^δ for some integer $\delta \geq 0$. Ciphertext bit c^n will contribute to $g(0),\ldots,g(2^{r-\delta}-1)$ exactly when $q_n < 2^{r-\delta}$. (Identify q_n with the integer that has binary expansion q_n).
Define $N_\delta := N \cdot 2^\delta$ and E_δ to be the expected number of states q_{k+n}, $0 \leq n \leq N_\delta - 1$, with $q_{k+n} < 2^{r-\delta}$. The expected value is taken over all initial states including the all-zero state. We have the following

LEMMA: $E_\delta = N$

PROOF: For all initial states q_k we have to count the number of states q_{k+n} with $q_{k+n} < 2^{r-\delta}$, $0 \leq n \leq N_\delta - 1$. To do so we count for all positions i, $0 \leq i \leq N_\delta - 1$, the number of initial states q_k so that $q_{k+i} < 2^{r-\delta}$. There are overall N_δ positions and $2^{r-\delta}$ such initial states for each position when we include the all-zero state. This gives

$$E_\delta = 2^{-r} \cdot N_\delta \cdot 2^{r-\delta} = 2^{-r} \cdot N \cdot 2^\delta \cdot 2^{r-\delta} = N \quad .$$

□

Thus we can expect a sufficient number of ciphertext bits to be considered in the computation of the cross correlation coefficients when we use the vector ($g(0),\ldots,g(2^{r-\delta}-1)$) and the Hadamard-Matrix $H_{r-\delta}$. Thereby we have a trade-off between the length of the ciphertext and the dimension of the Hadamard-Matrix. Increasing the length of the ciphertext by the factor 2^δ will reduce the dimension of the Hadamard-Matrix by the same factor. It is therefore possible to analyse the LFSR in

$$O\left(R((r-\delta)2^{r-\delta} + r \cdot N \cdot 2^\delta)\right)$$

steps. The best choice for δ is close to $(r-\log_2 N)/2$ (see Table 1). Note that this attack will only find the correct feedback polynomial . There are still 2^δ possible initial states corresponding to each correlation coefficient.

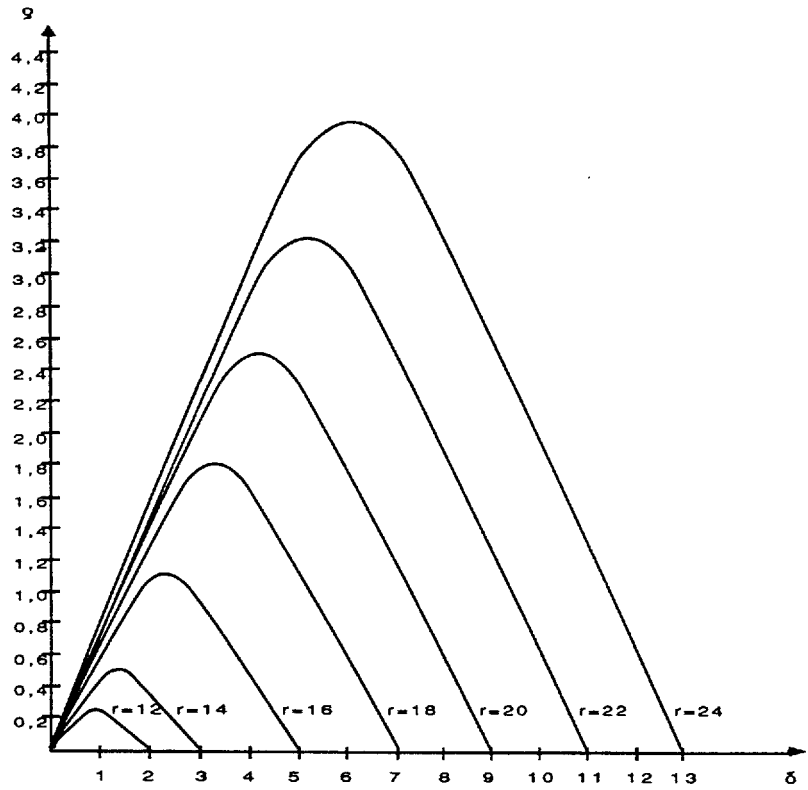


Table 1. Schematic plot of the ratio $g := \ln (R(r2^r+rN) / R((r-\delta)2^{r-\delta}+rN2^{\delta}))$

3. THE ANALYSIS OF THE MSR-GENERATOR

The MSR-generator (Fig.2) consists of two LFSR and a multiplexer that takes the state of the first register to decide which position of the second register shall be tapped to give the output. An algebraic analysis of the MSR-generator can be found in [2].

For each register cell we have to compute the probability that its content and the output of the generator are equal. This probability has to be different from 0.5 to allow a cross correlation attack. We assume that x_j and y_k are statistically independent for all j,k , $1 \le j \le m$, $1 \le k \le n$. We also assume that the positions y_k are tapped with equal probability.

Let m be the length of the first register and let $x_1,...,x_m$ be the contents of its register cells. Let n be the length of the second register and $y_1,...,y_n$ the contents of its register cells. Let z be the output of the MSR-generator.

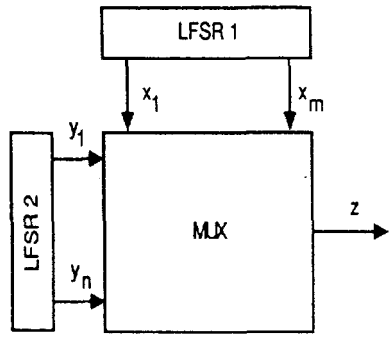


Fig.2 The MSR-Generator

We have for the first register

$$P(x_j=z) = \sum_{i=1}^{n} P(x_j = y_i)\cdot P(y_i \text{ is tapped}) = \sum_{i=1}^{n} \frac{1}{2}\cdot\frac{1}{n} = \frac{1}{2} \quad .$$

Thus the first register cannot be attacked directly by correlation analysis. The analysis of the second register gives

$$P(y_k=z) = P(y_k \text{ is tapped}) + \sum_{i=1, i\neq k}^{n} P(y_i \text{ is tapped})\cdot P(y_i=y_k)$$

$$= \frac{1}{n} + \sum_{i=1, i\neq k}^{n} \frac{1}{n}\cdot\frac{1}{2} = \frac{1}{2} + \frac{1}{n}\cdot\frac{1}{2} \quad .$$

As $P(y_k = z) \neq 0.5$ the correlation analysis is feasible.

We will demonstrate the cross correlation attack on a MSR-generator with m=12, n=14 and feedback polynomials $x^{12}+x^6+x^4+x+1$ and $x^{14}+x^{10}+x^6+x+1$. The initial states are 0000 0000 0001 and 0000 0000 0000 10 respectively. The multiplexer reads positions 3,7,4 of register 1. This 3-bit number serves as an index for the array (2,10,4,8,11,5,7,9). The entries in this array determine which position of register 2 will be tapped. The error probabilities for the test are $p_m = 0.9$ (dismissing the correct LFSR) and $p_f = 1/(R_{14}\, 2^{14})$ (accepting a wrong LFSR). We have $R_{14} = 756$ and $N = 2^{12}$.

The result of the correlation attack for the correct choice of the feedback polynomial is shown in Table 2. The attack was performed for δ=2, δ=1, and δ=0. The bound for significant Walsh-Coefficients is 430. Note that all significant positions are already highlighted by the attack with $\delta = 2$.

vectors s for δ=0 : δ=1 : δ=2	initial states for δ=0	Walsh-Coefficients δ=0 : δ=1 : δ=2
0 : 0 : 0000 0011 0000	0000 0100 0001 11	504/493/572
0 : 0 : 0000 0110 0000	0000 0010 0000 11	536/551/516
0 : 0 : 0000 1100 0000	0000 0001 0000 01	436/525/546
0 : 0 : 0001 0001 0001	0111 1001 0010 11	584/583/642
0 : 0 : 0001 1000 0000	0000 0000 1000 00	536/585/612
0 : 0 : 0011 0000 0000	0000 0000 0100 00	480/483/546
0 : 0 : 1100 0000 0000	0000 0000 0001 00	500/465/542
0 : 1 : 1000 0000 0000	0000 0000 0000 10	-528/575/614

Table 2. Value and position of the significant Walsh-Coefficients for δ=2, δ=1, δ=0. The initial states are only given for δ=0.

4. ANALYSIS OF THE DES S-BOXES

The S-boxes in the DES are non-linear functions with six inputs, say x_1, x_2, x_3, x_4, x_5, x_6 , and four outputs, say z_1,z_2,z_3,z_4 . The inputs x_1 and x_6 choose one of four different mappings from x_2,x_3,x_4,x_5 to the outputs. These mappings can be described as permutations of the numbers 0 to 15. The 8 DES S-boxes are given in Appendix 1.

The Walsh-Coefficients of input/output pairs:

We examine the amount of information that leaks through an S-box from one input to an output. These cross correlations can be measured by the Walsh-Coefficients with Hamming-Weight 1. Table 3 gives the Walsh- Coefficients for all input/output pairs (x_i,z_j) , i=2,..,5 , j=1,..4 . The Walsh-Coefficients corresponding to the inputs x_1 and x_6 are zero for all S-boxes and all outputs.

Paths in the DES:

We investigate the amount of information leakage through several rounds of the DES. We consider therefore the full set of S-boxes and look for instances where an input/output pair with large Walsh-Coefficient is linked to another such input/output pair. Enumerate the inputs x_2, x_3, x_4, x_5 of the 8 S-boxes by 0 to 31. The outputs are enumerated likewise. Let i->j denote an input/output pair and j<=>k the permutation due to wire crossing (see Appendix 2). The largest Walsh-Coefficients for input/ output pairs are of size 12. Considering the permutation in each round they correspond to the transitions 1->16 and 9->29. Obviously there is no way of getting a path of length two with Walsh-Coefficients of size 12 assigned to all transitions. For Walsh-Coefficients of size 8 one obtains at most paths of length two. Only Walsh-Coefficients of size 4 will give cycles and thus complete paths through the DES.

S-Box 1: $x_i \backslash z_j$	z_1	z_2	z_3	z_4
x_2	0	-4	4	4
x_3	-8	-12	-4	8
x_4	-4	-4	-4	4
x_5	4	-4	-4	-4

S-Box 2: $x_i \backslash z_j$	z_1	z_2	z_3	z_4
x_2	0	0	-8	0
x_3	-4	-4	4	0
x_4	0	4	4	-4
x_5	0	0	0	0

S-Box 3: $x_i \backslash z_j$	z_1	z_2	z_3	z_4
x_2	4	4	4	0
x_3	-4	0	12	0
x_4	0	4	0	-8
x_5	0	0	-4	4

S-Box 4: $x_i \backslash z_j$	z_1	z_2	z_3	z_4
x_2	-4	4	-4	-4
x_3	8	0	0	-8
x_4	4	-4	-4	-4
x_5	0	8	8	0

S-Box 5: $x_i \backslash z_j$	z_1	z_2	z_3	z_4
x_2	0	0	-4	4
x_3	-4	0	4	0
x_4	0	0	-4	4
x_5	8	-4	-4	8

S-Box 6: $x_i \backslash z_j$	z_1	z_2	z_3	z_4
x_2	-8	0	0	4
x_3	4	-8	-4	4
x_4	4	4	4	0
x_5	4	4	0	0

S-Box 7: $x_i \backslash z_j$	z_1	z_2	z_3	z_4
x_2	-4	0	4	-4
x_3	-8	-4	0	-8
x_4	8	0	4	8
x_5	4	8	4	0

S-Box 8: $x_i \backslash z_j$	z_1	z_2	z_3	z_4
x_2	0	0	0	0
x_3	-4	0	4	0
x_4	4	0	-4	-4
x_5	-4	-4	0	4

Table 3. The Walsh Coefficients for input/output pairs. Inputs x_1 and x_6 have been omitted. The coefficients have not been scaled by 2^{-6}.

4 -> 6 <=> 1 -> 3 <=> 30	13 -> 12 <=> 25 -> 27 <=> 6
4 -> 6 <=> 1 -> 0 <=> 8	15 -> 13 <=> 19 -> 16 <=> 7
4 -> 6 <=> 1 -> 1 <=>16	15 -> 13 <=> 19 -> 19 <=> 2
13 -> 12 <=> 25 -> 24 <=> 31	15 -> 14 <=> 9 -> 10 <=> 29

Table 4. Paths through two rounds of DES for input/output pairs with Walsh-Coefficients of size 8

Walsh-Coefficients of randomly selected S-boxes:

We have generated random S-boxes and computed their Walsh-Coefficients to examine whether the DES S-boxes had been chosen specifically to minimize cross correlation effects (conf. [4]). The S-boxes were constructed according to the following criteria .

P0. The mappings selected by x_1 and x_6 are permutations of <0,..,15> .
P1. No S-box is a linear or affine function of the input.

Further criteria used in the definition of the DES S-boxes (see e.g. [4]) have not been considered in this analysis. Because of property P0 the (unscaled) Walsh-Coefficients are always divisible by 4. Table 5 gives the relative frequency of the Walsh-Coefficients of input/output pairs for these randomly selected S-boxes and for the DES S-boxes. Table 6 gives the distribution of the maximal Walsh-Coefficients for each row and for each output of an S-box (*conf.* [6]). The statistics are based on 88 random S-boxes. In both instances the distributions for DES S-boxes and random S-boxes are rather similar. This suggests that non-linearity already explains the cross-correlation properties of the DES S-boxes.

5. CONCLUSION

The Walsh-Transform is a useful tool for the cross correlation analysis of pseudo random generators. We have shown how a further speed-up can be achieved in the analysis of a class of generators built from shift registers. The complexity of the attack still depends exponentially on the length of the register. However, this makes even a small reduction in the size of the exponent all the more interesting for practical applications.
We have also examined the correlation properties of the DES S-boxes. No obvious weakness was encountered. Experiments with randomly generated S-boxes seem to indicate that cross correlation properties have not been a special criterion in the design of the DES S-boxes. It remains a research problem to check this assumption in a proper statistical framework.

LITERATURE

[1] K.G.Beauchamp, "Applications of Walsh and Related Functions", Academic Press, 1984

[2] T.Beth, P.Heß, K.Wirl, "Kryptographie", Teubner, Stuttgart,1983

[3] T.Beth, "Verfahren der Schnellen Fourier-Transformation", Teubner, Stuttgart,1984

[4] E.F.Brickell, J.H.Moore, M.R.Purtill, "Structures in the S-Boxes of the DES (extended abstract)", Springer, LNCS 263, pp.3-8, 1987

[5] A.G.Konheim, "Cryptography, a Primer", Wiley, New York, 1981

[6] R.A.Rueppel, "Analysis and Design of Stream Ciphers", Springer, Heidelberg, 1986

[7] T.Siegenthaler, "Decrypting a Class of Stream Ciphers Using Ciphertext Only", IEEE Trans.Comp.,Vol. C-34, No.1, pp.81-85,1985

[8] Guo-Zhen Xiao, J.L.Massey, " A Spectral Approach to Correlation-Immune Combining Functions", to appear

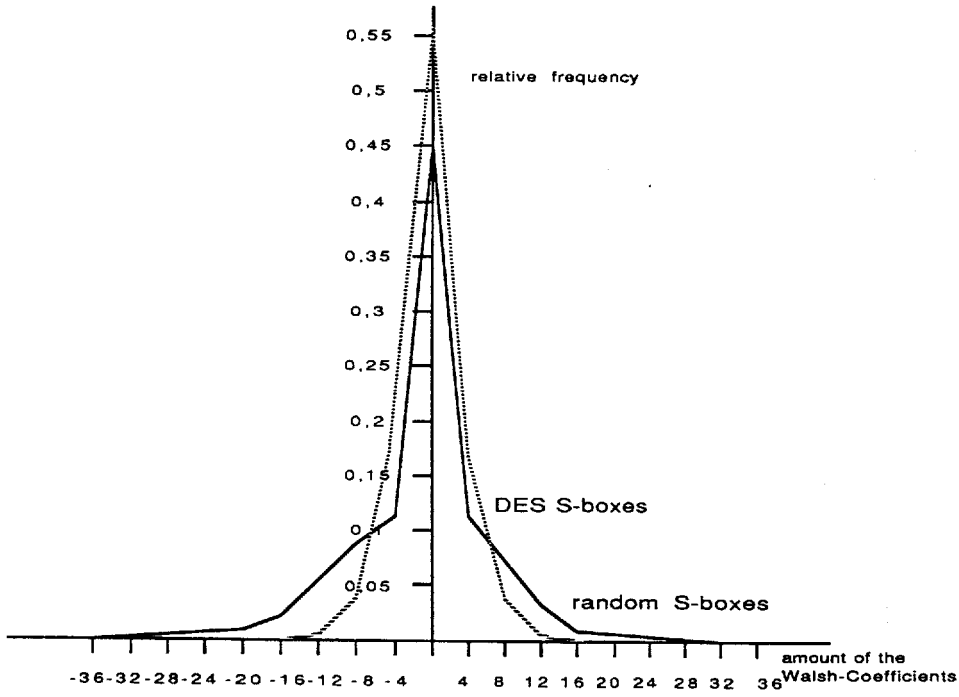


Table 5. Distribution of Walsh-Coefficients of input/output pairs for DES S-boxes and for randomly selected S-boxes.

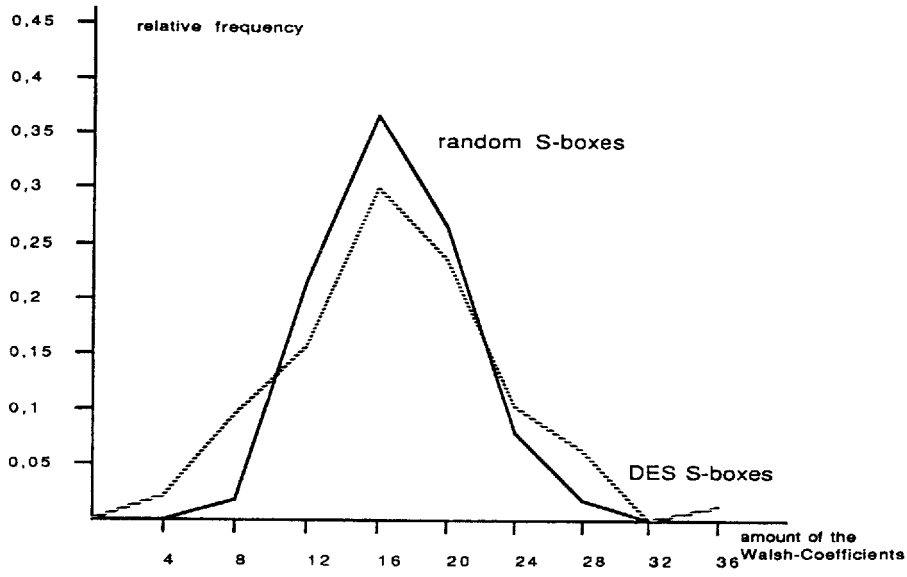


Table 6. Distribution of the maximal Walsh-Coefficients for DES S-boxes and randomly selected S-boxes.

Appendix 1: The DES S-boxes [5]:

S-Box 1:	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S-Box 2:	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
	0	14	7	11	10	4	13	1	5	8	12	6	9	3	12	15
	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S-Box 3:	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12
S-Box 4:	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14
S-Box 5:	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S-Box 6:	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S-Box 7:	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S-Box 8:	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

Appendix 2: The DES wire crossing [5]:

P : DES Wire Crossing

15	6	19	20
28	11	27	16
0	14	22	25
4	17	30	9
1	7	23	30
31	26	2	8
18	12	29	5
21	10	3	24

SEQUENCES WITH ALMOST PERFECT LINEAR COMPLEXITY PROFILE

Harald Niederreiter

Mathematical Institute, Austrian Academy of Sciences
Dr. Ignaz-Seipel-Platz 2
A-1010 Vienna, Austria

1. INTRODUCTION

Stream ciphers are based on pseudorandom key streams, i.e. on deterministically generated sequences of bits with acceptable properties of unpredictability and randomness (see [4, Ch. 9], [9]). From the cryptographic viewpoint, a useful measure for unpredictability and randomness is the linear complexity profile of a sequence (see [8], [9]). Sequences with a linear complexity profile similar to that of truly random sequences may be viewed as pseudorandom sequences. In this paper we establish connections between the linear complexity profile of a sequence and the continued fraction expansion for the generating function of the sequence and we use these connections to analyze randomness properties of the sequence.

We start with some basic definitions. If F is an arbitrary field, then a sequence $s_1,s_2,\ldots$ of elements of F is called a kth-order (linear feedback) shift register sequence if there exist constant coefficients $a_k,\ldots,a_0 \in F$ with $a_k \neq 0$ such that

$$a_k s_{i+k} + \ldots + a_1 s_{i+1} + a_0 s_i = 0 \quad \text{for } i = 1,2,\ldots . \tag{1}$$

The zero sequence $0,0,\ldots$ is viewed as a shift register sequence of order 0. A shift register sequence is uniquely determined by the recursion (1) and by the initial values $s_1,s_2,\ldots,s_k$.

Definition 1. Let $s_1,s_2,\ldots$ be an arbitrary sequence of elements of the field F and let n be a positive integer. Then the linear complexity $L(n)$ is defined as the least k such that $s_1,s_2,\ldots,s_n$ form the first n terms of a kth-order shift register sequence.

Definition 2. With the notation of Definition 1, the sequence $L(1), L(2),\ldots$ is called the linear complexity profile of the sequence $s_1,s_2,\ldots$.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 37-51, 1988.
© Springer-Verlag Berlin Heidelberg 1988

It is clear that $0 \leq L(n) \leq n$ and $L(n) \leq L(n+1)$ for all $n \geq 1$. Therefore the linear complexity profile is a nondecreasing sequence of nonnegative integers. The linear complexity $L(n)$ can be calculated by the Berlekamp-Massey algorithm (see [3, Ch. 8], [4, Ch. 6], [5]). In the applications to stream ciphers the field F is usually the binary field, i.e. the finite field with two elements.

In Section 2 we discuss formal power series viewed as generating functions of sequences. The core of the paper is Section 3 where it is shown that the linear complexity profile of a sequence can be described in terms of the continued fraction expansion for its generating function. This yields in particular a new approach to the Berlekamp-Massey algorithm. In Section 4 we show how this approach leads to a fairly simple proof for the characterization given by Wang and Massey [11] of binary sequences with a perfect linear complexity profile. In Section 5 we discuss sequences with almost perfect linear complexity profile and the construction of sequences with a prescribed linear complexity profile.

2. GENERATING FUNCTIONS

If $s_1, s_2, \ldots$ is an arbitrary sequence of elements of the field F, then we associate with it the formal power series

$$S = \sum_{i=1}^{\infty} s_i x^{-i}$$

in x^{-1} as its generating function. We view S as an element of the field $G = F((x^{-1}))$ of formal Laurent series over F in x^{-1}. The field G consists of the elements

$$S = \sum_{i=r}^{\infty} s_i x^{-i},$$

where all $s_i \in F$ and r is an arbitrary integer (positive, negative, or 0); the algebraic operations in G are defined in the usual way. The field G contains the field $F(x)$ of rational functions over F as a subfield, and $F(x)$ is the quotient field of the polynomial ring $F[x]$. Elements of G that belong to $F(x)$ are called rational, all other elements of G are called irrational.

Shift register sequences can easily be characterized in this context. If $s_1, s_2, \ldots$ is a shift register sequence satisfying (1), then the polynomial

$$f(x) = a_k x^k + \ldots + a_1 x + a_0 \in F[x] \tag{2}$$

is called a characteristic polynomial of the sequence (or of the recursion (1)). By definition, a nonzero constant polynomial is also viewed as a characteristic

polynomial of the zero sequence. As usual we define $\deg(0) = -\infty$.

Lemma 1. Let $f \in F[x]$ be a nonzero polynomial. Then the sequence $s_1, s_2, \ldots$ of elements of F is a shift register sequence with characteristic polynomial f if and only if

$$\sum_{i=1}^{\infty} s_i x^{-i} = \frac{g(x)}{f(x)}$$

with $g \in F[x]$ and $\deg(g) < \deg(f)$.

Proof. This is trivial if $\deg(f) = 0$. If f is as in (2) with $\deg(f) = k \geq 1$, then consider

$$f(x) \sum_{i=1}^{\infty} s_i x^{-i} = (a_k x^k + \ldots + a_1 x + a_0)(s_1 x^{-1} + s_2 x^{-2} + \ldots) .$$

The right-hand side is a polynomial of degree $< k$ if and only if the coefficient of each x^j with $j < 0$ vanishes, which means that

$$a_k s_{-j+k} + \ldots + a_1 s_{-j+1} + a_0 s_{-j} = 0 \quad \text{for all } j < 0 .$$

But this is equivalent to the validity of (1). □

Therefore shift register sequences are exactly those sequences whose generating functions are rational elements of G. The rational generating function of a shift register sequence has a reduced form g/m with $g, m \in F[x]$, $\deg(g) < \deg(m)$, and $\gcd(g,m) = 1$. The polynomial m is called a minimal polynomial of the shift register sequence. It can also be described as a characteristic polynomial of the shift register sequence of least degree. If one assumes that the minimal polynomial is monic, as is often done (see e.g. [3, Ch. 8]), then the minimal polynomial is uniquely determined; we will not make this assumption. The following lemma follows immediately from Lemma 1 and the definition of a minimal polynomial.

Lemma 2. Let $m \in F[x]$ be a nonzero polynomial. Then the sequence $s_1, s_2, \ldots$ of elements of F is a shift register sequence with minimal polynomial m if and only if

$$\sum_{i=1}^{\infty} s_i x^{-i} = \frac{g(x)}{m(x)}$$

with $g \in F[x]$, $\deg(g) < \deg(m)$, and $\gcd(g,m) = 1$.

If $s_1, s_2, \ldots$ is an arbitrary sequence of elements of F and n is a positive integer, then consider the least k such that $s_1, s_2, \ldots, s_n$ form the first n terms of a kth-order shift register sequence. A minimal polynomial m_n of such a kth-order

shift register sequence is called an nth minimal polynomial of the sequence $s_1, s_2, \ldots$. It follows from the definition of linear complexity that $L(n) = \deg(m_n)$ for all n.

3. LINEAR COMPLEXITY AND CONTINUED FRACTIONS

Let F be an arbitrary field and let the field $G = F((x^{-1}))$ be as in Section 2. Every $S \in G$ has a unique continued fraction expansion

$$S = A_0 + 1/(A_1 + 1/(A_2 + \ldots)) =: [A_0, A_1, A_2, \ldots] ,$$

where the A_j, $j \geq 0$, are polynomials over F and $\deg(A_j) \geq 1$ for $j \geq 1$. This expansion is finite for rational S and infinite for irrational S. For $S = \sum_{i=r}^{\infty} s_i x^{-i} \in G$ we define its polynomial part by

$$\mathrm{Pol}(S) = \sum_{i=r}^{0} s_i x^{-i} .$$

Then the polynomials A_j are obtained recursively by the following algorithm:

$$A_0 = \mathrm{Pol}(S), \qquad B_0 = S - \mathrm{Pol}(S),$$

$$A_{j+1} = \mathrm{Pol}(B_j^{-1}), \qquad B_{j+1} = B_j^{-1} - \mathrm{Pol}(B_j^{-1}) \quad \text{for } j \geq 0 ,$$

which can be continued as long as $B_j \neq 0$. If the continued fraction expansion is broken off after the term A_j, $j \geq 0$, we get the rational convergent P_j/Q_j. The polynomials P_j and Q_j can be calculated recursively by

$$P_{-1} = 1, \quad P_0 = A_0, \quad P_j = A_j P_{j-1} + P_{j-2} \quad \text{for } j \geq 1 ,$$

$$Q_{-1} = 0, \quad Q_0 = 1, \quad Q_j = A_j Q_{j-1} + Q_{j-2} \quad \text{for } j \geq 1 .$$

The following formulas are shown by straightforward induction on j:

$$\deg(Q_j) = \sum_{h=1}^{j} \deg(A_h) \quad \text{for } j \geq 1 , \tag{3}$$

$$P_{j-1} Q_j - P_j Q_{j-1} = (-1)^j \quad \text{for } j \geq 0 , \tag{4}$$

$$S = \frac{P_j + B_j P_{j-1}}{Q_j + B_j Q_{j-1}} \quad \text{for } j \geq 0 . \tag{5}$$

From (4) we get $\gcd(P_j, Q_j) = 1$ for $j \geq 0$. For rational S we interpret $\deg(A_j) = \deg(Q_j) = \infty$ whenever A_j and Q_j do not exist.

It is convenient to introduce the (exponential) valuation v on G which

extends the degree function on $F[x]$. For $S \in G$, $S \neq 0$, we put

$$v(S) = -r \quad \text{if} \quad S = \sum_{i=r}^{\infty} s_i x^{-i} \quad \text{and} \quad s_r \neq 0 .$$

For $S = 0$ we put $v(S) = -\infty$. We have the following properties for $S, T \in G$:

$$v(ST) = v(S) + v(T) ,$$
$$v(S + T) \leqq \max(v(S), v(T)),$$
$$v(S + T) = \max(v(S), v(T)) \quad \text{if} \quad v(S) \neq v(T) .$$

For $f \in F[x]$ we have $v(f) = \deg(f)$, hence

$$v(\frac{f}{g}) = \deg(f) - \deg(g) \quad \text{for} \quad f, g \in F[x], \ g \neq 0.$$

From (3), (4), (5), and the properties of v it follows easily that

$$v(Q_j S - P_j) = -v(Q_{j+1}) \quad \text{for} \quad j \geqq 0 . \tag{6}$$

Lemma 3. If $f, g \in F[x]$ are such that $v(fS-g) < 0$, then

$$f = \sum_{h=0}^{j} C_h Q_h \quad \text{and} \quad g = \sum_{h=0}^{j} C_h P_h$$

for some $j \geqq 0$ and $C_h \in F[x]$ with $\deg(C_h) < \deg(A_{h+1})$ for $0 \leqq h \leqq j$. If in addition $f \neq 0$, then

$$v(fS-g) = \deg(C_i) - \deg(Q_{i+1}) ,$$

where i is the least index with $C_i \neq 0$.

Proof. Using (3) we see that every $f \in F[x]$ can be represented in the indicated form. By (6) we have

$$v(C_h(Q_h S - P_h)) = v(C_h) - v(Q_{h+1}) < 0 \quad \text{for} \quad 0 \leqq h \leqq j ,$$

hence

$$v(\sum_{h=0}^{j} C_h(Q_h S - P_h)) < 0 .$$

Using $v(fS-g) < 0$ we get

$$v(\sum_{h=0}^{j} C_h P_h - g) = v(fS - g - \sum_{h=0}^{j} C_h(Q_h S - P_h)) < 0 .$$

But $\sum_{h=0}^{j} C_h P_h - g$ is a polynomial, hence it must be 0. To prove the second part, we note that if $f \neq 0$, then there exists a least index i with $C_i \neq 0$. From (6)

we get

$$v(C_i(Q_i\,S - P_i)) = v(C_i) - v(Q_{i+1}) \geq - v(Q_{i+1}) ,$$

$$v(C_h(Q_h\,S - P_h)) < \deg(A_{h+1}) - v(Q_{h+1}) = - v(Q_h) \leq - v(Q_{i+1}) \quad \text{for} \quad i < h \leq j ,$$

hence

$$v(fS-g) = v(\sum_{h=i}^{j} C_h(Q_h\,S - P_h)) = v(C_i(Q_i\,S - P_i)) = v(C_i) - v(Q_{i+1}). \square$$

We can now establish the main result of this section which gives an explicit description of the linear complexity profile of a sequence in terms of the polynomials Q_j that are obtained as above from the continued fraction expansion for the generating function S. Note that for generating functions S we always have $A_0 = \mathrm{Pol}(S) = 0$.

Theorem 1. Let $s_1, s_2, \ldots$ be an arbitrary sequence of elements of F and let $S = \sum_{i=1}^{\infty} s_i\, x^{-i}$ be its generating function. Then for any $n \geq 1$ the linear complexity $L(n)$ of $s_1, s_2, \ldots$ is given by

$$L(n) = \deg(Q_j) ,$$

where $j \geq 0$ is uniquely determined by the condition

$$\deg(Q_{j-1}) + \deg(Q_j) \leq n < \deg(Q_j) + \deg(Q_{j+1}) .$$

Furthermore, the nth minimal polynomials of $s_1, s_2, \ldots$ are exactly all polynomials m_n of the form

$$m_n = aQ_j + CQ_{j-1} ,$$

where $a \in F$, $a \neq 0$, and $C \in F[x]$ with $\deg(C) \leq 2 \deg(Q_j) - n - 1$.

Proof. Write $q_j = \deg(Q_j)$ for $j \geq 0$ and $q_{-1} = 0$. For $j \geq 0$ we get from (6),

$$v(S - \frac{P_j}{Q_j}) = - v(Q_j) - v(Q_{j+1}) = - q_j - q_{j+1} .$$

Hence if

$$\frac{P_j}{Q_j} = \sum_{i=1}^{\infty} t_i\, x^{-i} ,$$

then $t_i = s_i$ for $1 \leq i < q_j + q_{j+1}$. Since $\deg(P_j) < \deg(Q_j)$ and $\gcd(P_j, Q_j) = 1$, it follows from Lemma 2 that $t_1, t_2, \ldots$ is a shift register sequence with minimal polynomial Q_j. Therefore

$$L(n) \leq q_j \quad \text{for} \quad 1 \leq n < q_j + q_{j+1} . \tag{7}$$

Now let $q_{j-1} + q_j \leq n < q_j + q_{j+1}$. By the definition of $L(n)$ there exists a shift register sequence $u_1, u_2, \ldots$ with minimal polynomial $m_n \in F[x]$ of degree $L(n)$ and $u_i = s_i$ for $1 \leq i \leq n$. By Lemma 2 the generating function of $u_1, u_2, \ldots$ has the form g_n/m_n with $g_n \in F[x]$. Using (7) we get

$$v(m_n S - g_n) = L(n) + v(S - \frac{g_n}{m_n}) \leq L(n) - n - 1 < 0 . \tag{8}$$

Since $m_n \neq 0$, Lemma 3 yields the formulas

$$m_n = \sum_{h=i}^{j} C_h Q_h \quad \text{and} \quad g_n = \sum_{h=i}^{j} C_h P_h \quad \text{with} \quad C_i \neq 0 ,$$

$$v(m_n S - g_n) = \deg(C_i) - q_{i+1} . \tag{9}$$

Together with (8) this implies

$$L(n) \geq n + 1 + \deg(C_i) - q_{i+1} \geq q_{j-1} + q_j + 1 - q_{i+1} .$$

In view of (7) this is only possible if $i = j$ or $i = j - 1$. If $i = j$, then $m_n = aQ_j$ with $a \in F$, $a \neq 0$, and $L(n) = q_j$. If $i = j - 1$, then $m_n = C_j Q_j + C_{j-1} Q_{j-1}$ and

$$L(n) \geq n + 1 + \deg(C_{j-1}) - q_j \geq q_{j-1} + \deg(C_{j-1}) + 1 .$$

This shows that $C_j \neq 0$, hence $C_j = a \in F$, $a \neq 0$, and $L(n) = q_j$. Furthermore, (8) and (9) imply

$$\deg(C_{j-1}) - q_j = v(m_n S - g_n) \leq q_j - n - 1 ,$$

hence $\deg(C_{j-1}) \leq 2q_j - n - 1$.

We have now shown that an nth minimal polynomial m_n is necessarily of the form $m_n = aQ_j + CQ_{j-1}$ given in the theorem. It remains to prove that any such polynomial can serve as an nth minimal polynomial. Put $g_n = aP_j + CP_{j-1}$. Then $\deg(g_n) < \deg(m_n)$ and

$$P_{j-1} m_n - Q_{j-1} g_n = (-1)^j a$$

by (4), so that $\gcd(g_n, m_n) = 1$. By Lemma 2, m_n is a minimal polynomial of the

shift register sequence $u_1, u_2, \ldots$ with generating function g_n/m_n. If $C \neq 0$, then by (9)

$$v(S - \frac{g_n}{m_n}) = v(m_n S - g_n) - v(m_n) = \deg(C) - 2q_j \leq -n - 1 ,$$

and if $C = 0$, then

$$v(S - \frac{g_n}{m_n}) = v(m_n S - g_n) - v(m_n) = -q_{j+1} - q_j \leq -n - 1 .$$

Thus $u_i = s_i$ for $1 \leq i \leq n$, and so m_n is an nth minimal polynomial of $s_1, s_2, \ldots$. □

We remark that in the usual situation in which the Berlekamp-Massey algorithm is applied we have a finite field F and $n = 2\deg(m_n) = 2\deg(Q_j)$. In this case, Theorem 1 shows that the nth minimal polynomials are exactly given by $m_n = aQ_j$ with $a \in F$, $a \neq 0$. If one wants m_n to be monic, then a is uniquely determined.

Connections between the Berlekamp-Massey algorithm, the continued fraction algorithm, and the Euclidean algorithm have already been observed earlier (see e.g. the references in [3, p. 530] and the recent work of Dai and Wan [2]), but Theorem 1 seems to give the most transparent connection between these algorithms.

4. PERFECT LINEAR COMPLEXITY PROFILE

Rueppel [8], [9, Ch. 4] has shown that for random sequences of bits (viewed as elements of the binary field F_2) the expected value of the linear complexity $L(n)$ is $\frac{n}{2} + c_n$ with $0 \leq c_n \leq \frac{5}{18}$. This has given rise to the notion of a sequence with a perfect linear complexity profile: this is a sequence whose linear complexity profile follows the expected behavior of random sequences as closely as possible. Extending this notion to arbitrary fields, we get the following definition. We write $\lfloor t \rfloor$ for the greatest integer $\leq t$.

Definition 3. A sequence $s_1, s_2, \ldots$ of elements of a field F is said to have a perfect linear complexity profile (PLCP) if

$$L(n) = \lfloor \frac{n+1}{2} \rfloor \quad \text{for all } n \geq 1 .$$

Theorem 2. The sequence $s_1, s_2, \ldots$ of elements of F has a PLCP if and only if its generating function $S = \sum_{i=1}^{\infty} s_i x^{-i}$ is irrational and has a continued fraction expansion

$$S = [0, A_1, A_2, \ldots]$$

with $\deg(A_j) = 1$ for all $j \geq 1$.

Proof. If S is irrational and $\deg(A_j) = 1$ for all $j \geq 1$, then by (3) we have $\deg(Q_j) = j$ for all $j \geq 0$, and so Theorem 1 shows immediately that the sequence has a PLCP. Conversely, if the sequence has a PLCP, then $\lim_{n\to\infty} L(n) = \infty$ implies that S is irrational. If we had $\deg(A_j) > 1$ for some $j \geq 1$, then with $n = \deg(Q_{j-1}) + \deg(Q_j) = 2\deg(Q_j) - \deg(A_j)$ we get from Theorem 1

$$L(n) = \deg(Q_j) > \left\lfloor \frac{n+1}{2} \right\rfloor ,$$

which is a contradiction. □

Theorem 2 was shown for the binary field in Niederreiter [6] and for arbitrary finite fields in Niederreiter [7]. In the important special case where F is the binary field F_2 with two elements, Wang and Massey [11] have given a more explicit characterization of sequences with a PLCP. It was already pointed out in Niederreiter [6] that this characterization follows also from Theorem 2 and a result of Baum and Sweet [1]. We show now that the Wang-Massey characterization can be deduced from Theorem 2 by a fairly simple argument that uses an idea of Taussat [10].

Theorem 3 (Wang and Massey [11]). The sequence $s_1, s_2, \ldots$ of elements of F_2 has a PLCP if and only if it satisfies

$$s_1 = 1 \quad \text{and} \quad s_{2i+1} = s_{2i} + s_i \quad \text{for} \quad i = 1, 2, \ldots .$$

Proof. For $T \in G = F_2((x^{-1}))$ put

$$D(T) = x^{-1} T^2 + (1 + x^{-1}) T + x^{-1} \in G .$$

We have the following properties:

$$D(T + U + V) = D(T) + D(U) + D(V) \quad \text{for} \quad T, U, V \in G ,$$

$$D(T^{-1}) = D(T)\, T^{-2} \quad \text{for} \quad T \in G,\ T \neq 0 ,$$

$$D(x) + D(c) = c + 1 \quad \text{for} \quad c \in F_2 .$$

If $s_1, s_2, \ldots$ has a PLCP, then by Theorem 2 its generating function S has an infinite continued fraction expansion $S = [0, A_1, A_2, \ldots]$, where $A_j = x + a_j$ with $a_j \in F_2$ for all $j \geq 1$. For $j \geq 0$ we have by the continued fraction algorithm

$B_j^{-1} = A_{j+1} + B_{j+1}$, hence

$$D(B_j)\, B_j^{-2} = D(B_j^{-1}) = D(x + a_{j+1} + B_{j+1}) = a_{j+1} + 1 + D(B_{j+1}) .$$

By induction on j and using $S = B_0$ this yields

$$D(S) = \sum_{i=1}^{j} (a_i + 1)\, B_0^2\, B_1^2 \dots B_{i-1}^2 + B_0^2\, B_1^2 \dots B_{j-1}^2\, D(B_j) \quad \text{for } j \geq 1 .$$

We have $v(B_j) < 0$ and $v(D(B_j)) < 0$ for $j \geq 0$, thus

$$\lim_{j\to\infty} B_0^2\, B_1^2 \dots B_{j-1}^2\, D(B_j) = 0$$

in the topology on G induced by the valuation v. Hence

$$D(S) = \sum_{i=1}^{\infty} (a_i + 1)\, B_0^2\, B_1^2 \dots B_{i-1}^2 = \Big(\sum_{i=1}^{\infty} (a_i + 1)\, B_0\, B_1 \dots B_{i-1}\Big)^2 =: U^2$$

with $U \in G$, $v(U) < 0$. By the definition of $D(S)$ we obtain

$$S^2 + (x+1)\, S + 1 = xU^2 . \tag{10}$$

Comparing constant terms we get $s_1 = 1$ and comparing the coefficients of x^{-2i} for $i \geq 1$ we get $s_i + s_{2i+1} + s_{2i} = 0$.

Conversely, if $s_1 = 1$ and $s_{2i+1} = s_{2i} + s_i$ for $i \geq 1$, then we have (10) with a suitable $U \in G$, $v(U) < 0$. If S is either rational or $\deg(A_j) > 1$ for some $j \geq 1$, then by (6) there is a $j \geq 0$ with

$$v(Q_j\, S - P_j) < -\, v(Q_j) - 1 .$$

It follows that

$$v(Q_j^2\, S^2 - P_j^2) = v((Q_j\, S - P_j)^2) < 0 ,$$

$$v((x+1)\, Q_j\, (Q_j\, S - P_j)) = 1 + v(Q_j) + v(Q_j\, S - P_j) < 0 .$$

Therefore

$$T := P_j^2 + (x+1)\, P_j\, Q_j + Q_j^2 + xU^2\, Q_j^2 = Q_j^2\, S^2 - P_j^2 + (x+1)\, Q_j\, (Q_j\, S - P_j)$$

satisfies $v(T) < 0$. In particular, the constant term of T is 0. With $a = P_j(0)$, $b = Q_j(0)$ this means that $a + ab + b = 0$. In F_2 this is only possible if $a = b = 0$, and this is a contradiction to $\gcd(P_j, Q_j) = 1$. Therefore S satisfies the conditions in Theorem 2 and $s_1, s_2, \dots$ has a PLCP. □

5. ALMOST PERFECT LINEAR COMPLEXITY PROFILE

Theorem 3 shows that, at least in the field F_2, sequences with a PLCP are far from being unpredictable. Indeed this result says that in a sequence of bits with a PLCP every second term depends in a known manner on previous terms, an untenable state of affairs in a purportedly pseudorandom sequence.

The moral of this is that requiring a PLCP is too restrictive a condition. Therefore one should consider sequences for which somewhat larger deviations of the linear complexity $L(n)$ from its expected value (which is about $\frac{n}{2}$) are allowed. Such sequences may be called sequences with an *almost perfect linear complexity profile.* The deviation of $L(n)$ from $\frac{n}{2}$ can be controlled by the following simple consequence of Theorem 1.

Theorem 4. Let $s_1, s_2, \ldots$ be an arbitrary sequence of elements of the field F. Then, with the notation of Theorem 1, for any $n \geq 1$ we have

$$\frac{n+1}{2} - \frac{1}{2}\deg(A_{j+1}) \leq L(n) \leq \frac{n}{2} + \frac{1}{2}\deg(A_j)$$

with the interpretation $\deg(A_0) = -1$, where $j \geq 0$ is uniquely determined by the condition

$$\deg(Q_{j-1}) + \deg(Q_j) \leq n < \deg(Q_j) + \deg(Q_{j+1}) .$$

Proof. By (3) the bounds for n can be rewritten in the form

$$2\deg(Q_j) - \deg(A_j) \leq n \leq 2\deg(Q_j) + \deg(A_{j+1}) - 1 ,$$

which checks also for $j = 0$. This is equivalent to

$$\frac{n+1}{2} - \frac{1}{2}\deg(A_{j+1}) \leq \deg(Q_j) \leq \frac{n}{2} + \frac{1}{2}\deg(A_j) .$$

Since $L(n) = \deg(Q_j)$ by Theorem 1, the desired result is shown. □

It is convenient to use a quantity introduced in Niederreiter [6], [7]. For an irrational generating function $S = \sum_{i=1}^{\infty} s_i x^{-i} \in F((x^{-1}))$ let

$$S = [0, A_1, A_2, \ldots]$$

be its continued fraction expansion. Then we define

$$K(S) = \sup_{j \geq 1} \deg(A_j) ,$$

where we can have $K(S) = \infty$. Our interest will, however, be in the case where

$K(S) < \infty$. We note that by Theorem 2 the sequence $s_1, s_2, \ldots$ has a PLCP if and only if S is irrational and $K(S) = 1$. The following is an immediate consequence of Theorem 4.

Corollary 1. Let $s_1, s_2, \ldots$ be a sequence of elements of F which has an irrational generating function S (or, equivalently, which is not a shift register sequence). Then

$$\frac{1}{2}(n + 1 - K(S)) \leq L(n) \leq \frac{1}{2}(n + K(S)) \quad \text{for all } n \geq 1 .$$

The result of Corollary 1 was shown for $F = F_2$ in Niederreiter [6] and for arbitrary finite fields in Niederreiter [7]. If $K(S) < \infty$, then the bounds for $L(n)$ in Corollary 1 are in general best possible. Choose $j \geq 1$ with $\deg(A_j) = K(S)$ and put $n = \deg(Q_{j-1}) + \deg(Q_j)$, then $L(n)$ equals the upper bound by Theorem 1. Choose $j \geq 1$ such that $\deg(A_j) = K(S)$ and $n = \deg(Q_{j-1}) + \deg(Q_j) - 1 \geq 1$, then $L(n)$ equals the lower bound by Theorem 1.

Sequences with an almost perfect linear complexity profile can be constructed on the basis of Corollary 1, by considering irrational generating functions S with a relatively small $K(S) > 1$. The precise information on the linear complexity profile given in Theorem 1 can be used to produce a desired pattern in the linear complexity profile by an appropriate choice of the polynomials $A_1, A_2, \ldots$. By a formula of Rueppel [8], [9, Ch. 4], the standard deviation of $L(n)$ for random sequences of bits is asymptotically equal to $\frac{1}{9}\sqrt{86} \approx 1.03\ldots$. Hence by Corollary 1, choosing an irrational generating function $S \in F_2((x^{-1}))$ with $K(S) = 2$ or 3 would be roughly in accordance with the asymptotic behavior of the standard deviation of $L(n)$.

Sequences of elements of an arbitrary field F with a prescribed linear complexity profile can be constructed by an appropriate choice of the polynomials $A_1, A_2, \ldots$ determining the continued fraction expansion for the generating function $S = \sum_{i=1}^{\infty} s_i x^{-i}$. If we have chosen the j polynomials $A_1, A_2, \ldots, A_j$ for some $j \geq 1$, then it is important to know to what extent this determines the sequence $s_1, s_2, \ldots$. We note that by (6) we have

$$v\left(S - \frac{P_j}{Q_j}\right) = -\deg(Q_j) - \deg(Q_{j+1}) . \tag{11}$$

Therefore all the terms s_i with

$$1 \leq i \leq \deg(Q_j) + \deg(Q_{j+1}) - 1 ,$$

and so at least all the terms with $1 \leq i \leq 2\deg(Q_j)$, are determined by $A_1, A_2, \ldots, A_j$ and they are not affected by later choices of $A_{j+1}, A_{j+2}, \ldots$. These terms agree with

the corresponding terms in the power series expansion of P_j/Q_j. The polynomials P_j and Q_j are easily obtained by the recursions in Section 3 with $A_0 = 0$. The calculation of the coefficients in the expansion of P_j/Q_j is facilitated by the fact that these coefficients form a shift register sequence with minimal polynomial Q_j (by Lemma 2). Therefore we only need the $\deg(Q_j)$ initial values of this shift register sequence, the remaining terms can be calculated quickly by the recursion of the form (1) with characteristic polynomial Q_j. But since

$$\deg(Q_j) \leq \deg(Q_{j-1}) + \deg(Q_j) - 1 \quad \text{for} \quad j \geq 2 ,$$

the $\deg(Q_j)$ initial values can be obtained from the expansion of P_{j-1}/Q_{j-1}.

This leads to the following algorithm for calculating the terms s_i with $1 \leq i \leq 2\deg(Q_j)$, given the polynomials $A_1, A_2, \ldots, A_j$. We assume that the polynomials $Q_1, Q_2, \ldots, Q_j$ have already been calculated by the recursion in Section 3 and we put $q_h = \deg(Q_h)$ for $1 \leq h \leq j$.

Step 1: We have

$$\frac{P_1}{Q_1} = \frac{1}{Q_1} = c^{-1} x^{-q_1} + \text{smaller powers of } x ,$$

where c is the leading coefficient of Q_1. Therefore we can calculate the terms s_i with $1 \leq i \leq q_1 + q_2 - 1$ by the recursion (1) with characteristic polynomial Q_1 and initial values

$$s_i = 0 \quad \text{for} \quad 1 \leq i \leq q_1 - 1, \qquad s_i = c^{-1} \quad \text{for} \quad i = q_1 .$$

Step h $(2 \leq h \leq j)$: Suppose the terms s_i with $1 \leq i \leq q_{h-1} + q_h - 1$ have already been calculated. Then we calculate the terms s_i with $q_{h-1} + q_h \leq i \leq q_h + q_{h+1} - 1$ from the previously calculated terms by the recursion (1) with characteristic polynomial Q_h. If $h = j$, then we replace $q_j + q_{j+1} - 1$ by $2q_j$.

If a sequence $A_1, A_2, \ldots$ of polynomials is given, then the algorithm above can be continued indefinitely. The third sentence in Step h can then be deleted. A special situation occurs in the case where F is the binary field F_2. It is due to the trivial fact that if $a \in F_2$ is given and we know that $b \in F_2$ satisfies $b \neq a$, then b is uniquely determined as the binary complement of a, i.e. $b = a + 1$. It follows from this fact that for $F = F_2$ the relation (11) also determines the term s_i with $i = \deg(Q_j) + \deg(Q_{j+1})$. Consequently, in the algorithm above the numbers $q_h + q_{h+1} - 1$ can be replaced by $q_h + q_{h+1}$ for $1 \leq h \leq j - 1$ whenever the underlying field is F_2, provided the term s_i with $i = q_h + q_{h+1}$ is defined as the binary complement of the corresponding term obtained from the recursion (1).

We show now that if the sequence $s_1, s_2, \ldots$ of elements of F is constructed by the algorithm above with polynomials $A_1, A_2, \ldots$, then its generating function S has the continued fraction expansion

$$S = [0, A_1, A_2, \ldots] .$$

For suppose we had a different expansion $S = [0, A'_1, A'_2, \ldots]$ with corresponding convergents P'_h/Q'_h for $h \geq 0$. Then there exists a least $k \geq 1$ with $A'_k \neq A_k$. By the construction we have

$$v(S - \frac{P_h}{Q_h}) \leq - \deg(Q_h) - \deg(Q_{h+1}) \quad \text{for all } h \geq 0 . \tag{12}$$

In particular $v(Q_k S - P_k) < 0$, hence by Lemma 3 we get

$$Q_k = \sum_{h=i}^{j} C_h Q'_h \quad \text{and} \quad P_k = \sum_{h=i}^{j} C_h P'_h$$

with $C_i \neq 0$, $C_j \neq 0$, and $\deg(C_h) < \deg(A'_{h+1})$ for $i \leq h \leq j$. By the second part of Lemma 3 and (12),

$$- \deg(Q'_{i+1}) \leq v(Q_k S - P_k) \leq - \deg(Q_{k+1}) < - \deg(Q_k) \leq - \deg(Q'_j) ,$$

hence we must have $i = j$. From $Q_k = C_j Q'_j$, $P_k = C_j P'_j$, and $\gcd(P_k, Q_k) = 1$ it follows that $C_j = c \in F$. Since $Q'_h = Q_h$ and $P'_h = P_h$ for $0 \leq h < k$, we must have $j \geq k$. On the other hand, (6) and (12) yield

$$- \deg(Q'_k) = v(Q'_{k-1} S - P'_{k-1}) = v(Q_{k-1} S - P_{k-1}) \leq - \deg(Q_k) = - \deg(Q'_j) ,$$

hence $j \leq k$, and so $j = k$ and $Q_k = cQ'_k$, $P_k = cP'_k$. Then by (4),

$$(- 1)^k = P_{k-1} Q_k - P_k Q_{k-1} = cP'_{k-1} Q'_k - cP'_k Q'_{k-1} = (- 1)^k c ,$$

thus $c = 1$. From $Q_k = Q'_k$ it follows that

$$A_k Q_{k-1} + Q_{k-2} = A'_k Q'_{k-1} + Q'_{k-2} = A'_k Q_{k-1} + Q_{k-2} ,$$

hence $A_k = A'_k$, which is a contradiction.

REFERENCES

[1] L. E. Baum and M. M. Sweet: Badly approximable power series in characteristic 2, Ann. of Math. 105, 573–580 (1977).

[2] Z.-D. Dai and Z.-X. Wan: A relationship between the Berlekamp-Massey and the Euclidean algorithms for linear feedback shift register synthesis, Preprint, Academia Sinica, Beijing, 1986.

[3] R. Lidl and H. Niederreiter: Finite Fields, Addison-Wesley, Reading, Mass., 1983.

[4] R. Lidl and H. Niederreiter: Introduction to Finite Fields and Their Applications, Cambridge Univ. Press, Cambridge, 1986.
[5] J. L. Massey: Shift-register synthesis and BCH decoding, IEEE Trans. Information Theory 15, 122-127 (1969).
[6] H. Niederreiter: Continued fractions for formal power series, pseudorandom numbers, and linear complexity of sequences, Contributions to General Algebra 5 (Proc. Conf. Salzburg, 1986), Teubner, Stuttgart, to appear.
[7] H. Niederreiter: Cryptology - The mathematical theory of data security, Proc. Internat. Symp. on Prospects of Math. Science (Tokyo, 1986), to appear.
[8] R. A. Rueppel: Linear complexity and random sequences, Advances in Cryptology - EUROCRYPT '85 (F. Pichler, ed.), Lecture Notes in Computer Science, Vol. 219, pp. 167-188, Springer-Verlag, Berlin, 1986.
[9] R. A. Rueppel: Analysis and Design of Stream Ciphers, Springer-Verlag, Berlin, 1986.
[10] Y. Taussat: Approximation diophantienne dans un corps de séries formelles, Thèse, Université de Bordeaux, 1986.
[11] M.-Z. Wang and J. L. Massey: The characterization of all binary sequences with a perfect linear complexity profile, Paper presented at EUROCRYPT '86, Linköping, 1986.

The author gratefully acknowledges support for this research project by the Austrian Ministry for Science and Research.

WHEN SHIFT REGISTERS CLOCK THEMSELVES

Rainer A. Rueppel

Crypto AG
6312 Steinhausen
Switzerland

Abstract:

A new class of sequences, which we term [d,k] self-decimated sequences, is investigated. For appropriate choices of [d,k] these sequences possess large periods, balanced k-distributions, large linear complexities, and moderate out-of-phase autocorrelation magnitudes. Furthermore, they are easy to generate. These properties suggest that [d,k] self-decimated sequences may have some applications in cryptography and spread spectrum communication.

1 INTRODUCTION

Imagine we let the output sequence of a binary linear feedback shift register (LFSR) determine its own clock in the following way: whenever the output symbol is a '0', d clock pulses are applied to the LFSR, and, in case the output symbol is a '1', k clock pulses are applied to the LFSR. Figure 1 illustrates the system.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 53-64, 1988.
© Springer-Verlag Berlin Heidelberg 1988

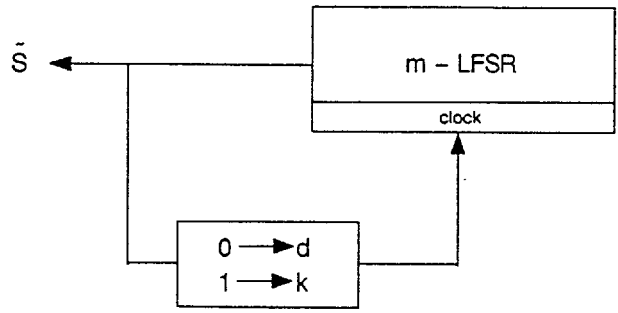


Fig. 1. Self-clocking LFSR

Suppose the above LFSR has a primitive connection polynomial $C(D) = 1 + D + D^2 + D^3 + D^4 + D^5$ and is started in state [1 1 1 1 1]. When the self-clocking rule [d,k] is chosen to be [1,2] (i.e., for a '0', the LFSR is clocked once, and for a '1' the LFSR is clocked twice), then the following periodic sequence will appear at the output of the system:

$$\underline{s} = (1111010100001101100 11)_0^{\infty}$$

This sequence has remarkable properties: (1) the distribution of k-tuples is 'balanced' (to be more precise: for $1 \le k \le 3$, the frequencies of k-tuples differ by at most 2); (2) the linear complexity (or linear span) of $\underline{s}$ is 20, which is the maximum possible for a sequence of period 20; (3) the periodic autocorrelation function of $\underline{s}$ has a peak out-of-phase magnitude of 0. This self-clocking operation can be interpreted as a generalization of the well-known and widely-studied decimation operation for LFSR-sequences ([1],[2],[3],[4]). The conventional decimation of a sequence $\underline{r}$ by a constant d is defined as the extraction of every d-th digit of $\underline{r}$, usually denoted as $\underline{r}[d]$. When a binary sequence $\underline{r}$ is [d,k]-"self-clocked", then it is no longer decimated by a constant but by a function of the previous sequence digit; we will term the resulting sequence a [d,k] self-decimated sequence. Let $\underline{r}$ be the original m-sequence

produced by the LFSR in Figure 1. Then the following example compares ordinary decimation by 2, and [1,2] self-decimation of r into s.

$$r = 1111100100110000101101010 0\ldots$$
$$r[2] = 111001001 1000\ldots$$
$$r[1,2] = 11101010000110110\ldots$$

Throughout this paper we will restrict ourselves to the case where the original sequences are maximum-length sequences over GF(2). Furthermore, without loss of generality, it is assumed that $0 < d,k < 2^L-1$, since, as with ordinary decimation, any d or k greater than 2^L-1 can be reduced mod 2^L-1.

If d is a unit mod 2^L-1, (i.e. d has an inverse mod 2^L-1), then

$$r[d,k] = r[d][1,k']$$

where

$$k' = k \cdot d^{-1} \mod 2^L - 1$$

That is, the self-decimation operation can be broken up into a constant decimation by d followed by a self-decimation of the special form [1,k']. If d is a unit mod 2^L-1 then $r[d]$ is again an m-sequence of same degree and period. It is to be expected that for given self-clocking rules [d,k] certain properties like period or bit distribution are invariant over the set of all m-sequences of same degree. In general, there are $\phi(2^L-1)\cdot(2^L-2)$ pairs [d,k] with d being a unit; $\phi(n)$ denotes Euler's totient function. If 2^L-1 is a Mersenne prime then all pairs [d,k] can be reduced to [d][1,k']. Thus, almost all cases can be covered by investigating self-decimation rules of the form [1,k].

Clearly the state diagram of the self-decimated m-LFSR will contain (one or more) cycles and tails. Depending on the initial state there may be a preperiod in the self-decimated sequence. A

resetting sequence is a subsequence of the original m-sequence which guarantees that the digit directly following the subsequence belongs to the self-decimated sequence. As an example let [d,k] be [1,2], then 0 is a resetting sequence. For, if the 0 itself belongs to the self-decimated sequence, so must its successor by the fact d=1; if, on the other hand, the 0 does not belong to the self-decimated sequence, its successor must by the fact k=2. This implies that, if a resetting sequence can be identified in the original m-sequence, then there exists only one cycle in the state diagram of the self-decimated m-LFSR, or equivalently, there exists only one self-decimated sequence (disregarding the preperiods for the moment).

Let us hypothetically assume that the original sequence is not an m-sequence but is comprised of N random bits which are repeated periodically. For [d,k] = [1,2], in the average every 1.5th digit is selected for the self-decimated sequence, or in other words, 2/3 of the original N random bits will appear in the self-decimated sequence. The first 0 among the N random bits will be a resetting subsequence. Thus, the period of the self-decimated sequence is expected to be approximately 2/3 N. As we will see in section 2 this is in perfect agreement with the theoretical results obtained for m-sequences. In section 3. some experimental data about characteristic properties like linear complexity and autocorrelation is shown.

2 THEORETICAL RESULTS

This section shall serve to demonstrate that despite the highly nonlinear setup of a self-clocking LFSR some analytical results can be obtained. The first topic of interest is the period.

Theorem 1: A [d,k]-self-decimated m-sequence of degree L has period

$$T_L = \left\lfloor \frac{2}{3}\left(2^L - 1\right) \right\rfloor$$

if [d,k] = g[1,2] mod 2^L-1 with gcd(g,2^L-1) = 1.

Proof: 0 is a resetting sequence; any following digit in the original m-sequence must belong to the periodic part of the self-decimated sequence.

It follows that for any subsequence $01^m x$ of the m-sequence the digit x will belong to the self-decimated sequence if and only if m is even.

From the theory of m-sequences the frequencies of such subsequences are known:

$$\#\{0x\} = 2^{L-1} - 1$$
$$\#\{01^m x\} = 2^{L-m-1} \qquad m = 1, .., L-1$$
$$\#\{01^L x\} = 1$$

Thus, the number of digits that will appear in the periodic part of the self-decimated sequence can be found by a simple counting argument.

If L is odd we have

$$T_L = 2^{L-1} - 1 + 2^{L-3} + 2^{L-5} + \ldots + 2^0$$

If L is even we have

$$T_L = 2^{L-1} - 1 + 2^{L-3} + 2^{L-5} + \ldots + 2^1 + 1$$

Using the fact that

$$\sum_{i=0}^{n-1} q^i = \frac{q^n - 1}{q - 1}$$

we obtain

$$T_L = \frac{2}{3}\left(2^L - 2\right) \qquad \text{L odd}$$

$$T_L = \frac{2}{3}\left(2^L - 1\right) \qquad \text{L even}$$

which proves the theorem.

The identical argument can be carried out for [d,k] = g[1,2^{L-1}] since 2[1,2^{L-1}] ≡ [2,1] mod 2^L-1. In this case 1 is a resetting sequence and

$$T_L = \left\lceil \frac{2}{3}\left(2^L - 1\right) \right\rceil$$

Note that for odd L [1,2^{L-1}] self-decimation yields a period which is one digit larger than for [1,2] self-decimation. The reason lies in the fact that the number of ones in an m-sequence exceeds the number of zeros by one.

Theorem 2: The absolute frequency of ones, $N_L(1)$, in the periodic part of a [d,k] self-decimated m-sequence of degree L is given as

$$N_L(1) = \left\lceil \frac{1}{3}\left(2^L - 1\right) \right\rceil$$

if [d,k] = g[1,2] mod 2^L-1 with gcd(g,2^L-1) = 1.

Proof: 0 is a resetting sequence; any following 1 in the original m-sequence must belong to the periodic part of the self-decimated sequence.

Thus, for any subsequence 01^{m+1} of the m-sequence the final 1 will belong to the self-decimated sequence if and only if m is even.

From the theory of m-sequences the frequencies of such subsequences are known:

$$\#\left\{01^{m+1}\right\} = 2^{L-m-2} \qquad m = 0,\ldots,L-2$$
$$\#\left\{01^{L}\right\} = 1$$

If L is odd we have

$$N_L(1) = 2^{L-2} + 2^{L-4} + \ldots + 2^1 + 1$$

If L is even we have

$$N_L(1) = 2^{L-2} + 2^{L-4} + \ldots + 2^0$$

We obtain

$$N_L(1) = \frac{1}{3}(2^L + 1) \qquad L \text{ odd}$$

$$N_L(1) = \frac{1}{3}(2^L - 1) \qquad L \text{ even}$$

which proves the theorem.

Note that for even L the bit distribution is perfectly balanced, i.e., $N_L(0) = N_L(1) = T_L/2$.

Theorem 3: Let [d,k] = g[1,2] mod 2^L-1 with $\gcd(g,2^L-1) = 1$. Then the absolute frequencies of bit pairs, $N_L(b_1,b_2)$, within one period of a [d,k] self-decimated m-sequence of degree L are bound by

$$N_L^0 \le N_L(b_1,b_2) \le N_L^0 + 2$$

with

$$N_L^0(00) = \left\lfloor \frac{1}{6}(2^L - 1) \right\rfloor - 1$$

$$N_L^0(01) = \left\lfloor \frac{1}{6}(2^L - 1) \right\rfloor$$

$$N_L^0(10) = \left\lfloor \frac{1}{6}(2^L - 4) \right\rfloor$$

$$N_L^0(11) = \left\lfloor \frac{1}{6}(2^L - 4) \right\rfloor$$

Proof: *case a*: 00

0 is a resetting sequence; any following pair '00' in the original m-sequence must belong to the periodic part of the self-decimated sequence.

It follows that for any subsequence 01=00 of the m-sequence the pair 00 will belong to the self-decimated sequence if and only if m is even.

From the theory of m-sequences the frequencies of such subsequences are known:

$$\#\{000\} = 2^{L-3} - 1$$
$$\#\{01^m00\} = 2^{L-m-3} \qquad m = 1,..,L-3$$

Subsequences longer than L may or may not exist as long as the number of consecutive 1's does not exceed L. Thus

$$\#\{01^m00\} = 0 \text{ or } 1 \qquad m = L-2,..,L$$

It follows that

$$N_L(00) \geq -1 + \sum_{\substack{m=0 \\ m \text{ even}}}^{L-3} 2^{L-m-3} = -1 + \left\lfloor \frac{1}{6}(2^L - 1) \right\rfloor$$

which proves the lower bound.

From the uncertain overlong subsequences at most 2 could contribute an entry, which proves the upper bound.

case b: 01
For any subsequence 01^m01 of the original m-sequence the final pair 01 will belong to the self-decimated sequence if and only if m is even.

$$\#\{01^m01\} = 2^{L-m-3} \qquad m = 0,..,L-3$$
$$\#\{01^m01\} = 0 \text{ or } 1 \qquad m = L-2,..,L$$

It follows that

$$N_L(01) \geq \sum_{\substack{m=0 \\ m \text{ even}}}^{L-3} 2^{L-m-3} = \left\lfloor \frac{1}{6}(2^L - 1) \right\rfloor$$

case c: 10
For any subsequence 01^mx0 (x arbitrary) of the original m-sequence the final pair 10 (x dropped) will belong to the self-decimated sequence if and only if m is odd.

$$\#\{01^mx0\} = 2^{L-m-2} \qquad m = 1,..,L-3$$
$$\#\{01^mx0\} = 0 \text{ or } 1 \qquad m = L-2,..,L$$

Therefore

$$N_L(10) \geq \sum_{\substack{m=1 \\ m\,odd}}^{L-3} 2^{L-m-2} = \left\lfloor \frac{1}{6}(2^L - 4) \right\rfloor$$

case d: 11
This case is analogous to case c with the final 0 in the subsequence replaced by 1.

Since the lower bounds differ by at most 1, theorem 3 implies that the absolute frequencies of bit pairs cannot differ by more than 3.

The trace from GF(2^L) into GF(2) is defined as

$$Tr(\beta) = \beta + \beta^2 + \ldots + \beta^{2^{L-1}}$$

where β is an element of GF(2^L). With the help of the trace function the jth digit of an m-sequence can be compactly expressed [4] as

$$r_j = Tr(A\alpha^j)$$

where α is a root of the minimal polynomial of r, and A relates to the initial phase of r.

For a [d,d+1] self-decimated m-sequence we obtain

$$s_j = Tr(A\alpha^{e_j})$$

with

$$e_j = d \cdot j + \sum_{k=0}^{j-1} s_k$$

This leads to the following (nonlinear) recursion of the exponents

$$e_{j+1} = e_j + d + Tr(A\alpha^{e_j})$$

As was mentioned before, the state diagram of a [d,k] self-decimated m-LFSR contains (one or more) cycles and tails. Every tail must contain an initial state (which we call a root in this context), and must finally join either another tail or a cycle. These junction

states are particular in the sense that they have two predecessors but only one successor. This implies that for a junction state's exponent e_{j+1} there exist two distinct exponents e_j and e_j', corresponding to the two distinct predecessors. Consequently

$$e_j - e'_j = Tr\left(A\alpha^{e_j}\right) - Tr\left(A\alpha^{e'_j}\right)$$

Without loss of generality assume $e_j > e_j'$. Then, for [d,d+1] self-decimation, we obtain

$$1 = Tr\left(A\alpha^{e_j-1}\right) - Tr\left(A\alpha^{e'_j}\right)$$

This equation tells us that a transition from 1 to 0 has occurred in the original m-sequence. The number of such transitions is 2^{L-2}. This proves the following theorem.

Theorem 4: The number of roots, (i.e. states with no predecessor) in the state diagram of a [d,d+1] self-decimated m-LFSR of length L is

$$N_L(\text{roots}) = 2^{L-2}$$

Since roots cannot be part of a cycle the following corollary is obvious.

Corollary 5: The period of a [d,d+1] self-decimated m-sequence of degree L is bound from above by

$$T_L \leq 2^L - 1 - 2^{L-2} = \left\lfloor \frac{3}{4}(2^L - 1) \right\rfloor$$

3 EXPERIMENTAL RESULTS

Extensive simulations have been run for [1,2] self-decimated m-sequences of degrees L=3,...,11. They showed that, for given L,

also the pair distributions, (beside period and bit distributions), were independent of the minimal polynomial of the m-sequence (see table 1).

L	T_L	$N_L(0)$	$N_L(1)$	$N_L(00)$	$N_L(01)$	$N_L(10)$	$N_L(11)$
4	10	5	5	2	3	3	2
5	20	9	11	4	5	5	6
6	42	21	21	10	11	11	10
7	84	41	43	20	21	21	21

Table 1. Periods, bit, and pair distributions.

Exhaustive searches over all primitive polynomials of degree L = 5,6,7,8 revealed the following agerages and minimum values for the linear complexities of [1,2] self-decimated m-sequences:

L	T_L	L_{avg}	L_{min}
5	20	19,3	16
6	42	38,7	33
7	84	82	78
8	170	169,3	166

Table 2. Linear complexities

The proximity of L_{avg} to the period length T_L and the largeness of the minimal encountered linear complexity L_{min} speak for themselves.

Another topic of interest is the periodic autocorrelation function. Exhaustive searches over all primitive polynomials of degrees L = 4,5,6,7 revealed the following averages R_{avg} and minimum values R_{min} for the peak out-of-phase autocorrelation magnitude of [1,2] self-decimated sequences:

L	R_{avg}	R_{min}
4	4	2
5	4	0
6	9.3	6
7	20.5	12

Table 3. Out-of-phase autocorrelation magnitudes

4 CONCLUSION

[d,k] self-decimated m-sequences are (almost) as easy to generate as m-sequences; for appropriately chosen [d,k] they exhibit similar properties as m-sequences with respect to period, k-distributions, and autocorrelation. But they behave much more like 'truly' random sequences as is indicated by the high linear complexity values. Therefore [d,k] self-decimated sequences may have some applications in cryptography and spread spectrum communication.
But a word of caution has to be added; if a [d,k] self-decimated m-LFSR is employed alone and [d,k] are made public, then from its output sequence the feedback polynomial and the initial state of the LFSR are easily retrieved (a system of linear equations has to be solved).

5 REFERENCES

[1] N. Zierler, "Linear recurring sequences", J. Soc.. Indust. Appl. Math., Vol. 7, 1959.

[2] D.V. Sarwate, M.B. Pursley, "Crosscorrelation properties of pseudorandom and related sequences", Proc. of the IEEE, Vol. 68, May 1980.

[3] J.L. Massey, R.A. Rueppel, "Linear ciphers and random sequence generators with multiple clocks", Proc. of Eurocrypt 84, Paris, Lecture Notes in Computer Science, Vol. 209, Springer-Verlag.

[4] R.A. Rueppel, "Analysis and design of stream ciphers", Springer-Verlag, 1986.

FINITE STATE MACHINE MODELLING OF CRYPTOGRAPHIC SYSTEMS IN LOOPS

Franz Pichler
Laboratory for Systems Theory and Information Engineering
Institute of Systems Science
Johannes Kepler University Linz
A-4040 Linz, Austria

Introduction

Finite State Machines and Sequential Switching Circuits are important system types for modelling cryptographic devices. This is especially true for the case of secret key systems for which automata-theoretical models can be very easily found, either for the whole system or for certain parts of it. Typical examples are stream-cipher systems with a Pseudo-Noise Machine PNM which can be modelled by an aggregate of coupled shift-registers. However, the concept of a Finite State Machine is also very useful for block-cipher operations, such as realized by the DES-Algorithm. An example is the sequential generation of the DES substitution boxes.

This paper is based upon the assumption that Finite State Machine Theory (FSM theory) can be effectively applied to cryptologial investigations, such as the design of ciphering devices or in cryptanalysis.

However, it is well known, that many problems of FSM theory are considered computationally hard. Some have even been proven to be NP complete. Furthermore the state sets of FSM-models of cryptographic devices are often inevitably very large. Therefore we have to take into account that the application of FSM-theory to cryptology is generally very limited. This is especially true whenever we have to use the usual algorithms described in textbooks and articles on FSM-theory. As a consequence, very little attention has been paid to FSM theory in cryptology.

To ameliorate this situation we suggest that the following studies be undertaken:

- the investigation of special types of finite state machines, which have a mathematical structure rich enough to allow a "speed up" of related algorithms,
- the construction of interactive data-bases for FSM methods to support the user in explorative programming.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 65-73, 1988.
© Springer-Verlag Berlin Heidelberg 1988

Our laboratory is now conducting studies in both of these fields. This paper mainly describes our efforts to establish an interactive database for FSM methods, which we designate as CAST.FSM (Computer Aided System Theory in the case of Finite State Machines).
CAST.FSM is implemented by the object oriented programming environment LOOPS which is based on INTERLISP-D. It runs on workstations SIEMENS 5815 (equivalent to XEROX 1108). Today, CAST.FSM is still in an early stage of development. Currently it is mainly being used for educational purposes. We hope to perfect our system so that it can be successfully applied to chip design ("Design for Testability") and to cryptography.

Finite State Machines and Cryptography

Before we describe CAST.FSM we would like to explore the general situation for the application of finite state machines in cryptology. We will limit ourselves to secret key systems. This category of cryptographic systems is still the principal one which is being used in practical applications. Secret key systems can be divided into stream cipher systems and block cipher systems.

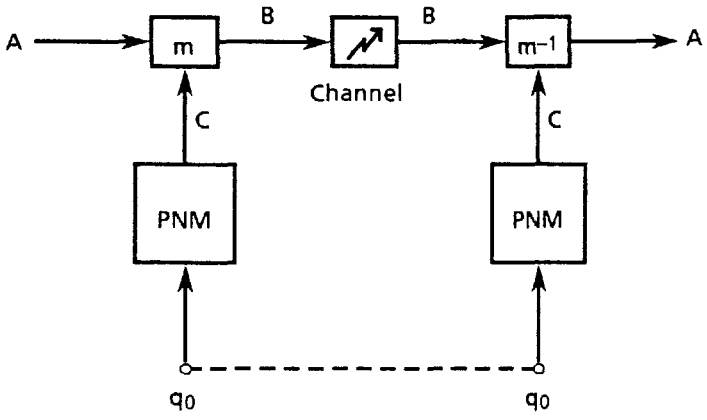


Figure 1: Stream-cipher system (E,D)

Figure 1 shows the block diagram of a stream cipher system (E,D). The encryption operation E consists of the mixing function m:AxC→B which transforms letter by letter from the plaintext into the ciphertext. This is controlled by a keying-sequence generated by the

pseudo-noise machine PNM. The decryption operation D is constructed by the inverse mixing operation m^{-1} together with a copy of PNM.
It is a well known fact that the pseudo noise machine PNM can very often be successfully modelled by FSM concepts. Therefore CAST.FSM can be applied to the design and analysis of stream-cipher systems.

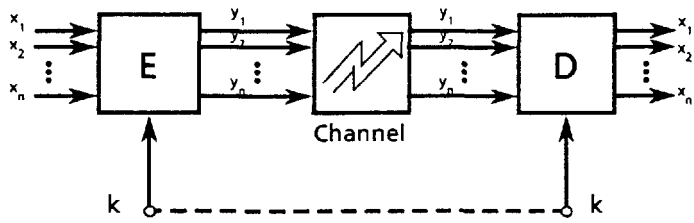


Figure 2: Block-cipher system (E,D)

In Figure 2 we show the diagram of a block-cipher system.
The encryption operation E receives the plaintext x in vector form $x=(x_1,x_2,\ldots,x_n)$ and computes under the control of the session key k the ciphertext $y=(y_1,y_2,\ldots,y_n)$.
A prominent example of a block-cipher system is the DES-algorithm in book-cipher mode.

The "inners" of the operations E and D is usually constructed by an iterative array as shown in Figure 3 (for the case of E).

Functions are the most frequently used mathematical models for E and D of a block-cipher system with a structure such as seen in Figure 3. However, the functions involved have to be, in a certain sense, very "complex" and the cryptographic design is, as a consequence, then also very difficult. The program system CAST.FSM has proven to be a useful tool for the analysis of such functions. To provide the reader with a concrete example of a block cipher operation with an internal iterative array structure we refer to the DES, in which m=16 (= number of "rounds") and the individual functions E(i) which are iterated have an internal structure such as shown in Figure 4.

We applied CAST.FSM successfully to the investigation of the symmetry of the S-boxes used to realize the function of Figure 4.

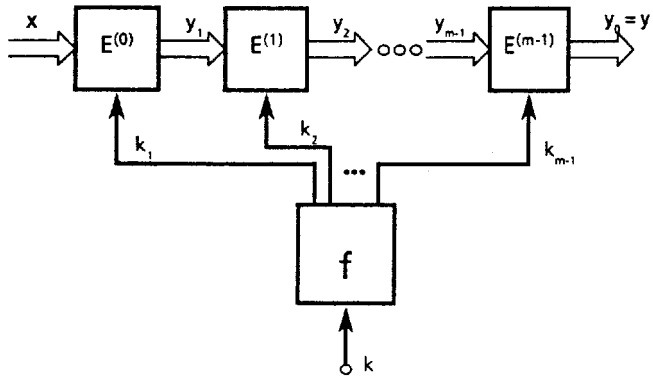


Figure 3: Iterative array structure of a block-cipher operation E

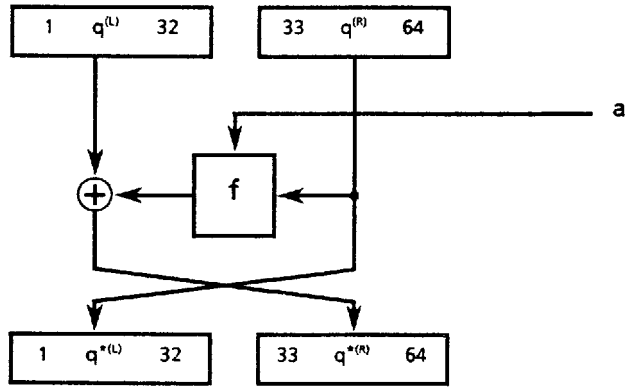


Figure 4: Internal structure of the functions E(i) of the DES

CAST-Computer Aided Systems Theory

Many important results of systems theory have not yet found any practical applications. One of the main reasons for this is that software for applying a systems method is often not available for the practitioner. If available, it frequently has a low degree of portability. Therefore, systems theorists have recently directed a lot of attention towards the improvement of this situation. They are striving to make it easier for the practitioner to gain access to operationally usable systems-theoretical knowledge.

One undertaking of this kind has been designated as CAST. An important difficulty which must be overcome is the establishment of a framework applicable for the definition of the shell of concrete CAST method bases.

We have developed such a framework in the form of the STIPS-machine STIPS.M (STIPS stands for Systems Theory Instrumented Problem Solving).
STIPS.M consists of a set of well-defined general systems-types, a set of systems-transformations which map systems types to each other and production rules which define the (type, transformation)→type transition. For CAST we need, in addition to the conceptional CAST shell STIPS.M, a programming environment which can be conveniently used for the development of CAST algorithms (= a well-selected sequence of systems-transformations) and for "ad hoc" programming tasks.

CAST.FSM an interactive data base in LOOPS for finite state machines

CAST.FSM is a finite state machine problem solver which is based on the CAST-shell STIPS.M. It is implemented in LOOPS/INTERLISP-D on SIEMENS 5815 workstations.
The currently available system classes can be selected from the Systems Theory Class Browser of CAST.FSM shown in Figure 5. This browser constitutes a tool on the first command level for solving FSM problems.

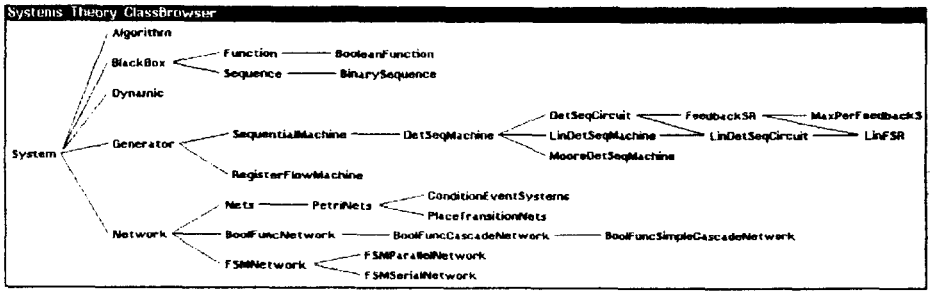


Figure 5: Systems Theory Class Browser

The development of a systems algorithms for solving FSM problems is supported by the realization tree browser. Figure 6 shows a typical example of a realization tree; it was developed during the computation of a parallel-decomposition of a finite state machine M.

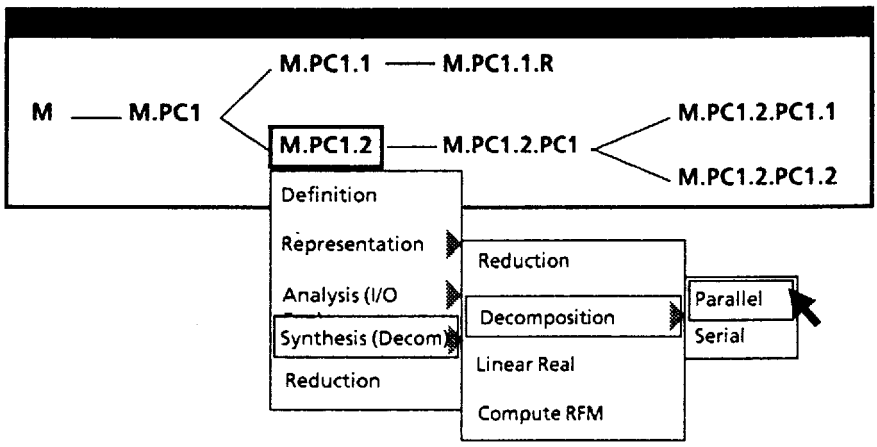


Figure 6: Realization tree

To elucidate the kinds of systems types and systems transformations which have been incorporated in CAST.FSM we provide the following examples:

systems types

- finite state machine FSM
  represented by table or state graph
- linear finite state machine LFSM
  represented by matrices (A,B,C) or by data-flow graph
- parallel network of FSM's
  represented by block diagram. The relative size of the individual machines is shown by the block-size.
- serial network of a FSM
- Petri-net
  represented by a bi-partite graph with token transitions
- register flow machine
  represented by tables or data flow graph

- lattice of a FSM
  represented by state-set partitions together with the related Hasse-diagram
- homing tree, diagnosing tree, synchronizing tree
  represented by tables

systems transformations

- FSM decomposition
  (method of Hartmanis-Stearns)
- linear realization of FSM's
  (method of B. Reusch)
- register flow machine realization of FSM's
  (method of Bainbridge)
- shiftregister realization of FSM's
  (method of Böhling)
- computation of the minimal length LFSR for a given binary sequence
  (method of Massey-Berlekamp)
- parallel-decomposition of LFSM's
  (by computation of the rational-canonical form of the matrix A)
- cascade-decomposition of switching functions
  (method of Ashenhurst)
- FSM state-identification
  (by homing-, diagnosis- and synchronizing experiments)
- FSM inversion

Figure 7 shows a typical display obtained when a certain finite state machine M is investigated with CAST.FSM.

CAST.FSM has already proven to be, even at its current stage of development, as a useful tool for dealing which cryptographic tasks. Besides the implementation of additional transformations we are also interested in studying the different possibilities for implementing CAST.FSM on other, more powerful workstations such as the SIEMENS WS 30 (Apollo DN 3000) enhanced by the PARWELL multiprocessor-system.

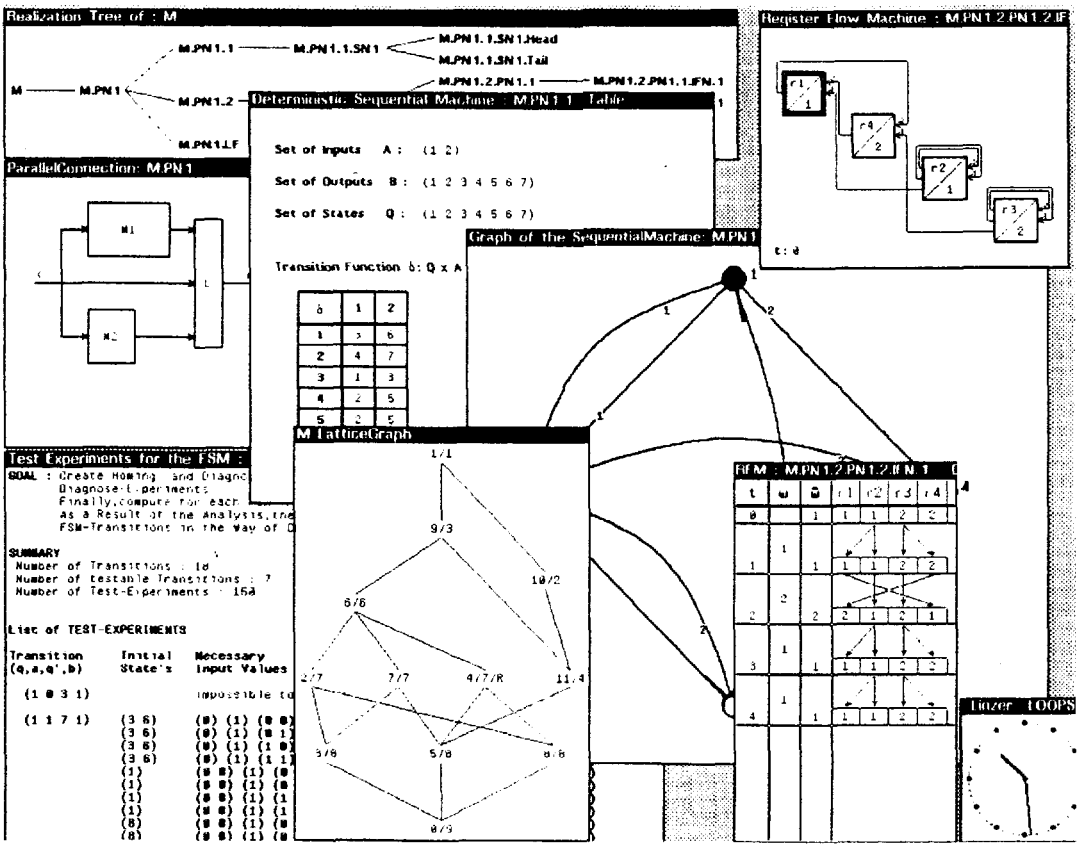


Figure 7: Typical CAST.FSM display

Conclusions

The paper emphasizes the importance of finite state machine theory for cryptology. A fundamental requirement for the application of this theory is the availability of relevant computerized tools.
The software system CAST.FSM constitutes such a tool. The LOOPS language and the user-friendliness of the SIEMENS 5815 workstation enables one to concentrate on the problem solving task and frees him from cumbersome programming and data handling. Although most of the researchers in cryptography seem to ignore the "good old stuff" of finite state machine theory, the author believes that this theory, together with appropriate CAST implementations, could prove to be a valuable tool in crpytographical research.

References

Pichler Franz and Herbert Prähofer:
CAST.FSM
A program system to support Finite State Machine methods
Proceedings of EMCSR 88 (ed. R. Trappl)
North Holland, Amsterdam 1988 (in preparation)

Pichler Franz and Heinz Schwärtzel:
CAST: Computerunterstützte Systemtheorie
Springer-Verlag Berlin (in preparation)

Prähofer Herbert:
LOOPS-Implementierung von automatentheoretischen Methoden
für "Design for Testability" Anwendungen
Master thesis, University Linz, September 1986

Pichler Franz:
Anwendung der Automatentheorie in der Kryptologie
in: E und I (Elektrotechnik und Informationstechnik),
105. Jahrgang, Heft 1, Jänner 1988, Springer-Verlag Wien
(in press)

Random Sources for Cryptographic Systems

G.B. Agnew
Dept. of Electrical Engineering
University of Waterloo
Waterloo, Ontario, Canada

Introduction

Random and pseudorandom sources have long been of interest in the study of statistical processes. Applications include the simulation of computer networks and communication channels. In these systems, pseudorandom sources (PRS) are preferred over true random sources (TRS) as repeatability (recreatability) of observations is important.

In cryptographic systems, true random sources are preferred in some instances to prevent penetration or influence of the system (some cryptographic systems in fact assume the availability of a true random source [1], [2]). It is observed that the strongest cryptographic system may be rendered insecure if an attacker can influence the generation of keys (presumably either by a PRS or TRS).

In this study we define a binary true random source (BTRS) as a device which generates an output pattern of 1's and 0's such that they are bitwise iid and all 2^n sequences of n bits are equally likely for any integer n. BTRS are generally based on measuring naturally occurring events such as sampling diode shot noise [3], time between radioactive particle decay, etc.. These devices are generally built external to the device using the random source. This is done for a number of reasons. First, the technology used to realize the random source is generally incompatible with the technology used for the cryptographic system (cryptodevice). Second, due to the nature of the process observed, temperature control or compensation is generally required. Third, shielding to prevent influence or observation may be required. Finally, if the random source is proven bias at a later date, it can be replaced by another source without redesign of the cryptodevice. We make the distinction at this point, between binary true random sources for general applications and sources for cryptographic purposes. In the former, the device operates in a 'friendly' environment, while in the latter, we must assume that an active attacker may try to observe or bias the output of the device.

There are several drawbacks in using an external random source. If the system is subject to observation or influence along the path from the source to the cryptodevice, then the system may be insecure (in some cryptographic systems, it may not be necessary or desirable to expose the outcome of the random source to the external environment, e.g., Diffie-Hellman key exchange protocol [2]). To provide a cryptographically strong BTRS for implementation on a cryptodevice, several criteria must be met:

i) Compatibility with device technology
ii) Immunity to observation or influence
iii) Stability of source output and freedom from bias.

By including the BTRS as part of the cryptodevice, most observation attacks can be avoided. Outside influence on the other hand, can take on several forms. A good BTRS should provide immunity from outside influence due to:

i)Modulation of grounds, power, input or output lines
ii) electromagnetic fields or radiation
iii) temperature manipulation
iv) forced resetting of the device to a known starting state.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 77-81, 1988.
© Springer-Verlag Berlin Heidelberg 1988

If we look carefully at the above requirements, we observe that requirements (i) and (ii) can be realized by a device exhibiting common mode rejection [4]. This can be explained in the following way: if we use two similar devices, both subject to the same environment (external stimulus), and measure the *difference* between them, any common effects will be reduced or nullified. This observation leads us to the design of a VLSI implementation of a BTRS as discussed in the next section.

A Metal Insulator Semiconductor Capacitor

A common structure in semiconductor systems is a metal insulator semiconductor capacitor (MISC). While a thorough discussion of the operation of such devices is beyond the scope of this paper, a brief explanation of the concepts involved will be given. Figure 1 shows the general physical structure of a MISC. It consists of a p-type semiconducting substrate material covered by insulating silicon oxide (S_iO_2) with a metalized pad placed over the insulation (the area covered by the metalization defines the area of the MISC). The p-type substrate material has, by nature of the impurities planted in it, a deficiency of free electrons. If a positive voltage V_g is applied to the metal pad, a *potential well* will be formed under the metalization. This potential well is the result of electrons being drawn to the positive charge. Since the p-type material is deficient in free electrons, there will not be enough free electrons to *fill the well* and a net shortage will exist. The depth of the well is determined by V_g and the size of the charge (number of electrons) required to fill the well is determined by ϕ_s. This is shown in Fig. 2a.

No physical system will support such an imbalance in charge for an indefinite period. Electrons will eventually migrate into the potential well and fill it up (see Fig. 2b). The period over which electrons are collected is referred to as the integration period. If we now remove the voltage V_g from the metalization, a net surplus of electrons will be present and this "charge" can be measured.

There are two major mechanisms by which free electrons can be generated to fill the potential well: i)by radiation or, ii) by dark current generation. If the top of the semiconductor device is left open and many cells are coupled together, light can be used to illuminate the cells and generate free electrons. In this case, the number of electrons generated is proportional to the intensity of the light. This forms the basis of today's imaging Charge Coupled Devices (CCDs) used to replace the old tube-type television cameras.

The second process involves electrons generated by thermal processes (noise). While this effect is small as compared to the radiation effect, if the device is sealed against light, this effect is considerable. This effect is highly sensitive to the temperature of the device. It is generally agreed by theory and experimental observation, that the number of electrons generated by dark current over the integration period follows a Poisson process (the variance equals the mean) [5]. It also appears that the dark current generation between devices even in close physical proximity, is uncorrelated. This leads us to a proposed structure for a BTRS.

A VLSI Realization

The implementation consists of two identical structures, cells X and Y, in close physical proximity. We allow them to "charge" over the same integration period, then measure the difference in charge between them and assign either a 1 or 0 to the outcome. This amounts to the construction of a device with high common mode rejection since any attempt at influence will be common to both devices and removed in the comparison. In addition, the close physical proximity of the devices will result in consistent temperature affects in both cells.

To analyze the suitability of such a system as a BTRS, we will use some observed values from actual test CCDs (it should be noted that these devices were designed as "good" devices which exhibit low dark current characteristics, our purpose is opposite, to build cells which promote dark current effects). The dark current generated in the device is a function

of the size (area) of the cell and the dark current density (J_{gen}). The dark current density is highly dependent on the level of impurities in the p-type material and the temperature of the device. J_{gen} for a good device may range from 5 $nanoamps/cm^2$ to 1 $milliamp/cm^2$ at 20°C. For a cell with cross-sectional area of approximately 10^{-7} cm^2, the dark current generated is shown in Table 1 for various temperatures.

To create a random result, we must measure the difference in charge between the two cells. This will require the development of a differential sensing amplifier. Current sense amplifiers used in memory devices can resolve a difference down to about 100 electrons. If we model the Poisson distribution of the number of electrons arriving during the integration period as a normal distribution, then the characteristics of the process should be:

$$mean = \lambda_X - \lambda_Y \equiv 0$$
$$variance = \lambda_X + \lambda_Y \equiv 2\lambda$$

While the above would be the ideal case, it would be physically impossible to exactly match two cells even under the strictest control. In addition, we assume that if the difference in charge is too small for the differential amplifier to resolve, it will always be bias in one direction. Thus the generation of 1's and 0's will not be symmetric and we must put up with some bias. This bias will be inversely proportional to the number of electrons collected (i.e., the unresolved area of 100 electrons becomes less likely as the number of electrons collected increases). (Note: the simple lack of symmetry does not present a real problem in the generation of random sequences as the output of several such devices can be combined.)

In Table 2, we show the number of electrons which must be collected in order to provide various levels of accuracy (deviation from symmetry of 1's and 0's). In Table 3, we use the dark currents from Table 1 and calculate the integration period required to produce the various accuracies. These results can be interpreted in two ways: either the integration period must be increased at lower temperatures or, if the integration period is fixed, then the accuracy will decrease with temperature.

Summary

In this paper, we have presented a new approach to the generation of a random binary bit stream. This structure is compatible with current VLSI technology. We have shown that by using two cells subject to the same dark current generation process and taking the difference of charge between the two cells, a random result can be created. While much work remains to be done in this area, we feel the initial investigation shows great promise.

References

1. R. Rivest, A. Shamir, L. Adleman, 'On digital signatures and public key cryptosystems', Comm. of ACM, Vol. 21, Feb. 1978, pp.120-126

2. W. Diffie, M. Hellman, 'Privacy and authentication : An introduction to cryptography', Proc. of the IEEE, Vol. 67, March 1979, pp. 397-427.

3. D. Kahn, 'Cryptography and the origin of spread spectrum', IEEE Spectrum, Vol. 21, no. 9, Sept. 1984, pp. 70-80.

4. IEEE Standard Dictonary on Electrical and Electronic Terms, IEEE Inc., New York, 1984.

5. G. Hobson, 'Charge Transfer Devices', Edward Arnold, London, 1978.

Table 1

Dark Current for Typical Cell

TEMP	I_{gen}
20°C	100 pA
-10°C	10 pA
-30°C	1 pA

Table 2

Number of Electrons Required During Integration

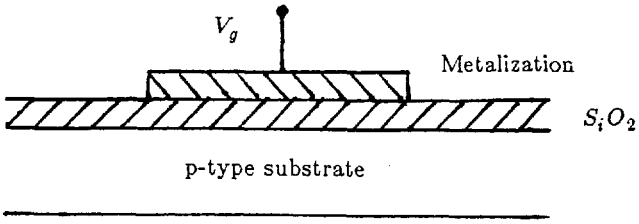
Accuracy	Number of Electrons λ
1%	10^7
5%	$5 * 10^5$
10%	10^5

Table 3

Integration Time

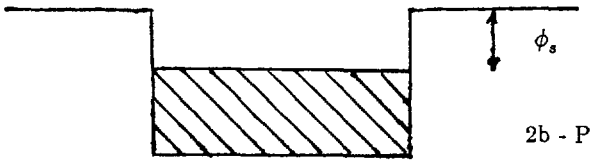
TEMP	1%	5%	10%
20°C	10 msec	0.3 msec	60 μsec
-10°C	100 msec	3 msec	600 μsec
-30°C	1 sec	30 msec	6 msec

Figure 1 - Metal Insulator Semiconductor Capacitor





2a - Empty Potential Well



2b - Partially Full Potential Well

Figure 2 - Potential Well

PHYSICAL PROTECTION OF CRYPTOGRAPHIC DEVICES

Andrew J. Clark

Computer Security Limited
31/32 High Street, Dorset Place
Brighton, East Sussex. BN2 1RP
United Kingdom

ABSTRACT

With the growth of user awareness for the need to protect sensitive computer data by cryptographic means, this paper explains the need to protect critical cryptographic variables (particularly keys, and in some cases algorithms) in a secure environment within cryptographic equipment, particularly those used in the area of high value funds transfer transactions.

Design principles are outlined, leading to the concept of tamper resistant and not tamper proof devices to protect key data, whether the data be retained within physically large devices or on small portable tokens.

Criteria for the detection of attempts to gain access to sensitive data rather than attack prevention are outlined, together with two types of attack scenario - invasive and non-invasive.

The risks of attack on cryptographic devices are surveyed and intruder attack objectives are outlined, together with some typical scenarios. The available counter-measures are discussed. Several discreet mechanisms are described.

Typical detection mechanisms and sensor systems are discussed plus the design trade-offs that must be made in implementation, in particular manufacturing and maintenance costs versus scope of attack protection.

Once an attack is detected, various data destruction mechanisms may be employed. The desirability of active data destruction by "intelligent" means is proposed, together with a discussion of alternative techniques with particular reference to the data storage device characteristics.

Some experiences of tamper resistant research and development highlight the potential manufacturing problems - particularly in respect of quality assurance, product fault analysis and life-testing.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 83-93, 1988.
© Springer-Verlag Berlin Heidelberg 1988

The desirability of tamper resistant standards and independent assessment facilities is expressed, the applicability of such standards and large scale protection methods on intelligent tokens, in particular smart cards and personal authenticators, is discussed.

What Do We Mean by Physical Protection?

Physical protection as applied to cryptographic equipment does not necessitate locking devices within mechanical safes or enclosing their electronics within thick steel or concrete shields, i.e. making them tamper-proof. It does, however, involve using sound design practices to construct a system capable of attack detection by a comprehensive range of sensors, i.e. tamper resistant. Few documents are available covering tamper resistant requirements and standards, among them are the Code of Practice for Cryptographic Equipment Security[1] and the U.S. Federal Standard[2]. They both require the incorporation of tamper-resisting, tamper- indicating, tamper-detecting and tamper-responding physical security measures. The level of physical security suggested should be such that unauthorised attempts at access or use will either be unsuccessful or will have a high probability of being detected during or after the event. Additionally, the standards recommend that cryptographic equipment should be prominently situated in operation so that its condition (outward appearance, indicators, controls etc.) is easily visible to minimise the possibility of undetected penetration.

This paper discusses both the concepts and detail of tamper resistant design to meet these requirements.

Why Do We Need It?

General principles of commercial cryptographic system design suggest that the security of data should not depend on the secrecy of the encryption methods used (for example, the algorithm) but rather on the secrecy of the key data. The generation and distribution of all key variables within a tamper-resistant environment such that no human being has knowledge of the key data in plain text is a necessary safeguard.

Classical Key Management hierarchies as described by Davies and Price[3] illustrate the need for high integrity key storage and

protection of top level 'master' keys. Clearly if these master keys are compromised by an intruder without detection the security of the whole system is brought into doubt. Attacks designed to discover such keys might involve physical penetration of the device to gain access to the internal components in which the keys are stored. This is particularly significant where many devices contain the same key, as may be the case in the design of some point-of-sale systems or networks of automatic teller machines. ATM's are almost certainly sufficiently well protected in their physical environment to defy attacks aimed at key discovery. Point-of-sale terminals however can be found on the counters of shops, large and small, and generally do not include particularly strong physical protection, largely because of the question of cost. It is quite possible that some point-of-sale terminals may be stolen with a view to compromising the secret parameters by an intruder. The fact that a terminal may be made unusable by the act of gaining access is immaterial to the intruder if profit is possible as a result of the discovery of the secret parameters, possibly by attacking the transactions at other terminals. What is required of the protection system is that the secret parameters should be lost or destroyed before access to them is gained.

Attack Objectives

An intruder's attack objectives are many and varied, but perhaps three of the best recognised are to:

i) Read cryptographic keys in plaintext

ii) Force cryptographic keys to a known or predictable value

iii) Render the crypto system useless and force the operators to revert to insecure methods.

Attack Scenarios

In order to achieve the objectives outlined above, various means may be employed - three typical methods are:-

i) Remotely monitor the mains supply to the crypto devices and attempt to analyse any conducted RFI to determine the plaintext data.

ii) Using a directional antenna and RF generator, produce a large RFI field in the vicinity of the crypto device to force the internal random key generators to latch-up and produce predictable keys.

iii) Gain access to the operations area and substitute a look-alike for the crypto device and steal the real device for subsequent attack.

This last scenario is probably the most real threat, particularly if one considers potential theft of devices containing live keys during shipment to operational sites or return for maintenance or, as in the case of POS terminals, from insecure shop premises outside normal office hours. Bearing this in mind, let us consider the response.

Defence Strategy

Clearly, as already stated, the major criterion is to design defence mechanisms that provide a very high confidence of intruder detection either during or after the attack, and in doing so make the cost of mounting such an attack greater than the intruder's potential gains.

Naturally, since different systems have different levels of potential gain, a "layered" approach to tamper resistance where varying levels of protection are available, is flexible and by its nature affords effective solutions in a cost conscious environment.

Defence mechanisms are broadly split into three main areas:-

i) Access control - for normal operator activity to ensure that unauthorised personnel may not operate the equipment.

(ii) Physical protection - generally mechanical, to ensure that casual substitution and non-invasive attack is difficult.

(iii) Electronics - mechanisms to detect intrusion - both invasive and non-invasive.

Let us address these areas in turn:-

Meeting the Challenge

a) Access Control

Since operational requirements will dictate that from time to time audit logs of cryptographic module behaviour must be dumped, master key updates initiated etc., operator access to cryptographic devices must be carefully controlled. This is generally achieved by the fitting of physical keylocks of the appropriate standard[2] requiring two or more trusted keyholders to initiate the top level security commands. For more stringent system operational requirements, these physical keylocks may be replaced with intelligent token interfaces, for example for smart cards or personal authenticators.

b) Physical

Generally, cryptographic units should be designed to be unique in appearance to avoid the possibility of casual substitution by a lookalike device. Physically strong mounting methods may be provided, although no direct access should be provided to the inside of the crypto unit. Ideally, there should be no ventilation holes, although if these are unavoidable they should be so constructed that it is impossible to gain access to sensitive areas within the device.

c) Electronics

In the majority of cases keys and other sensitive data are stored in Random Access Memory (RAM) with power supplied by independent battery sources, physically located close to the sensitive electronic devices. Alarm circuits are provided to detect intrusion and cause destruction of secret data. Some typical detector mechanisms are described in the next Section.

d) Detectors

(i) Dismantling

A wide range of sensors is available including simple micro-switches to detect removal of external case screws or lid assemblies, these may be supplemented by magnetic reed switches and permanent magnet actuators on mating surfaces. Active techniques of ultrasonic or infra-red space signature may be utilised, although because of power constraints it may be necessary to pulse these

detectors to conserve battery power. After an extended period on battery power, performance of these detection circuits may degrade and it is difficult to make them fail 'safe'.

(ii) Mains Power Variation/Monitoring

In order to ensure that no vestigial signal representing secret data appears on the mains power interface to the device, filtering should be employed between the device mains input and the power supply input point, and the power supply low voltage outputs should be adequately filtered and decoupled. Passive transorbs and fuses provide protection against deliberate over-voltage and reverse-voltage attacks on the device while good design practices must be observed when implementing power up/down monitoring circuits designed to protect the integrity of secure data.

(iii) Physical Removal

Unauthorised attempts at moving the device can be detected by tilt and jitter sensors which operate when the device is, for example, tilted more than 20° from the horizontal or subjected to the sort of vibrations generated by a normal power tool. Additionally, to protect against illegal removal of the power or communications cables, closed-loop alarms should be connected through both security devices and peripherals via the connecting cable assemblies.

(iv) Drilling and Grinding

Encapsulation of the sensitive electronic components holding secure data in a potting resin is a well-known process which certainly acts as a good physical barrier to an intruder wishing to probe the key storage electronics. The simplest method to gain access to the sensitive components is to drill, mill, grind or plane the potted area until sufficiently close to the target and then proceed more carefully using fine hand tools. In order to successfully attack in this way, knowledge of the layout of the PCB and the associated components is desirable and this is best accomplished using X-Rays, the drilling procedure may then be undertaken more accurately. Embedding a fine mesh of multiple layers of randomly located fine wires within the potting or, alternately, integrating a flexible PCB with multiple orientation alarm tracks on it, is a useful detection mechanism against these attacks. It is interesting to note that if the wires are fine enough, accurate detection of their location by

X-Ray means is a relatively difficult task. Obviously, all components accessing secure data paths must be enclosed within this encapsulation. In a classical bus-oriented micro computer solution, this obviously applies to all devices having access to the main data and address busses.

(v) Solvents

Since the embedded shield methods of Section (iv) render drilling, grinding and planing relatively difficult, a suitable chemical solvent attack on the encapsulation would prospectively seem attractive. If the potting compound has been carefully selected by the manufacturer such that any appropriate solvents for it are also solvents of the chip fabrication materials and PCB fabric, together with probably having special handling problems owing to its volatility, then solvent attack becomes more difficult. Embedding fusible links within the potted area such that mass flooding or immersion is impractical is an added safeguard.

(vi) Temperature

Since the majority of electronic components perform within a temperature specification of, typically, -30°C to +85°C and these would generally include the alarm detection and key destruction circuitry, rendering these circuits inactive by raising or, more generally accepted, lowering the unit temperature to, typically, -80°C would render these circuits inactive. Hot and cold temperature attacks are relatively easily detected by the inclusion of temperature sensors within the alarm circuitry which operate at, say, -25°C and +70°C although the effect of thermal shock on these devices and the units themselves, due to sudden change in temperature (e.g. by immersion in liquid nitrogen (-195°C)), must be carefully calculated to ensure correct failsafe operation. The choice of temperature detection thresholds is important if false alarming of a device in transit (e.g. an aircraft hold or a car boot) is to be avoided.

(vii) X-Ray

As mentioned in Section (iv), the use of X-Rays as a mechanism for locating critical components and data paths is extremely useful. Including X-Ray detection in alarm circuitry at first sight seems attractive, although in practice when devices are despatched from

manufacturers premises for subsequent shipment by air freight, they are likely to be X-Rayed under normal security procedures and hence alarm systems activated. As an alternative the sensitive component areas may be screened against X-Ray surveillance by a lead shield. Practical experience shows that, to be effective, the thickness of lead should not be less than typically 3mm, and the surfaces should be stippled and scratched in random pattens to enhance the deflection effects.

(viii) EMI/RFI

In considering the effects of electromagnetic and radio frequency interference, it is apparent that these effects are bi-directional i.e. radiation of signals from the device should not be capable of interpretation to reveal secret data, nor should any external interference source directed at the unit cause it to malfunction or 'latch-up' into a predictable state. This latter effect is particularly important in considering the behaviour of white noise seeded random number circuits which generate encryption keys. In designing device enclosures, material choice and bonding techniques which affect EMI behaviour are naturally important. It is generally accepted that metal case construction is preferable, and good electro/mechanical designs should be employed to ensure minimum escape of radiated energy. Additional barriers around sensitive component areas may be provided using copper screening cans with modular 'onion skin' construction techniques. Recent advances in spray-on conductive graphite, nickel and silver coatings give EMI/RFI attenuation performance figures of typically greater than 70dB which approach good design objectives of, for example, 100dB. A combination of these spray techniques and metal case construction can lead to good EMI/RFI resilience and a reasonable level of physical strength.

Data Destruction

It is generally accepted that all sensitive data storage requires a "zeroisation capability".[1] This implies that all data bits are actively set to zero following an alarm condition. Although at first this appears a simple task which may be accomplished by disconnecting the power supply of the volatile storage devices or loading a single set of zero's into non-volatile storage, in practice this is unsatisfactory. Certainly random access memory (RAM) devices have

residual data retention characteristics which, under controlled conditions, permit reconstitution of the original data contents prior to the attempt at erasure.

In order to be confident of destroying the data successfully, all storage cells must be actively purged by overwriting with all '1's, and then all '0's at least three times in rapid succession, followed by the shorting of the device power supply input pins to ground. In cases of extreme sensitivity it is possible that the only acceptable method of destroying the data is by non-reversible physical destruction of the storage devices themselves, although this is naturally rather a difficult strategy to follow in a production test environment!

Tradeoffs

In any system containing detection and destruction methods as described here, there is naturally a cost penalty for providing very high levels of tamper resistance, due to construction and test requirements by the manufacturer. It is naturally important to analyse the risks of key disclosure against cost of protection and specify a suitable implementation. For some of the methods described here, where the tamper resistance cannot easily be removed for maintenance purposes, the implications of a throw away replacement maintenance policy should not be overlooked.

Intelligent Tokens

The attack scenarios and associated countermeasures discussed so far have been oriented towards physically 'large' devices. With the growth in EFTPOS schemes and the increased use of smart cards and personal authenticators, the applicability of these techniques is worth considering.

Since intelligent tokens are 'stand alone' devices prone to theft and/or substitution, good cryptographic system designs should ensure that compromise of one token should not threaten the security of the whole system. Generally these units contain user related key data stored in battery backed-up RAM and mask programmed ROM. Naturally, size constraints dictate that protection against X-Ray inspection are impractical and hence mechanical attacks using precision micro-manipulation techniques are the biggest threat. Recent advances in

token design have taken this into account, and the sensitive data areas on the device silicon structure have been 'buried' under several layers of metallisation and convoluted data paths constructed to add confusion. These simple techniques are probably the best that can be practically applied in volume manufacture and hence the risk analysis of token-related key compromise must be stringently assessed.

Independent Assessment

Few independent assessment houses or laboratories seem to address the rather particular needs of tamper resistance testing. The author has had experience of two such organisations, they are:

The National Physical Laboratory,
Teddington, Middlesex, UK,
and
TNO Division of Technology for Society,
Delft, The Netherlands.

The role of such bodies is to act as an independent test and assessment facility for organisations specifying and procuring items of cryptographic equipment with tamper resistant capabilities.

Future Trends

As markets develop and grow within the financial sector the need for secure product design to keep up is unquestionable. The potentially explosive growth of EFT/POS schemes has led to research in the area of low-cost tamper resistant modules where either the unit cost is so low that the secure components can be thrown away if they fail, or the tamper resistant mechanisms are reusable allowing return to factory maintenance.

The level of protection afforded and complexity of attacks catered for must naturally increase with time - particularly since current trends in computer system design are moving more into a distributed architecture approach which makes physical computer protection, particularly personal computers, much more important.

Some manufacturers have already looked to miniaturise their crypto key storage components, and the inclusion of data destruction circuitry on-

chip is an integrated part of this process. To date these techniques have primarily been non-recoverable, i.e. they physically destroy the silicon - adaptive systems will no doubt appear in due course.

Underlying all of these trends however must be the message that as systems designers we must continue our R & D programmes to devise new techniques and strategies to meet the ever increasing sophistication of computer crime.

References

(1) "Cryptographic Equipment Security: A Code of Practice" S.C. Serpell, British Telecom Research Laboratories, Martlesham Heath, Ipswich.
Published by Institution of Electronic and Radio Engineers, 99 Gower Street, London WC1E 6AZ

(2) "U.S. Federal Standard 1027 Telecommunications: general security requirements for equipment using the data encryption standard"

(3) "Security for Computer Networks - An Introduction to Data Security in Teleprocessing and Electronic Funds Transfer". D.W. Davies & W.L. Price
Published by John Wiley & Sons

THE RSA CRYPTOGRAPHY PROCESSOR

Holger Sedlak

Institut für Theoretische Informatik
Technische Universität Braunschweig
D-3300 Braunschweig
Federal Republic of Germany
(531) 391-2384

1. Introduction

In commercial applications, a minimum ciphering rate of 64 $Kbit/sec$ is required which will be the transmission rate of public digital networks. In contrast, the RSA method has a very slow ciphering rate particulary when using software implementations of the algorithm. The solution of this problem is a hardware implementation of the RSA algorithm. A cryptography processor, however, consisting of standard chips like bit slice processors again does not achieve the speed necessary. Moreover, in a multi-chip processor, the security of the key management system cannot be guaranteed. Therefore, a single-chip implementation of the RSA algorithm seems to be the only solution. Such a solution is presented as an RSA *Cryptography Processor* (CP).

2. The Implementation of the RSA Method

The bottleneck of an implementation of the RSA method is the handling of very long numbers. If one wishes to cipher a 200 digit decimal number, then the length of each key is 660 bit (the CP supports numbers up to a length of 780 bit). With this number the CP has to execute the function

$$C = M^E \bmod N,$$

where C is the code generated, M the data (message), and (E, N) is the public encoding key. The same function is used for decoding (only another exponent is used), so the fast execution of this function is the heart of the problem. In this form, the function is not implementable, but after some transformations the function is reduced to a sequence of additions and subtractions. This is demonstrated in Fig. 1, at first sight in the well known way.

First, the powering is reduced to a sequence of multiplications (Fig. 1a), and it is executed in the ring of residual classes of N. Next, the multiplication is reduced to a sequence of additions. Also the modulo operation is reduced to a sequence of subtractions as in the classical algorithm of division (Fig. 1b). Last, for reducing the maximum size of the registers of the CP, the multiplication is computed in the ring of residual classes of N.

Up to this point it is the known way. But now the implementation leaves this way with the look-ahead algorithms which decrease the maximum number of additions for the multiplication and subtractions for the modulo operation. It is important to recognize the necessity of simultaneous acceleration of multiplication and modulo operation. If, for

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 95-105, 1988.
© Springer-Verlag Berlin Heidelberg 1988

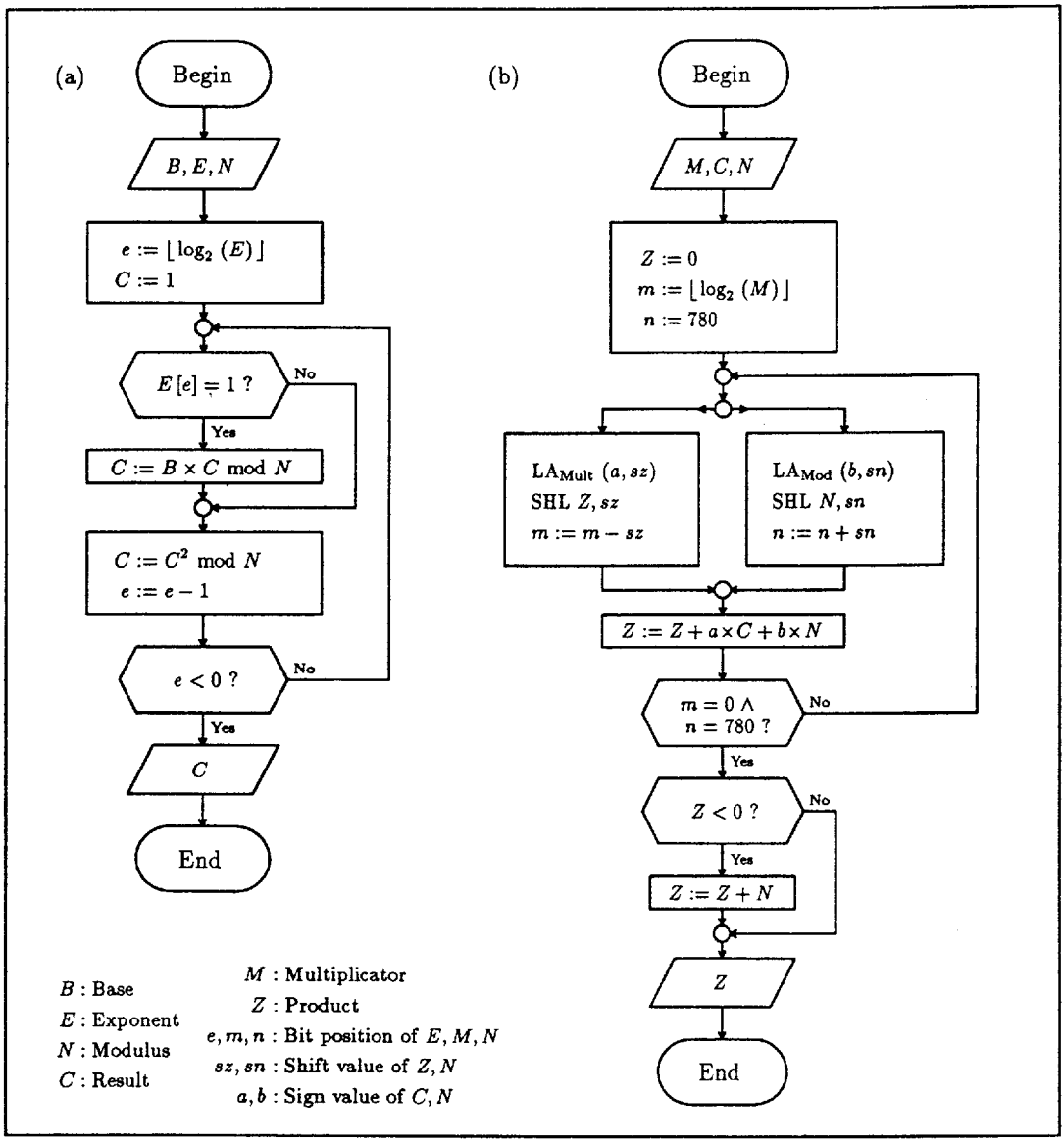


Figure 1: Flowchart of exponentiation (a) and multiplication mod N (b). The two LA-Functions calculate in parallel with the adding circuits the next look-ahead values for shifting and addition. This main loop of multiplication takes only one processor cycle, e.g. 32 ns, except for some quite infrequent cases.

example, only the multiplication was accelerated, this would not result in an improvement of the algorithm because the number of subtractions for the modulo operation would remain unchanged. Since the multiplication and the modulo operation are executed in turn, the number of subtractions becomes the time determining part.

While the look-ahead algorithm of the multiplication is well known, a new look-ahead procedure for the division and hence the modulo operation had to be developed in [8]. The major condition was that the average reduction of the look-ahead of division is approximately the same as the one of the look-ahead of multiplication. In this case, the two look-ahead algorithms would cooperate optimally.

In the serial multiplication with the classical look-ahead, chains of maximal length of 0s and 1s are processed

alternatingly. When multiplying, for example, a binary number with

$$\begin{array}{ll} \mathrm{M} = \ldots 0 & 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad \ldots \\ \text{bit} & \ldots m_{11}\, m_{10}\, m_9\, m_8\, m_7\, m_6\, m_5\, m_4\, m_3\, m_2\, m_1\, m_0 \ldots, \end{array}$$

the algorithm starts in a state *0* processing the 0-chain at bits m_0 to m_3 as a whole. Then it changes to state *1* processing the 1-chain at bits m_4 to m_6 and bit m_7 analogously. At bit m_7 it remains in state *1* processing the 1-chain at bits m_8 to m_9 and bit m_{10} where it finally changes back to state *0*. For the computation of the next change of state, it is sufficient to recognize the end of the present chain. For an optimal efficiency it suffices to consider the two bits after the end of a chain. In this case, the computation time as compared with the multiplication without look-ahead is reduced to an average of 1/3.

In the CP the multiplication is executed in the descending direction, and not as explained in the ascending order. Nevertheless, the look-ahead algorithm works the same way. The amount of computation reduction is identical, only the decision about a change of state is a little bit different but is of no interest for this concern.

The division of the modulo operation with look-ahead operates in quite a different manner. In the following, comparisons of Z with $1/3, 1/6 \ldots$ of N play an important role in the computation of Z mod N. Let us assume that during the division of Z by N, N was already shifted and *subtracted* several times from Z and let the remainder be stored in Z. The special look-ahead algorithm ensures that

$$|Z| \leq \frac{1}{3} N.$$

This will also be true for the following steps as will be shown now. By assumption, Z falls into one of the following ranges:

$$\begin{array}{lrcl} \text{range} \quad si = 2: & \frac{1}{6} * N & < |Z| & \leq \frac{1}{3} * N, \\ \text{range} \quad si = 3: & \frac{1}{12} * N & < |Z| & \leq \frac{1}{6} * N, \\ & \vdots & < & \leq \quad \vdots \\ \text{i.e.,} & \frac{2}{3} * 2^{-si} * N & < |Z| & \leq \frac{4}{3} * 2^{-si} * N, \end{array}$$

for some $si > 1$. Now N is shifted si bits towards the lower bits. For the result $N' = 2^{-si} * N$, we have

$$\begin{array}{rcl} \frac{2}{3} * 2^{-si} * (2^{si} * N') < |Z| & \leq \frac{4}{3} * 2^{-si} * (2^{si} * N), \\ \frac{2}{3} * N' < |Z| & \leq \frac{4}{3} * N' \text{ or} \\ -\frac{1}{3} * N' < |Z| - N' & \leq \frac{1}{3} * N'. \end{array}$$

For the resulting $Z' = Z \pm N'$, $|Z'| \leq 1/3 * N'$ holds again. Since this condition is always met, N is shifted in every step by at least $si = 2\,bit$. However, the expected value of si is 3 thus reducing the computation time to an average of 1/3.

But what is the advantage of computing the modulo operation with the look-ahead algorithm compared with the normal computing? It seems that only subtractions are substituted with comparisons which take approximately the same time as the removed subtractions. Generally, this statement is correct but not with this look-ahead algorithm since it is only necessary to compare for example the highest $10\,bit$ of Z and N. Occasionally (in this example every 2^{10} steps in average), the algorithm makes a wrong decision, namely if the highest $10\,bit$ of Z and N are identical. However, if this happens, the computation of the multiplication does not fail but only slows down the next step of the modulo operation to a shift value $si = 1\,bit$. Thus it is possible to reach nearly the complete theoretical advantage in a practical implementation of the look-ahead algorithm. The few $10\,bit$ comparisons can be done in parallel with the same number of comparators while the addition circuits compute the next intermediate result Z.

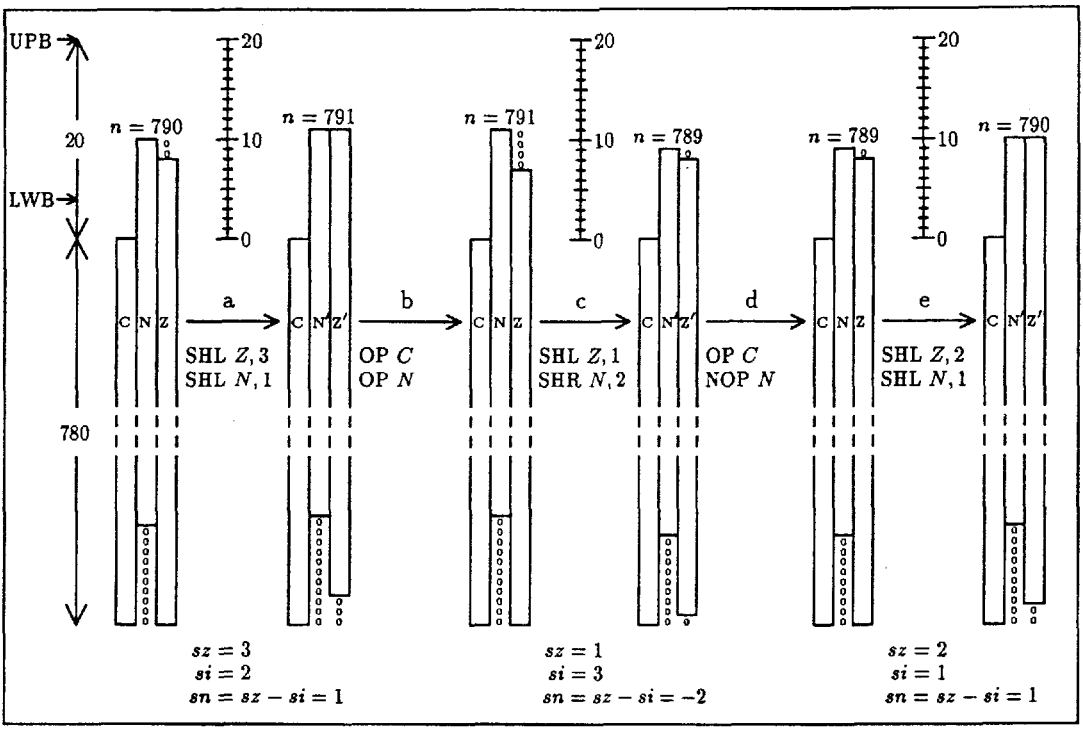


Figure 2: On principle three steps of the *flow buffer* are shown. One shift step (SHL or SHR) and one operation step (ADD, SUB or NOP) together last one processor cycle, i.e. $32ns$. The towers represent the registers. SHL $Z, 2$ means, shift left register Z by $2\,bit$, i.e. multiply Z by 4. This is demonstrated in increasing the tower representing Z by 2 units. The abbreviations are: *UPB* → upper bound of flow buffer, *LWB* → lower bound of flow buffer, *SHL* → shift left, *OP* → addition *or* subtraction and *NOP* → no operation.

In reality, however, the improvement of the multiplication and modulo operation by an average of 1/3 is not reached because a realistic algorithm cannot look ahead an unbounded number of bits. In an implementation of this algorithm, a look-ahead bound k is given. If a change of state cannot be decided by considering the next k bits, the algorithm shifts k bits in this step without taking a further action and tries again in the next step. With a bound k, the average reduction of the number of operations is developed in [8] as

$$\frac{1}{3} * \frac{2^k + 1}{2^k - 1}.$$

Since only the expected shift values of the two look-ahead algorithms coincide, it is necessary to uncouple them in the CP. In both cases, shift values sz and sn determine the number of bits by which the intermediate result of multiplication Z and the modulus N are to be shifted in each cycle. Z is shifted absolutely, and N is shifted relatively to Z. The two shifts are uncoupled by N *floating* on Z. This property which is formally deduced in [8], is explained in Fig. 2.

The state of the CP is shown for three consecutive steps involving transitions (a) to (e) of Fig. 2. Transitions (a), (c), and (e) demonstrate the shift operations while (b) and (d) are additions or subtractions not discussed further at this point. The towers represent the registers. Their size is $780 + 20\,bit$. 780 is the maximal word length, and 20 is the size of the buffer. This buffer decouples the look-ahead procedures. If, for example, the shift value of the multiplication is greater than the one of the modulo operation, then N is shifted by the difference partially into the buffer (transitions (a) and (e)). In the opposite case, N is shifted towards the low end (transition (c)). The observation of the buffer bounds is taken care of but not discussed here.

Before transition (a), N was shifted 10 *bit* into the buffer. Shift value sz subsequently takes the values 3, 1, and 2. The shift value si mentioned above which determines the shift of N relative to Z, takes the values 2, 3, and 1 meaning that N is absolutely shifted by $sn = sz - si$ hence 1, -2, and 1. After transitions (a), (c), and (e), N is shifted into the buffer by 11, 9, and 10 *bit*, respectively.

In transition (c), the influence of the look-ahead bound $k = 3$ is demonstrated. Although the algorithm might have asked for an $si = 4$ shift, N is moved by 3 *bit* relative to Z, and in the transition (e) by 1 *bit*. In transition (d), N is neither added nor subtracted from Z.

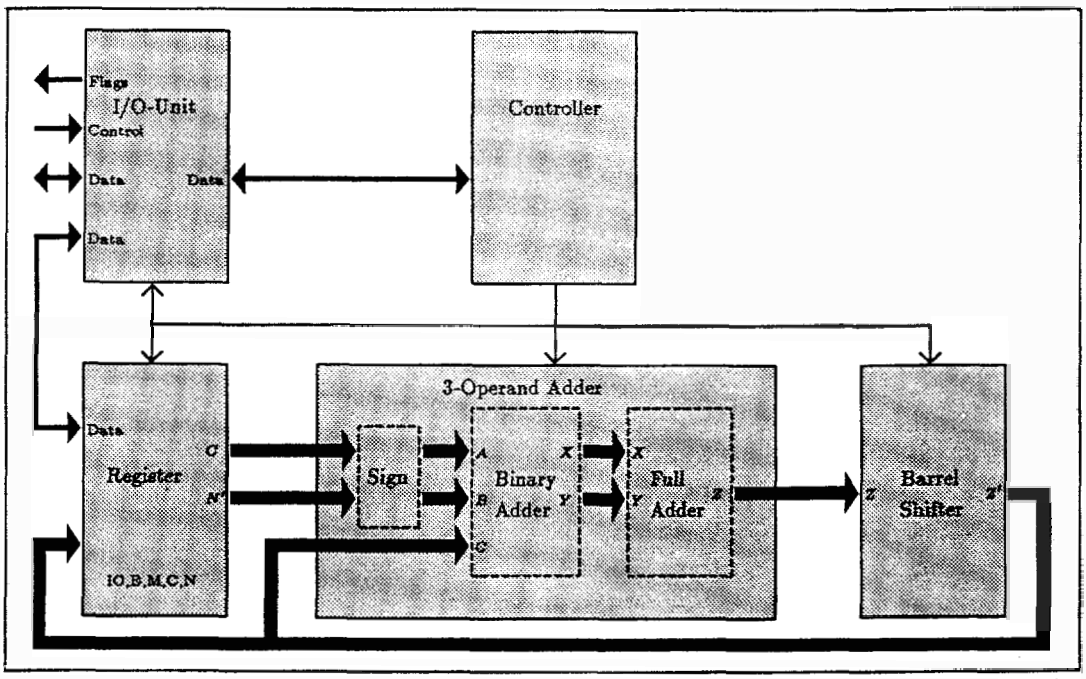


Figure 3: The block structure of the CP. In addition to the components discussed so far, the CP consists of an *I/O-Unit*, a *Controller*, a *Register Field* and a *Barrel Shifter*. One task of the Controller is to compute the look-ahead values of exponentiation (cp. [4]), multiplication and modulo operation. The Barrel Shifter is necessary to perform the shifting by several bits in one step.

Two additional features are worth mentioning. First, there are two different adders in the data path (Fig. 3). The first of them is a *one bit slice* adder with three data inputs but without a carry input. It generates two sum bits. In the CP these two bits of every bit slice are fit together to form two new numbers, called X and Y. These numbers are directly passed to the second adder in the data path which is a *full adder*. Thus it is possible to perform one multiplication step requiring an addition and one modulo operation step requiring a subtraction in one processor cycle.

Second, the critical phase of each addition is the delay of the carry bit as long as the lowest bit position may influence the highest one. So for the full adder a new *carry look ahead* logic (CLA) was designed reducing the time for carry bit calculation to a minimum. In this CLA, the addition is normally calculated in a single processor cycle. For long carry bit calculations, the CLA increases the calculation time for this addition to more than one cycle. The major advantage of this proceeding lies in the fact that the rate of addition is not determined by the longest lasting addition, but by the average adding time.

The resulting block structure with the major components discussed so far is shown in Fig. 3.

3. The Ciphering Rate

The ciphering rate of the CP is determined by the structure of the inplementation of exponentiation. Thus the ciphering is influenced by

- the number of cycles needed for one addition in the full adder,
- the number of steps needed for one multiplication which is determined by the classical look-ahead algorithm of multiplication,
- the number of steps needed for one modulo operation which is determined by the new look-ahead algorithm of modulo operation, and
- the number of modular multiplications needed to perform one exponentiation.

All these points should not be classified by constant values but by probabilistic functions. Before describing these functions we make two assumptions. First, the probability of having a binary *0* or *1* is equal at every binary position of the modulus N, the exponent E and the message M, the code C, respectively, e.g. $\mathcal{P}(X_i = 0) = \mathcal{P}(X_i = 1) = 1/2$. Second, it is meaningful to assume that one step of addition is not depending on the result of the previous addition.

The first assumption is obvious but it is harder to understand why the second one is rich in meaning. What happens if the opposite is true? Of course it is much more difficult to construct the probabilistic functions but this is not important. Important is that now an attacker has the possibility of building a probabilistic algorithm using the dependence between consecutive steps. Thus he had a good chance of breaking all cryptographic methods using modular multiplications without solving the basic problem of this method. Of course it is wrong making the conclusion that the second assumption is true because this attack must be permitted. But it is meaningful to assume it when using the modular multiplication to construct cryptographic methods. Thus five probabilistic functions can be defined which characterize the behaviour of the CP and therefore the ciphering rate completely:

$f_t\,(x) : \mathcal{P}(\#(\text{cycles per addition}) = x)$,

$f_a\,(x) : \mathcal{P}(\#(\text{additions per multiplication}) = x)$,

$f_s\,(x) : \mathcal{P}(\#(\text{cycles per modular multiplication}) = x)$,

$f_m(x) : \mathcal{P}(\#(\text{multiplications per exponentiation}) = x)$, and

$f_e\,(x) : \mathcal{P}(\#(\text{cycles per exponentiation}) = x)$.

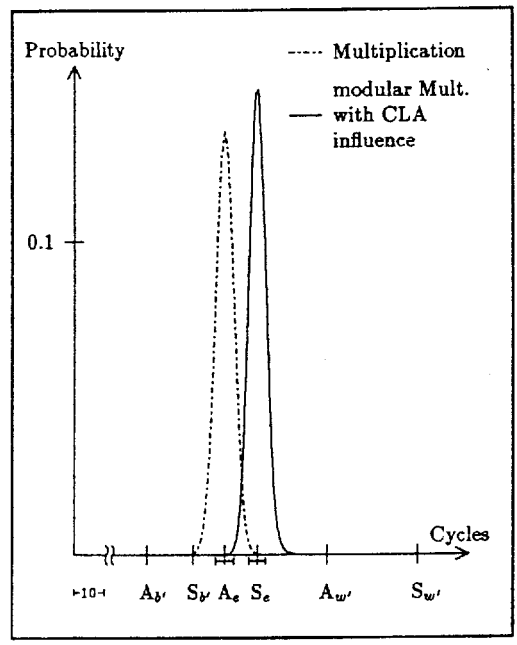


Figure 4: The probability distribution of f_a : *Cycles needed for Multiplication* and f_s : *Cycles needed for modular Multiplication on CP.* Rendered prominent are the best case $A_{b'}$,$S_{b'}$, the expexted case (with the standard deviation) A_e,S_e and the worst case $A_{w'}$,$S_{w'}$ of execution. The exact values of these cases are listed in Tab. 2.

The function f_s depends on f_t and f_a while f_e depends on f_s and f_m. Here is not enough space to describe these dependencies and the mathematical formula of all these functions. Thus the two functions f_t and f_e are described exactly but only an overview is given of building up the three functions f_a, f_s and f_m. To construct the latter ones, the basic idea is to use the *probabilistic finite state machines* (PFSM) which describe the three look-ahead algorithms for accelerating the multiplication, modulo operation and exponentiation (cp. [8], [9]). Although these machines only

Function f_x	Best Case X_b	Expected Case X_e	Worst Case X_w
f_t	1	$1 + \mathcal{P}(\text{Panic}) * \left(\left\lceil \frac{1}{4 \times 20} \log_2(N) \right\rceil - 1\right)$	$\left\lceil \frac{1}{4 \times 20} \log_2(N) \right\rceil$
f_a	$\frac{1}{k}\lceil \log_2(N) \rceil$	$\frac{1}{3}\lceil \log_2(N) \rceil * \frac{2^k+1}{2^k-1}$	$\frac{1}{2}\lceil \log_2(N) \rceil$
f_s	A_b	$(A_e + c(k) * B_L) * T_e$	$A_w * T_w$
f_m	$\lceil \log_2(N) \rceil$	$\frac{4}{3}\lceil \log_2(N) \rceil$	$\frac{3}{2}\lceil \log_2(N) \rceil$
f_e	$M_b * S_b =$	$M_e * S_e \approx$	$M_w * S_w \approx$
	$\frac{1}{k}\lceil \log_2(N) \rceil^2$	$\frac{4}{9}\lceil \log_2(N) \rceil^2 * \frac{2^k+1}{2^k-1} + \frac{4}{3}\lceil \log_2(N) \rceil * c(k) * B_L$	$\frac{3}{320}\lceil \log_2(N) \rceil^3$

Table 1: The exact values of the best, the expected and the worst case (X_b, X_e and X_w) of the probability distributions f_t, f_a, f_s, f_m and f_e. The values are computed by complexity theory. However, it is not a good choice to characterize the abilities of the CP by E_b and E_w. (The function $c(k)$ returns a result in the range $[0,1]$, B_L is the length of the flow buffer.)

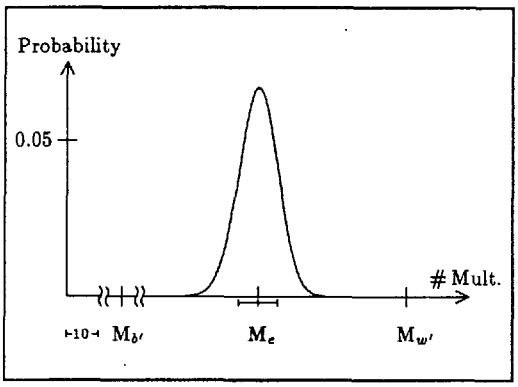


Figure 5: The probability distribution of f_m : *Multiplications needed for Exponentiation.* Rendered prominent are the best case $M_{b'}$, the expexted case (with the standard deviation) M_e and the worst case $M_{w'}$. The exact values of these cases are listed in Tab. 2.

allow to compute the best case X_b, the expected case X_e and the worst case X_w of execution (Tab. 1), they can be used to construct a new PFSM which characterizes the dynamic and not only the static behaviour of the CP. Based on these new machines one is able to compute the required functions (Fig. 4, 5). During this computation, the probability is represented not only as a scalar but as a vector due to the PFSM. The functions return as result the sum of all vector *components*.

The function f_t depends on the implementation of the full adder. In the CP this 780 *bit* adder is realized as a chain of 20 *bit* sized adders, called *block adders*. Each block adder has an attached *block CLA* element. Generally, both together are able to compute in one processor cycle of 32 *ns* the 20 *bit* result and early enough the *block carry bit* so that the next higher block adder is able to compute its result in attention to the mentioned carry bit. So the complete addition of 780 *bit* sized numbers takes only one processor cycle.

But what happens if this carry block bit depends on the carry block bit of the next lower block adder? Now there is not enough time to calculate the block carry bit early enough for the next higher block adder. Therefore this block CLA gets into *panic*! But there is a magician in the CP who avoids the failing of computation. He notices the panic of this unlucky block adder, flourishes his magic wand and suddenly there is even enough time for traveling the block carry bit from the lowest block adder to the highest one. In a more technical language this wonderful magician is simply an interrupter which notices the normally unused *block carry propagate bit* and, when activated by this bit, it disconnects for a certain amount of cycles the triggering clock lines of the *Ciphering Units* (CU). The probability of this scenario is coupled to the probability that one block adder gets into panic which is

$$\mathcal{P}(\text{One block adder gets into panic}) = 2^{-20}.$$

Due to the acting look-ahead algorithms of multiplication and modulo operation this probability is not as obvious as

it seems to be. The algorithms cause the binary *0*s and *1*s to appear not with the same probability at the input of the binary adder and even at one input line of the full adder (Fig. 3). But as in [9] deduced this inequality does not influence the appearence of *0*s and *1*s in the addition result Z, just as the carry propagate signal of each bit slice is not affected. Because only these carry propagate signals determine the block carry propagate bit the derived probability is correct. Hence the probability of one CU geting into panic is (considering that the lowest block adder cannot get into panic)

$$\begin{aligned}
\mathcal{P}(\text{Panic}) &= 1-(1-\mathcal{P}(\text{One block adder gets into panic}))^{\#(\text{block adders of CP})-1} \\
&= 1-(1-2^{-20})^{\lceil \frac{1}{20}\log_2(N)-1\rceil} \\
&= 1-\sum_{i=0}^{\lceil \frac{1}{20}\log_2(N)-1\rceil}\binom{\lceil \frac{1}{20}\log_2(N)-1\rceil}{i}(-2^{-20})^i \\
&= 2^{-20}\left\lceil \frac{1}{20}\log_2(N)-1\right\rceil - \sum_{i=2}^{\lceil \frac{1}{20}\log_2(N)-1\rceil}\binom{\lceil \frac{1}{20}\log_2(N)-1\rceil}{i}(-2^{-20})^i \\
&= 2^{-20}\left\lceil \frac{1}{20}\log_2(N)-1\right\rceil - \mathrm{O}(\varepsilon) \\
&\simeq 2^{-20}\left\lceil \frac{1}{20}\log_2(N)-1\right\rceil .
\end{aligned}$$

Now it is easy to derive the required function f_t (the block CLA chain is able to travel a block carry bit over four 20 *bit* CLA elements in one processor cycle)

$$f_t(x)=\begin{cases} 1-\mathcal{P}(\text{Panic}) & \text{if } x=1; \\ \mathcal{P}(\text{Panic}) & \text{if } x=\left\lceil \frac{1}{4\times 20}\log_2(N)\right\rceil; \\ 0 & \text{otherwise.}\end{cases}$$

For the computation of f_m (Fig. 5), only the corresponding PFSM representing the look-ahead algorithm of exponentiation is used as base (cp. [4], [9]). This function and the above-mentioned f_s determine f_e as follows. Required is the probability that one exponentiation takes x cycles of time which is $f_e(x)$. The exponentiation is divided into an indefinite number of multiplications assuming the exponent is statistically distributed. This probability distribution of the number of multiplications is described by f_m. Each of these multiplications is independent of the other ones. The number of cycles needed to perform every multiplication is therefore independent, too. Furthermore, the number of cycles to perform one multiplication is unknown and only described by the probability distribution f_s. The sum of all cycles for performing all multiplications has to be x. Thus the derivation from f_e is

$$\begin{aligned}
f_e(x)=\mathcal{P}(\mathrm{E}=x) &= \sum_{m=1}^{\infty}\left\{\mathcal{P}(\mathrm{M}=m)*\mathcal{P}(\sum_{i=1}^{m}\mathrm{S}_i=x)\right\} \\
&= \sum_{m=\mathrm{M}_b}^{\mathrm{M}_w}\left\{\mathcal{P}(\mathrm{M}=m)*\sum_{(s_1,\ldots,s_{m-1})\in\mathcal{N}^{m-1}}\left\{\mathcal{P}(\mathrm{S}_m=x-\sum_{i=1}^{m-1}s_i)*\prod_{i=1}^{m-1}\mathcal{P}(\mathrm{S}_i=s_i)\right\}\right\} \\
&= \sum_{m=\mathrm{M}_b}^{\mathrm{M}_w}\left\{f_m(m)*\sum_{(s_1,\ldots,s_{m-1})\in[\mathrm{S}_b,\mathrm{S}_w]^{m-1}}\left\{f_s(x-\sum_{i=1}^{m-1}s_i)*\prod_{i=1}^{m-1}f_s(s_i)\right\}\right\}.
\end{aligned}$$

The curve of f_e is very funny, it looks like a cookscomb (Fig. 6). This is true for a statistically independent distribution of the exponent. For a selected key the exponent is static and therefore the curve reduces to one peak of the cookscomb.

The above-mentioned functions f_a and f_s are shown in Fig. 4 whereby f_a is computed in the described way of using as base only the PFSM representing the look-ahead algorithm of multiplication. An addition time of one cycle per step is assumed, too. On the other hand, for the computation of f_s the influence of flow buffer bounds (Fig. 2) and the functions f_t and f_a are used in addition to the PFSM of modulo operation. Amazingly, the standard devi-

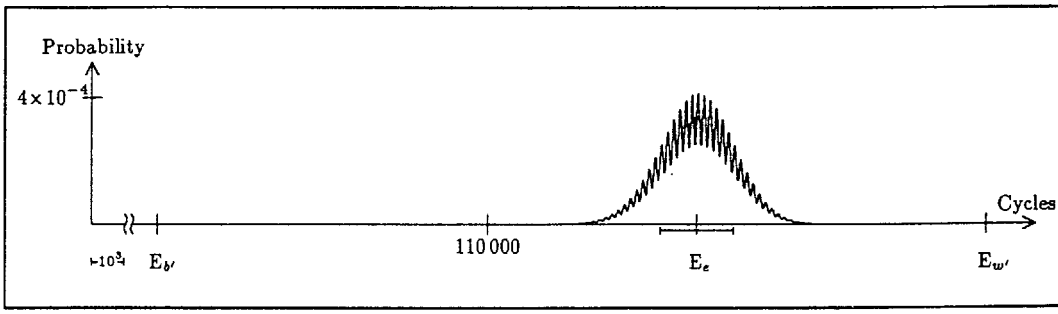


Figure 6: The probability distribution of f_e : *Cycles needed for modular Exponentiation on CP.* Rendered prominent are again the best case $E_{b'}$, the expexted case (with the standard deviation) E_e and the worst case $E_{w'}$. The exact values of these cases are listed in table 2.

$\lceil \log_2(N) \rceil = 440; \quad k = 3; \quad B_L = 20$			
Function f_x	Best Case $X_{b'}$	Expected Case X_e	Worst Case $X_{w'}$
f_t	1	1.00013	6
f_a	164	189.102	213
f_s	179	199.693	250
f_m	499	584.778	631
f_e	99 660	116 776.181	125 955

Table 2: The values are computed by analyzing the probability distributions and defining the worst and the best case in a more technical than in the former mathematical manner (see the text). The new values of $X_{b'}$, $X_{w'}$ and specially $E_{w'}$ are very suitable to characterize the abilities of the CP.

ation of f_s is smaller than the standard deviation of f_a although assuming not an addition time of one cycle per step due to f_t. This aspect results in the flow buffer and the statistical independence of multiplication steps and modulo operation steps. If the CP gets a fast multiplication the probability that the intermediate result is shifted toward the upper bound of flow buffer is high so in average the CP has to perform some additional modulo operation steps. If on the other hand the CP gets a slow multiplication, the probability that the intermediate result is near the lower bound of flow buffer is high, so in average the CP has to perform much less additional modulo operation steps than before. This coherence effects the decrease of standard deviation.

Now one has all suppositions to compute the ciphering rate. However, it is important in which way the worst case is defined because it determine the ciphering rate. From the view of complexity theory it is necessary to use the values of Tab. 1. But these values do not characterize very well the abilities of CP in a practical employment. Of course the absolute worst case of execution is possible. But in every transmission line a protocol is performed to detect several different situations in which one of the communication partners is not able to transmit or receive data. Also this protocol is used to detect transmission errors. If the CP is not able to compute the required value fast enough it can be conceived as a transmission error by the receiver. Thus the sender is solely invited to transmit the value again.

However, it is important to fix the probability of this scenario to a value much less than the normal error rate of transmission lines. The latter probability is assumed to be 10^{-12} (a fantasticly good value). Thus for the computation of the values in Tab. 2, the technically best case $X_{b'}$ and worst case $X_{w'}$ are defined as

$$X_{b'} : \mathcal{P}(x \leq X_{b'}) = 10^{-18}, \text{ and}$$
$$X_{w'} : \mathcal{P}(x \geq X_{w'}) = 10^{-18}.$$

As described later, the CP consist of two independent CUs. The larger one has a size of 440 *bit*. They are able to decode parallelly a 780 *bit* sized number. Therefore the technical ciphering rate of CP is

$$\text{Ciphering Rate (CP)} = \frac{\text{Size (Code)}\,[Kbit]}{\text{E}_{w'}[cycle] * 32\,[ns/cycle]} \simeq 195\,[Kbit/sec].$$

4. Features of the RSA Cryptography Processor

All RSA implementations known to the author suffer from one or more of the following restrictions: the ciphering rate is too low (e.g. the "Security Processor C.R.I.P.T." in [5] with a rate of less than $10\,Kbit/sec$); the key length is too short; or several chips are required (e.g. the "NEC/Miyaguchi" design in [4] with a rate of $29\,Kbit/sec$, a key length of $660\,bit$ and a solution of 333 chips).

In contrast to all of them is the CP. The heart of it consists of two independent CUs with a size of 340 and $440\,bit$. For encoding, both CUs cooperate as a single $780\,bit$ CU, for decoding each of them works independently using the advantages of the Chinese Remainder Theorem. This results in an encoding rate of $3\,Mbit/sec$ when using as the exponent Fermat's 4th number and in a decoding rate of $0.2\,Mbit/sec$ if the two primes involved do not exceed the size of the CUs.

Furthermore, the CP has a shell around its kernel. So it is not only a very fast ciphering processor but a "Cryptograhic Area" which means that

- "Cryptographic Actions" like "Load a new key and accept it only if the certificate is correct" can only be started but not manipulated from outside the chip,
- signatures (certificates) are automatically generated respectively checked,
- secret memory is neither readable nor writeable from outside the chip, and
- it exists the possibility of generating new keys.

The generation and checking of signatures requires a hash function with the abilities of a One-Way function. Such a function is implemented. It is the one discussed at ISO ([6], [7]) and not as formerly described the one developed at SEPT [5]:

1. Insert between every four bits of M four binary 1's,
2. Divide M' into m blocks B_i less than N,
3. Perform
$$H_0 = 0,$$
$$H_i = (H_{i-1} \oplus_2 B_i)^2 \bmod N,$$
4. H_m is the result.

The key generation is done by the Monte Carlo method using only the strong pseudoprime test. The two primes p and q were computed in the known way: $p-1$ has a large factor p', $p'-1$ has a large factor p'', and the same for q, respectively. But in addition to that caution, q is calculated such that

$$\gcd(N-1, E) = \gcd(p \times q - 1, E) = E = 2^{2^4} + 1.$$

This property prevents an illegal installation of a key $N_{Attacker}$ which is not certificated by the secret key $D_{Authority}$ but by the manipulated "secret" key

$$D'_{Attacker} \equiv E^{-1} \bmod (N_{Authority} - 1).$$

This attack is possible if $N_{Authority}$ is a pseudoprime to the base $H(N_{Attacker})$ and if $\gcd(N_{Authority} - 1, E) = 1$. The possibility of the last condition is very high if the two primes are computed in the known way. Although the first condition of finding such a pseudoprime is very low the presented way of key generation is implemented in the CP to prevent this attack.

For users, the following features of the CP are of interest:

- protected "Cryptographic Area",

- key generation takes an average time of 2 *sec*,
- operation as a co-processor,
- one DMA channel for a data stream of 200 $Kbit/sec$, or
- three DMA channels for three independent data streams of 64 $Kbit/sec$,
- 8 *bit* wide data bus,
- independent operation of the CU and the I/O Unit,
- about 160 000 transistors in 1.5μ-CMOS technology, and
- chip size of approximately $5mm \times 4.8mm$.

5. Conclusions

So far, a prototype consisting of about 5000 5μ-NMOS transistors has been realized. This chip contains all major parts of the CP in a small version. Furthermore, based on the experience with the 5μ-NMOS process, the performance rates of the 1.5μ-CMOS process were obtained by theoretical considerations as well as by full length simulations. The development of a 1.5μ-CMOS design of the CP is now completed and therefore, first prototypes will be available in summer 1988.

6. References

[1] Hellmann,M.E., Die Mathematik neuer Verschlüsselungssysteme, Spektrum der Wissenschaft 10, 1979, 92-101.

[2] Meyer,C.H., Matyas,S.M., Cryptography - A new Dimension in Computer Data Security, Cryptography Competence Center, IBM, Kingston, New York, Wiley-Interscience Publication, 1982.

[3] Pomerance,C., Recent Developments in Primality Testing, The Mathematical Intelligencer 3, 1981, 97-104.

[4] Rivest,R.L., RSA Chips (Past/Present/Future), in "Advances in Cryptology", Proc. Eurocrypt 84, Paris, France, 1984, 160-164.

[5] Pailles,J.C., Girault,M., The security processor C.R.I.P.T., Information Security: The Challenge, Preprints of the Fourth IFIP Security on Information Systems Security, Monte-Carlo, December 2-4, 1986.

[6] A Hash function for use in digital signature and authentication (Exclusive OR and squaring), Working Document - First Draft, Secrétariat ISO/TC 97/SC 20/WG 2 N75, Paris, France, 1986.

[7] TeleTrusT/OSIS, The TeleTrusT/OSIS Hash-Function, TeleTrusT/OSIS - WG1 Security Aspects, GMD, Darmstadt, Germany, 1986.

[8] Sedlak,H., Konzept und Entwurf eines Public-Key-Code Kryptographie-Prozessors, Institut für Theoretische und Praktische Informatik, Technische Universität Braunschweig, Germany, 1985.

[9] Sedlak,H., Theory and Single-Chip Implementation of a Cryptography Processor, Institut für Theoretische Informatik, Technische Universität Braunschweig, Germany, to be submitted as a PhD-Thesis.

EXTENSION OF BRICKELL'S ALGORITHM FOR BREAKING HIGH DENSITY KNAPSACKS

F. Jorissen, J. Vandewalle, R. Govaerts

Katholieke Universiteit Leuven,
Department of Electrical Engineering, ESAT Laboratory
K. Mercierlaan 94, B-3030 Heverlee, Belgium

INTRODUCTION

A knapsack (or subset-sum) problem that is useful for cryptographic purposes, consists of a set of n positive integers $a = \{a_1, a_2, \ldots a_n\}$, called the knapsack a, and a sum s. The density d of a knapsack is defined to be $n/\log_2(a_i)_{max}$. The knapsack problem then consists of finding the set, if any, of binary numbers $x = \{x_1, x_2, \ldots, x_n\}$, such that $\Sigma x_i.a_i = s$.

In {2}, E.F.Brickell presented an algorithm for breaking knapsacks of low density. It was expected {2} that an improved version of it would solve most knapsacks of density less than .54. For knapsacks of increasing density, the probability of having so-called "small coefficient identities" (SCI's) in the knapsack also increases. The presence of these SCI's appears to be the reason for the failure of Brickell's algorithm for knapsack problems of high density.

In this paper, a simple technique is proposed for circumventing this problem effectively, so that the algorithm becomes capable of solving knapsacks of higher densities.

Tests for low dimensions have shown that the effectiveness of the algorithm can thus be increased to solve knapsacks of densities even >.9 with high probability and reasonable supplementary computer power. Since the number of available knapsacks decreases very quickly with its density (cfr. fig. p.7), only a relatively small set of high density knapsacks of low dimension remains difficult to solve.
Moreover, all the advantages of Brickell's algorithm are preserved. The most important of these is that the algorithm is applicable to any knapsack public-key cipher based on the knapsack problem mentioned above, whether the cipher already exists or is to be invented.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 109-115, 1988.
© Springer-Verlag Berlin Heidelberg 1988

THE EXTENSION OF BRICKELL'S ALGORITHM

We shall now give a very brief overview of Brickell's algorithm and the improvements that have been accomplished. More details can be found in {2},{1}. Brickell made the basic observation that a knapsack problem $\Sigma x_i.a_i=s$ is in fact a linear equation in n binary unknown x_i and with non-zero sum s. He concluded that if n linear equations ($\Sigma x_i.y_{ji} = s_j(k)$) could be found such that:
- they are not linearly dependent of each other
- they all conform to the solution x of the knapsack problem
- that finding such set of equations is computationally feasible

that then each knapsack (problem) for which this is possible may be regarded as solved.

In the first part of his algorithm, a technique is described to construct a nxn matrix Y from which the rows y_j correspond to the coefficients of the n equations in the n unknown x_i. The first row y_1 of Y consists of the elements $b_i = a_i.W \bmod M$. The other rows of Y are derived by applying modular mappings with the small sum property on previous rows of Y.{2},{1} These modular mappings are calculated with the L.L.L.-algorithm {3}.

In the second part of his algorithm it is proven that $\Sigma x_i.b_i \in \{s', s'+M, \ldots, s'+(n-1)M\}$, with $s' = s.W \bmod M$. Thus n integers $s'(k) = s'+k.M$ $(0 \leq k \leq n-1)$ are derived, exactly one of which corresponds to the correct binary solution x. Because all rows of Y, except for the first one, have been calculated by applying modular mappings with the small sum property on previous rows, we can now calculate {2},{1} from the set of possible sums of the first row s'(k), a new set of possible sums $s_j(k)$ for every other row j, with: $s_j(k) = \Sigma x_i.y_{ji}$, assuming that $\Sigma x_i.b_i = s'(k)$.
(for exactly one constant value of k, $s_j(k)$ will be the sum of the new knapsack problem $\Sigma x_i.y_{ji} = s_j(k)$ with the same x as the original knapsack problem, and this for every row y_j of Y.) For each of the values k we can thus compose a nx1 matrix S(k) and we can calculate the possible solutions x as nx1 matrices X from the matrix problems Y.X = S(k). If indeed s is the sum of a subset of the a_i's then one of the vectors S(k) will give a correct <u>binary</u> solution X for $Y^{-1}.S(k)$.

If the first part of the algorithm can be completed then it is always straightforward to execute the second part and to solve the knapsack problem.

In experiments run by Brickell, his algorithm appeared to be always successful except when there were identities satisfied by the a_i's of the form:

$\Sigma\alpha_i.a_i = 0$, with $\Sigma|\alpha_i| \leq n$.

Such an identity is called a "small coefficient identity" (SCI). It appears that some "dangerous" types of these SCI's are easily "inherited" from one vector to another through the use of modular mappings, and in particular by modular mappings with the small sum property.{1} Brickell also demonstrated that the expected number of SCI's of a given knapsack increases with its density.

Through experiments it has come clear to us that the basic algorithm of Brickell fails for knapsacks of high densities because the given knapsack contains one or more SCI's. Since the first row y_1=b of Y, consisting of the numbers b_i, is derived from the numbers a_i through a modular mapping, some of the SCI's present in the knapsack a are inherited by the first row of Y. Since all the other rows of Y are analogously derived through modular mappings with the small sum property of the previous rows, it may happen that one or more of the dangerous SCI's of the initial knapsack are present in all the possible rows of Y. As a consequence of this, it becomes impossible to find a matrix Y with linearly independent rows and the algorithm fails.

In order to be able to still use Brickell's algorithm in these cases, we propose to construct the first row b of Y such that its expected number of SCI's is smaller than unity. Hereto we proved that the density of b is so small for practical dimensions n that it would normally not contain any SCI's if it was constructed randomly. It is also proven that, by construction, b can exclusively inherit SCI's present in a, for practical dimensions {1}. It may therefore be concluded that the SCI's of b are mainly introduced by the non-random character of b.

We therefore propose to construct b as:

$$b_i = a_i.W \mod M + k_i.M.$$

The numbers k_i can initially be regarded as relatively small random numbers.

It can be proven that as a consequence of this:

$$\Sigma x_i.b_i \in \{s', s'+M, \ldots, s'+(n-1+\Sigma k_i)\}.$$

So it remains possible for this choice of b to determine $\Sigma x_i.b_i$, but $n+\Sigma k_i$ vectors S(k) have to be tested instead of n.

It is therefore advantageous to keep Σk_i small, e.g. by restricting all the k_i's to $k_i \in [1,K]$, in which K is a small positive integer value.

It has been mathematically and experimentally verified that this new technique has several advantages:

- b can still exclusively contain SCI's of the knapsack a.
- Therefore, b can only contain SCI's if at least one of the SCI's of a is still present in b, the probability of which decreases with increasing K.
- The average supplementary computer power needed, however, can be roughly estimated as the processing of (K-1).n/2 extra vectors S(k), and the implications of larger values of the b_i's, mainly on the execution time of the L.L.L.-algorithm.

These results have been verified experimentally for knapsacks of low dimensions n, in two ways.

1. Knapsack problems with knapsacks of low dimensions and high density have been tested.

For example the knapsack problems with knapsack:

a = { 2 , 5 , 6 , 10 , 20 }, of - dimension n = 5
- high density d = 1.16

could not be solved with the original algorithm of Brickell due to the inheritance of the SCI α = { 0 , 0 , 0 , 2 , -1 } from a to b and from there to all possible rows of the matrix Y.
This knapsack was however solved with the better choice of b.

2. Computer tests have been run on the effectiveness of both the original and the new version of Brickell's algorithm.

The purpose of these Monte-Carlo tests was to gain experimental results on the probability that b contains SCI's, for both algorithms.
As parameters were chosen the dimension n and the density d, plus, for the new algorithm, the value of K and the number of trials T (i.e. the number T of vectors k that may be chosen for a given knapsack to try to find a SCI-free vector b for it). While considering the figures, we must bear the following in mind:

- All figures represent dimension 5. Calculations were also made for dimension 8. For this higher dimension, the figures appear to remain flat till higher densities but then fall steeper.
  This may indicate worse efficiency for high dimensions.
  This observation is in line with results obtained by Lagarias and Odlyzko.
- The figures, however, give rather pessimistic results for low dimensions, since:
  - Pessimistic approximations had to be made to calculate them.
  - It has been assumed that finding a SCI-free vector b is a necessary condition for solving a knapsack. To our experience however, a lot of knapsacks with b contaminated by SCI's could be solved,since only for dangerous SCI's the probability is high that they are inherited by all possible rows of Y.
  - In case only one SCI is transported through all possible rows of Y, a special countermeasure can be taken. {2}
  - The set of knapsacks that will remain unsolved will contain a relatively large fraction of "useless" knapsacks, since for increasing density, the probability also increases that knapsacks are not one-to-one.

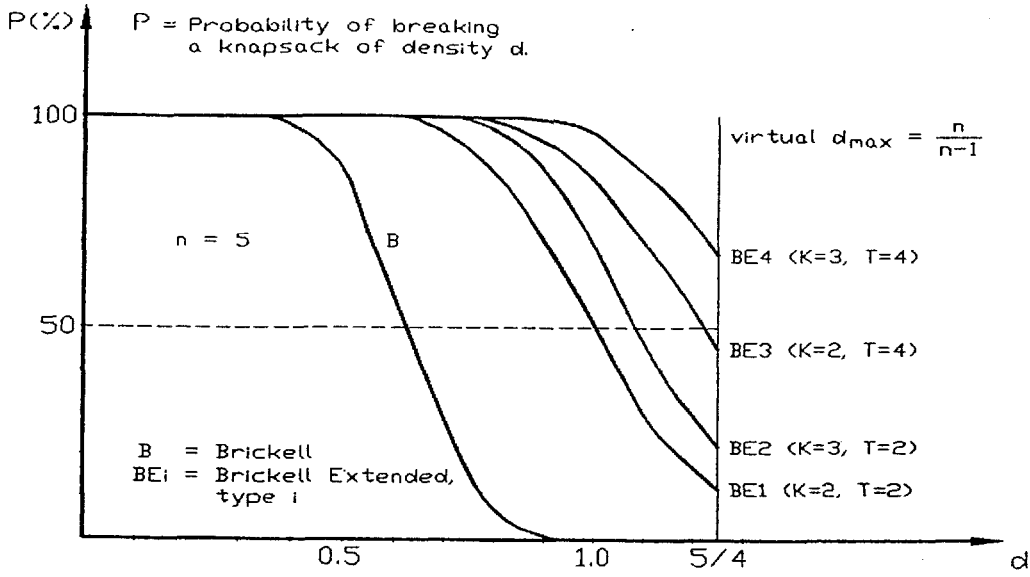
CONCLUSION

In our work, Brickell's algorithm {2} has been extended to the use of better vectors b. {1} This makes the algorithm capable of solving low dimensional knapsacks of high densities larger than .9, with reasonable supplementary computer power.
Since the efficiency of this improvement has only been tested for low dimensions, and considering contradictory results from Lagarias and Odlyzko for high dimensions, it remains to be tested to what extent our improvement remains efficient for increasing dimensions.

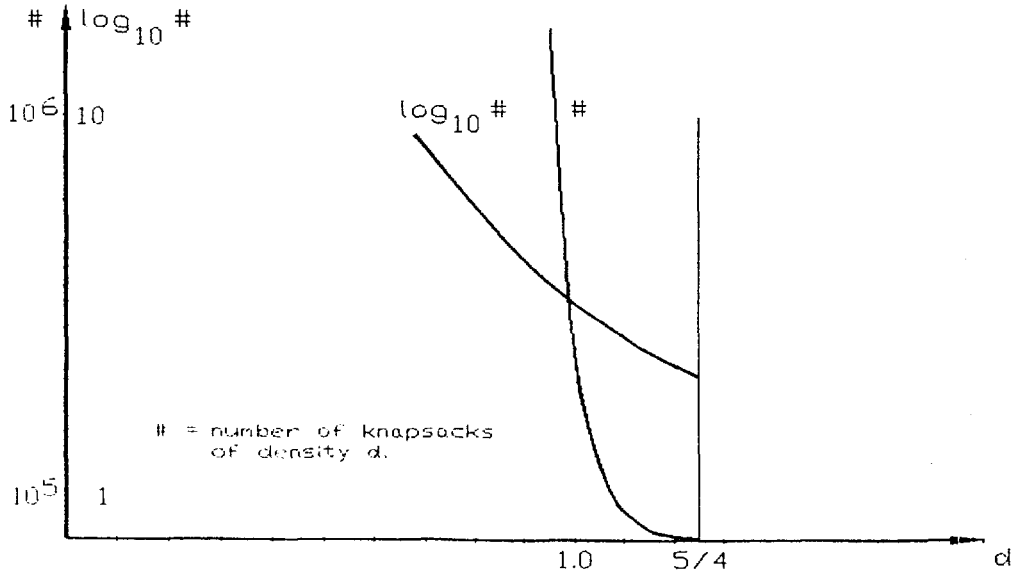
ACKNOWLEDGEMENTS

We are grateful to Y. Desmedt for many suggestions during the thesis work and to S. Claeys for his help with the test programs. We also wish to thank the company CRYPTECH for its support and stimulating interaction. Last but not least, we thank E. Brickell and A. Odlyzko for the discussion on this work.

Figures



Test results of the extended algorithm of Brickell: Probability of breaking a knapsack, with both algorithms, and number of available knapsacks as functions of its density.



REFERENCES

{1} F. Jorissen, "De cryptanalyse van knapzak publieke sleutel geheimschriftvormende algoritmes", M.Sc. Thesis, Katholieke Universiteit Leuven, Belgium, May 1985.

{2} E.F. Brickell, "Solving Low Density Knapsacks", Sandia National Laboratories, Albuquerque, USA.

{3} A.K. Lenstra, H.W. Lenstra Jr., and L. Lovasz, "Factoring Polynomials with Rational Coefficients", Mathematische Annalen, Vol.261, no.4, pp.515-534, 1982.

On Privacy Homomorphisms
(Extended Abstract)

Ernest F. Brickell
Yacov Yacobi

Bell Communications Research
435 South Street
Morristown, New Jersey 07960

Abstract

An additive privacy homomorphism is an encryption function in which the decryption of a sum (or possibly some other operation) of ciphers is the sum of the corresponding messages. Rivest, Adleman, and Dertouzos have proposed four different additive privacy homomorphisms. In this paper, we show that two of them are insecure under a ciphertext only attack and the other two can be broken by a known plaintext attack. We also introduce the notion of an R-additive privacy homomorphism, which is essentially an additive privacy homomorphism in which only at most R messages need to be added together. We give an example of an R-additive privacy homomorphism that appears to be secure against a ciphertext only attack.

1. Introduction

A privacy homomorphism is an encryption function which allows the encrypted data to be operated on without knowledge of the decryption function. Privacy homomorphisms were introduced by Rivest, Adleman, and Dertouzos [RAD]. Secure privacy homorphisms could be applied to protect data-bases against eavesdropping by the system manager.

[RAD] mentioned that privacy homomorphisms could be used to establish data base systems in which an encrypted total of a list of items could be computed using only the encrypted values of the list of items. Privacy homomorphisms could also be used to establish a secure conference telephone call. In a typical conference call, a central facility adds together the signals of the speakers. If the signals were encrypted using an additive privacy homomorphism, then the central facility could "add" the signals together without decrypting them.

We must now give a more formal definition. A privacy homomorphism is a family of functions $(e_k, d_k, \alpha, \gamma)$ such that $d_k(\gamma(e_k(m_1), ..., e_k(m_r)) = \alpha(m_1, ..., m_r)$ for each k in key space $\mathbf{K}$ and any $m_1, ..., m_r$ in message space $\mathbf{M}$. The

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 117-125, 1988.
© Springer-Verlag Berlin Heidelberg 1988

definition given in [RAD] is more general. In considering the security of these systems, we will assume that the cryptanalyst knows the functions e_k, d_k, α, γ but does not know the key k. Ahituv, Lapid & Neumann [ALN] showed that any privacy homomorphism, which has addition as its ciphertext domain operation is insecure under chosen ciphertext attack.

One privacy homomorphism mentioned in [RAD] is based on the multiplicative property of the RSA encryption function [RSA]. Let $n = pq$ where p and q are large primes. Let $\alpha(m_1, ..., m_r) = m_1 \cdots m_r \bmod n$ and $\gamma = \alpha$. Define e and d in the usual manner for RSA. This privacy homomorphism is as secure as RSA.

There are four more privacy homomorphisms mentioned in [RAD]. There are weaknesses in each of these which will be discussed in section 2.

We now introduce the notion of an R-additive privacy homomorphism, in which only the addition of at most R messages is allowed and α is addition over the integers. Let $\mathbf{M}$ be a subset of the integers. An R-additive privacy homomorphism is a family of functions $(e_k, d_k, \alpha, \gamma)$ such that $d_k(\gamma(e_k(m_1), ..., e_k(m_r)) = m_1 + \cdots + m_r$ for $r \leq R$ and for each k in key space $\mathbf{K}$ and any $m_1, ..., m_r$ in message space $\mathbf{M}$. We do not know of any example of an R-additive privacy homomorphism that it thought to be secure against a known plaintext attack. It would be very interesting to know whether such a function exists that is provably secure in the sense that cryptanalysis can be shown equivalent to some well studied problem.

An R-additive privacy homomorphism would provide a nice solution to the problem of designing a secure conference call [BLY]. For this application, it would only be necessary for R to be 2 or 3, but it would be desirable to limit the data expansion. For large enough R, R-additive privacy homomorphisms for which γ is addition will be insecure under the chosen ciphertext attack of [ALN]. However for small R, this attack does not apply. This will be explained in more detail in section 3. It is possible that a privacy homomorphism that is succeptible to a known plaintext attack or a chosen ciphertext attack could still be secure for encrypting digitized speech. Therefore, we think that it is worthwhile to examine the security of R-additive privacy homomorphisms against a ciphertext only attack.

In section 3, we will describe an example of an R-additive privacy homomorphism. At first glance, it appears that the attacks that broke the knapsack type cryptosystems would also break this system using a ciphertext only attack. In section 4, we will show that this is probably not the case. In section 5, we will describe how this system could be modified to possibly improve the security.

2. Cryptanalysis of [RAD] privacy homomorphisms

We present cryptanalysis of four privacy homomorphism systems which appear in [RAD] (examples 1,5,3,4 in this order).

System 1

Let g be a generator modulo a prime p, where $p-1=\prod_{i=1}^{k} p_i^{d_i}$, and for all i, $p_i \le B$, for some small B. Let q be a large prime and let $n=p\cdot q$. Message M is encrypted by computing: $C \equiv g^M \bmod n$. Decryption is $M \equiv \log_g C \bmod p$. The structure of p enables computation of the discrete logarithm by the method of Pohlig and Hellman [PH] in time $O(B^{1/2})$. This is a privacy homomorphism with α being addition mod $p-1$ and γ multiplication mod n.

Cryptanalysis

This system is insecure because the modulus n can be factored by the Pollard $p-1$ method [P]. This method is effective because $p-1$ is highly composite. To explain the basic idea of this factoring algorithm, let $d \ge \max\{d_i \mid 1 \le i \le k\}$.

Lemma 1. $p \mid gcd(a^{B!^d}-1, n)$.

Proof. $a^{p-1} \equiv 1 \bmod p$, hence $a^{\prod_{i=1}^{k} p_i^{d_i}} \equiv 1 \bmod p$. Since $\prod_{i=1}^{k} p_i^{d_i} \mid B!^d$; $a^{B!^d} \equiv a^{\prod_{i=1}^{k} p_i^{d_i}} \equiv 1 \bmod p$. □

The cryptanalyst will not know the values of B and d. He can just choose some B' and d'. If $gcd(a^{B'!^{d'}}-1, n)=1$, then he knows that he picked B' or d' too small, and he could just increase them, perhaps by doubling them. The gcd could also be n, but this would only happen if $q-1$ was also highly composite. In this case, the cryptanalyst could just modify the values of B' and d' until he found one that would give a gcd of either p or q. The expected running time for this cryptanalysis is $O(dB \log B \log n)$.

System 2

Let $a_0, a_1, \ldots, a_{k-1}$ be randomly chosen positive integers. Let A be the matrix

$$\begin{pmatrix} 1 & a_0 & a_0^2 & \cdots & a_0^{k-1} \\ 1 & a_1 & a_1^2 & \cdots & a_1^{k-1} \\ & & \vdots & & \\ 1 & a_{k-1} & a_{k-1}^2 & \cdots & a_{k-1}^{k-1} \end{pmatrix}$$

Given an integer x, with binary representation $x_0, x_1, \ldots, x_{k-1}$, let $\bar{x} = (x_0, \ldots, x_{k-1})$. The encryption of x is the vector $\bar{y} = A\bar{x}$. Decryption is preformed by multiplying by A^{-1}. This is a privacy homomorphism with γ being addition or subtraction over the integers, and α being componentwise addition or subtraction over the integers.

Cryptanalysis

Suppose j is the largest index such that $x_j = 1$, i.e., $x_{j+1}, \ldots, x_{k-1} = 0$. Let z be any positive integer. Then $z^j \le \sum_{i=0}^{j} x_i z^i \le \sum_{i=0}^{j} z^i < (z+1)^j$. Thus $\left\lfloor \left(\sum_{i=0}^{j} x_i z^i \right)^{\frac{1}{j}} \right\rfloor = z$. This leads to an obvious ciphertext only attack. Given a cipher $y = (y_0, \ldots, y_{k-1})$, guess a value for j, the largest index such that $x_j = 1$. Compute $\lfloor y_0^{1/j} \rfloor = b_0$. Write y_0 in base b_0 notation. If all the coefficients are 0 and 1, then probably $b_0 = a_0$ and x is easily found. If not try a different choice for j. The values of $y_1, \ldots, y_{k-1}$ can be used as an additional check.

System 3

Let p and q be large primes. Encryption of message M is an ordered pair $(M \bmod p, M \bmod q)$. This is a privacy homomorphism with γ being addition, subtraction or multiplication componentwise, and α being addition, subtraction or multiplication mod pq. Decryption is done using the Chinese remainder theorem.

Cryptanalysis

We will argue that this can be broken by a known plaintext attack. Let the encryption of a message M be represented by a pair (C, D), where $C \equiv M \bmod p$ and $D \equiv M \bmod q$. Assume that the cryptanalyst has plaintext-ciphertext pairs M_i, C_i, D_i, $i = 1, 2, 3, \ldots, r$. $p \mid C_i - M_i$. Let p' be the $gcd\ \{C_i - M_i : 1 \le i \le r\}$. Clearly, $p \mid p'$. A q' such that $q \mid q'$ can be found in a similar manner. If $p' = p$ and $q' = q$, the cryptanalyst can decrypt any ciphertext. Even for small r, there is a high probability that $p' = p$ and $q' = q$. Even if this is not the case, then for any new ciphertext the cryptanalyst can either determine the plaintext, or if he is given the plaintext, he can improve his knowledge of either p or q. Specifically, given ciphertext (C, D), the cryptanalyst can find M' such that $M' \equiv C$

mod p' and $M' \equiv D \mod q'$. If $M' \neq M$, then either $M \not\equiv C \mod p'$ or $M \not\equiv D \mod q'$. So if the cryptanalyst is given this M, he can improve either p' or q' by replacing p' by $gcd(M - C, p')$ and q' by $gcd(M - D, q')$. The cryptanalyst would have to be given at most $\log p + \log q$ such messages before he knows p and q.

System 4

In this system encryption is just writing the message in a secret radix system. Referring to the least significant digit, this reduces to the cryptanalysis of the previous system.

3. An R-additive privacy homomorphism

In this section, we introduce the notion of an R-additive privacy homomorphism in which only the addition of at most R messages is allowed. We will also give an example that appears to be secure against a ciphertext only attack. We will call this example the modular multiplication R-additive privacy homomorphism or MM-RAPH.

Let $\mathbf{M}$ be a subset of the integers. An R-additive privacy homomorphism is a family of functions $(e_k, d_k, \alpha, \gamma)$ such that $d_k(\gamma(e_k(m_1), ..., e_k(m_r)) = m_1 + \cdots + m_r$ for $r \leq R$ and for each k in key space $\mathbf{K}$ and any $m_1, ..., m_r$ in message space $\mathbf{M}$.

The [ALN] chosen ciphertext attack on privacy homomorphisms in which γ is addition will also apply to R-additive privacy homomorphisms if R is large enough. In particular, if C is the set of integers between 0 and 2^n, then the chosen ciphertext attack will work as follows. For $i = 0, 1, ..., n-1$, find m_i such that $e_k(m_i) = 2^i$. Given a cipher $c = \sum_{i=0}^{n-1} x_i 2^i$, for $x_i \in \{0, 1\}$, then $d_k(c) = \sum_{i=0}^{n-1} x_i m_i$ if $R > \sum_{i=0}^{n-1} x_i$. So if $R > n$, this attack will always work. However, if R is a small constant, then this attack would need an exponential number of chosen ciphertext-plaintext pairs to be successful.

Suppose that the messages that we actually want to encrypt are the integers between 0 and B. Let ℓ be a fixed positive integer. Let $N = RB$. Let z be an integer randomly chosen in the interval $[N^\ell, 2N^\ell]$, and let y be chosen relatively prime to z.

A block of ℓ messages $m_0, ..., m_{\ell-1}$ will be encrypted by computing $t = \sum_{i=0}^{\ell-1} m_i N^i$, and then computing $c = ty \bmod z$.

The decryption process is obvious. Given a cipher c form $t = cy^{-1} \bmod z$. Write t base N to obtain $m_0, ..., m_{\ell-1}$. Showing that this is an R-additive privacy homomorphism is straightforward as well.

We can modify System 3 of [RAD] to make an RAPH similar to the MM-RAPH. We merely choose p and q so that pq is in the interval $[N^\ell, 2N^\ell]$. Then to encrypt a block of messages $m_0, ..., m_{\ell-1}$ we again compute $t = \sum_{i=0}^{\ell-1} m_i N^i$, and then form the ordered pair $(t \bmod p, t \bmod q)$. The security of this system is similar to the security of the MM-RAPH, but we will not discuss the details here.

4. Attacks on the R-additive privacy homomorphism

The modular multiplication RAPH is immediately suspect to the attacks that broke the knapsack cryptosystems. However, it appears that these attacks will not be successful against the modular multiplication RAPH when z is large enough relative to R. We will not specify how large z must be, because it depends upon the performance of the Lovasz basis reduction algorithm [LLL], and there have not been enough computational experiments to estimate what this performance is.

For $a_1, ..., a_n, A, b_1, ..., b_n, B$ all positive integers, we will say that a vector $\beta = \left(\frac{b_1}{B}, ..., \frac{b_n}{B} \right)$ is a δ-quality simultaneous Diophantine approximation (or just a δ-approximation) to $\alpha = \left(\frac{a_1}{A}, ..., \frac{a_n}{A} \right)$ if $B < A$ and $\max_{1 \le i \le n} | B \frac{a_i}{A} - b_i | \le A^{-\delta}$. See [L].

For most tuples $a_1, ..., a_n, A$ there are no δ-approximations for fixed $\delta > \frac{1}{n}$. On the other hand, by Dirichlet's theorem, there is always a δ-approximation for fixed $\delta < \frac{1}{n}$, specifically if $A^{-\delta} > \left(\lceil A^{\frac{1}{n}} \rceil - 1 \right)^{-1}$.

Suppose now that we are given $n+1$ ciphers $c_0, c_1, ..., c_n$. The attack that we will examine involves finding simultaneous diophantine approximations to the vector $\left(\frac{c_1}{c_0}, ..., \frac{c_n}{c_0} \right)$. We will assume that c_0 is the largest of the c_i. If it is not, we would need to reduce the $c_i \bmod c_0$ and this would complicate our discussion. If the c_i were random, we would expect the best δ-approximation to be for $\delta \approx \frac{1}{n}$. However, these c_i are not random. In particular there exist

integers k_i, such that $c_i\ y^{-1} - k_i\ z = t_i < \frac{z}{R}$. This implies that

$$\frac{y^{-1}}{z} - \frac{k_i}{c_i} = \frac{t_i}{c_i z}$$

$$\frac{k_0}{c_0} - \frac{k_i}{c_i} = \frac{t_i}{c_i z} - \frac{t_0}{c_0 z}$$

$$k_0 \frac{c_i}{c_0} - k_i = \frac{t_i}{z} - \frac{t_0 c_i}{z\ c_0}$$

$$\left| k_0 \frac{c_i}{c_0} - k_i \right| < \frac{1}{R}. \qquad (4.1)$$

If $\ell > 1$, there are other approximations that are of about this same quality. The reason for these other approximations is similar to the reason for other approximations existing in the multiple iterated knapsacks, [B], but we will not go into the details here. We will simply indicate that these other approximations exist because each $t_j = \sum_{i=0}^{\ell-1} m_{ji} N^i$, and $m_{ji} < N/R$. If we had only $m_{ji} < N$, these other approximations would not exist. It can be shown that if ℓ such interesting approximations can be found then the privacy homomorphism can be broken.

In n is large enough so that $\frac{1}{R} << c_1^{\frac{-1}{n}}$, i.e. $R^n >> c_1$, then the interesting approximations will probably be best approximations, but only by a factor of $\frac{1}{R}$. So the security of this system depends on whether we can find these approximations.

The Lovasz algorithm can be used to find good simultaneous diophantine approximations. It is guaranteed to find a δ-approximation within a factor of 2^n of the best approximation for a given vector of integers. Computational experience has shown that it often does much better. The attacks on some knapsack systems relied on the Lovasz algorithm finding approximations that are within n of the best approximations. However computational experience indicates that the Lovasz algorithm is unlikely to find approximations that are only within a constant of the best. Further computational experiments on our privacy homomorphism will be necessary to conclude that the Lovasz algorithm will not break it.

5. A modification to the MM-RAPH

In this section, we discuss a modification to the MM-RAPH that appears to increase security, although it is still

susceptible to a known plaintext attack.

Again, let the integers between 0 and B be the messages that we want to encrypt. Let $N = R\,B$. Let p be a prime larger than N. (The first prime larger than N will be satisfactory.) Let A be an ℓ by ℓ matrix chosen randomly such that A is nonsingular mod p. Let q be an integer $> Rp$. Let z be an integer randomly chosen in the interval $[q^{\ell}, 2q^{\ell}]$ and let y be chosen relatively prime to z.

A block of ℓ messages $m_0, \ldots, m_{\ell-1}$ will be encrypted by taking the vector $\bar{m} = (m_0, \ldots, m_{\ell-1})$ and forming $\bar{s} = A\,\bar{m} \bmod p$. Let $\bar{s} = (s_0, \ldots, s_{\ell-1})$. Compute $t = \sum_{i=0}^{\ell-1} s_i\, q^i$. Finally set $c = ty \bmod z$.

The decryption process is obvious. Given a cipher c form $t = cy^{-1} \bmod z$. Write t base q to obtain $\bar{s} = (s_0, \ldots, s_{\ell-1})$. Then $A^{-1}\,\bar{s} \bmod p$ gives the desired message $\bar{m}$.

Showing that this is an R-additive privacy homomorphism is straightforward as well. Suppose $\bar{m}_1, \ldots, \bar{m}_r$ are r messages for $r \le R$. Let $\bar{s}_j = (s_{j,0}, \ldots, s_{j,\ell-1})$, t_j, and c_j be the values computed in the encryption of $\bar{m}_j$. We need to show that when $c = c_1 + \cdots + c_r$ is decrypted the result is $\bar{m} = (m_{1,0} + \cdots + m_{r,0}, \ldots, m_{1,\ell-1} + \cdots + m_{r,\ell-1})$.

Clearly $t = cy^{-1} \equiv t_1 + \cdots + t_r \bmod z$. But $\sum_{j=1}^{r} t_j = \sum_{j=1}^{r} \sum_{i=0}^{\ell-1} s_{j,i}\, q^i = \sum_{i=0}^{\ell-1} \left[\sum_{j=1}^{r} s_{j,i} \right] q^i$. Also $\sum_{j=1}^{r} s_{j,i} < q$, for $0 \le i \le \ell-1$. So $\sum_{j=1}^{r} t_j < z$ and writing t base q gives the values $\sum_{j=1}^{r} s_{j,i}$, for $0 \le i \le \ell-1$. Let $\bar{s} = \left[\sum_{j=1}^{r} s_{j,0}, \ldots, \sum_{j=1}^{r} s_{j,\ell-1} \right]$. Then $\bar{s}\,A^{-1} \equiv \bar{m}_1 + \cdots + \bar{m}_r \bmod p$. But since $m_{1,i} + \cdots + m_{r,i} < p$, for $0 \le i \le \ell-1$, the result follows.

We will not give all of the details of why this modified system is still susceptible to a known plaintext attack. The basic idea is that known plaintext-ciphertext pairs can be used to produce vectors that have simultaneous Diophantine approximations that are better than the approximations described in (4.1).

6. Open problems

We have introduced an R-additive privacy homomorphism for which we have argued that it cannot be broken by an obvious application of the attacks that broke the knapsack cryptosystems. Is this scheme in fact secure? It would be interesting to find other privacy homomorphisms.

References

[ALN] N. Ahituv, Y. Lapid, S. Neumann, "Processing Encrypted Data", CACM, Sept. 1987, Vo. 30, No. 9, pp. 777-780.

[B] E. F. Brickell, "Breaking Iterated Knapsacks", *Advances in Cryptology, Proc. Crypto 84,* Santa Barbara, August 19-22, 1984, Lecture Notes in Computer Science, vol. 196, Springer-Verlag, Berlin, 1985, pp. 342-358.

[BLY] E. F. Brickell, P. J. Lee, Y. Yacobi, "Secure Audio Teleconference", to appear in *Advances in Cryptology, Proc. Crypto 87,* Springer-Verlag, New York.

[L] J. C. Lagarias, "Knapsack Public Key Cryptosystems and Diophantine Approximation" (Extended Abstract), *Advances in Cryptology, Proc. Crypto 83*, Plenum Publ. Co., New York, 1984, pp.3-24.

[LLL] A. K. Lenstra, H. W. Lenstra, Jr., and L. Lovasz, "Factoring Polynomials with Rational Coefficients", *Mathematische Annalen 261*, pp. 515-534, 1982.

[PH] Stephen C. Pohlig and Martin E. Hellman, "An Improved algorithm for computing Logarithms over $GF(p)$ and its cryptographic significance", IEEE Trans. on Inf. Th. Vol. IT-24, No. 1 Jan. 1978. pp. 106-110.

[P] J. M. Pollard, "Theorems on factorization and primality testing", Proc. Cambridge Philos. Soc. vol. 76 (1974), pp. 521-528.

[RAD] Ronald L. Rivest, Len Adleman and Michael L. Dertouzos, "On data banks and privacy homomorphisms", in Foundation of Secure Computations, Academic Press 1978.

[RSA] R. L. Rivest, A. Shamir and L. Adleman, "A Method for Obtaining Digital Signatures and Public Key Cryptosystems", *Commun. ACM,* vol. 21, pp. 294-299, April 1978.

[S] A. Shamir, "A Polynomial Time Algorithm for Breaking the Basic Merkle-Hellman Cryptosystem", IEEE Trans. Inform. Theory, vol. IT-30, No. 5, September 1984, pp. 699-704.

AN IMPROVED PROTOCOL FOR DEMONSTRATING POSSESSION OF DISCRETE LOGARITHMS AND SOME GENERALIZATIONS

David Chaum
Jan-Hendrik Evertse
Jeroen van de Graaf

Centre for Mathematics and Computer Science
Kruislaan 413 1098 SJ Amsterdam The Netherlands

Abstract:
A new protocol is presented that allows A to convince B that she knows a solution to the Discrete Log Problem—i.e. that she knows an x such that $\alpha^x \equiv \beta \pmod N$ holds—without revealing anything about x to B. Protocols are given both for N prime and for N composite.

We also give protocols for extensions of the Discrete Log problem allowing A to show possession of:

— multiple discrete logarithms to the same base at the same time, i.e. knowing $x_1, \ldots, x_K$ such that $\alpha^{x_1} \equiv \beta_1, \ldots, \alpha^{x_K} \equiv \beta_K$;
— several discrete logarithms to different bases at the same time, i.e. knowing $x_1, \ldots, x_K$ such that the product $\alpha_1^{x_1}\alpha_2^{x_2} \cdots \alpha_K^{x_K} \equiv \beta$;
— a discrete logarithm that is the simultaneous solution of several different instances, i.e. knowing x such that $\alpha_1^x \equiv \beta_1, \ldots, \alpha_K^x \equiv \beta_K$.

We can prove that the sequential versions of these protocols do not reveal any "knowledge" about the discrete logarithm(s) in a well-defined sense, provided that A knows (a multiple of) the order of α.

1. Introduction

Consider the following problem:

- Alice (party A) knows a solution to the **Discrete Log (DL)** problem: for particular α, β and N, she knows the exponent x such that $\alpha^x \equiv \beta \pmod N$ holds.
- Alice wants to convince Bob (party B) that she knows x.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 127-141, 1988.
© Springer-Verlag Berlin Heidelberg 1988

- Alice is not willing to reveal the value of x.
- Bob accepts an exponentially small chance that Alice is cheating, i.e. that she pretends to know an x but in fact does not. More precisely, the probability that Alice succeeds in cheating without being detected by Bob, is 2^{-T}, where T is proportional to the time and space required.

This paper presents a protocol which solves this problem, both for the cases that N is a prime and that $N = P_1 P_2$, where P_1 and P_2 are prime numbers of roughly the same size. In the second case, it is assumed that A knows the factorization of N. When A does not know this factorization, however, our protocol is still of interest, since given α and N she can choose $x \in \{1, \ldots, N-1\}$ at random and then compute β simply by exponentiation (or a third party could supply A with x and β). It is assumed that B has only polynomial (in log N) computational power, whereas no restrictions are imposed on A's computational resources. No probabilistic polynomial time algorithm is known for finding x given α, β and N, if N is a prime or a composite that is difficult to factor.

In [CEGP86] protocols were presented that solve the same problem. Compared to those protocols, the basic protocol presented here is perhaps easier to understand, to use, and to generalize. The existence of a protocol with the same functionality is implied by general results of [BrCr86], [Ch86] and [GMW86]. However, these protocols are not very useful in practice. In [Ch87] efficient protocols that solve this problem are needed; this was the major motivation for our research.

We also present protocols for proving possession of a solution to some generalizations of the Discrete Log problem:

(1) **Multiple Discrete Log (MDL):**
A shows to B that, given α and $\beta_1, \ldots, \beta_K$, she knows $x_1, \ldots, x_K$ such that $\alpha^{x_1} \equiv \beta_1, \ldots, \alpha^{x_K} \equiv \beta_K$. This protocol is more efficient than applying the basic DL-protocol for the pairs $(x_1, \beta_1), \ldots, (x_K, \beta_K)$ while it gives B the same probability of catching a cheating A. When a third party creates the x_i's at random and supplies A with the x_i's and β_i's, this protocol also offers the possibility to use DL as the basis for an authentication scheme in a way similar to Fiat & Shamir [FiSh86], whose scheme is based on the difficulty of factoring.

(2) **Relaxed Discrete Log (RDL):**
A shows to B that, given $\alpha_1, \ldots, \alpha_K$ and β, she knows $x_1, \ldots, x_K$ such that $\alpha_1^{x_1} \alpha_2^{x_2} \cdots \alpha_K^{x_K} \equiv \beta$.

(3) **Simultaneous Discrete Log (SDL):**
A shows to B that, given $\alpha_1, \ldots, \alpha_K$ and $\beta_1, \ldots, \beta_K$, she knows x such that $\alpha_i^x \equiv \beta_i$ for $i = 1, \ldots, K$.

The Discrete Log problem is stated above in Z_N^* (the multiplicative group of residue classes modulo N of integers coprime with N) with N prime or composite. However, the Discrete Log problem can be stated in any finite group: let G be a finite group, $<\alpha>$ the

subgroup generated by $\alpha \in G$, and $\beta \in <\alpha>$; then find x such that $\alpha^x = \beta$. The protocols presented in this paper are feasible in any group G in which both A and B can apply the group operation in an efficient way, e.g. in time polynomial in the logarithm of the order of G. (For the RDL-protocol we also have to assume that G is commutative). The properties of the DL-protocol over Z_N^* which are proved in this paper (namely that it allows A to convince B with high probability that she knows the discrete logarithm of β with respect to α without revealing any knowledge about that discrete logarithm) remain true for the DL-protocol over any group G, such that A knows (a positive multiple of) the order of α in G and B knows a "good" approximation of (a positive multiple of) the order of α, i.e. if m is some multiple of the order of α then B knows an integer m' such that $|m - m'| \leq m^c$, where c is some number with $0 \leq c < 1$. For instance, if $G = Z_N^*$, then B knows the exact order of G if N is a prime, while if $N = P_1 P_2$ with P_1 and P_2 primes of order $O(N^{1/2})$, then G has order $\phi(N) = (P_1 - 1)(P_2 - 1)$, B knows N and $|N - \phi(N)| = O(N^{1/2})$. The DL-protocol can be used also if B does not know a good approximation for (a multiple of) the order of α; however, B may be able to obtain such an approximation by examining the messages which he receives from α while participating in the protocol. Further, with a slight modification, the DL-protocol is still feasible if A does not know a multiple of the order of α in G, but then the protocol leaks information about x.

Of course, these protocols are of interest only if no efficient algorithm for computing the Discrete Log in G exists. Apart from the case $G = Z_N^*$, with N prime or composite, we can take the K-fold direct product of Z_N^*, giving rise to the Simultaneous Discrete Log protocol, or the set of points of an elliptic curve over $GF(P)$ for some prime P, imposed with the usual group structure. It was argued in [Mi85] that discrete logarithms in the group of points of an elliptic curve over $GF(P)$ might be even harder to compute than "ordinary" discrete logarithms.

For describing the protocols, we use the same protocol notation throughout the paper. The meaning of this notation is straightforward; only the next few things might need explanation:

- T is the **security parameter**, agreed upon before the protocol starts. Increasing T reduces A's chance of successfully cheating exponentially, but increases the amount of communication and computation only linearly.
- In the zeroth step of the protocol, A and B agree on α, β and N.
- If not indicated otherwise, the expressions appearing in the protocol have to be reduced modulo N.
- By $a :\equiv$ *expression* (mod M) we mean that the expression at the right-hand side must be computed and reduced modulo M and that the resulting value is assigned to a; if $M = N$ we omit the suffix "(mod N)".
- $e \in_R S$ indicates that an element e is chosen at random from the set S, i.e. all elements of S have an equal probability of being chosen and that the choice is

independent of all previous events.

— In some steps of the protocol a party checks if a particular equality holds; this is denoted as: check $a \stackrel{?}{=} b$. If the check fails, cheating is detected and the protocol halts.

— Expressions shown on the left or right are known to the corresponding party only, and are secret from the other party.

— A party cannot learn anything about the computations that are done by the other party, except from the messages which (s)he received from that party.

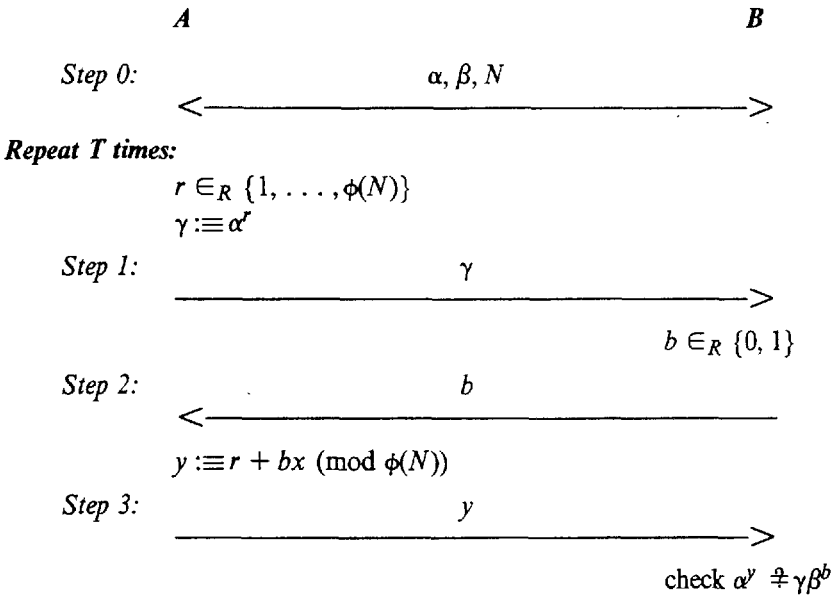
2. The basic protocol: Discrete Log

Instance: N, $\alpha \in Z_N^*$, $\beta \in <\alpha>$
Solution: x such that $\alpha^x \equiv \beta \pmod N$

In order for the protocol to make sense, one has to assume that there are no efficient (polynomial in $\log N$ time) algorithms to compute discrete logarithms modulo N for N prime or composite. It is generally believed, that for large primes N satisfying certain weak restrictions, it is infeasible to compute discrete logarithms in Z_N^*. In this paper we assume that computing discrete logarithms is also hard when N is a product of two primes that is difficult to factor. Our motivation behind this assumption is that any fast method to compute for each pair $\alpha \in Z_N^*$ and $\beta \in <\alpha>$ an integer x with $\alpha^x \equiv \beta \pmod N$, enables one to efficiently find the factorization of N with high probability. Indeed, choose γ at random from Z_N^* and pick a "probable prime" p between N and $2N$. Compute $\alpha :\equiv \gamma^{2p}$, $\beta :\equiv \gamma^2$. Then with high probability, p is a prime number coprime with $\phi(N)$, whence $\beta \in <\alpha>$. Suppose that the discrete log algorithm computes an x with $\beta \equiv \alpha^x$. Then $\gamma^{2(px-1)} \equiv 1$, hence γ^{px-1} is a square root of 1. With 50% chance, this square root is not equal to 1 or -1 and yields the factorization of N. It is in fact possible to prove the following stronger (and from a cryptographic point of view more convincing) statement. Let N be a given product of two large primes and suppose that there is a random polynomial time algorithm (i.e. an algorithm whose running time is polynomial in the length of the input and which can do unbiased coinflips) with the following property: when the algorithm is given the pair α, β as input, where α is uniformly distributed on Z_N^* and β is uniformly distributed on $<\alpha>$, then the probability that that algorithm outputs an integer x with $\alpha^x \equiv \beta$ is at least $1/Q(\log N)$ for some polynomial Q. Then there is a random polynomial time algorithm that outputs the factorization of N with probability at least ½. We do not work this out here.

We develop the protocols simultaneously for both the cases N prime and N composite, and point out the differences. If N is composite, we assume that A knows its factorization.

Protocol 1: Discrete Log: $\alpha^x \equiv \beta \pmod{N}$



2.1. Remarks about the underlying mathematical model

Our purpose is to prove that Protocol 1 (and the other protocols that will be described in this paper) have the following properties:

- *correctness:* even a cheating party A is unable (or with a very small chance able) to send messages to B satisfying all of B's checks;
- *security:* B cannot obtain any useful "knowledge" about the discrete logarithm from the protocol other than from the initializing information α, β and N, even if he cheats.

In this subsection we explain more precisely what is meant by "cheating" and what it means that no knowledge is revealed. In the remainder of this paper we assume that the computational power of B is polynomially bounded in $\log N$. For the computational power of A, we do not make any assumption since it does not matter in our arguments whether A's computational power is polynomially bounded or not.

We say that A cheats if she constructs her messages by means of some probabilistic algorithm, in another way than that described in the protocol. For instance, if A does not know the discrete logarithm, then she could try to construct her messages in such a way that they still satisfy B's checks. B cheats if he generates his bits in step 2 using a random polynomial time algorithm that does not choose them at random.

In several papers, e.g. [GMR85], [BKP85], [GMW86], and [CEGP86], it was argued that the security of a protocol can be proved by showing the existence of a random polynomial time "simulator" that simulates the interaction between A and B using as input only what B knows at the beginning of the protocol. For convenience of the reader, we explain below the notion of such a simulator, and why its existence suffices.

Informally speaking, we would like to prove that in whatever way B tries to cheat, the data he obtains during his participation in the protocol do not help him find a solution to any equation (*) $f(\alpha,\beta,N,z)=0$ in the unknown z. Before the protocol starts, B gets α, β and N. In step 1, B gets $\gamma \in Z_N^*$ from A. In step 2, B generates a bit b. If B cheats, then he generates b in another way than just choosing it at random; he might use all messages that he computed or received before (in the first round of the protocol these are only N, α, β, and γ). During the execution of the algorithm that produces b, B might obtain intermediate results, some of which he would like to store for later purposes; let $\mathbf{b}$ comprise the intermediate results stored by B. Finally, in step 3, B receives an integer y from A such that $\alpha^y \equiv \gamma\beta^b$. Thus B gets a tuple $(\gamma,\mathbf{b},b,y)$. After steps 1, 2, and 3 have been executed T times, B has obtained a tuple $W_B = (\gamma_1,\mathbf{b}_1,b_1,y_1,\ldots,\gamma_T,\mathbf{b}_T,b_T,y_T)$ containing all data obtained by B during his participation in the protocol. Note that W_B is stochastic, and that its probability distribution depends on the initializing information $I_A=(\alpha,\beta,N,x)$.

Suppose that B has a probabilistic algorithm M_f that computes a solution to equation (*) with some positive probability. Further, suppose that there is a "simulator" S, with small (polynomial in $\log N$) running time, which produces a tuple W_B^* with about the same probability distribution as W_B, on input $I_A^*=(\alpha,\beta,N)$. This simulator may depend on B's way of cheating. Let M_f^* be the algorithm that first computes W_B^* in the same way as S, on input I_A^*, and then computes a solution to (*) by applying M_f to I_A^* and W_B^*. M_f^* outputs a solution to (*) with about the same probability as M_f (since W_B and W_B^* have about the same probability distribution) and M_f^* has about the same running time as M_f. This shows that the protocol does not reveal any useful knowledge to B: algorithm M_f when input the data gathered by B during the performance of the protocol does not output a solution to (*) faster or with higher probability than algorithm M_f^* when input the initialization data I_A^* only. Hence in order for the protocol to be secure, it suffices that there is a simulator with small running time for each way of cheating by B.

It is possible to give the notion of a simulator, informally described above, a formal meaning similar to [GMR85], [BKP85] or [CEGP86]. We assume that the reader is familiar with the formal definition of a protocol and with the underlying computational model, as described in [BKP85]. We use a slightly different model that is briefly described below.

We consider cryptographic protocols with two parties, a "prover" A and a "verifier" B. Both A and B use probabilistic Turing machines T_A and T_B, respectively, with a work tape, a random tape and a "mailbox". The machines use the same alphabet Σ. Each machine can read only from its own work tape, random tape, and mailbox, but it can write on its own work tape as well as on the other machine's mailbox. Each step executed by a machine is determined by the machine's state and the contents of its three tapes, and does not depend on the other machine's state. Whenever a machine has to send a message to the other machine, it copies that message from its own work tape to the other machine's mailbox; then the other machine may copy this message from its mailbox to its work tape. For convenience we assume that the machines do not run simultaneously. Thus after a machine has written a complete message string on the other machine's mailbox, it stops and is reactivated again only after it has received a message from the other machine.

Before the protocol starts, both machines are in a fixed initialization state, and the work tapes of these machines are filled with certain initialization data I_A^*. Further, T_A's work tape contains the secret x. Put $I_A = (I_A^*, x)$; then I_A is a string of length l, say, over Σ. Further, in the beginning both random tapes are filled with an infinite number of symbols, each uniformly chosen from Σ. At the end of the protocol, both machines are supposed to be in an end state. We suppose that the number of steps performed by T_B between the initialization state and the end state is bounded above by a polynomial in l; for our purposes it does not matter whether or not the number of steps executed by T_A between the initialization state and the end state is polynomially bounded in l.

Denote by W_B the contents of T_B's work tape in the end state. W_B contains all data stored by T_B while the protocol was running; these data might contain the messages sent and received by T_B and some final or intermediate results of T_B's computations. Because of the use of random tapes, W_B is a stochastic variable whose probability distribution depends on I_A. We assume that for each I_A, W_B assumes its values in some enumerable set Ω; let $\mathbf{P}_{I_A}$ denote the probability distribution of W_B on Ω. An A-simulator, based on machine T_B, is defined as a probabilistic Turing machine which produces a tuple W_B^* with almost the same probability distribution as W_B (but depending only on I_A^*); more precisely, if $\mathbf{P}_{I_A}^*$ denotes the probability distribution of W_B^* then for each I_A with sufficiently large length l we have

$$\sum_{\omega \in \Omega} \left| \mathbf{P}_{I_A}(W_B = \omega) - \mathbf{P}_{I_A}^*(W_B^* = \omega) \right| \leq C^{-l},$$

where C is some absolute constant with $C > 1$.

2.2. Correctness and security of Protocol 1

In this subsection we prove that Protocol 1 is correct and secure. In the theorem below we assume that T is polynomially bounded. (By "polynomial" we always mean polynomial in log N.)

Theorem 1.
(a) If B does not cheat, and if A does not know the discrete logarithm x, then any cheating by A in Protocol 1 is detected by B with probability $\geq 1-2^{-T}$.
(b) If A does not cheat, then for any random polynomial time machine used by B in Protocol 1, there exists a polynomial time A-simulator.

Proof:
(a) Correctness: If A does not know x, then each time that step 3 is executed, she is unable to send the proper answer to B in at least one of the cases $b = 0$ or $b = 1$. Hence, in each round of the protocol, she will be caught with probability at least ½. Thus B will detect that A does not know x with probability at least $1 - 2^{-T}$.
(b) Security (sketch): Let T_B be the random polynomial time machine used by B. Suppose for the moment that the number of rounds T is equal to 1. We have $I_A = (\alpha,\beta,N,x)$, $I_A^* = (\alpha,\beta,N)$ and $W_B = (\gamma,\mathbf{b},b,y)$ where: γ is the message received by B in step 1; b is the bit computed by T_B in step 2, using γ; $\mathbf{b}$ comprises the intermediate steps in the computation of b stored by T_B; and y is the integer received by B in step 3, satisfying $\alpha^y \equiv \gamma\beta^b \pmod N$. Then the polynomial time A-simulator is described as follows (all expressions have to be reduced modulo N):

Repeat at most $L := \log N / \log 2$ times:
(1) choose c at random from $\{0, 1\}$
(2) choose y at random from $\{0, \ldots, N-2\}$
(3) compute $\gamma := \alpha^y \beta^{-c}$
(4) compute $b \in \{0, 1\}$ using T_B; let $\mathbf{b}$ comprise the saved intermediate results
(5) if $b = c$ then output $W_B^* = (\gamma,\mathbf{b},b,y)$
until $b = c$
If $b \neq c$ in all L executions of steps (1)-(5), then output W_B^* = "badluck"

Note that this simulator has polynomial running time.

Suppose first that N is a prime number and consider one execution of steps (1)-(5) described above. In this execution, γ is uniformly distributed over $<\alpha>$, and γ and c are mutually independent. Further, in the computation of b, only γ is used, hence b is also independent of c. Therefore, $b = c$ with probability ½. This implies that the probability that $b = c$ in at least one of L executions of steps (1)-(5) is at least $1 - N^{-1}$. Note that $\alpha^y \equiv \gamma\beta^b$. Let Ω be the set of values which can be assumed by W_B^*, including the message "badluck". It is easy to verify that for each $\omega \in \Omega$ with $\omega \neq$ "badluck" we have

$$\mathbf{P}_{I_A}^{*}(W_B^{*}=\omega \mid W_B^{*}\neq \text{badluck}) = \mathbf{P}_{I_A}(W_B=\omega).$$

Together with the fact that $\mathbf{P}_{I_A}^{*}(W_B^{*} = \text{badluck}) \leqslant N^{-1}$ this shows that

$$S := \sum_{\omega \in \Omega} \left| \mathbf{P}_{I_A}(W_B=\omega) - \mathbf{P}_{I_A}^{*}(W_B^{*}=\omega) \right| \leqslant 2N^{-1}.$$

Since the length of I_A is proportional to $\log N$, this implies part (b) of Theorem 1 if N is a prime.

Now suppose that $N = P_1 P_2$ where P_1 and P_2 are primes of order $N^{1/2}$. Then $N-1$ is not a multiple of the order of α in Z_N^{*}, hence the number γ computed in step 3 is not uniformly distributed over $<\alpha>$. However, all arguments given above remain valid if we consider conditional probabilities given that $0 \leqslant y \leqslant \phi(N)-1$. Using that $\mathbf{P}_{I_A}^{*}(\phi(N) \leqslant y \leqslant N-2) = O(N^{-1/2})$, it follows that S is bounded above by $O(N^{-1/2})$.

If $T > 1$, the simulator described above has to be repeated T times. This increases the running time by a factor T, and S by a factor $\leqslant T$. But since T is bounded above by a polynomial in $\log N$, this completes the proof of Theorem 1. □

3. Generalization 1: Multiple Discrete Log

Instance: $N,\ \alpha \in Z_N^{*},\ \beta_1, \ldots, \beta_K \in <\alpha>$
Solution: $x_1, \ldots, x_K$ such that $\alpha^{x_1} \equiv \beta_1 \pmod N, \ldots, \alpha^{x_K} \equiv \beta_K \pmod N$

Protocol 2: Multiple Discrete Log: $\alpha^{x_1} \equiv \beta_1 \pmod N, \ldots, \alpha^{x_K} \equiv \beta_K \pmod N$

	A		*B*
Step 0:		$\alpha, \beta_1, \ldots, \beta_K, N$	
		$<$——————————$>$	
Repeat T times:			
	$r \in_R \{1, \ldots, \phi(N)\}$		
	$\gamma :\equiv \alpha^r$		
Step 1:		γ	
		——————————$>$	
			$b_i \in_R \{0, 1\},$
Step 2:		$b_1, \ldots, b_K$	
		$<$——————————	
	$y :\equiv r + b_1 x_1 + \cdots + b_K x_K \pmod{\phi(N)}$		
Step 3:		y	
		——————————$>$	
			check $\alpha^y \stackrel{?}{=} \gamma \beta_1^{b_1} \cdots \beta_K^{b_K}$

We assume that T and 2^K are bounded above by some polynomial in $\log N$.

Theorem 2.
(a) If B does not cheat, and if A does not know at least one of the discrete logarithms $x_1, \ldots, x_K$, then any cheating by A in Protocol 2 is detected by B with probability $\geqslant 1-2^{-T}$.
(b) If A does not cheat, then for any random polynomial time machine used by B in Protocol 2, there exists a polynomial time A-simulator.

Proof:
(a) Correctness: Consider one round of the protocol, consisting of steps 1, 2, and 3. By assumption, A does not know the discrete logarithm of at least one β_i (with respect to α). Hence for whatever γ she computes in step 1, she is not able to compute the discrete logarithm of $\gamma \beta_1^{b_1} \cdots \beta_K^{b_K}$ for at least one vector $(b_1, \ldots, b_K) \in \{0, 1\}^K$. Together with the lemma below this implies that, in each round, A is caught cheating with probability at least ½. Hence her cheating is detected by B with probability at least $1-2^{-T}$.

Lemma: *Suppose that A does not know the discrete logarithm of $\gamma(\vec{b}) :\equiv \gamma \beta_1^{b_1} \cdots \beta_K^{b_K} \pmod N$ for at least one vector $\vec{b} = (b_1, \ldots, b_K) \in \{0, 1\}^K$. Then she does not know the discrete logarithm of $\gamma(\vec{b})$ for at least half the vectors $\vec{b} \in \{0, 1\}^K$.*

Proof: We proceed by induction on K. For $K = 1$ the lemma is trivial. Suppose now that the lemma is true for $K = L - 1$, where $L \geqslant 2$ (induction hypothesis). We shall prove the lemma for $K = L$. We distinguish three cases. In what follows, $\vec{b}$ always denotes a vector $(b_1, \ldots, b_L) \in \{0, 1\}^L$, and $\gamma(\vec{b})$ has the same meaning as above with L replacing K.

In the first case, A knows the discrete logarithms of all the products $\gamma(\vec{b})$ with $b_L = 0$. Thus, she cannot know the discrete logarithm of β_L. Hence she cannot form the discrete logarithm of any product $\gamma(\vec{b})$ with $b_L = 1$.

In the second case, A knows the discrete logarithm of each product $\gamma(\vec{b})$ with $b_L = 1$. Then, by the same argument as in case 1, it follows that A cannot form the discrete logarithm of any product $\gamma(\vec{b})$ with $b_L = 0$.

In the last case, A does not know the discrete logarithm of at least one of the products $\gamma(\vec{b})$ with $b_L = 0$ and also not the discrete logarithm of at least one of the products $\gamma(\vec{b})$ with $b_L = 1$. Then by the induction hypothesis, she does not know the discrete logarithm of at least half the products $\gamma(\vec{b})$ with $b_L = 0$ and also, by the induction hypothesis with $\gamma \beta_L$ instead of γ, she does not know the discrete logarithm of at least half the products $\gamma(\vec{b})$ with $b_L = 1$.

We conclude that in each of the three cases A cannot know the discrete logarithm of at least half the products $\gamma(\vec{b})$. This completes the induction step. □

(b) Security. The proof is essentially the same as that of Theorem 1, part (b). We only describe the A-simulator. B uses machine T_B.

For $i = 1$ to T:
  repeat at most $L' := \log N / \log (1-2^{-K})$ times:
    choose $\vec{c}_i = (c_{1i}, \ldots, c_{Ki})$ at random from $\{0, 1\}^K$
    choose y_i at random from $\{0, \ldots, N-2\}$
    compute $\gamma_i := \equiv \alpha^{y_i} \beta_1^{-b_{1i}} \cdots \beta_K^{-b_{Ki}}$
    compute $\vec{b}_i \in \{0, 1\}^K$ with T_B; let $\mathbf{b}_i$ comprise the intermediate results of T_B's computations
    if $\vec{b} = \vec{c}$ then output $(\gamma_i, \mathbf{b}_i, \vec{b}_i, y_i)$
  until $\vec{b} = \vec{c}$
  if $\vec{b}_i \neq \vec{c}_i$ in all L' iterations, then output "badluck"
If not at least once "badluck" then output $W_B^* = (\gamma_1, \mathbf{b}_1, \vec{b}_1, y_1, \ldots, \gamma_K, \mathbf{b}_K, \vec{b}_K, y_K)$

Note that the running time of this simulator is proportional to T and 2^K, but by assumption these numbers are bounded above by some polynomial in $\log N$. □

Remark 1. It is possible to use Protocol 2 as an interactive "identification scheme," a concept introduced by Fiat and Shamir [FiSh86]. Suppose that not A, but some mutually trusted "center" generates the x_i's at random, supplies these to A (but to nobody else) and stores the corresponding β_i's in some public directory. Then A can identify herself to B by showing that she knows the discrete logarithms of the β_i's without revealing any knowledge about their values, using Protocol 2. Thus, the data obtained from his interaction with A will not enable B to identify himself to a third party as A. The Fiat-Shamir scheme uses a public composite number, whose factorization is known only to the center. In that scheme, the β_i's for a user A are squares modulo that composite, constructed by the center, and A has to convince B that she possesses square roots of these β_i's. Contrary to our scheme used with a prime modulus, in the Fiat-Shamir scheme the center must keep some trapdoor information secret (namely the factorization of the modulus). On the other hand, Fiat and Shamir argued that there scheme allows the center to form the β_i's of some user A by applying some public function to A's name and address or the like. Thus, any verifier B can compute the β_i's by himself and they do not have to be stored in a public file. The function that is used to construct the β_i's should be such that only the center, knowing the factorization of the modulus, is able to compute a square root of some output of the function. However, it is currently not known how to prove that any such public function prevents people from constructing names for which they can find corresponding square roots themselves. The scheme of Fiat and Shamir is more efficient than ours, because it requires only squaring whereas our scheme requires exponentiations of $\log N$-bit numbers.

Remark 2. If we assume that not 2^K but K is bounded above by a polynomial in $\log N$, then the running time of the simulator described above is not polynomial any more since it is proportional to 2^K. It seems impossible to construct a simulator whose running time depends only polynomially on K for *each* machine used by B, since B might generate its bits by some one-way function. However, there does exist a simulator (described below) for the machine that chooses the bits to be sent from B to A uniformly from $\{0, 1\}$. In order to prevent B from choosing the bits to be sent to A not uniformly, one could modify the protocol so that the bits are chosen not by B alone, but by A and B together, using a coin flipping protocol like that in [Bl82]. The protocol thus modified is called "verifier-passive" (cf. [CEGP86]) because B can do nothing but checking that A sends the correct answers. The simulator is described below:

For $i = 1$ to T:
  choose $\vec{bi} = (b_{1i}, \ldots, b_{Ki})$ at random from $\{0, 1\}^K$
  choose y_i at random from $\{0, \ldots, N-2\}$
  compute $\gamma_i := \equiv \alpha^{y_i} \beta_1^{b_{1i}} \cdots \beta_K^{b_{Ki}}$
Output $W_B^* = (\gamma_1, \vec{b}_1, y_1, \ldots, \gamma_T, \vec{b}_T, y_T)$

4. Generalization 2: Relaxed Discrete Log

Instance: $N, \alpha_1, \ldots, \alpha_K \in Z_N^*, \beta \in Z_N^*$
Solution: $x_1, \ldots, x_K$ such that $\alpha_1^{x_1} \cdots \alpha_K^{x_K} \equiv \beta \pmod N$

It is easy to see that if there exists an efficient algorithm which computes a solution to the Relaxed Discrete Log problem for each instance, then there is also a fast way to compute discrete logarithms for each possible instance: in order to find the discrete logarithm of β with respect to α one has merely to solve the Relaxed Discrete Log problem for the instance $N, \alpha, 1, \ldots, 1, \beta$. It is possible to prove the following stronger result. Let N, K be given integers such that N is either a prime or the product of two primes and that K is bounded above by a polynomial in $\log N$ and suppose that there exists a random polynomial (in $\log N$) time algorithm with the following property: if $\alpha_1, \ldots, \alpha_K$ and β are given as input to the algorithm, where $\alpha_1, \ldots, \alpha_K$ are uniformly distributed over Z_N^* and β is uniformly distributed over $<\alpha_1, \ldots, \alpha_K>$, then that algorithm outputs integers $x_1, \ldots, x_K$ such that $\alpha_1^{x_1} \cdots \alpha_K^{x_K} \equiv \beta \pmod N$ with probability at least $1/Q(\log N)$ for some polynomial Q. Then there is a random polynomial time algorithm that computes for each pair $\alpha \in Z_N^*$ and $\beta \in <\alpha>$ with probability $\geq$ ½ an integer x such that $\alpha^x \equiv \beta \pmod N$. This statement is not proved here.

Protocol 3: Relaxed Discrete Log: $\alpha_1^{x_1} \cdots \alpha_K^{x_K} \equiv \beta \pmod{N}$

	A		*B*
Step 0:		$\alpha_1, \alpha_2, \ldots, \alpha_K, \beta, N$	
		$\longleftrightarrow$	

Repeat T times:

$r_1, \ldots, r_K \in_R \{1, \ldots, \phi(N)\}$

$\gamma_1 :\equiv \alpha_1^{r_1}, \ldots, \gamma_K :\equiv \alpha_K^{r_K}$

Step 1:		$\gamma_1, \ldots, \gamma_K$	
		$\longrightarrow$	
			$b \in_R \{0, 1\}$
Step 2:		b	
		$\longleftarrow$	

$y_i :\equiv r_i + bx_i \pmod{\phi(N)}$ for $1 \leqslant i \leqslant K$

Step 3:		$y_1, \ldots, y_K$	
		$\longrightarrow$	
			check $\alpha_1^{y_1} \cdots \alpha_K^{y_K} \equiv$
			$\equiv \gamma_1 \cdots \gamma_K \beta^b$

If K and T are bounded above by a polynomial in $\log N$ then we have:

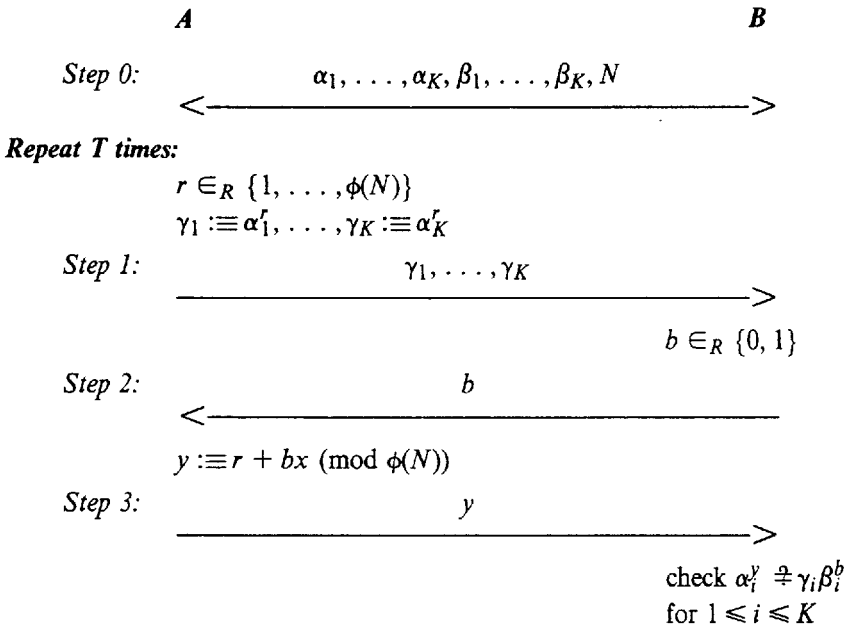
Theorem 3.
(a) If B does not cheat and if A does not know at least one of $x_1, \ldots, x_K$, *then any cheating by A in Protocol 3 is detected by B with probability* $\geqslant 1 - 2^{-T}$.
(b) If A does not cheat, then for any random polynomial time machine used by B in Protocol 3 there exists a polynomial time A-simulator.

The proof of this result is essentially the same as that of Theorem 1 and we do not give it here.

5. Generalization 3: Simultaneous Discrete Log

Instance: $N, \alpha_1, \ldots, \alpha_K, \beta_1, \ldots, \beta_K \in Z_N^*$
Solution: x such that $\alpha_1^x \equiv \beta_1 \pmod{N}, \ldots, \alpha_K^x \equiv \beta_K \pmod{N}$

Protocol 4: Simultaneous Discrete Log: $\alpha_1^x \equiv \beta_1 \pmod N, \ldots, \alpha_K^x \equiv \beta_K \pmod N$



We assume again that both K and T are bounded above by a polynomial in $\log N$.

Theorem 4.
(a) If B does not cheat and if A does not know x, then any cheating by A in Protocol 4 is detected by B with probability $\geqslant 1 - 2^{-T}$.
(b) If A does not cheat then for any random polynomial time machine used by B in Protocol 4, there exists a polynomial time A-simulator.

The proof of this theorem is essentially that of Theorem 1.

Remark. From an algebraic point of view, this protocol is just Protocol 1 applied to the K-fold direct product $(Z_N^*)^K$, with $\alpha = (\alpha_1, \ldots, \alpha_K)$ and $\beta = (\beta_1, \ldots, \beta_K)$.

Acknowledgements

We would like to thank René Peralta and Bert den Boer for their valuable suggestions.

References

[BKP85] R. Berger, S.Kannan, and R. Peralta, "A Framework for the Study of Cryptographic Protocols," *Proc. CRYPTO 85*, pp. 87-103 H.C. Williams, ed., Lecture Notes in Computer Science 218, Springer Verlag, Berlin etc., (1986).

[Bl82] M. Blum, "Coin Flipping by Telephone," *Proc. IEEE COMPCON*, pp. 133-137, (1982).

[BrCr86] G. Brassard, and C. Crépeau, "Zero-Knowledge Simulation of Boolean Circuits," *Proc. CRYPTO 86*, pp. 223-233, A.M. Odlyzko, ed., Lecture Notes in Computer Science 263, Springer Verlag, Berlin etc., (1987).

[Ch86] D. Chaum, "Demonstrating that a Public Predicate can be Satisfied Without Revealing Any Information About How," *Proc. CRYPTO 86*, pp. 195-199.

[Ch87] D. Chaum, "Blinding for unanticipated signatures," *To appear in proc. EUROCRYPT 87.*

[CEGP86] D. Chaum, J.-H. Evertse, J. van de Graaf, and R. Peralta, "Demonstrating possession of a discrete logarithm without revealing it," *Proc. CRYPTO 86*, pp. 200-212.

[FiSh86] A. Fiat, and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," *Proc. CRYPTO 86*, pp. 186-194.

[GMR85] S. Goldwasser, S.Micali, and C. Rackoff, "The Knowledge Complexity of Interactive Proof Systems," *Proc. 17th Annual ACM Symp. on Theory of Computing*, pp. 291-304, (1985).

[GMW86] O. Goldreich, S. Micali, and A. Wigderson, "How to Prove all NP-statements in Zero-Knowledge, and a Methodology of Cryptographic Protocol Design," *Proc. CRYPTO 86*, pp. 171-185.

[Mi85] V. Miller, "Elliptic curves and cryptography," *Proc. CRYPTO 85*, pp. 417-428.

A Public Key Analog Cryptosystem

George I. Davida & Gilbert G. Walter

University of Wisconsin-Milwaukee
Milwaukee, WI 53201
USA

In this paper we present a public key cryptosystem based on error correcting codes [1, 7, 15]. The new public key system is obtained by extending the public key cryptosystem of McEliece [6, 12].

In this scheme a message M, consisting of a column vector of k elements from a finite field is first scrambled by multiplying it by a non singular matrix Q to get

$$M' = Q\ M$$

This scrambled message has parity check variables added to it, by multiplying it by a generator matrix G and then has all the variables reordered by multiplication by a permutation matrix P. Noise is then added to obtain the encrypted message

$$C = P\ G\ Q\ M + Z \tag{1}$$

The product of the three matrices $G' = P\ G\ Q$ is made public, but the factors are not.

The analogs of these matrix operations are integral transforms, while the column vectors are functions which we take to be square integrable on (0,1) or (0,2) [2-5, 11, 13, 14, 18]. The message M will now be denoted by $x = x(t)$, and $x' = Q\ x$ will mean

$$x'(t) = \int_0^1 q(t,s)\, x(s)\, ds \tag{2}$$

where $q(t,s)$ is the kernel of the transformation. P will be a similar operator except that it must be an orthogonal operator to avoid changing the magnitude of the noise. This noise will consist of a realization of a random process $z(t)$ on (0,2). The operator G will have a special form to be discussed below.

In the McEliece digital scheme, the matrix $G = \begin{bmatrix} I \\ -A \end{bmatrix}$ where the submatrix A introduces the new variables [12]. Decryption is accomplished by multiplying both sides of

The research reported in this paper was supported in part by NSF grant DCR-8504620.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 143-147, 1988.
© Springer-Verlag Berlin Heidelberg 1988

$$P^T C = G\ Q\ M + P^T Z \tag{3}$$

by a matrix $H = [A\ I]$. This annihilates the term $G\ Q\ M$ and leaves the syndrome $S = H\ P^T\ C$ on the left. Coding Theory is used to estimate P^{TZ} which in turn is used with (3) to estimate $G\ Q\ M$. An estimate for M then is obtained by a projection operator followed by Q^{-1}.

In the analog scheme G is based on a kernel $k(t,s)$, $t,s \in (0,1)$ and is given by

$$(Gx)(t) = \int_0^1 \{\delta(t-s) - k(t-1,s)\}\, x(s)\, ds, \quad 0 < t < 2$$

and H is given by

$$(Hy)(t) = \int_0^1 k(t,s)\, y(s)\, ds + \int_1^2 \delta(t+1-s)\, y(s)\, ds, \quad 0 < t < 1.$$

Clearly both are continuous operators and $H\ G = 0$. The analog encryption consists of first operating with Q, then G, then P on x and then adding noise, i.e. the ciphertext is

$$y = P\ G\ Q\ x + Z \tag{4}$$

where $y = y(t)$ is in $L^2(0,2)$. The kernel $q(t,s)$ of the operator Q may be taken to be a Green's function of appropriate differential operator. The kernel $p(t,s)$ of P may be taken to be

$$p(t,s) = \sum_n \phi_{i(n)}(t)\, \phi_n(s)$$

where $\{\phi_n\}$ is a complete orthonormal system in $L^2(0,2)$, and $i(n)$ is a permutation of the integers. In general this will be a singular kernel, (i.e. not an L^2 function) but in practical cases will be approximated by a finite sum.

The kernel $k(t,s)$ is chosen to be of the form

$$k(t,s) = \sum_n \psi_n(s)\delta(t-t_n)$$

where $\{\psi_n\}$ is again an orthonormal system, the Rademacher system, which takes the values $+-1$ and $\{t_n\}$ is a dense sequence of numbers in $(0,1)$. The number of terms in the series will again be a finite number in practical situations and the impulse δ will be approximated by a

function. Furthermore the integral transform

$$\int_0^1 k(s,t)\, y(s)\, ds = \sum_n \psi_n(t)\, Y(n)$$

(in which is $y(s)$ happens to be white noise) gives us a finite sum of independent normal random variables.

Decryption begins by operating with H on

$$P^{-1}\, y = G\ Q\ x + P^{-1}\, Z$$

to obtain the syndrome $S = HP^{-1}y$. The value of $P^{-1}Z$ of minimum error which satisfies

$$H\ Z' = H\ P^{-1}Z = S$$

is then found. This can be done by using the generalized inverse

$$\hat{Z}' = H^*\ (H\ H^*)^{-1}S$$

where H^* is the adjoint operator. This then is used to estimate $P^{-1}Z$ and $P^{-1}y - \hat{Z}'$ restricted to (0,1) is used to estimate Qx which is then inverted to obtain the estimate of the message.

The final estimate cannot be made completely noise free as in the digital case but the noise can be reduced by this method. The reason noise could be eliminated completely in the digital case is that there is a minimum distance between a correct and an incorrect message. This is nor longer true in the analog case in which an incorrect message can be arbitrarily close to the correct message. Thus the best that can be done is to reduce the variance of the noise by means of a decoding algorithm.

The error between the true noise and the estimated noise may be shown, after a few manipulations, to be given by

$$e = G(G^*G)^{-1}G^*Z$$

This in turn can be shown to be the average of a finite number of independent normal random variables with variance σ^2 which has a variance approaching zero as the number of terms increases.

Fortunately in analog signals it is unnecessary to obtain the exact correct message since in most cases it suffices to obtain a message close to the correct one. In voice transmissions, for example, messages slightly corrupted by noise can be understood.

The three operators P, G and Q can be combined in a single integral transform which can then serve as a public key. The security of this arrangement will be at least as good as the original scheme of McEliece. This is particularly true if Q is taken to be a Green's function

operator as in [5]. However in this case the inverse would involve noise amplification and must be balanced against the noise reduction of G.

References

[1] Berlekamp, E., *Algebraic Coding Theory*, McGraw Hill, New York (1968).

[2] Blum, J., Susarla, V., and Walter, G., "Estimation of the Prior Distribution Using Differential Operators," *Colloquia Mathematica Societatis*, (1980).

[3] Coddington, E.A. and Levinson, N., *Theory of Ordinary Differential Equations*, McGraw Hill, New York (1955).

[4] Davida, G., Gilbertson, C., and Walter, G., "Analog Cryptosystems," *Eurocrypt85*, (1985).

[5] Davida, G. and Walter, G., "A Class of Analog Cryptosystems," *SECURICOM 87*, (1987).

[6] Denning, D., *Cryptography and Computer Security*, Addison Wesley, Reading, MA (1982).

[7] Diffie, W. and Hellman, M., "New Directions in Cryptography," *IEEE Transactions on Information Theory* IT-22(6)(1976).

[8] E., Hellman, M., "An Extension of the Shannon Theory Approach to Cryptography," *IEEE Trans. on Info. Theory* IT-23 pp. 289-294 (May 1977).

[9] Hellman, M., "A Cryptanalytic Time-Memory Tradeoff," *IEEE Transactions on Information Theory* IT-26(4)(1980).

[10] Kak, S. and Jayant, N., "On Speech Encryption Using Waveform Scrambling," *Bell System Technical Journal*, (1977).

[11] Kak, S., "Overview of Analog Signal Encryption," *IEEE Proceedings*, (1983).

[12] McEliece, R., "A Public Key Cryptosystem Based on Algebraic Coding Theory," DSN Progress Rep. 42-44, Jet Propulsion Lab.,Cal Inst. of Tech, Pasadena, CA (Jan., Feb. 1978).

[13] Nashed, M., "Operator-Theoretic and Computational Approaches to Ill-Posed Problems with Apaplications to Antenna Theory," *IEEE Transactions on Antennas and Propogation*, (March 1981).

[14] O'Bryan, T. and Walter, G., "Mean Square Estimation of the Prior Distribution," *Sankya, The Indian Journal of Statistics*, (1979).

[15] Rivest, R., Shamir, A., and Adleman, L., "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," *Comm. ACM* Vol 21(2) pp. 120-126 (Feb. 1978).

[16] Shannon, C., "Communcation Theory of Secrecy Systems," *Bell Systems Technical Journal* 28(1949).

[17] Stakgold, I., *Green's Functions and Boundary Value Problems*, Wiley, New York (1975).

[18] Tikhonov, A. and Arsenin, V., *Solutions of Ill-Posed Problems*, Winston-Wiley, New York (1977).

[19] Wyner, A., "An Analog Scrambling Scheme Which Does Not Expand Bandwidth: Part II -Continuos Time," *IEEE Transactions on Information Theory* IT-25(6)(1979).

[20] Wyner, A., "An Analog Scrambling Scheme Which Does NOt Expand Bandwidth: Part I -Discrete Time," *IEEE Transactions on Information Theory* IT-25(3)(1979).

MESSAGE AUTHENTICATION WITH ARBITRATION OF TRANSMITTER/RECEIVER DISPUTES*

Gustavus J. Simmons
Sandia National Laboratories
Albuquerque, NM 87185

In the most general model of message authentication, there are four essential participants: a transmitter who observes an information source†, such as a coin toss, and wishes to communicate these observations to a remotely located receiver over a publicly exposed, noiseless, communications channel; a receiver who wishes to not only learn the state of the source (as observed by the transmitter) but also to assure himself that the communications (messages) he accepts actually were sent by the transmitter and that no alterations have been made to them subsequent to the transmitter having sent them, and two other parties, the opponent and the arbiter. The opponent wishes to deceive the receiver into accepting a message that will misinform him as to the state of the source. We assume, in accordance with Kerckhoffs' criteria in cryptography, that the opponent is fully knowledgeable of the authentication system and that in addition he is able to both eavesdrop on legitimate communications in the channel and to introduce fraudulent communications of his own choice. We also assume that he has unlimited computing power, i.e., that any computation which can be done in principal can in fact be done in practice. Given this, the opponent can achieve his objective in either of two ways:

1) he can impersonate the transmitter and send a fraudulent message when in fact no message was sent by the transmitter,

or

2) he can wait to intercept a legitimate message from the transmitter and substitute in its stead some other message of his own devising.

---

* This work performed at Sandia National Laboratories supported by the U. S. Dept. of Energy under contract no. DE-AC04-76DP00789.

† Ideally we would call the states of the source "messages" as is the practice in communications theory. However, if we did this we would be forced to introduce terminology to designate the collection of sequences that are actually transmitted through the channel, perhaps "authenticating codewords," paralleling "error detecting and correcting codewords" from communications theory. Unfortunately, the natural contraction "codeword" already has an accepted meaning in communications theory so that we would either have to coin a new term to designate the specific sequence of symbols transmitted to convey and authenticate a message -- none of which seem very natural -- or else use the cumbersome term "authenticating codeword." The term "authenticator," which is usually used to denote an authenticating suffix appended to the information that is to be authenticated, has too restricted a connotation for the general case. We have opted instead to use the term "message" to designate the sequence of symbols actually transmitted and to tolerate the rather artificial device that the information conveyed by a message is the state of a hypothetical source.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 151-165, 1988.
© Springer-Verlag Berlin Heidelberg 1988

In either case, the opponent wins if the receiver accepts the fraudulent message as being a legitimate and unmodified communication from the transmitter and ends up being misinformed as to the state of the source as a result.

In the simplest model of authentication, the transmitter and receiver are assumed to be mutually trusting and trustworthy and to act with the joint purpose of detecting attempted deceptions by the opponent. Authentication codes have been devised by Brickell [1], Simmons [2,3,4], Stinson [5,6] and others [7,8,9] that not only achieve this end, but also make perfect use of the information content of the transmitted message in the process. Unfortunately, until now this has required that the transmitter and receiver be privy to precisely the same secret (from the opponent) information about the specific authentication protocol being used and hence, that each be able to do anything the other can do, i.e., the transmitter will be able to disavow a message that he actually sent to the receiver or the receiver can fraudulently attribute a message of his own devising to the transmitter. What this means, of course, is that if the assumption of mutual trustworthiness doesn't hold, that either will be able to defraud the other in a way that cannot be verified -- or demonstrated -- to a third party. It is here that the fourth party, the arbiter, comes in. The arbiter is provided with secret (known only to him and the transmitter) information as to which messages the transmitter is supposed to use in the communications protocol and may also include information that the arbiter shares in secret with the receiver as to which messages the receiver will accept. His sole function is to certify on demand whether a particular message presented to him is one that the transmitter could have used under the established protocol. He cannot say that the transmitter did send it, only that he could have under the established protocol. In this setting, the transmitter can cheat if

3) he can cause the receiver to accept and act on a message that he (the transmitter) can later disavow. To be successful, he must not only choose a message that the receiver will accept, but also one which the arbiter will not certify, because it is not a message that would have been used by him (the transmitter) under the authentication protocol established by the arbiter.

If the transmitter succeeds in disavowing the message, the receiver will be, according to the terms of the protocol, held (unjustly) liable.

The receiver can cheat if he can successfully attribute a message of his own devising to the transmitter, i.e., a message not sent by the transmitter, but one which the arbiter will certify as being one that could have been sent by the transmitter under the established authentication protocol. There are two strategies for cheating available to the receiver, paralleling the two strategies available to the opponent:

4) He can claim to have received a message (which he fabricated) from the transmitter, when in fact no message was sent by the transmitter,

or

5) he can wait until he receives a legitimate message from the transmitter and then claim to have received some other message conveying different information than that communicated by the transmitter's message: replacing an order to buy with one to sell, for example.

In either case, if the arbiter later certifies the fraudulent message as being one that the transmitter might have sent under the established authentication protocol, the transmitter will, according to the terms of the protocol, be held (unjustly) liable.

The presence of an arbiter has no effect on the outcome of the opponent's attempted deception. If the opponent is successful in deceiving the receiver, the transmitter will, of course, appeal to the arbiter when he is later held liable (by the receiver) for a message that he (the transmitter) didn't send. The arbiter in this case, depending on whether the opponent chose a message that was not only acceptable to the receiver but which also might have been sent by the transmitter under the established authentication protocol, will assign the liability for the receiver's actions to the transmitter or otherwise to the receiver. The assignment of liability by the arbiter is unjust in either case since neither the transmitter nor the receiver cheated or failed to properly use the authentication protocol, however the arbiter can only verify whether a message is or is not consistent with the protocol in use, and not what its source might be. He can, therefore, never ascribe the liability to the opponent, even though there is always some nonzero probability that this is the source of the message.

The smallest example of an authentication code [3,4], capable of detecting attempted deceptions by an opponent, but providing no protection against deception by either the transmitter or the receiver, i.e., without arbitration, is the following: The source is a fair coin toss, by the transmitter, whose outcome denoted H or T we take to be the state of the source. There are four encoding rules, e_i, which encode states of the source into one of four possible messages, m_j, according to the scheme

(1)

	m_1	m_2	m_3	m_4
e_1	H		T	
e_2	H			T
e_3		H	T	
e_4		H		T

For the details of how authentication schemes are constructed in general see [2,3,4], but for the present discussion it suffices to note that in (1) each encoding rule uses only two out of the four possible messages and that each message appears in only two out of the four encoding rules. Consequently the probability that a randomly chosen message will be in a particular encoding rule, chosen with uniform probability, is 1/2 as is the probability of identifying the chosen encoding rule given the fact that it contains a particular message.

In the protocol for authenticating the outcome of a coin toss, the transmitter and receiver choose (in secret from the opponent) an encoding rule with the uniform probability distribution on the e_i (their optimal authentication strategy) in advance of the communication that they wish to authenticate. The opponent knows (1) and their strategy for choosing an e_i but not then choice. If he chooses to impersonate the transmitter and send an unauthentic message when no message has yet been sent by the transmitter, it should be obvious that irrespective of which message, m_j, he chooses, the probability that it will correspond to an encoding of a source state under encoding rule e_i, and hence that it will be accepted by the receiver as an authentic message, is 1/2. Similarly, if the opponent waits to observe a legitimate communication by the transmitter, his uncertainty about the encoding rule being used will drop from one out of four equally likely possibilities to one out of two. However, his probability of choosing an acceptable (to the receiver) substitute message will still be 1/2. For example, if the opponent observes m_1 he knows that the transmitter and receiver are using either encoding rule e_1 or e_2. In the first case m_3 would be an acceptable message to the receiver while m_4 would be rejected as unauthentic, while in the second case, exactly the opposite would be true. Hence, the opponent's probability of deceiving the receiver is 1/2 irrespective of whether he impersonates the transmitter or substitutes (modifies) legitimate messages.

Clearly since the transmitter and receiver must both know the chosen encoding rule -- the transmitter so that he can encode the source state into a message the receiver will accept and the receiver so that he can decode and authenticate the message -- either can do anything the other can. In particular the receiver can claim to have received a message when none was sent and the transmitter will be unable to prove to a third party that he didn't send it, or the transmitter can disavow a message that he did send and the receiver will be unable to prove that he received the message through the communications channel.

The essence of this paper is illustrated in an extension of this simple one-bit source example that in addition to one providing bit of protection against each of the two possible outsider (the opponent) deceptions, also provides one bit of protection against each of the three forms of insider (transmitter or receiver) cheating described earlier [10]. In this extended protocol, using the same source example as before, the receiver first chooses one of the 16 encoding rules defined by the Cartesian product

$$\begin{pmatrix} H & H & - & - \\ H & - & H & - \\ - & H & - & H \\ - & - & H & H \end{pmatrix} \times \begin{pmatrix} T & T & - & - \\ T & - & T & - \\ - & T & - & T \\ - & - & T & T \end{pmatrix}$$

with a uniform probability distribution. For example, the first row of the product would be

	m_1	m_2	m_3	m_4	m_5	m_6	m_7	m_8
a_1	H	H	-	-	T	T	-	-
⋮								

which says that a head outcome to the transmitter's coin toss could be communicated by the transmitter using either message m_1 or m_2. Similarly, messages m_5 or m_6 would communicate source state "tails", while messages m_3, m_4, m_7 and m_8 would be rejected by the receiver as unauthentic. The important point to note is that in each of the encoding rules there are exactly two acceptable (to the receiver) messages available for each state of the source. The receiver communicates his choice of an authenticating rule to the arbiter in secret (from the transmitter and the opponent(s)). What the receiver has done is to commit himself to accept as authentic any of the messages used in the authenticating rule he chose, and to reject as unauthentic any not used there. The arbiter next chooses one of the four vectors defined by the Cartesian product

$$\begin{bmatrix} 1 & - & - & 1 \\ - & 1 & 1 & - \end{bmatrix} \times \begin{bmatrix} 1 & - & - & 1 \\ - & 1 & 1 & - \end{bmatrix} ,$$

again, with a uniform probability distribution, and forms the Schur product† of the chosen vector with the authenticating rule selected by the receiver. The net result is, for the example of a_1 having been the authenticating rule chosen by the receiver, that one of the four possible encoding rules

H - - - T - - -
H - - - - T - -
- H - - T - - -
- H - - - T - -

will be selected (with a uniform probability distribution) as a result of the concatenated choices of the receiver and arbiter. The arbiter communicates, in secret (from the receiver and the opponent(s)), the resulting encoding rule to the transmitter. This rule is then the established protocol that the transmitter is supposed to use to encode the observed state of the source into the message that is to be transmitted to the receiver. The transmitter knows that only the messages used in the encoding rule given to him by the arbiter will be certified by the arbiter in the event of a later dispute. He also knows that there are other messages which the receiver will accept as authentic since they appear in the authenticating rule that he (the receiver) chose, but which the arbiter will not certify as authentic since they do not appear in the encoding rule he (the arbiter) selected. Assume, for example, that the arbiter chose the vector

- 1 1 - 1 - - 1

† Given vectors $A = (a_i)$ and $B = (b_i)$ the Schur product is the vector $C = (a_i b_i)$.

so that the resulting encoding rule is

$$e \left| \begin{array}{cccccccc} m_1 & m_2 & m_3 & m_4 & m_5 & m_6 & m_7 & m_8 \\ - & H & - & - & T & - & - & - \end{array} \right. .$$

In this example, source state "heads" is to be communicated by the transmitter sending message m_2 while "tails" is to be communicated by sending message m_5.

Using this authentication scheme we now show that the immunity provided to each of the five types of cheating described earlier is to hold the cheater to a probability of 1/2, i.e., one bit of protection, as claimed irrespective of which type of cheating is considered. The easiest of the deceptions to analyze is the case of the outsider (opponent) who only knows the "system," i.e., he knows what the procedures are but does not know the receiver's or arbiter's choices. It should be clear that if he attempts to impersonate the transmitter and send a message when none has been sent, his probability of choosing one of the four (out of eight) messages that the receiver has agreed to accept (in his choice of an encoding rule) is 1/2 since in each case there are four equally likely messages that will be accepted as authentic and four that will be rejected as unauthentic. On the other hand, if he waits to observe a message, say m_1, his uncertainty about the encoding rule chosen by the receiver drops from one out of sixteen equally likely candidates to one out of four, however these four leave him with four equally likely possibilities for the message that the transmitter is to use to communicate the other state of the source, and much more importantly, with four equally likely pairings of messages that the receiver would accept as communicating the other state of the source, with each message occurring in precisely two of the pairs. The net result is that the opponent's probability of success in substituting a message that the receiver will accept as communicating the other state of the source is still 1/2.

Consider next, the next simplest case to analyze, the transmitter disavowing a message that he actually sent. In order to succeed, the transmitter must choose a message that the receiver will accept but that is not used in the established protocol forwarded by the arbiter. In other words he must choose a message that was used in the encoding rule that the receiver chose, but not used in the final encoding rule generated by the arbiter's choice. Continuing with the example used above, the transmitter knows from the final encoding rule that was given to him by the arbiter:

$$- H - - T - - -$$

that the arbiter must have chosen vector

$$- 1 1 - 1 - - 1$$

and hence can infer that the receiver must have chosen one of the four encoding rules.

m_1	m_2	m_3	m_4	m_5	m_6	m_7	m_8
H	H	-	-	T	T	-	-
H	H	-	-	T	-	T	-
-	H	-	H	T	T	-	-
-	H	-	H	T	-	T	-

Since messages m_3 and m_8 do not appear in any of these rules, the transmitter can be certain that they would be rejected by the receiver as unauthentic, and hence he will not send either of these. Each of the remaining four messages, m_1, m_4, m_6, and m_7, appear in two of the equally likely choices of an encoding rule, hence he cannot do better than choose one out of these four messages with equiprobability. Irrespective of which of the four he chooses, the probability that it will be accepted by the receiver is 1/2. If it is accepted, the transmitter can disavow having sent it, since he knows that the arbiter will not certify it as a message that would have been used under the established protocol.

Finally, we consider the two types of cheating available to the receiver. Of the four messages that he has agreed (with the arbiter) that he will accept as authentic, since they are used in his choice of an authenticating rule, two will be certified as being messages that could have been used under the established protocol and two will not be certified. The receiver will succeed in fraudulently attributing a message to the transmitter if he is able to choose one of the pair that the arbiter will certify and will fail otherwise. It should be clear that his probability of success is 1/2 since the arbiter's selection procedure chooses among the acceptable (to the receiver) messages with a uniform probability distribution. If he waits until he receives a message from the transmitter, say m_2, he can reduce his uncertainty about the vector that the arbiter choose from one of four equally likely cases to one of two:

- 1 1 - 1 - - 1

or

- 1 1 - - 1 1 -

in the example. The result however is that either message m_5 or m_6 is equally likely to be the one that will be certified, and his probability of successfully substituting a message conveying a different state of the source than was communicated in the message sent by the transmitter, i.e., of substituting one which will both communicate a different state of the source and will subsequently be certified by the arbiter as a message the transmitter could have sent under the established authentication protocol is 1/2.

This small example illustrates all of the essential features of authentication codes that permit arbitration. In the resulting code three bits of information must be communicated to identify one of eight equally likely messages. According to the protocol, this communication provides one bit of information about the source state,

one bit of protection against deception by outsiders and one bit of protection against cheating by insiders. In earlier papers on authentication codes without arbitration [3,4], codes were defined to be perfect in the natural sense that all of the information transmitted was either used to communicate the state of the source or else to confound one of the cheating parties. Consequently, it seems reasonable to also describe the code illustrated here as perfect.

Our object will be to generalize the one-bit example of the preceding section to construct an infinite class of perfect authentication codes that permit arbitration. To do this, we will carry over various properties of the small example that were essential to that construction, but without a rigorous proof that all of these conditions are necessary to construct authentication codes that permit arbitration.

We start by insisting that the extended codes also be of the form of a k-fold Cartesian product of an $m \times n$ array, A, in which each row contains r entries and each column contains $t = mr/n$ entries. k is the number of states the source can assume, i.e, the total number of distinct pieces of information that the transmitter may need to communicate to the receiver. A must satisfy several other conditions which we will derive in order for it to be suitable as a basis for constructing an authentication with arbitration code whose security is easy to calculate. The first of these is a regularity condition which can be most easily described by saying that the hypergraph defined by the rows of A is symmetric†.

Given that the authentication code **A** is a Cartesian product, $\mathbf{A} = A^k$, each of the km messages is used in kt of the authenticating rules. The fact that A is symmetric is sufficient to insure that the optimal strategy for the receiver to use in selecting an authenticating rule is to choose with a uniform probability distribution on the rows of **A**, and hence because of the Cartesian product construction of **A**, to equivalently choose rows from the k factors A in the same manner. In this case, the opponent's probability of choosing a message which appears in the authenticating rule chosen by the receiver based only on his knowledge of the structure of **A**, i.e., of impersonating the transmitter, will be:

$$P_{Opp}(\text{impersonation}) = \frac{kr}{kn} = \frac{r}{n} = \frac{t}{m} \quad . \tag{1}$$

On the other hand, if he waits to observe a legitimate message he will have learned something about the collection of messages which the receiver will accept (conveying a particular source state) but because of the independence attributable to the Cartesian product construction of **A**, he can do no better than choose with a uniform probability distribution among the (k-1)r messages used by the other factors, hence;

† A hypergraph is symmetric if there exists an automorphism carrying any edge into any other edge, or equivalently any row of A can be interchanged with any other row and the resulting incidence array A' can be made identical to A by appropriate interchanges of columns and the other rows in A'.

(2) $$P_{Opp}(\text{substitution}) = \frac{(k-1)r}{(k-1)n} = \frac{r}{n} = \frac{t}{m}$$

therefore

(3) $$P_{Opp} = \frac{r}{n} = \frac{t}{m} \quad .$$

We arbitrarily specialize to the case in which the arbiter is constrained to choose, with a uniform probability distribution, only one message from among the r messages that the receiver has indicated he will accept as authentic and interpret as conveying a particular state of the source. In general, the arbiter may choose some number α, $1 \leq \alpha < k$, of the acceptable messages but the analysis is only made more difficult by this generality, and with no apparent gain in security, so we have chosen to set $\alpha = 1$ for the extended authentication codes described here. The receiver knows, of course, that the transmitter will use (according to the protocol) only the k messages appearing in the encoding rule communicated to him by the arbiter. If he can find one of these, he can falsely attribute it to the transmitter and be assured that the arbiter would later certify it to be one that the transmitter could have sent under the existing protocol, i.e., the transmitter would be held liable. Clearly, the receiver's probability of successfully attributing a fraudulent message to the transmitter will be

(4) $$P_{Rx}(\text{impersonation}) = \frac{k}{kr} = P_{Rx}(\text{substitution}) = \frac{k-1}{(k-1)r} = \frac{1}{r}$$

or

(5) $$P_{Rx} = \frac{1}{r} \quad .$$

The equality in (4) follows again from the independence of the factors in the Cartesian product construction for **A**.

Since our object is to make all forms of cheating equally improbable of success, we find that

(6) $$n = r^2$$

by setting $P_{Rx} = P_{Opp}$ in (3) and (5).

The analysis of the transmitter's probability of successfully being able to disavow a message which the receiver accepted as authentic is more difficult to analyze. To do this, the transmitter must choose one of the r-1 messages in one of the factors of the authenticating rule chosen by the receiver, and hence which he will accept as authentic, but which the arbiter will not certify since they do not appear in the encoding rule that he (the arbiter) constructed. Given any message M, it occurs in t rows of the corresponding factor A of **A** and hence in t^k authenticating rules. Because of the independence due to the Cartesian product construction of

A we can restrict attention to the factor A that uses the message M. Since each row of A contains r entries, there are t(r-1) occurences of other messages paired with M over all of the rows of A. It probably follows from the requirement that A represent a symmetric graph, however, instead of trying to prove this, we simply impose the constraint that any message that occurs with M in some row of A also must occur the same number of times, s, as any other message that occurs with M. The number of distinct messages that occur with M is therefore

$$\frac{t(r-1)}{s} \quad , \tag{7}$$

(r-1) of which occur with M in the row of A that the receiver chose in constructing the encoding rule he communicated to the arbiter. Therefore, the probability that the transmitter chooses (at random) a message the receiver will accept as authentic but which the arbiter will not certify is;

$$P_{Tx} = \frac{\frac{r-1}{t(r-1)}}{s} = \frac{s}{t} \quad . \tag{8}$$

Setting $P_{Tx} = P_{Rx}$, we get

$$\frac{s}{t} = \frac{1}{r} \quad \text{or} \quad s = \frac{t}{r} = \frac{m}{n} = \frac{m}{r^2} \tag{9}$$

Figure 1 should help make clear the canonical structure of the array A that has been forced by the conditions imposed thus far.

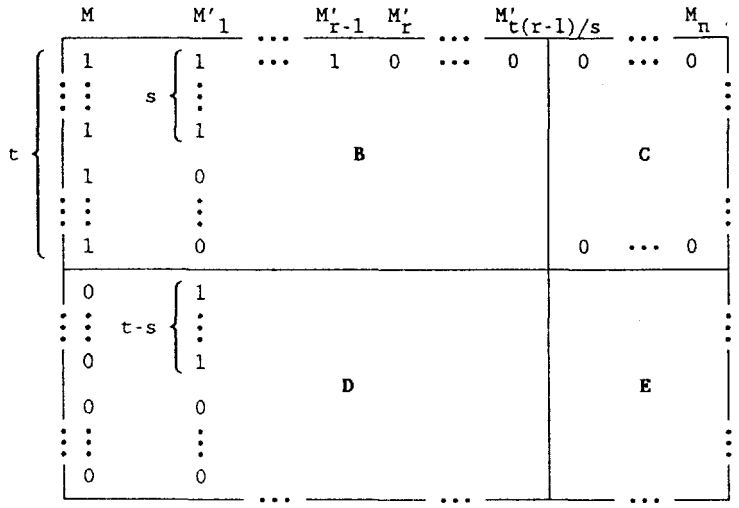


Figure 1.

The subarray C is completely filled with 0's since all of the tr 1's in the first t rows of A are accounted for in subarray B. The transmitter, knowing the structure of A and the legitimate message M cannot do better than choose one of the t(r-1)/s M' messages with a uniform probability distribution, since each occurs paired s times with M. His probability of success using this strategy would be

$$P_{Tx} = \frac{r-1}{\frac{t(r-1)}{s}} = \frac{s}{t}$$

as already noted. He would not, of course, choose any of the n-t(r-1)/s messages that do not occur paired with M since there is no chance that such a message would either be accepted by the receiver nor certified by the arbiter. In the small example we had the array A (in canonical form):

M	M'_1	M'_2	
1	1		
1		1	
	1		1
		1	1

where t = 2, s = 1 and r = 2. From (6) we know that n is a square, therefore the next smallest array must have r = 3 and n = 9. One such array is the following

	1	2	3	4	5	6	7	8	9
1	1	1	1						
2	1			1	1				
3	1					1	1		
4		1		1					1
5		1				1		1	
6			1		1			1	
7			1				1		1
8				1			1	1	
9					1	1			1

Figure 2.

where t = 3, s = 1 and r = 3.

The reader will have probably already recognized that the two incidence arrays exhibited for r = 2 and 3 and s = 1 are examples of what Bruck [12,13] has defined to be a finite net. A finite net (i-net), **N**, is a set of $n = r^2$ elements (points) and ir r-subsets of points (lines) which can be grouped into i parallel classes of r lines each, such that distinct lines of the same parallel class have no points in common, while any two lines from different classes have exactly one point in common. Finite nets are closely related to finite affine planes. Under very general conditions, a finite net can be extended (but not always) to be embedded in an affine

plane, however, every subset of i spreads of parallel lines from an affine plane is necessarily an i-net. It is this result that we will use to construct general authentication with arbitration codes. The parallel classes in the case r = 3 shown above are the sets of lines (row headings); (1,8,9)(2,5,7)(3,4,6), or in a more familiar form the sets of triples of messages (column headings)

$$\begin{array}{ll} (123)(478)(569) & \pi_1 \\ (145)(268)(379) & \pi_2 \\ (167)(249)(358) & \pi_3 \end{array}$$

The general means of constructing perfect authentication codes that permit arbitration is now obvious. We will restrict s = 1 which, although not necessary, makes the constructions easier. Start with any finite affine plane with $n = r^2$ points (whose existence is assured for all $r = p^\alpha$, p a prime) and form an i-net by choosing i parallel spreads of lines. Identify lines of the i-net with authenticating rules to form the factor array A. For example, there are three non-isomorphic A arrays possible when r = 2, corresponding to choosing one, two or all three of the parallel spreads from

$$\begin{array}{ll} (12)(34) & \pi_1 \\ (13)(24) & \pi_2 \\ (14)(23) & \pi_3 \end{array}$$

i.e., A in this case is one of the three forms

$$\left(\begin{array}{cc|cc} 1 & 1 & & \\ \hline & & 1 & 1 \end{array}\right), \quad \left(\begin{array}{cc|cc} 1 & 1 & & \\ 1 & & 1 & \\ \hline & 1 & & 1 \\ & & 1 & 1 \end{array}\right) \quad \text{or} \quad \left(\begin{array}{cc|cc} 1 & 1 & & \\ 1 & & & 1 \\ 1 & & & 1 \\ \hline & 1 & 1 & \\ & 1 & & 1 \\ & & 1 & 1 \end{array}\right).$$

All of these arrays are in the standard form of Figure 1. Clearly, by arguments given earlier

$$P_{Opp} = P_{Rx} = \frac{1}{r} \quad . \tag{10}$$

However P_{Tx} is dependent on i. For i = 1, given any acceptable message the transmitter is certain of the other message the receiver will accept, i.e., once he is informed which message he is to use in the encoding rule constructed by the arbiter he knows the other acceptable message, since there is only one authenticating rule containing any given message. Similarly, he knows that one of two possible messages must be the other acceptable message when i = 2, however they occur uniquely in two equally likely authenticating rules, so that his probability of guessing which one

the receiver will accept is only 1/2. In the general case for A an i-net, the transmitter's probability of success will be

$$P_{Tx} = \frac{(r-1)}{i(r-1)} = \frac{1}{i} \tag{11}$$

Since we wish to force

$$P_{Opp} = P_{Rx} = P_{Tx} \tag{12}$$

and by (10)

$$P_{Opp} = P_{Rx} = \frac{1}{r} \quad ,$$

$i = r$. An affine plane contains $r + 1$ parallel spreads, i.e., $i \leq r + 1$, so that we can construct the desired factor A by deleting an arbitrary spread from an affine plane.

Conclusion

To summarize, we have described a construction for an infinite family of authentication codes that permit arbitration in which

$$P_{Opp} = P_{Rx} = P_{Tx} = \frac{1}{r} \tag{13}$$

whose existence is assured for r a prime power. These codes are of the form

$$\mathbf{A} = A^k$$

where k is the number of distinct pieces of information that may need to be authenticated and A is an $r^2 \times r^2$ array whose rows are identified with the lines in an r-net derived from the affine plane EG(2,r) by deleting an arbitrary parallel spread of lines.

The procedure for authentication is that the receiver will choose an authenticating rule, **a**, (row of **A**) with a uniform probability distribution and communicate this choice (in secret from the opponent and the transmitter) to the arbiter who will choose (also with a uniform probability distribution) one out of the r messages in each block of the authenticating rule to form the encoding rule, **e**, which he communicates (in secret from the opponent and the receiver) to the transmitter. The authentication protocol is that the receiver will accept as authentic only messages appearing in **a**, the transmitter is supposed to use only messages appearing in **e**, but

in any event, the arbiter will certify only those messages appearing in e. All such codes satisfy (13) as was desired:

$$P_{Opp} = P_{Rx} = P_{Tx} = \frac{1}{r} = \frac{1}{p^{\alpha}} \tag{14}$$

p a prime, $\alpha \geq 1$. To communicate $\log_2 k$ bits of information and provide security against all five forms of deception of $\frac{1}{p^{\alpha}}$ requires that

$$\log_2 k + 2\log_2 r$$

bits of information be communicated through the channel, i.e., just enough information to identify which one of the kr^2 equally likely messages the transmitter is using.

References

1. E. F. Brickell, "A Few Results in Message Authentication," Proceedings of the 15th Southeastern Conference on Combinatorics, Graph Theory and Computing, Baton Rouge, LA, March 5-8, 1984, Congressus Numerantium, Vol. 43, Dec. 1984, pp. 141-154.

2. G. J. Simmons, "A Game Theory Model of Digital Message Authentication," Proceedings of the 11th Annual Conference on Numerical Mathematics and Computing, Univ. of Manitoba, Winnipeg, Canada, Oct. 1-3, 1981, Congressus Numerantium, Vol. 34, June 1982, pp. 413-424.

3. G. J. Simmons, "Message Authentication: A Game on Hypergraphs," Proceedings of the 15th Southeastern Conference on Combinatorics, Graph Theory and Computing, Baton Rouge, LA, Mar. 5-8, 1984, Congressus Numerantium, Vol. 45, December 1984, pp. 161-192.

4. G. J. Simmons, "Authentication Theory/Coding Theory," Proceedings of Crypto'84, Santa Barbara, CA, August 19-22, 1984, in Advances in Cryptology, Ed. by R. Blakley, Springer-Verlag, Berlin (1984), pp. 411-431.

5. D. R. Stinson, "Some Constructions and Bounds for Authentication Codes," presented at Crypto'86, Santa Barbara, CA, Aug. 12-15, 1986, to appear in Journal of Cryptology, 1988.

6. D. R. Stinson, "A Construction for AuthenticationSecrecy Codes from Certain Combinatorial Designs," presented at Crypto'87, Santa Barbara, CA, Aug. 16-20, 1987, to appear in Journal of Cryptology, 1988.

7. E. N. Gilbert, F. J. MacWilliams, N.J.A. Sloane, "Codes which Detect Deception," The Bell System Tech. Journal, Vol. 53, No. 3, March 1974, pp. 405-424.

8. J. L. Massey, "Cryptography -- A Selective Survey," presented at Int'l. Tirrenia Workshop on Digital Communications, Tirrenia, Italy, Sept. 2-6, 1985. Alta Frequenza, Vol. LV #1, Jan.-Feb., 1986, pp. 4-11.

9. P. Schoebi, "Perfect Authentication Systems for Data Sources with Arbitrary Statistics," presented at Eurocrypt'86, Linköping, Sweden, May 20-22, 1986.

10. G. J. Simmons, "Authentication Codes that Permit Arbitration," to appear Proceedings of the 18th Southeastern Conference on Combinatorics, Graph Theory and Computing, Boca Raton, FL, Feb. 23-27, 1987.

11. D. Raghavarao, Constructions and Combinatorial Problems in Design of Experiments, John Wiley & Sons, New York, NY (1971).

12. R. H. Brock, "Finite Nets I: Numerical Invariants," Canadian Journal of Math., Vol. 3 (1951), pp. 94-107.

13. R. H. Brock, "Finite Nets II: Uniqueness and Embedding," Pacific Journal of Math., Vol. 13 (1963), pp. 421-457.

PERFECT AND ESSENTIALLY PERFECT AUTHENTICATION SCHEMES

Extended Abstract

Albrecht Beutelspacher

Siemens AG
ZT ZTI SYS 4,
D-8000 München 83
Federal Republic of Germany

Suppose that *A* wants to send a message M to *B*. It is important that *B* receives the message without any alteration. On the other hand, a bad guy **X** looks for his chance to alter M in his favour. In order to make the bad guy's life difficult, *A authenticates* the message M.

For this, **A** and **B** have to agree on an authentication function f and a secret key K. The function f has M and K as its input, and the *authenticator* (also called *message authentication code*) f(M,K) as its output.

Now the procedure is as follows. *A* sends the message M along with the authenticator A = f(M,K). *B* receives a message, say M'. Then *B* computes A' = f(M',K); only if A' = A, *B* accepts the received message as it stands.

What can a bad guy do? He wants to delete M and to insert another message M*. Since he does not know the secret key K, he has no method to forge M, he can only try. But the bad guy's chances are not as bad as it may seem. Gilbert, MacWilliams and Sloane [2] have proved the following

Result. Assume that all messages and all keys occur with the same probability. Denote by k *the total number of keys. Then, in any authentication system, the bad guy's chance of success is at least* $1/\sqrt{k}$.

An authentication system in which the bad guy's chance is exactly $1/\sqrt{k}$ is called *perfect*. Gilbert, MacWilliams and Sloane [2] have constructed perfect authentication systems using projective planes (see below). These examples lack on the fact that there are very few messages (compared with the number of keys). The aim of this paper is to present many authentication systems, in particular those with 'many' messages. Some of our schemes are not perfect in a strong sense, but *essentially perfect*. By this we mean that the bad guy's chance of success is only $O(1/\sqrt{k})$. Our constructions are based on geometric structures, in particular finite projectivce spaces. Definitions and results can be found for instance in in [1].

Construction 1. Let **P** be a d-dimensional finite projective space of order n and fix a hyperplane **H** of **P**. We define the authentication system **A** = **A**(t,d) as follows:

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 167-170, 1988.
© Springer-Verlag Berlin Heidelberg 1988

The *messages* are the t-dimensional subspaces of **H** ($t \leqq d-1$),

the *keys* are the points of **P–H**,

the *authenticator* belonging to message M and key K is the $(t+1)$-dimensional subspace $<M,K>$.

Theorem 1. The authentication system **A**(t,d) *is essentially perfect if and only if* $d = 2t+2$; *it is perfect if and only* if $d = 2, t = 0$.

Proof. In any case, the number of keys is $k = n^d$. Assume that the bad guy wants to forge an authenticated message. For this, we may assume that he has a valid authenticator A (which is a $(t+1)$-dimensional subspace of **P–H**), which intersects **H** in the message M. He wants to substitute M by the message M*, which is also a t-dimensional subspace of **H**. Since almost all t-dimensional subspaces of **H** are skew to M, we may assume for the moment that M and M* are disjoint.

In the worst case, the bad guy is clever. He observes that he has not to check all keys, but only those which are points of A–**H**. Since there are only n^{t+1} such points, his chance of success is at least $1/(n^{t+1})$.

If our system is essentially perfect, we have therefore

$$O(1(\sqrt{n^d})) = O(1/n^{t+1})),$$

that is $d = 2t+2$.

Suppose now $d = 2t+2 > 2$. Then there are messages $M^* \neq M$ which intersect M in a subspace W of dimension $i \geqq 0$. Then any hypothetical (but reasonable) authenticator A* of M* intersects A in a subspace of dimension $i+1$, the bad guy's chance of success is

$$1/(n^{t-i}) > 1/(n^{t+1}).$$

So, the system is essentially perfect, not perfect in the strong sense.□

Remark. The systems **A**(0,2) are exactly the systems constructed by Gilbert, MacWilliams and Sloane [2].

Another construction, which also yields perfect authentication systems is as follows.

Construction 2. Denote by $P = PG(s+t+1,q)$ the projective space of dimension $s+t+1$ ($s,t \geqq 0$) and order q. Fix an s-dimensional subspace U of **P**. Define the system **A**(s,t) as follows:

The *messages* are the points on U,

the *keys* are the t-dimensional subspaces of **P** which are skew to U,

the *authenticator* of the message M corresponding to the key W is the subspace $<M,W>$ of dimension $t+1$.

Theorem 2. *Let be the above defined authentication system.*

(a) *The probability of forging is* $1/(q^{t+1})$.

(b) *The system is perfect if and only* if $s = 1$.

Proof. (a) Let A be an authenticator, that is a $(t+1)$-*dimensional* subspace which intersects U in just one point M. The assertion follows since there are exactly q^{t+1} hyperplanes of A which do not contain M, that is are skew to U.

(b) It is sufficient to prove the following assertion: The number a_t of t-dimensional subspaces disjoint to U equals

$$a_t = q^{(s+1)(t+1)}.$$

This formula is not difficult to prove.□

Construction 3. Denote by $\mathbf{P} = PG(d,q)$ the finite projective space of dimension d and order q. A *partial* t-*spread* of $\mathbf{P}$ is a set S of mutually skew t-dimensional subspaces of P. A t-*spread* of $\mathbf{P}$ is a partial t-spread S with the property that every point of $\mathbf{P}$ lies on (precisely) one element of S. It is well known that $\mathbf{P}$ has a t-spread if and only if $t+1$ divides $d+1$. Any t-spread in $PG(2t+1,q)$ has $q^{t+1}+1$ elements; a partial t-spread S of $PG(2t+1,q)$ has *deficiency* $\delta = q^{t+1}+1-|S|$. The set of points of $\mathbf{P}$ not covered by the partial t-spread S is denoted by $D(S)$.

Theorem 3. *Let* S *be a partial* t-*spread of* $\mathbf{P} = PG(2t+1,q)$. *Define the authentication system* $\mathbf{A} = \mathbf{A}(S)$ *as follows:*

The *messages* are the elements of S;

the *keys* are the points in $D(S)$;

the *authenticator* for the message M under the key K is the $(t+1)$-dimensional subspace $<M,K>$.

Claim: This authentication system is essentially perfect if and only if the definciency δ *of* S *equals* $\delta = q^t + \ldots + q + 1$. *In this case, the number of messages is* $q^{t+1} - q(q^{t-1} + \ldots + 1)$ *and the total number of keys is* $(q^t + \ldots + 1)^2$.

Proof. The number k of keys equals $k = \delta(q^t + \ldots + 1)$. On the other hand, any $(t+1)$-dimensional subspace through an element of S has exactly points in common with D(S). So, the bad guy can forge a message with probability $1/\delta$.□

Remark. The case $t = 1$ is of particular interest. In the perfect case, we have q^2-q messages, but only $(q+1)^2$ keys. One example of such a system is obtained if a regulus is removed from a "regular spread" (alias an "elliptic congruence").

Construction 4. Here, we would like to address the problem of the lucky bad guy. So far we considered our systems under the unspoken hypothesis that the same key was used only once. In other words, we assumed that a change of keys takes place after every message. Now we would like to discuss a more realistic situation in which there are several messages authenticated with the same key. Is there any security, if the bad gay knows two or more valid authenticators belonging to the same key? For most of the above discussed authentications schemes the answer is "no". For instance, in the fundamental example [2] two different authenticators determine the key uniquely.

Let us consider authentication systems in which all messages have the same number n of authenticators. Then the bad guy's chance of success is at least $1/n$, since for his favourite message he simply has to choose one of the n authenticators at random.

Such an authentication system A is said to be *s-fold secure*, if

– knowledge of any s authenticators belonging to the same key gives the bad guy only a chance of sucess of $1/n$;

– knowledge of some $s+1$ authenticators gives the bad guy a considerably better chance.

We conclude by presenting the following s-fold secure authentication system, which generalizes the (dual version) of the example constructed in [2].

Let $\mathbf{P} = PG(s+1,n)$ be the $(s+1)$-dimensional projective space of order n. Fix a point P_0 of $\mathbf{P}$.

Messages are the $n^s + \ldots + n + 1$ lines of $\mathbf{P}$ through P_0,

keys are the n^{s+1} hyperplanes of $\mathbf{P}$ not through P_0,

the *authenticator* belonging to the message ℓ and the key H is the point $\ell \cap H$. In other words, the authenticators are precisely the points $\neq P_0$ of $\mathbf{P}$.

We claim that this system is s-fold secure. In fact, even if the bad guy knows s authenticators, i.e. s points $\neq P_0$, then through these points there are at least n hyperplanes which do not pass through P_0. So his chance to guess the correct key is still not better than $1/n$. On the other hand, $s+1$ points in general position uniquely determine a hyperplane.□

References

[1] Dembowski, P: Finite Geometries. Springer-Verlag, 1968.

[2] E.N. Gilbert, F.J. MacWilliams, N.J.A. Sloane: Codes which detect deception. Bell. Syst. Tech. J. **53** (1974), 405-424.

MESSAGE AUTHENTICATION AND DYNAMIC PASSWORDS

Professor H J Beker & Mr G.M Cole
Racal-Guardata Limited, UK

The security of transactions flowing across a communications network is of ever increasing importance. In many such circumstances it is important not only to protect the messages from passive interception but also, and often of greater importance, to be able to detect any active attack against messages. An active attack may take the form of an interceptor tampering with the message: altering it, adding information, removing information and so on. While it is almost impossible to prevent an active attack there are many mechanisms to ensure, with a high probability, that such an attack may be detected and hence rendered harmless. The techniques to allow detection and thus audit take many forms of which the most common are normally cryptographically based and depend upon the generation, before transmission of the message, of a check-sum which is then appended to the message. The theory underlying this approach works on the basis that if a would-be fraudster changes any part of the message in any way then the check-sum will no longer be correct and thus the recipient of such message can compute, for himself, the expected check-sum, compare it with that received in the message and if they disagree will know the message has been altered. If on the other hand the expected and received check-sums agree then he knows with a high probability that the message has not been altered. This probability is dependent upon the amount of information within the check-sum (i.e. the longer it is) the lower the probability of an undetected alteration.

Many such systems exist. Some of these depend only upon an algorithmic check-sum, often called a test-key or authentication parameter. In this case the security level is often relatively low since someone attacking the system with knowledge of this algorithm may be aware of ways in which he can alter the message without affecting the check-sum computation. A trivial example of this is as follows: suppose the check-sum on a numeric message is computed solely

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 171-175, 1988.
© Springer-Verlag Berlin Heidelberg 1988

as the modulo-10 sum of all digits in the message. An attack upon the system which simply involves altering the order of the digits in the message would not be detected by the check-sum.

A normally more secure technique involves the use of a cryptographic check-sum, often termed a message authentication code. In this case the check-sum is dependent not only upon the cryptographic algorithm, but also a cryptographic key. An example of this, in common usage, is the system described within the American National Standards Institute (ANSI) standards X9.9 and X9.19. Within these standards the cryptographic algorithm is the Data Encryption Algorithm as described in FIPS 45 and ANSI X3.92. The cryptographic key is a 56-bit DEA key. The check-sum or message authentication code (MAC) is a 32-bit value appended to the message. It is currently generally accepted that provided the cryptographic key is kept secret then any alteration to the message will be detected by the recipient with a probability of 0.9999999998.

Within some communications systems protection of messages in the above manner is considered adequate. However, there do also exist many systems within which it is important not only to detect any alterations to the message, and thus be able to provide alarms and an audit system of these, but also to identify the person or group of persons from which such a message originated. This is in some sense equivalent to requiring a signature on the message. We shall now go on to describe how, by use of another commonly used technique of dynamic passwords, such messages can be signed and thus far greater protection afforded. We begin by describing the technique of dynamic passwords as it is commonly used for access control. We shall then go on to show how the technique can be combined with a check-sum to provide a 'signature'. As we shall see the combination of the two techniques, in the manner described, will provide far greater levels of security.

Computer access control systems often depend upon a static user password. These systems are notorious for their insecurity. Recently, dynamic password systems have become more popular. There are many variations on this particular theme. By way of example we describe one such method.

Having entered his user identity, the user is presented, by the system, with a challenge. The user must then provide the correct response to this challenge in order to be granted access. The theory behind this system is that since the system has control of the challenge and the response will be unique, for that user, to that challenge, the system is running, essentially, a one-time password system. Any unauthorised person will not know how to respond to the challenge in the correct way and will thus be denied access to the system. Similarly anyone recording the challenge and response will be unable to directly use this information since ideally that challenge will never be used for that user again.

There are many techniques possible to allow the user to produce the correct response. These vary from biometric techniques to user tokens. A typical method involves a user token similar to a small calculator which can be correctly activated by the user via entry of a Personal Identification Number (PIN). Once the device has been correctly activated entry of the challenge will result in the correct response being generated by the token. This may be achieved, for instance, via a one-way function of the challenge and a cryptographic key unique to that user and embedded in his token. Thus, loss of the token does not enable an unauthorised user to enter the system since he requires the PIN to correctly activate the system. Indeed, a would-be hacker requires both the PIN and token as well as the user ID or the algorithm and cryptographic key corresponding to a user ID in order to enter the system. In the case of a token being used in this way it may well take the form of a 'smart card'. Biometric means may also be used.

We shall now give an example of how a check-sum can be combined with a dynamic password system in order to provide message and user authentication within a system.

For example we shall consider a user, issued with a dynamic password token, using a terminal which can provide a cryptographic check-sum or message authentication code (MAC). We shall also assume that the recipient of the message is in possession of the appropriate cryptographic keys to check both the MAC and 'response'.

Once the user has compiled his message the terminal will generate

an appropriate MAC, or some derivative of it which is presented to the user as his 'challenge'. Once he has produced the correct response to that challenge and appended this response or a function of it to the message the message has been 'signed' by the user. On receiving the message the recipient can not only check the MAC, but may also, via the user ID, check the response to that 'MAC challenge' thus also authenticating the originator of the message.

Such a system may have considerable benefits within a scenario within which a corporation or institution is allowing users to enter messages into its computer network. Typically this might be a corporate banking network where the institution is a bank and is accepting payments, transfers, etc from its customers. This system may be set up as follows:

The institution issues to the user the cryptographic MAC facility in a tamper resistant form. This may constitute the entire terminal or a part of it. The cryptographic key upon which MAC security depends is contained within the tamper resistant enclosure. The corresponding cryptographic key may be held by the institution itself encrypted under a master key which again is contained in a highly tamper resistant enclosure. Similarly, the user is also issued with his dynamic password token itself containing a cryptographic key in a tamper resistant manner while again held by the institution encrypted under a master key. Assuming the tamper resistant enclosure containing the master key can also carry out the appropriate cryptographic functions then the institution can only be compromised while the devices are being set up or through a breach of the tamper resistant module containing the master key.

At the user level, the system can only be compromised via an attack upon both the user's cryptographic facility and his token. Bearing in mind that should he lose his token it will normally be in his interests to report this as soon as possible the system provides a high level of security.

Since this procedure is centred around the concept of using the MAC (or check-sum) as the 'challenge' to the user let us see what extra security benefits are thus achieved.

1. Since the response now depends on the MAC it depends upon those sensitive parts of the message which the MAC was itself protecting and thus is a message dependent response. It is in this way that it provides a similar facility to a signature. In particular this response cannot be removed from this message and appended to another since it will no longer be appropriate and will therefore be detected by the recipient.

2. Even if some unauthorised person were able to discover the cryptographic key associated with the MAC, by breaking into the user's terminal or otherwise, this would not be sufficient to penetrate the system since any alteration of the MAC in turn would mean the response on the message would now be inappropriate and would therefore be detected by the recipient.

3. An implication of the above remark is that theoretically the institution could give all its users the same cryptographic key for the cryptographic MAC facility and still be assured a high level of security through the response confirmation.

4. Clearly if the user's identity was incorporated into the message and the MAC calculation, then only the holder of that corresponding dynamic password token (and corresponding PIN, if used) could 'sign' the message.

We therefore see that the system now has two interrelated security mechanisms: the MAC and the response. As we stated above an attack by an unauthorised user would need to be directed either at the institution's highly tamper resistant facility or at both user's cryptographic facility and that user's token. We believe such an attack to be extremely difficult.

IC-CARDS IN HIGH-SECURITY APPLICATIONS

I. Schaumüller-Bichl
VOEST-ALPINE AG
P.O. Box 2
A-4031 Linz

IC-cards, which are credit-card-size plastic cards with integrated CPU and memory, have increasingly attracted public interest in recent years.

Mainly used as "electronic money" in the business of banking and as a storage medium at first, the IC-card is gaining more and more importance as a secure and user-optimised component for cryptographic systems.

The following article analyses IC-cards with regard to their own security and their applications in the field of "EDP security".

The paper is concluded with a glance at the requirements to be met by future card generations and on possible developments.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 177-199, 1988.
© Springer-Verlag Berlin Heidelberg 1988

Contents

I) IC-cards

II) Security demands on the card, security analysis

III) A new card concept and its applications

IV) Future requirements

I) IC-CARDS

IC-cards are plastic cards of the dimensions of conventional credit cards (85.6X54X0.76mm). One or several ICs as well as a system interface are implanted in the plastic card.

Different card types

Depending on the number and design of the implanted chips, the cards are classified according to various criteria:

Number of chips

- "Single-chip cards"
  containing exactly one chip

- "Multi-chip cards"
  containing two or more chips which are connected with each other within the card

Types of chips

- "Passive cards"
  The chips implanted in these cards are merely storage modules. Therefore, the cards are frequently referred to as "memory cards".

- "Active cards"
  containing a CPU in addition to the memory, which
  . secures the access to the data in the memory, and
  . can execute special functions.

Thus, cards with an implanted CPU are often designated as "intelligent cards".

Memory technology

- Erasable cards
  based on EEPROM technology

- Non-erasable cards
  generally based on EPROM technology

For applications in the fields of "electronic money" and "cryptographic systems", mainly active single-chip cards are used for safety reasons. They are often briefly called IC-cards.

System interface

The interface to the IC-card is determined by the ISO Draft International Standard DIS 7816/2 "Identification cards - Integrated circuit(s) cards with contacts - Part 2: Dimension and location of contacts".

This standard defines 8 contacts (C1 to C8), which are located on the left card side, either in the centre or in the upper edge.

Pin assignment:

C1: VCC, circuit supply voltage
C2: RST, reset signal
C3: CLK, clock signal
C4: RFU, reserved for future use
C5: GND, zero voltage
C6: VPP, programming voltage
C7: I/O, Data Input/Output
C8: RFU, reserved for future use

The exact location and arrangement of the contacts is specified in ISO 7816/2.

II) SECURITY DEMANDS ON THE CARD, SECURITY ANALYSIS

Unlike many other high-security systems, which are often developed for a special problem and which are used by trained specialists in relatively small numbers, the IC-card is intended for large-scale use in a broad range of applications.

Currently, the range of applications chiefly comprises the following fields:

- Electronic money
  (ATM, POS, telephone cards, credit cards, ...)

- Security
  (Personal identification, access control, cryptographic carrier medium in cryptographic systems, ...).
  These interrelated topics are dealt with in detail in chapter III.

- Portable personal files
  (Medical files, study records, ...)

- Take-over of routine functions
  (Start-up of devices, service cards, inventory control, ...)

Thus, the card applications are numerous and manifold, and so are the demands on the cards as far as security, ease of use and flexibility are concerned.

The "ideal" IC-card must meet a number of high requirements:

- Resistance, physical properties:

IC-cards are designed for frequent use. A typical user will carry his IC-cards with him like his credit cards or his bunch of keys. Special protective measures cannot be taken. For this reason, the cards must show a relatively high stability and high resistance to bending, torsion, heat, radiation, electromagnetic fields, chemicals, etc. These requirements are fully specified in the ISO Draft International Standard 7816/1 "Identification Cards - Integrated Circuit(s) Card with Contacts - Part 1: Physical Characteristics".

- Flexibility:

Especially in the commercial or private sector, it cannot be reasonably expected of the user to handle each system based on cards, e.g. ATM, credit card, access to building and rooms, etc., differently.
The acceptance of the system will largely depend on the successful development of a card concept that is flexible enough to be used in a great variety of applications, in spite of differing security requirements.

- Ease of use:

In former times, security problems were basically confined to the military and diplomatic service, where specialists carried out the necessary security operations.
Today, the situation is completely different. Due to the common use of computers, networks and telecommunication media, the protection of data is increasingly becoming a real concern to everyone.

Thus, the demands on the protective systems change. Since a special training in this field cannot be required of a user in the commercial or private sector, the system must be provided with a clear interface that is easy to handle. IC-cards are excellently suited for this purpose.

- Security:

Naturally, paramount importance is attached to the security requirements to be met by the cards.
A proper card concept must be suitable for various applications. Therefore, it must be also protected against the entire scope of possible attacks as well as a great variety of potential attackers.

Security analysis

The following considerations prove that the group of possible "attackers" of the system as well as all potential attacks are hardly limited:

A) Potential attackers

Basically, it has to be assumed that every individual person as well as every institution may be considered a potential "attacker". Even trustworthy institutions run the risk of employing personnel who misuse the special knowledge available for their own purposes. So, even if the employees have been selected extremely carefully, there is always the danger of an "attack from inside".

According to their knowledge about the card, the potential attackers can be subdivided into 4 main groups:

a) Manufacturers

This group comprises e.g. chip manufacturers and card producers (or their staff members respectively), which might carry out manipulations in the production sequence

b) Card issuers

Companies or organisations, which issue cards for their customers or employees (e.g. banks, credit card organisations, ...)

c) Authorised card users

d) Unauthorised third parties,

which find or steal cards or try to forge cards.

In order to reduce the risks, during the life cycle of a card, i.e. chip production, card manufacture, issuing of the cards, use, taking out of service, it should be seen to it in general that means of production and information on individual cards may be made available only to persons who need them by all means.

B) Potential attacks/protection requirements

IC-cards are exposed to the entire range of possible cryptoanalytic attacks.

The most important protective mechanisms with which the cards have to be provided if they are to be used in a broad range of applications, are as follows:

a) Protection against unauthorised reading

This corresponds to the "classic" data protection problem. Since usually confidential data (cryptographic keys, passwords, personal information, ...) are stored on the IC-card, it has to be ensured that these are read by authorised persons only.

In principle, there are two possibilities of protection:

i) via a logical or physical "barrier", which permits access to the data only if certain criteria are fulfilled, such as biometric characteristics like finger-prints or voice identification, or the input of "personal identification numbers" (PINs).

ii) Enciphered storage of the data to be protected: The data are enciphered on the card under a key that is known to the authorised user only. As compared to the method described above, this one offers the advantage that a "circumvention" of the barrier or "direct reading out" is made impossible - or actually senseless - by mechanical devices (e.g. electron microscope).

b) Protection against unauthorised modification of data

Not only confidential but also non-confidential data have to be frequently protected against unauthorised modification. "States of accounts", for instance, especially with minor amounts paid in advance, like with telephone cards, etc., need not necessarily be kept secret, but must be protected against unauthorised modification in any case. In this context, it is noticeable that such a modification, i.e. an "increase" of the current state of account, in special cases, may well be in the interest of the legitimate card holder, and thus the card - unlike most of all the other high-security systems - has to be protected even against manipulations by the legitimate user.

Basically, this problem can be solved in the following ways:

i) a logical or physical "barrier" analogous to Section a) i)

ii) Calculation of a "message authentication code" (MAC)
From the data to be protected, a "test sum" is calculated by applying cryptographic methods; this test sum indicates unauthorised, subsequent manipulation of the data.
Such a method has been standardised in the USA under the designation Ansi X9.9.

iii) Encryption of data
Analogous to Section a) ii)

In addition, the VOEST-ALPINE card concept provides two further security functions:

- PIN check
  Even though the PIN is not stored in the card, it can be checked for correctness upon request.

- Block locking
  Each block, and thus each application, can be locked after a certain number of wrong PIN inputs. The locking of a block has no effect on the operativeness of the other blocks on the card.

  These considerations result in the following card concept:

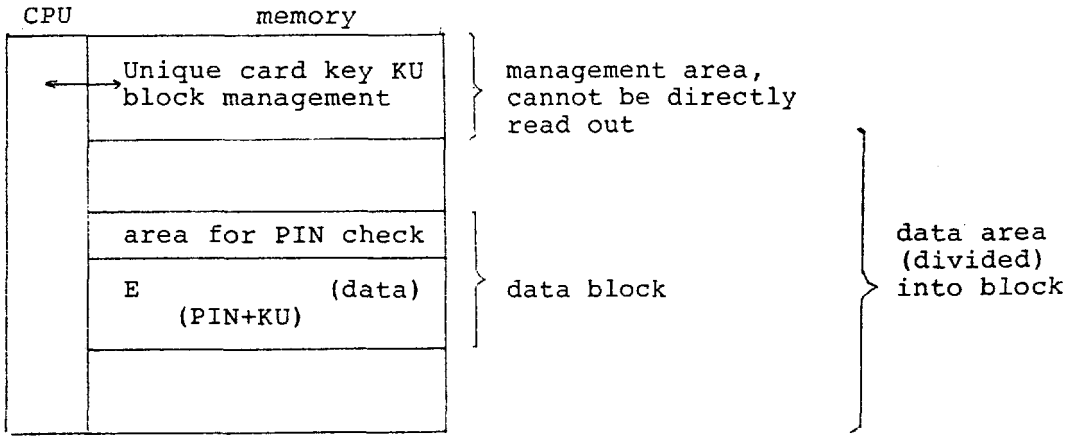


Fig.1: The VOEST-ALPINE IC-card concept

c) Protection against unauthorised copying of cards

In almost all IC-card applications, unauthorised copying of cards is a special security hazard. In high-security applications like the access control system, copying of cards corresponds to making of skeleton keys, and in "electronic money" applications to printing of counterfeit money.

In both cases, the possible attacker need not know the actual contents of the card, i.e. the meaning of the data; a bit-by-bit copying of the data onto another card would suffice.

In order to effectively prevent such an attack, every card must be provided with a unique key that cannot be modified or copied but checked.

Today, this key is usually realised in the form of a random number that is automatically generated for every card, stored in the card and protected by the microprocessor of the card. In the following, this key is called KU ("Unique Card key").

KU can be checked explicitly or implicitly.

i) Direct check
For direct checking, the KU would have to be input directly and compared with the stored value in the card, which involves considerable security hazards.

ii) Indirect check
For indirect checking, a (pseudo) random number is transmitted to the card to be checked for authenticity. By means of a

special function, the card calculates a value that depends on the random number (PRN) as well as on the Unique Card Key.
R: = f (PRN, KU)
The result R serves for checking of the correctness of the card.

In some cases, this method may entail difficulties. In order to be able to check R for correctness, and thus, the card for authenticity, either

- the secret card key KU must be known outside the card,

- a number of reference values must be stored, or

- a suitable "check card" must exist for every card, which contains the same KU.

Each of these 3 solutions involves a considerably great expenditure for the "key management", which is necessary to ensure a minimum of security, and which may cause great problems in the large-scale application of the cards.

iii) Implicit check

For implicit checking, a connection between the data stored on the card and the Unique Card Key is established. This is achieved by applying special cryptographic methods, for instance. Based on the concept described in a) ii), i.e. enciphered storage of data on the card, these data are enciphered under a key

which results from a combination of PIN and Unique Card Key.

The card can be copied only if the PIN is known; even data that can be read out by means of an electron microscope cannot be appropriately copied onto another card. In case several groups (e.g. bank/customer) are interested in the protection of data, the PIN proper must consist of the corresponding partial PINs.

d) Protection against simulation of the card

An attacker may - sometimes without major technical and organisational expenditure - intercept the connection between the IC-card and the master (card reader, PC or host), and thus store the request data and the corresponding responses of the card. A subsequent re-input of the data, and thus a simulation of the card, is possible. This attack can be effectively prevented by utilising the "intelligence" of the card, i.e. its abilitiy to execute computer operations.
Similarly to the generation of "session keys" with communication encryption, a pseudo random number is transmitted to the card upon every call. The card calculates the response as a function of this pseudo random number.

III) A NEW CARD CONCEPT AND ITS APPLICATIONS

Chapter II deals with the security of IC-cards with regard to various attacks, while this chapter gives examples of how the IC-card in turn helps to increase the security of systems.

IC-cards are effective especially in two functions:

- as carrier medium for confidential data, such as cryptographic keys and passwords, and

- as "special computer" for taking over selected security functions.

The following section describes an IC-card, which has been developed for high-security applications.

The basic card concept

The concept is based on the considerations of Chapter II, concluding that the cryptographic protection of the data stored on the card provide a maximum degree of security in general.

i) Block structure
The data memory is segmented into blocks of freely selectable lengths. Each block is allocated to a

specific application and protected by a separate PIN, i.e. the PINs are block-specific, and thus application-specific, but not card-specific.

ii) Encryption of data on the card
All (user) data on the card are basically stored in enciphered form. In order to fulfill the security requirements to be met by the card (cf. Chapter 1, security analysis), the encryption of the data must comply with a number of specifications.

- Dependence on the PIN
  In order to prevent misuse of a stolen or lost card, it has to be protected by some additional information that is known to the legitimate user only, i.e. usually a "PIN" (Personal Identification Number).
  In the VOEST-ALPINE concept, the PIN is highly involved in the protective mechanism; it serves as part of the key under which the data stored on the card are enciphered.

  The PIN can be replaced by other - user-related - parameters, such as biometric parameters, without the basic concept having to be modified.

- Dependence on the card
  In order to effectively prevent copying of the enciphered data onto another card, and thus, duplicating the card, the encryption must depend on a paramater which is different for each card, secret and not predicatable ("pseudo random").

  This "Unique Card Key", in the following referred to as "KU", is exclusively used for the encryption of data on the card and cannot be read out.

Other card functions

In addition to the basic functions of the card as described above, two other functions are provided, which are based on the cipher algorithm integrated in the card:

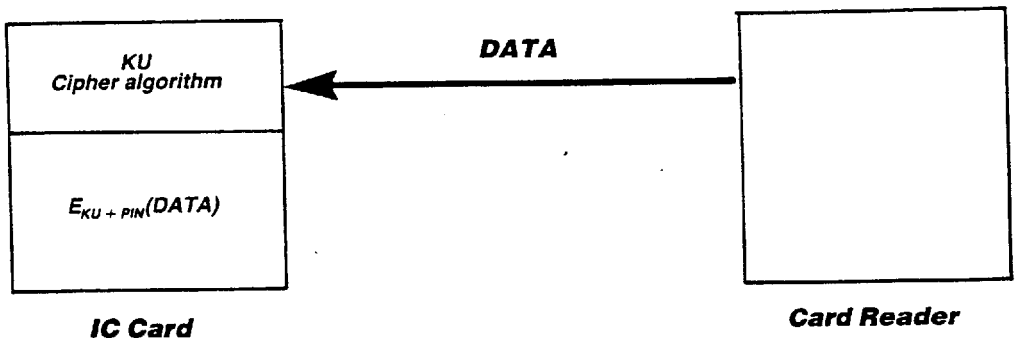
- Enciphered communication

  It is possible to encipher all data transmitted between the card and the card reader. This - expensive - function is intended for special applications.

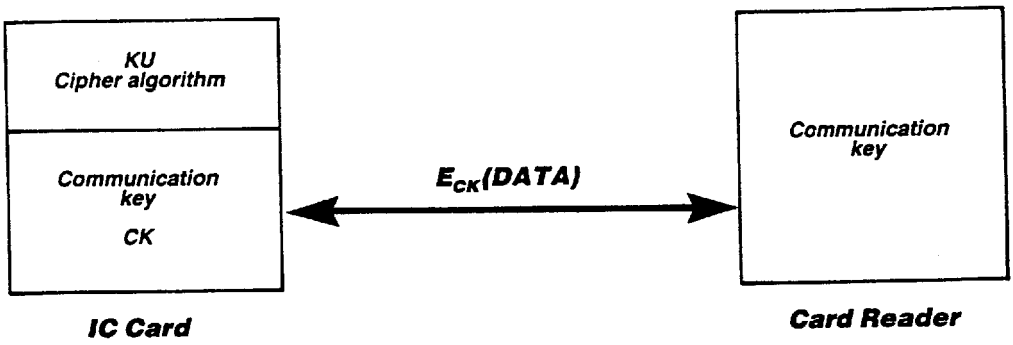
- Encryption of external data ("Black Box Cipher")

  This functions enciphers external data under key stored on the data and retransmits them to the card reader. It is especially used for the realisation of key management functions.

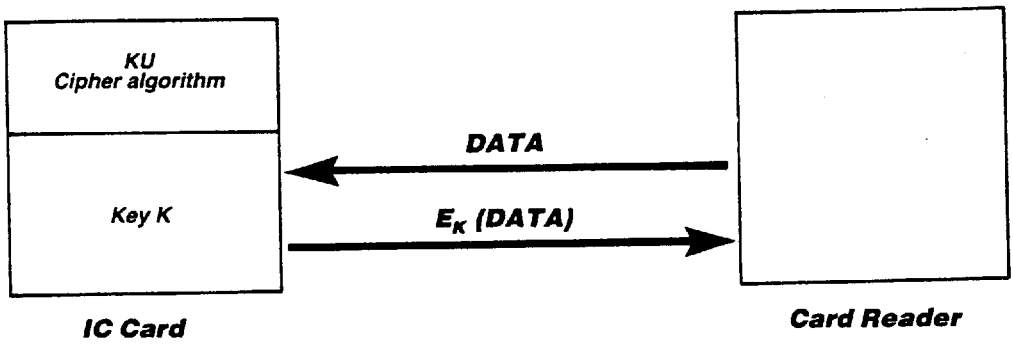
  Thus, the cipher algorithm on the card is used for 3 different functions:



a) Data are stored on card in enciphered form



b) Enciphered communication between card and card reader



c) "Black Box"- Encryption of external data

Fig.2

Range of applications

Basically, there are two possibilities of using IC-cards in an efficient way:

i) as "carrier medium" for confidential data:

Cryptographic keys, passwords, identification parameters, states of account, medical information and similar data can be securely stored on the card and retrieved in a user-friendly way. By storing several keys, it is possible to set up key hierarchies.

ii) as "special computer":

Special functions, such as encryption of external data, are taken over by the card.

Thus, it is ensured that
- these functions cannot be manipulated, and
- secret parameters cannot occur outside the card.

The card concept described above is mainly used in:

- Applications with high security requirements, e.g.

  - EDP security:

    Data protection: File encryption, database encryption, communication encryption

    Access protection: Identification, authentication, access control

. Software protection: Protection against software piracy, protection against unauthorised software applications

Electronic money: Credit cards, debit cards, telephone cards, POS, ATM, etc.

- Multi-functional cards:

If, in the future, IC-cards are to be applied to the degree planned today and accepted by the users, the use of multi-functional cards is indispensible.

The above card concept is an attempt to meet these requirements:

- The block organisation allows the use of a card in a number of different applications

- PIN depends on the application

- PIN can be selected by the user or preset by the issuer

- Locking of a block on the card has no effect on the other blocks (= applications)

- Varying number of allowed wrong inputs possible for the individual blocks.

IV. FUTURE REQUIREMENTS

In the unanimous opinion of technical engineers and market research specialists, the IC-card will spread widely in the future.

Even today, IC-cards are used especially in the fields of "electronic money" and "portable personal files" on a large scale; by 1988, several million IC-cards will be in circulation.
With the continuous spreading of the cards and new fields of applications, however, the requirements to be met by the cards increase, too.

In the next few years, further developments in the card technology are to be expected particularly in the following 3 fields:

a) Memory expansions

In general, the current (single-chip) IC-cards have a memory size of 1, 2 or 8 kilobytes. According to the progress made in IC-technology, a gradual expansion of the data memory of the card is to be expected. Moreover, the combination of IC-cards with laser cards is taken into consideration. The card resulting would unite an increased security of the IC-card and the high storage volume of the laser card.

b) The "Super Smart Card"

The "Super Smart Card" is an IC-card at which the keyboard and the display are already integrated in the card.

This extra equipment

- ensures an increased security of the entire system, and

- allows its application as an "Offline Security Device".

The security is increased primarily in applications in which the card reader or the keyboard respectively is unprotected, and thus exposed to the danger of manipulation.
It is possible, for instance, to intercept the connection between the keyboard and the card reader unnoticed and with a relatively small expense, and to store also the PINs typed in by the users.
If the keyboard is located on the card, and thus is controlled by the card user, such an attack is impossible.

As an "Offline Security Device", the Super Smart Card can be applied in fields in which peripheral devices are used to which a card reader cannot be connected - which is the case with the major part of the terminals used today.
The common direct data transfer between the card and the computer is replaced by manual typing in of the request or response data respectively by the user.
Such a procedure also permits the realisation of a homogenous security system even if different hardware (terminals and PCs) is used.

COLLISION FREE HASH FUNCTIONS AND PUBLIC KEY SIGNATURE SCHEMES

Ivan Bjerre Damgård[1]

Aarhus University, Inst. of Math.
Ny Munkegade,
DK 8000 Aarhus C,
Denmark

Abstract

In this paper, we present a construction of hash functions. These functions are collision free in the sense that under some cryptographic assumption, it is provably hard for an enemy to find collisions. Assumptions that would be sufficient are the hardness of factoring, of discrete log, or the (possibly) more general assumption about the existence of claw free sets of permutations.

The ability of a hash function to improve security and speed of a signature scheme is discussed: for example, we can combine the RSA-system with a collision free hash function based on factoring to get a scheme which is more efficient and much more secure.

Also, the effect of combining the Goldwasser-Micali-Rivest signature scheme with one of our functions is studied. In the factoring based implementation of the scheme using a k-bit modulus, the signing process can be speeded up by a factor roughly equal to $k \cdot O(\log_2(k))$, while the signature checking process will be faster by a factor of $O(\log_2(k))$.

1. Introduction

One of the most fascinating features of public key cryptography is the notion of digital signatures. However, for many of the so far proposed schemes a proof of security does not (yet) exist, or they have been shown to be breakable under sufficiently strong attacks. Moreover, a practical implementation of a signature scheme is often made very difficult by the complexity of the algorithms needed in the system. These problems suggest the use of a hash function, some suitable transformation which is applied to a message before signing it. In particular, we would like to mention the following:

- with a block oriented signature algorithm where messages are longer than a block, it is not safe to sign messages block by block: an enemy could remove blocks from a signed message or insert blocks of his choice into a message before it was signed. Thus, some transformation must be used to make a signature depend on all parts of a message.

[1]This research was supported by the Danish Natural Science Research Council.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 203-216, 1988.
© Springer-Verlag Berlin Heidelberg 1988

- if the message space has some algebraic structure, and the signing algorithm behaves too nicely with respect to this structure, the system can be vulnerable to a chosen message attack (see e.g. [De]). A hash function can be used to destroy this algebraic structure.
- usually, the output of the hash function is much shorter than the input, so if the signature algorithm is slower than the hash function used, a considerable amount of time can be gained in an implementation of the scheme.

The need for hash functions has been realized before ([De], [DP]), and several attempts have been made to construct such functions using fx. DES or RSA as building blocks. However, none of these suggestions have been proved to be secure, and several of the proposals using DES have been proven insecure ([Wi], [Co]). Other variations of hash functions have also been proposed [We]. But the use of these functions, like pseudo random functions [GGM], require that sender and receiver share a secret key. They are therefore suitable for authentication purposes, but do not fit into the scenario of a public key signature scheme. We would like to have publicly known hash functions that are easy for all users to compute.

2. Construction of Collision Free Hash Functions

In stead of considering just one hash function, we will consider families of them, in order to make a complexity theoretic treatment possible. Any member of such a family will have a value of *security parameter* attached to it. A number of things, such as the overall security of a system using the hash function, will depend on it. For any security parameter value k, we choose a finite alphabet Σ_k with $|\Sigma_k| = r_k$. A hash function with security parameter value k will be a map $h: M_k \rightarrow A_k$, where M_k is the set of all finite words over Σ_k, and A_k is some finite set. Note that when no confusion is possible, the subscript k's will sometimes be dropped.

The most basic demand to a good hash function is that it should be computationally infeasible for an enemy to find collisions, i.e. different messages hashing to the same value. There are a number of possible interpretations of what "computationally infeasible" means. In this paper we will choose circuit complexity to describe it, because it seems best suited in a cryptographic setting (cf. [BM], p.857). But note that other computational models, e.g. Turing machines, would allow our results to be proven in the same way, and would only result in a change in the intractability assumptions we make later.

Throughout this paper, a problem will be said to be *computationally infeasible* to solve, if the following is satisfied:

- Let $\{C_k\}$ be a boolean circuit family of polynomially bounded size. Let ε_k be the fraction of the instances of size k which are solved by C_k. Then, as a function of k, ε_k vanishes faster than any polynomial fraction.

Loosely speaking, a problem is hard in above sense if no polynomial size circuit can solve more than a negligible fraction of the instances. Whenever this definition is used in this paper, k will be interpreted as a security parameter value. "Polynomial time" will always

mean "computable by a polynomial (in k) size circuit family. Finally, a probability is "negligible" if it vanishes faster than any polynomial fraction as a function of the security parameter.

Definition 2.1
A *family of collision free hash functions* is a set of hash functions with the following properties:

- There is a probabilistic polynomial time algorithm, which on input a value of security parameter selects uniformly and randomly a member of the family with the given value attached.
- All functions in the family are computable in polynomial time.
- The problem of finding $x \neq y$ such that $h(x)=h(y)$ for a given h in the family is computationally infeasible to solve □

A member of a collision free family is also called collision free. If h is collision free, then h is also one-way in the following sense:

Lemma 2.2
Given a collision free family, and a member $h: M \rightarrow A$. For any finite $W \subseteq M$ with $|W| - |A|$ a nonnegligible fraction of $|A|$, it is computationally infeasible given $h(w)$, where $w \in W$, to find any $w' \in W$ with $h(w') = h(w)$, for more than a negligible fraction of the w in W.

Proof.
Assume the lemma is false. Choose $w \in W$ at random and compute $h(w)$. By assumption, we can efficiently find w' such that $h(w)=h(w')$. By assumption on $|W|$, the probability that $w \neq w'$ is nonnegligible, which means that this procedure creates a collision for h with nonnegligible probability □

It is important to note that since A might be negligibly small compared to W, the above lemma does not imply that it is hard to invert h on almost all elements in A: even though the set of elements $w \in W$ for which it is easy to invert h on $h(w)$ is negligibly small compared to W, their images might constitute a large fraction of A! It turns out that this last form of one way property is important in connection with chosen message attacks on signature schemes. Each hash function therefore has to be checked for this one way property. More details can be found in [Da].

In [GMR], the notion of claw free pairs of trapdoor permutations are introduced. We will use a generalisation of this idea, without the trapdoor property.

Definition 2.3
A *claw free family of permutations* is a family of sets of permutations with the following properties:

- Each set S in the family has a security parameter value attached to it, and there exists a function $g:\mathbf{N} \rightarrow \mathbf{N}$ such that when S has security parameter k, then $|S| = g(k)$.
- All members of a set S in the family have the same domain.
- There is a probabilistic polynomial time algorithm, which on input a security parameter value selects randomly and uniformly a member of the family with the given value attached.
- For each set $S = \{f_0, \ldots, f_{r-1}\}$, and each $x \in domain(f_0)$, it is easy to compute $f_i(x)$ for all $i = 0 \cdots r-1$, but it is computationally infeasible to create *a claw*, i.e. find x and y such that for some $i \neq j$, $f_i(x) = f_j(y)$ □

Assuming the existence of claw free permutations, we can construct collision free hash functions. First some notation:

Let an alphabet Σ with cardinality r and a finite word m over Σ be given. We now let $[m]$ denote a prefix free encoding of m over Σ. The choice of a particular encoding is not important to the results given here, except for the fact that it is possible to encode efficiently so that the length of $[m]$ is a linear function of the length of m. In binary, for example, 1 could be encoded as 11, 0 as 00, and all encodings could be terminated with 01. This is important because a short encoding will make the mechanisms shown here more efficient. In fact, theoretical results show that the encoding can be chosen such that the length of m is almost equal to the length of $[m]$. In this case, however, the encoding process itself might become inefficient. Further details on prefixfree encodings can be found in [BP].

Now, if $\{f_0, \ldots, f_{r-1}\}$ is a set of permutations, all with domain D, we define

$$f_{[m]}(I) = f_{m_1}(f_{m_2}(\cdots f_{m_s}(I)\cdots)),$$

where $I \in D$, $[m] = m_1 m_2 \cdots m_s$, and the letters in Σ are denoted by the numbers $0, \ldots, r-1$. A similar construction is used in [GMR] with $r=2$.

Theorem 2.4
The family **F** constructed below is a collision free family of hash functions.

Let **P** be a family of claw free permutations. For each value of the security parameter k, we let Σ_k be the alphabet of cardinality $r_k = g(k)$ given by $\Sigma_k = \{0,1, \ldots, r_k - 1\}$. For each set $S = \{f_0, \ldots, f_{r_k-1}\} \in \mathbf{P}$ and each $I \in domain(f_0)$, we define a member h of **F** with security parameter k by:

$$h(m) = f_{[m]}(I),$$

for all $m \in M_k$.

Proof

Assume for contradiction that F is not collision free. This means that for any $h \in \mathrm{F}$, we can efficiently find $m \neq m'$, such that

$$f_{[m]}(I) = f_{[m']}(I),$$

where $[m]$ and $[m']$ have lengths s and s', respectively. Note that since m and m' are assumed to be produced by a polynomial size circuit, both s and s' can be at most polynomial in k. If $m_1 \neq m'_1$, we have a claw for the set S. If $m_1 = m'_1$, the fact that the f's are injective implies that

$$f_{m_2}(\cdots f_{m_r}(I)\cdots) = f_{m'_2}(\cdots f_{m'_r}(I)\cdots).$$

The same argument now applies again, and since the encoding used is prefix free, this process must stop with the creation of a claw □

If we let T denote the time needed to evaluate one of the permutations used in the construction, it is clear that the time needed to compute h on a message of length L is $O(TL)$.

The motivation for working with sets of claw free permutations rather than just pairs (as in [GMR]) is now clear: any binary message can be seen as a word over a larger alphabet by treating s-bit chunks of it as symbols in an alphabet with 2^s elements. Thus if we have claw free sets with 2^s elements, the message can be hashed by processing s bits of it in stead of 1 at time.

2.1. Examples of Claw Free Permutations

We first give a construction of a claw free family of permutations under the assumption that factoring is hard.

Choose any polynomially bounded function $g:\mathrm{N} \to \mathrm{N}$. For each value of the security parameter k, we define the permutation set size to be $g(k)$, for short denoted by r_k. Now, let $n = p_1 p_2 \cdots p_t$, where all the p's are k bit prime numbers equivalent to 3 modulo 4, and where t is the smallest integer such that $2^{t-1} \geq r_k$. The set of integers of this form is denoted H_k. For each $n \in H_k$ we shall construct a set of claw free permutations with security parameter k.

For each a prime to n, define

$$J(a) = ((\frac{a}{p_1}), (\frac{a}{p_2}), \cdots, (\frac{a}{p_t})).$$

$QR(n)$ will be the set of quadratic residues mod n. Clearly,

$$QR(n) = \{a \mid J(a) = (1,1,\cdots,1)\}.$$

The set of all ± 1 t-tuples form a group under pointwise multiplication. Let G_t denote this group modulo the subgroup generated by $(-1,-1,\ldots,-1)$. Clearly J induces a surjective homomorphism $\phi_n: \mathbf{Z}_n^* \to G_t$. Choose a set of r_k elements in $\mathbf{Z}_n^*$, $A = \{a_0,\ldots,a_{r_k}-1\}$ such that $|\phi_n(A)| = |A|$. This is clearly possible by choice of t. A is called *an injective set of numbers*, whenever it satisfies this condition. We can now define our set of permutations $\{f_0^{(n)}, f_1^{(n)},\ldots,f_{r-1}^{(n)}\}$ to be the set of permutations of $QR(n)$ given by

$$f_i^{(n)}(x) = (a_i x)^2 \bmod n, \quad \text{for } x \in QR(n) \text{ and } i = 0,\ldots,r-1.$$

To prove that finding claws is as hard as factoring, it turns out to be essential that the a_i's (and not just their squares) are made public. It might be argued that this could endanger the factorization of n, since checking whether a set is injective requires knowledge of the factors, and since the set size grows exponentially with the number of factors. To prove that such release of an injective set is not dangerous, we need the following series of technical, but elementary lemmas:

Lemma 2.5.
Let G be a finite abelian group of exponent 2. Let $S = \{g_1,\ldots,g_s\} \subseteq G$. Then $\langle S \rangle$, the subgroup generated by S, has order at most 2^s, and equality occurs exactly when no g_i can expressed as a product of the others.

Proof.
Trivial from the fact that all g_i's have order 1 or 2 □

Lemma 2.6
Let G be a finite abelian group of exponent 2, $|G| = 2^s$. The probability that a randomly chosen subset S of G of cardinality s generates G is

$$p_s = \frac{2^s-1}{2^s}\cdot\frac{2^{s-1}-1}{2^{s-1}}\cdot\ldots\cdot\frac{2-1}{2}.$$

Moreover,

$$p_s \to p_\infty = \frac{1}{3} - \frac{1}{3\cdot 7} + \frac{1}{3\cdot 7\cdot 15} - \cdots \approx 0.289 \text{ for } s \to \infty$$

Proof.
Let $S = \{g_1,\ldots,g_s\}$. For all $1 \le i \le s$ we have that

$$Prob(g_{i+1} \notin \langle g_1,\ldots,g_i\rangle) = \frac{2^s-2^i}{2^s} = \frac{2^{s-i}-1}{2^{s-i}},$$

provided $\langle g_1,\ldots,g_i\rangle$ has maximal size, by Lemma 2.5. Thus we get

$$Prob(|\langle g_1,\ldots,g_{i+1}\rangle| = 2^{i+1}) = \frac{2^{s-i}-1}{2^{s-i}}\cdot Prob(|\langle g_1,\ldots,g_i\rangle| = 2^i).$$

This proves the first statement. The second follows from a long series of tedious and rather trivial manipulations with the expression for p_s. The reader will be spared the details □

Lemma 2.7

Consider a probabilistic polynomial size circuit family that on inputs an integer $n \in H_k$ and an injective set for n, factors n with probability p_k. For any such circuit family and any $\varepsilon>0$ there exists another probabilistic polynomial size circuit family that factors n with probability q_k, such that $|p_k-q_k|<\varepsilon$.

Proof.

The idea is to guess an injective set and to feed this and n to the factoring circuit we are given. It suffices to consider the case where the set size r_k equals 2^{t-1} (smaller sets are easier to guess). Choose $t-1$ numbers at random from Z_n^*, call them $a_1,\ldots,a_{t-1}$. Under ϕ_n, they correspond to $t-1$ randomly chosen elements in G_t. By Lemma 2.6, if we form the set

$$A = \{x \in Z_n^* \mid x = \prod_i a_i^{v_i},\ v_i = 0 \text{ or } 1\},$$

then the probability that $\phi_n(A) = G_t$ (and hence that A is an injective set) is at least p_∞. Since r_k is at most polynomial in k, the process of choosing A can be completed by a polynomial size circuit. By going through this procedure s times, we obtain for the probability of success, q_k, that

$$p_k \geq q_k \geq p_k(1-(1-p_\infty)^s).$$

Since p_∞ is independent of k, this proves the lemma.

Theorem 2.8

The family consisting of all sets of permutations constructed as above is a claw free family of permutations, provided factoring integers in $\{H_k\}$ is computationally infeasible.

Proof.

Using well known algorithms and the Chinese remainder theorem, the p's and the a's for each n can easily be selected. Further, each permutation can be evaluated with two modular multiplications. Thus it suffices to prove that finding claws is as hard as factoring. Suppose a claw for the set $\{f_0^{(n)},\ldots,f_{r-1}^{(n)}\}$ has been found. So for some $i \neq j$ we have:

$$f_i^{(n)}(x) = f_j^{(n)}(y)$$

or

$$(a_i x - a_j y)\cdot(a_i x + a_j y) \equiv 0 \bmod n.$$

By choice of the a's and basic properties of the Jacobi symbol, this means that

$GCD(a_i x + a_j y, n)$ will produce a nontrivial factor of n. By Lemma 2.7, we can now efficiently factor n if claws can be found efficiently □

With this construction of claw free sets, it will often be possible to improve performance of our hash functions by using larger sets, rather than fx. pairs. One should be aware, though, that with this construction a larger set requires more prime factors and hence a longer modulus - note that a modulus with many shorter prime factors will not be safe enough because of factoring algorithms like Lentra's elliptic curve method [St], whoose running time depends mostly on the size of the smallest prime factor.

There is, however, a slight variation on this construction which does not have this drawback. It was inspired by a construction of "blobs", which was presented in [BC]. In this case, let n be a modulus with exactly 2 k-bit prime factors equivalent to 3 mod 4. Next, choose a set $A = \{a_1, \ldots, a_{r-1}\} \subseteq QR(n)$. Define a set of permutations $\{f_1^{(n)}, \ldots, f_{r-1}^{(n)}\}$ by

$$f_i^{(n)}(x) = a_i x^2, \text{ for } x \in QR(n).$$

It is now trivial to prove:

Lemma 2.9
Suppose an adversary can compute a claw for $f_i^{(n)}$ and $f_j^{(n)}$ as defined above. Then he can compute a square root of $a_i \cdot a_j^{-1}$.

The practical advantage of this is that the set A can be enlarged without changing the modulus, so that one can use as many a's as one is willing to store (or generate pseudo randomly). Asymptotically, since we can only use a polynomial size A, this means that this variation is faster than the previous construction by a factor of $O(\log_2(k))$. It does have a drawback, which is only of theoretical significance, however: with the previous construction, we could prove that for *any* choice of injective set, knowledge of a claw implied knowledge of a factor of n. In the new construction, it is conceivable that an adversary could know some square roots of quotients of elements in A just "by chance", and not as a result of possessing an algorithm for computing square roots of randomly chosen elements in $QR(n)$. We may assume, however, that the probability of this is negligible in practice: if the elements of A are chosen at random, then the probability that some polynomial size circuit can find their square roots is negligible, unless square roots can be computed in random polynomial time. More details can be found in [Da].

The final example is based on the hardness of discrete log, and shows that claw free permutations need not be trapdoor. It is of theoretical significance for that reason, but of no great practical use, since one permutation evaluation needs a full modular exponentiation. Choose a large prime p, a generator g of Z_p^*, and a set $A = \{a_0, \ldots, a_{r-1}\} \subseteq Z_p^*$. Define

$$f_i(x) = a_i g^x, \text{ for } x \in Z_p^*, \text{ and } 0 \le i \le r-1.$$

In the same way as before, it is easy to prove that to find claws, one must be able to find the discrete log base g of quotients of elements in A.

3. Combining a Signature Scheme and a Hash Function

The model we shall use is the following: we have a signature scheme with

- message space A
- signature space B
- and a signature algorithm Δ,

which for any message $a \in A$ produces as output a signature $\Delta(S,a) \in B$, using the secret key S.

Δ could be probabilistic, and its output could depend on the number of messages previously signed, or how they were signed in which case the signature scheme is called non-memoryless. Since memorylessness is not relevant to the results proved below, the dependence on random inputs or previously signed messages has been omitted from the notation. Finally we have

- a verification predicate Γ,

which for any message a and signature s produces as output a boolean value $\Gamma(P,a,s)$ using the public key P. $\Gamma(P,a,s) = \text{TRUE}$ if and only if s is a valid signature for a using the secret key matching P.

This scheme is referred to as *the original scheme.* A full model of a signature scheme would also have to include an algorithm that generates matching pairs of public and secret keys, but this will not be important in this context.

As usual, a number of things such as the length of the keys, the running time of Δ and Γ, the overall security of the scheme depend on the value of a security parameter k.

We now combine this scheme with a hash function $h: M \rightarrow A$. A message is signed by computing $\Delta(S,h(m))$ and signatures are checked by computing $\Gamma(P, h(m), \Delta(S, h(m)))$. This scheme will be called *the combined scheme.*

When combining with a collision free hash function, we always assume for simplicity that the signature scheme and the hash function have the same value of security parameter.

Following [GMR], a signature scheme is called *existentially forgeable* if under some attack, an enemy can forge the signature of at least one message. This message cannot necessarily be chosen by the enemy.

A *chosen message attack* is an attack where the enemy can choose messages to be signed by the legitimate signer before trying to forge a signature. If he is also allowed to choose such messages *during* the process of forgery, we speak of an *adaptive chosen message attack.*

It is of course essential to be able to compare the security of the two schemes. A first result in this direction is:

Theorem 3.1
Suppose that the combined scheme is at least existentially forgeable under a chosen message attack, and that the hash function used is collision free. Then the original scheme can be existentially forged under a chosen message attack.

Proof.
Assume for contradiction that we have an algorithm Φ that forges at least one signature in the combined scheme after receiving signatures of messages of its choice. Assume also that we can mount a chosen message attack on the original scheme. While running Φ, we will be asked for signatures of messages in some subset N of M. They are easily found by computing the set $h(N)$ and using the chosen message attack to obtain $\Delta(S, h(N))$. With nonnegligible probability, Φ will produce as output a message $m \notin N$ and a valid signature for m. If $h(m) \in h(N)$ we have found a collision for h, but by assumption on h this can only occur with negligible probability. We may therefore assume that $h(m) \notin h(N)$ which means that we have forged a new signature in the original scheme □

Note that the proof goes through in the same way if the attack is adaptive. Therefore "adaptive chosen message attack" could substituted for "chosen message attack" in the theorem.

This result has two useful implications:

- If the original scheme is not existentially forgeable, then neither is the combined scheme.
- If the combined scheme has any kind of weakness, then the original scheme was already existentially forgeable.

The last fact states that the use of the hash function does not introduce any "completely new" weaknesses. This may make a security analysis of the combined scheme easier, but unfortunately it does not rule out the possibility that the combined scheme is much weaker than the original scheme! This will be illustrated by an example later.

The next result, however, shows that at least a large class of attacks directed against the original scheme are prevented by the use of a hash function with the right property:

Proposition 3.2
Suppose the hash function used is one way in the sense that it is hard to invert on a randomly chosen element in A (c.f. the discussion following Lemma 2.1). Consider an algorithm for forging signatures which is directed against *the original scheme*, i.e. the hash function used is not part of the input to the algorithm. No such algorithm which satisfies i) or ii) below will be of any use in attacking the combined scheme.

i) The algorithm only works under a chosen message attack, and will during the attack ask for signatures of messages uniformly distributed in A.

ii) The algorithm can only forge signatures for a small (i.e. polynomial) number of messages, which are non-chosen and uniformly distributed over the message space (where the distribution is taken over all choices of messages to sign made by the signer and all choices made by the enemy).

Proof

i): by the one way property, even a chosen message attack on the combined scheme does not allow a chosen message attack on the original scheme: Since h is not part of the input to the forgery algorithm, any procedure which could implement a chosen message attack on the original scheme based on a chosen message attack on the combined scheme, would in fact be an inversion algorithm for h. ii): Call the set of messages for which the algorithm has forged signatures X. Since X is in fact a random variable, then to use these signatures, the enemy must have an algorithm that on input any set of messages Y with $|Y| = |X|$ efficiently produces an element $m \in M$ with $h(m) \in Y$. But this would enable him to invert h on a randomly chosen element with non negligible probability □

Let us see whether the functions constucted in the previous section have the one way property required in the proposition. As far as the functions based on factoring are concerned the answer, curiously, is yes if discrete log is hard! A simple example: suppose we construct a hash function based on the pair of permutations

$$f_0(x) = x^2 \text{ and } f_1(x) = ax^2$$

where, as usual, all computations are modulo an appropriately chosen modulus n. We now chose a random $I \in SQ(n)$ and define $h(w) = f_{[w]}(I)$ But with these permutations, it is easy to see that in fact

$$h(w) = a^{[w]_l} I^{2^{l(w)}},$$

where $l(w)$ is the length in bits of $[w]$. Thus, even if an enemy when presented with an element in $Im(h)$ could guess the length of a preimage, he would still have to solve a discrete log problem base a. Moreover, the set of discrete log problems that arise in this way certainly constitute a non negligible fraction of all possible discrete log problems base a.

As far as hashfunctions based on claw free permutations in general are concerned, it seems to be hard to give actual proof that the one way property is always satisfied. It is possible, however, to give heuristic arguments suggesting that such functions will hit any element in their image with almost equal probability. This and the collision freeness would imply the one way property (more details on this can be found in [Da]).

Assumptions i) and ii) in the proposition above seem to be reasonable models of the multiplicative attacks on RSA mentioned in [De]. Thus a collision free function would prevent these. But let us stress once again that the proposition does not talk about attacks directed

against the *combined* scheme. Therefore an analysis of this combination is needed in each concrete case.

As an example of these problems, consider a combined scheme with RSA as the original scheme, and a hash function based on factoring as described in Section 2.

If the modulus used for the hash function is chosen *independently* of the RSA-modulus, then the following assumption seems reasonable:

If there exists an algorithm that forges RSA signatures under condition 1) below, then there exists an equally efficient algorithm that forges signatures under condition 2).

1) All messages signed are of the form $h(m)$, and h and m are known to the forgery algorithm.

2) All messages are chosen at random from $\mathrm{Im}(h)$.

In other words, as far as forging RSA-signatures is concerned, the hash function might as well be a random function. If this is indeed true, then by Proposition 3.2 the only way to break the combined scheme is to totally break RSA under a non chosen message attack (i.e. forge the signature of any message with nonnegligible probability). It therefore seems that this combination is much more secure than plain RSA.

A word of warning, however: if the two moduli used are not independent, then the above assumption can be seriously violated. For example, if for convenience we use the same modulus for the hash function as for the RSA, then by an attack due to Bert den Boer [Bo], the combined scheme can in fact be broken. The attack relies heavily on the fact that the multiplicative structure used in computing the hash function is exactly the same as the one used in computing signatures. It therefore does not generalise to the case where the moduli are independently chosen.

It is worth noting that in addition to being more secure, the combination with RSA has a number of practical advantages:

- Since modular arithmetic is used for both the hash function and RSA, the combination can be implemented without using more hardware than is needed for basic RSA.
- Using the most efficient construction mentioned in Section 2 will yield a scheme which is more efficient than plain RSA on long messages. For example, if one is willing to store a set of 512 constant coefficients, then the combined scheme will be 9 times faster than the original one.
- Signatures have length 1 RSA block, independent of the message length.

4. Speeding up the Goldwasser Micali Rivest Signature Scheme

In [GMR], the Goldwasser-Micali-Rivest (GMR) signature scheme is introduced and is proven to be not existentially forgeable, even under an adaptive chosen message attack. The basic building blocks in the scheme are two pairs of claw free trapdoor permutations, (f_0, f_1) and (g_0, g_1). To sign a message a, the following is done:

1) A random value R is chosen.
2) A value α is computed using the trapdoor information for the g-permutations such that $g_{[a]}(\alpha) = R$.
3) R is authenticated using the f-permutations.

The signature for a now consists of α, R, and the authentication for R. It is pointed out that the scheme is most practical when messages have length much greater than the value of security parameter used, since in this case signatures are short compared to the messages. Clearly, if α is computed in a straightforward way by inverting the g-permutations involved one by one, then the time needed to compute a signature for a message of length L is $O(LT_{inv})$, where T_{inv} is the time needed to invert one permutation. Moreover, if (as proposed in [GMR]) we use the factoring based construction of claw free pairs of permutations, then T_{inv} is greater than the time for one permutation evaluation by a factor of $\log_2(n)$, where n is the modulus used.

Using the constructions from Section 2, we can improve the efficiency of this scheme substantially:

1) It is trivial to check that the GMR scheme will work equally well with any size of sets of claw free permutations. Therefore we can use the most efficient construction from Section 2 in place of both the f- and the g-permutations. This will speed up both the signing and the signature checking process by a factor of $O(\log_2(\log_2(n)))$.
2) By combining with a collision free hash function based on factoring, we can avoid most of the permutation inversions. It is easy to check that this will speed up the signing process further by a factor of about $\log_2(n)$ for long messages. Moreover, by Theorem 2.4 and 3.1, the combined scheme is as secure as the original one.

Other methods for speeding up the GMR scheme have been proposed, both by Goldreich [Go] and Goldwasser, Micali and Rivest in the full version of their paper [GMR2]. They both apply only to the signing process and achieve a slightly smaller improvement than our method (the factor gained is $\log_2(n)$). They are also fundamentally different: the method from [Go] only applies to the factoring based implementation of claw free permutations, while our method is potentially useful with any implementation. The method from [GMR2] will not allow the use of non trapdoor permutations, while the construction of a hash function only uses the claw free property.

5. Conclusion

We have seen that hash functions can be useful, both in improving the security and the efficiency of signature schemes. We have also seen that if factoring or discrete log is hard, provably secure and reasonably efficient hash functions can be constructed.

References

[BC] Brassard and Crepeau: Non transitive transfer of confidence: a perfect zero-knowledge interactive protocol for SAT and beyond. Proc. of FOCS 86.

[Da] Damgård: Application of claw free functions in cryptography; unconditional protection in cryptographic protocols. Phd Thesis, in preparation.

[De] Denning: Digital signatures with RSA and other public key cryptosystems. CACM, v.27, nr 4, 1984, pp.388-392.

[GMR] Goldwasser, Micali and Rivest: A "paradoxical" solution to the signature problem. Proc. of 25th FOCS, 1984, pp.441-448.

[BP] Berstel and Perrin: Theory of codes, Academic Press, 1985.

[DP] Davis and Price: The application of digital signatures based on public key cryptosystems. Proc. of 5th Int. CompCom, 1980, pp.525-530.

[Wi] Winternitz: Producing a one-way hash function from DES. Proc. of Crypto 83, pp.203-208.

[Co] Coppersmith: Another birthday attack. Proc. of Crypto 85.

[St] Stephens: Lentras factoring method based on elliptic curves. Proc. of Crypto 85.

[WC] Wegman and Carter: New hash functions. JCSI, v.22, 1981.

[GGM] Goldreich, Goldwasser and Micali: How to construct random functions. MIT.LCS.TM-244, 1983.

[BM] Blum and Micali: How to generate cryptographically strong sequences of pseudorandom bits. SIAM J. of Computing, v.13, 1984.

[Go] Goldreich: Two remarks concerning the GMR-signature scheme. Proc. of Crypto 86.

[GMR2] Goldwasser, Micali and Rivest: A digital signature scheme secure against adaptive chosen message attacks. To appear.

[Bo] Bert den Boer: private communication.

HASH-FUNCTIONS USING MODULO-N OPERATIONS

Marc Girault

Service d'Etudes communes des Postes et Télécommunications
42 rue des Coutures
B.P. 6243
14066 Caen-Cedex, France

INTRODUCTION

Today, there is a need for one-way hash-functions, particularly for use in digital signatures [1]. Following R.S. Winternitz [2], we define that H is a one-way hash-function if it maps messages of arbitrary length to some small fixed length, such that it is computationally infeasible to find two different messages M and M' hashing to the same value H(M) = H(M'). Now, if Alice wishes to sign M using for example the public-key system RSA, she submits H(M) to her secret function S_{Alice}, and the signature of M is Sig = S_{Alice} [H(M)]. The functions H and P_{Alice} (her RSA public function) being public, anybody who received the plain message M along with its signature Sig, is able to verify the signature by matching P_{Alice}(Sig) against H(M). The one-way property of H is not the only one required [3], but is the essential one: it prevents anybody (including Alice) from claiming that Sig is Alice's signature of a message M', different from M.

Until now, much attention has been paid to hash-functions based on a conventional encipherment algorithm, generally DES [2,3,4,5,6,7]. Nevertheless, it is also useful to design hash-functions using modulo-n operations, particularly if (as above) the hash-result is intended to be signed by the secret function of a public-key system, because these systems use this type of operations. In 1984, D.W. Davies and W.L. Price proposed such a hash-function [8,9]. The main objective of this paper is to show the weakness of their scheme (referred to below as the DP-scheme) and to examine some variations of it. This weakness was pointed out (but without details) for the first time by A. Jung in [10].

I TWO BASIC MODULO-N SCHEMES

Let n be a public integer (the so-called modulus) and (+) denote the bit-by-bit modulo 2 addition. By "message", we mean any pattern of

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 217-226, 1988.
© Springer-Verlag Berlin Heidelberg 1988

bits (issued from the natural language, a random number generator or any other source). As a consequence, a message may or may not have some internal redundancy.

I.1 TYPE 1 SCHEMES

A first basic scheme has been introduced by R.R. Jueneman [11], though used by him in a somewhat different way. It consists in dividing the message M to be hashed into k blocks B_i smaller than n and in performing:

$$H_0 = I \text{ (an initialization vector)}$$
$$H_i = (H_{i-1} \ (+) \ B_i)^2 \mod n \qquad 1 \leq i \leq k \qquad (E)$$

The hash-result is $H = H_k$.

Such a scheme is similar to the CBC mode of a block-cipher algorithm. But so described it is not secure, for the enemy can choose all the blocks as he likes and correct the resulting alteration by properly choosing the last one.

Precisely, he can replace $(B_1,\ldots,B_{k-1})$ with $(B'_1,\ldots, B'_{q-1})$, which lead to partial hash-results $(H'_1,\ldots,H'_{q-1})$, then choose the last block B'_q equal to H_{k-1} (+) H'_{q-1} (+) B_k. In that way, the input of the last modulo-n squaring remains unchanged and the hash-result too. For commodity, we will call this attack the "correcting last block (CLB) attack".

The reply consists in forcing the blocks to have some explicitly defined (and sufficiently high) redundancy. So, the attack mentioned above becomes ineffective because the probability is close to 1 that the corrected block B'_q will not satisfy the redundancy rule. But the following parts of this paper show that some precautions must be taken when adopting such a rule, otherwise a CLB attack remains possible, though becoming a little less easy.

I.2 TYPE 2 SCHEMES

A second basic scheme consists in replacing the equations (E) with:

$$H_0 = I \text{ (an initialization vector)}$$
$$H_i = H_{i-1} \ (+) \ (B_i^2 \mod n) \qquad 1 \leq i \leq k \qquad (E')$$

The hash-result is $H = H_k$, equal to the "sum" of the squares modulo n of all the blocks of the message.

Of course, this scheme is also imperfect because the enemy can permute the B_i, insert a spurious B' an even number of times or insert

the null block any number of times without modifying H. He may also easily find blocks $(B'_1,\ldots,B'_q)$ such that H is the "sum" of their real squares (e.g. 40 = 36(+)9(+)4(+)1).

As for the previous scheme, we can reply to these attacks by imposing to the blocks some redundancy. But here, the redundancy will have to depend on the rank of each block, in order to avoid the first two mentioned attacks. For example, this rank can be inserted somewhere in the block.

The rest of the paper is dedicated to the type 1 schemes (equations (E)).

II THE DP-SCHEME DESCRIPTION

The Davies-Price scheme is a type 1 scheme with redundancy. The exact description is:
- choose a public 512 bit integer n;
- divide the message M into 448 bit blocks $B_1,B_2,\ldots B_k$ (that is equivalent to imposing 64 bits of redundancy at the head of each B_i, these bits being equal to 0);
- apply the equations (E).

The hope is that the probability of a successful CLB attack be equal to the probability that a random number satisfy the required redundancy (i.e. 2^{-64}, negligible). We now show that this hope must be abandoned.

III AN ATTACK ON THE DP SCHEME

Let us call "valid" a 512 bit block with the 64 m.s.b. (most significant bits) equal to zero. A successful attack consists in replacing the valid blocks $(B_1,\ldots,B_k)$ with other valid blocks $(B'_1,\ldots,B'_q)$ leading to the same hash-result.

For simplicity, we only deal here with the case: k=q=2 and I=0 (the general case is considered in section V); hence $H_1 = B_1^2 \pmod n$ and $H'_1 = B'^2_1 \pmod n$.

First, let us show that the trivial choice $B'_1 = B_1 + 1$ leads very often to the success. In that case, $H'_1 = H_1 + 2B_1 + 1 \pmod n$. If h denotes the integer composed of the 448 l.s.b. of H_1, then the probability for H_1 and H'_1 to have the same 64 m.s.b is equal to the probability for $(h + 2B_1 + 1)$ to be smaller than 2^{448}, i.e. approximatively 1/4, if we consider h and B_1 as uniformly distributed in the interval $[0,2^{448}]$. If this event occurs, and since B_2 is valid, it

follows that $B'_2 = H_1$ (+) H'_1 (+) B_2 is valid too and the CLB attack succeeds, without any subtlety!

In a general manner, the enemy must find a block B'_1, close to B_1 (here, "close" means "having the same 64 m.s.b.") and such that the squares modulo n of B_1 and B'_1 are close too. If he finds it, then B'_1 as long as $B'_2 = H_1$ (+) H'_1 (+) B_2 will be valid and the CLB attack will succeed again.

In order to find such a B'_1, let d denote the small rational integer such that $B'_1 = B_1 + d$ (where + denotes natural addition). H'_1 is equal to $(B_1^2 + 2B_1d + d^2)$ mod n and will be close to H_1 if (but not only if) $2B_1d$ (mod n) and d^2 (mod n) are both small (here, "small" means approximatively "smaller than 2^{448}"). And d^2 (mod n) will be small if (but not only if!) d^2 is itself small, i.e. d is a "very small" integer (approximatively smaller than 2^{224}).

So, the problem is now reduced to finding a very small integer d such that $2B_1d$ (mod n) is small. At this stage, we need a little arithmetic.

IV SOME ELEMENTARY ARITHMETIC

Let n be a positive (>0) integer and b,X,Y three positive integers smaller than n. Does the system

$$\begin{cases} bx = y \pmod n \\ \text{abs}(x) < X \\ \text{abs}(y) < Y \end{cases} \qquad (S)$$

admit solutions in $Z^2-(0,0)$?

The answer is "certainly yes" if XY>n. The essential of the proof comes from [12], and uses the "pigeon-hole principle". The pigeons are the pairs (x,y) where $1 \leq x \leq X$ and $0 \leq y \leq Y-1$, and the holes are the integers bx-y (mod n) calculated with these pairs. There are XY pigeons and at most n holes. Consequently, if XY>n, there are necessarily two pigeons in the same hole, i.e. two different pairs (x,y) and (x',y') such that bx-y = bx'-y' (mod n). So, denoting x'-x by x" and y'-y by y", we have found a pair (x",y") different from (0,0) such that bx" = y" (mod n). If we remark that x" cannot be zero, it is clear that 0<abs(x")<X and abs(y")<Y, hence that we found a solution to our system (and even two, since the opposite of a solution is another solution too).

Now, what about the effective construction of such solutions ? Of course, the above proof is of no help if n is 512 bit long! We should rather remark that finding a small y such that bx = y (mod n) comes to

finding a good approximation of the fraction b/n. Consequently, the continued fractions theory should be useful here (see an introduction to the subject e.g. in [13]).

In a few words, it is possible with this theory to find "best" rational approximations of any real number x. These approximations, denoted b_i/n_i, are called convergents of x. For example, the first convergents of the square root of 2 are 1/1, 3/2, 7/5, 17/12, 41/29, etc. This sequence is illimited except of course when x is itself a rational (say b/n). For example, if x=72/91, the convergents are 1/1, 3/4, 4/5, 15/19, 19/24 and 72/91. In the rational case, the convergents are very easy to calculate, because provided by the Extended Euclidean Algorithm, which is now described.

As all the pupils know, the basic Euclidian Algorithm is an effective way to calculate the gcd (greatest common divisor) of two positive integers (n,b) and works as follows:

Let $r_0=n$, $r_1=b$ and $r_{i+1} = r_{i-1} - q_i r_i$ ($i \geq 1$) where q_i is the quotient in the euclidean division of r_{i-1} by r_i (hence r_{i+1} is the remainder of this division). The sequence of remainders r_i is strictly decreasing and the algorithm stops when a remainder (say r_{k+1}) comes to be equal to zero. Then gcd(n,b) is equal to r_k, the last non-zero remainder.

Now, let us define two supplementary sequences (l_i) and (m_i) in the following way: $l_0=1$ and $l_1=0$; $m_1=0$ and $m_1=1$; $l_{i+1} = l_{i-1} - q_i l_i$ and $m_{i+1} = m_{i-1} - q_i m_i$ for $i \geq 1$. As the recurrent equations are the same for the three sequences (r_i), (l_i) and (m_i), it is easy to see that for all the i: $l_i n + m_i b = r_i$. Hence, for all the i, $m_i b = r_i \pmod n$. This algorithm is called Extended Euclidean Algorithm (EEA), and it can be shown that the absolute values of the coefficients l_i and m_i are equal to, respectively, the numerator b_i and the denominator n_i of the convergents of b/n (in fact, $b_i/n_i = -l_i/m_i$ for all the i).

Moreover, the continued fractions theory states that the absolute value of the difference between a number and one of its convergents b_i/n_i is smaller than $(1/n_i)^2$. So:

$$\text{abs}(b/n - b_i/n_i) < (1/n_i)^2 \iff \text{abs}(b/n + l_i/m_i) < (1/m_i)2$$
$$\iff \text{abs}(r_i/m_i n) < (1/m_i)2 \iff \text{abs}(m_i r_i) < n$$

Finally, we see that the EEA provides some pairs (m_i,r_i) such that:

$$\begin{cases} bm_i = r_i \bmod n \\ \text{abs}(m_i r_i) < n \end{cases}$$

If we come back now to the initial system (S) stated at the top of this section, it appears that the EEA provides some pairs (x,y) such that bx = y (mod n) and abs(xy)<n but not necessarily verifying abs(x)<X and abs(y)<Y. However, if the product XY is not too close to n, the hope is great that at least one of these pairs be a solution.

V THE DP-SCHEME ATTACK ALGORITHM

Now, we can state our attack against the DP-scheme precisely. Recall that we presently suppose that the enemy wants to replace a message M composed of two blocks B_1 and B_2 with another message composed of two blocks B'_1 and B'_2, without modifying the hash-result H, calculated from the message M and an initialization vector equal to zero. We stated in section III that the enemy has a good chance of success if he finds a very small integer d such that $2B_1d \pmod n$ is small (the terms "small" and "very small" being vaguely defined, because they depend on B_1). So his problem is a particular instance (S_1) of the system (S) stated in the section IV with $b=2B_1$, $x=d$, $X=2^{224}$, $Y=2^{448}$ (and n a 512 bit long integer). As XY is higher than n, we now that (S_1) has solutions. In addition, XY being <u>much</u> higher than n, we are almost sure that some of them can be found with the EEA applied to b and n.

So a simple algorithm to attack the DP scheme is:

(1): let $b = 2B_1 \pmod n$ and $H_1 = B_1^2 \pmod n$
(2): apply the EEA to (n,b); the i-th step is $l_in + m_ib = r_i$
(3): at each step of (2), let $B'_1 = B_1 + m_i$; $H'_1 = B'^2_1 \pmod n$;
$B'_2 = H_1 \ (+) \ H'_1 \ (+) \ B_2$
(4): if B'_1 and B'_2 are valid then output (B'_1,B'_2)

Such an algorithm, applied to some (n,B_1,B_2) picked at random, made us able to find 121 fraudulent pairs (B'_1,B'_2). It is enough to prove the weakness of the DP scheme. But in reality, the number of pairs which could be calculated is far greater, because linear combinations of solutions provided by the EEA are very good candidates to be solutions too, as explained in the following paragraph. Consequently, because the product XY is much higher than n, the number of solutions is finally astronomic, at least about 2^{150} !

Suppose for example that the EEA provided (x_1,y_1) and (x_2,y_2) satisfying the system (S_1) with $x_1 \# 2^{142}$, $y_1 \# 2^{370}$ and $x_2 \# 2^{146}$, $y_2 \# 2^{366}$ (a very plausible situation). Then all the pairs $(u_1x_1 + u_2x_2, u_1y_1 + u_2y_2)$ with $abs(u_1)$ and $abs(u_2) < 2^{77}$ satisfy (S_1) and therefore probably

provide 2^{146} fraudulent pairs (B'_1,B'_2) to the enemy, who did not request so many!

Now, the attack has been described only with an initialization vector equal to zero. If I is supposed to be any natural integer smaller than n, the algorithm is modified as follows:

(0,5): let $C_1 = I$ (+) B_1
(1) : let $b = 2C_1 \pmod n$ and $H_1 = C_1^2 \pmod n$
(2) : as above
(3) : at each step, let $C'_1 = C_1 + m_i$; $H'_1 = C'^2_1 \pmod n$;
$B'_2 = H_1$ (+) H'_1 (+) B_2
(3,5): let $B'_1 = I$ (+) C'_1
(4) : as above.

The trials that we made show that this algorithm is in average as effective as the previous one.

At last, it is easy to see that not only the first two blocks of a message M can be modified in an undetected manner. For example, the blocks B_i and B_{i+1} can be successfully altered by replacing $(B_1,C_1,H_1,B_2,B'_1,C'_1,H'_1,B'_2$ and I) with, respectively, $(B_i,C_i,H_i,B_{i+1},B'_i,C'_i,H'_i,B'_{i+1}$ and $H_{i-1})$ in the second algorithm.

VI IMPROVEMENTS OF THE DP-SCHEME

Many variations to the DP-scheme may be envisioned in order to make it stronger.

An exponent higher than 2 can be chosen. But it has a direct consequence on the performance (execution time) of the function. For this reason, the better choice seems to be a power of 2.

The redundancy length can be increased. If u redundancy bits are required (instead of 64), the system (S_1) becomes the system (S_2) with $X=2^v$ and $Y=2^{2v}$, where $v=(512-u)/2$. If XY becomes close to n, we know that the success of our attack is no more guaranteed. This corresponds to $3v=512$, i.e. about $u=v=170$. This suggests that a minimal number of redundancy bits be 256 (in order to preserve a safety margin).

The redundancy can be placed elsewhere. For example, the additional bits could be placed on the right of the block instead of on the left. But this suggestion is not satisfactory, because the attack described in section III can be adapted to such a case, as shown now:

Let us call "valid" a 512 bit block with the 64 l.s.b. (least significant bits) equal to zero. As in III, and for simplification, we only show here that a CLB attack is possible with k=q=2 and I=0. So, B_1

and B_2 being valid, we have to find valid blocks B'_1 and B'_2 leading to the same hash-result. The problem is hence to find a valid B'_1 such that H_1 and H'_1 have the same 64 l.s.b. . If we succeed, then $B'_2 = H_1$ (+) H'_1 (+) B_2 will be valid and the CLB attack will work.

In order to find such a B'_1, let d denote the small integer such that $B'_1 = B_1 + 2^{64}d$. H'_1 is equal to $[B_1^2 + 2^{64}(2B_1d + 2^{64}d^2)] \bmod n$ and will be as wanted if (but not only if) $2B_1d$ and $2^{64}d^2 \pmod n$ are both small (approximatively smaller than 2^{448}). Now, $2^{64}d^2$ will be small if (but not only if) d is itself particularly small (approximatively smaller than 2^{192}).

So the problem is now reduced to solving a particular instance (S_2) of the problem (S) stated in the section IV, with $b=2B_1$, $X=2^{192}$, $Y=2^{448}$ (and n a 512 bit integer). As XY>>n, we know that there are a lot of solutions, and the enemy will discover a lot of false messages. Let us just remark that the attack stops to be effective if the number of redundancy bits is 128 or more (instead of 170 for the original DP scheme).

So, if we think to move the additional bits, a better idea consists in placing them ...in the middle of the block! In that way, the attack seems to be no more effective. A still better idea consists in dispersing them in the block (for example: a few on the left, a few on the right, and the rest in the middle).

By combining two of the above suggestions (explicitly: 256 redundancy bits, which are regularly dispersed in the blocks), one obtains a hash-function proposed by the OSIS-TeleTrusT project [10] for standardization as an annex in CCITT/SG VII [14] and ISO TC97/SC21 [15]. Again, it is a type 1 scheme with a number of redundancy bits equal to the half of the modulus size, these bits being equal to 1, and placed four at the time.

Precisely, and assuming a modulus size equal to 512, the message M is first divided into 256 bit blocks B_i. Then, each octet of B_i is split in half, each of the halves being headed by binary ones (moreover, the first octet of block B_1 contains the low order octet of the length of the message and if the last block is incomplete, it is padded with "1"s).

Of course, such a hash-function is certainly robust enough to resist to any variation of our attack. But it is perhaps still possible to improve it (at least theoretically). For example, redundancy quartets could be dependent on the significant quartets they are associated to, instead of being fixed. Such a choice would make the enemy's work still more difficult, if it is possible.

VII OTHER MODULO-N SCHEMES

It has been told in section II that the type 1 schemes are viable only if some redundancy is inserted in the blocks. The sections which followed were only a discussion about the nature of this redundancy. But, whatever the choice may be, introduction of redundancy always implies an increase of the execution time. Particularly if there are as many redundancy bits as significant ones.

So, a question is: can we design secure schemes which do not impose redundancy to the blocks of the messages to be hashed? It seems as if the answer was not so easy. For example, in [16], the following scheme is proposed, very close to a type 1 scheme:

$$H_0 = I$$
$$H_i = ((H_{i-1} \ (+) \ B_i)^2 \mod n) \ (+) \ B_i \qquad (1 \leq i \leq k)$$

Unfortunately, there is (at least) one weakness in this proposal, due to the fact that the operation "squaring modulo" has fixed points (0, 1 and perhaps others). If P is such a point, it is clear that we can insert the block $H_i(+)P$ between B_i and B_{i+1} without modifying the hash-result.

Another possibility, quite different, consists in squaring the blocks before adding them to the current hash-result:

$$H_0 = I$$
$$H_i = [H_{i-1} \ (+) \ B_i^2 \ (\mod n)]^2 \mod n \qquad (1 \leq i \leq k)$$

But there is no gain in execution time, compared to the TeleTrusT scheme.

VIII CONCLUSION

This paper dealt with hash-functions using modulo-n operations. Two types have been presented, which are viable only if redundancy is inserted in the blocks of the message to be hashed. A proposal of the first type, by D.W. Davies and W.L. Price, was considered. It has been shown that an enemy can use the Extended Euclidean Algorithm to find more than about 2^{150} substitutes to a message of length superior to 448 bits, without modifying the hash-result. Some variations of this proposal have been analysed, in particular the OSIS-TeleTrusT scheme which seems very solid (but perhaps perfectible). Other schemes, which do not use redundancy, have been also presented and criticized.

REFERENCES

[1] Pinkas, D., "The need for a Standardized Compression Algorithm for Digital Signatures", presented at Eurocrypt 86.

[2] Winternitz, R.S., "Producing a One-Way Hash Function from DES", Advances in Cryptology, Proceedings of Crypto 83, ed. D.Chaum, Plenum Press, New York, 1984, pp.203-207.

[3] Denning, D.E., "Digital Signatures with RSA and Other Public-Key Cryptosystems", CACM, Vol. 27, N°4, April 84, pp. 388-392.

[4] Rabin, M., "Digital Signatures", Foundations of Secure Computation, Academic Press, New York, 1978.

[5] Davies, D.W., "Applying the RSA Signature to Electronic Mail", Computer, 1983.

[6] Akl, S.G., "On the Security of Compressed Encodings", Advances in Cryptology, Proceedings of Crypto 83, ed. D.Chaum, Plenum Press, New York, 1984, pp.209-230.

[7] Coppersmith, D., "Another Birthday Attack", Advances in Cryptology, Crypto '85 Proceedings, Lecture Notes in Computer Science, Vol.218, Springer-Verlag, Berlin, 1986, pp. 14-17.

[8] Davies, D.W. and Price, W.L., "Digital Signatures, an update", Proceedings of the International Conference on Computer Communications, Sydney, October 1984, pp. 845-849.

[9] "Modes of Operations and Hash-functions", ISO/TC97/SC20/WG2/N31, July 1985.

[10] "OSIS Security Aspects", OSIS European Working Group WG1, Final report, October 1985.

[11] Jueneman, R.R., Matyas, S.M. and Meyer, C.H., "Message Authentication with Manipulation Detection Codes", Proceedings of Security & Privacy, 1983, IEEE Catalog N° 83CH1882-0, pp. 33-54.

[12] De Jonge, W. and Chaum, D., "Attacks on some RSA Signatures", Advances in Cryptology, Crypto '85 Proceedings, Lecture Notes in Computer Science, Vol.218, Springer-Verlag, Berlin, 1986, pp. 18-27.

[13] Riesel, H., "Prime numbers and computer methods for factorization", Progress in Mathematics, vol.57, 1985, Birkaüser Ed., pp.300-317.

[14] "The Directory-Authentication Framework", CCITT/SGVII/Q35 Draft Recommendation X 509, June 1987.

[15] "The Directory-Authentication Framework", ISO/TC97/SC21 DP 9594/8, June 1987.

[16] Pailles, J.C. and Girault, M., "The security processor CRIPT", Preprints of the 4th IFIP'SEC, Monte-Carlo, December 2-4, 1986, pp.127-139.

Blinding for Unanticipated Signatures

David Chaum

Centre for Mathematics and Computer Science
Kruislaan 413 1098 SJ Amsterdam

Summary

Previously known blind signature systems require an amount of computation at least proportional to the number of signature types, and also that the number of such types be fixed in advance. These requirements are not practical in some applications. Here, a new blind signature technique is introduced that allows an unlimited number of signature types with only a (modest) constant amount of computation.

Background

The notion of "blind signatures" was first introduced in [Chaum 82]. Such systems typically have a party called the *signer* who is able to make certain digital signatures. The other parties, called *providers*, would like to obtain such signatures on messages they provide to the signer. What makes it interesting is that the providers wish to keep the signer in the dark as much as possible. Since the signatures will in general ultimately be shown widely, it is assumed that the signer will learn them. What providers can keep from the signer, however, is the exact correspondence between the actual signing operations performed by the signer and the signatures which are later made public. This essential property of blind signatures is called "unlinkability."

Blind signatures can be made using the RSA public key signature system [Rivest, Shamir & Adleman 78]. As is usual for such systems, the *signer* chooses two appropriate large primes p and q, and makes their product n public. In a slight generalization, which will be important later, the signer then issues l "public exponents" $e_1, ..., e_l$. The signer further computes corresponding "secret exponents" $d_1, ..., d_l$ satisfying $d_i \equiv e_i^{-1} \pmod{(p-1)(q-1)}$, $1 \leqslant i \leqslant l$. The signature of "type" i is formed on a number m by the signer as $m'_i \equiv m^{d_i} \pmod{n}$. Anyone can use the public n and e_i to verify the signature of type i on m by checking that $m \equiv (m'_i)^{e_i} \pmod{n}$ holds.

The first actual realization of a blind signature system [Chaum 85] worked as follows: The provider "blinds" a message m, using a key k chosen uniformly from $\{1,..,n\}$, producing $t \equiv [m]k^{e_i} \pmod{n}$, where the square brackets indicate the input to

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 227-233, 1988.
© Springer-Verlag Berlin Heidelberg 1988

the transformation. The signing of t yields $t' \equiv [mk^{e_i}]^{d_i} \equiv m^{d_i}k \pmod{n}$. Then the provider "unblinds" this returned t' by forming $m'_i \equiv [m^{d_i}k]k^{-1} \equiv m^{d_i} \pmod{n}$. It is easy to see informally why the unlinkability property mentioned above holds. The signer knows two sets: the set of signed blinded messages t' (or equivalently to the signer, t) and the set of signed messages m'_i (or equivalently, m_i). For any particular t' and m'_i that the signer suspects might correspond, the signer is able to determine exactly one $k \equiv t'm'^{-1}_i \pmod{n}$ that would have been the provider's secret blinding key if they did in fact correspond. But since the k's are chosen independently and uniformly by the providers, the signer learns nothing about whether any guessed correspondence holds.

Notice that a provider must "anticipate" the particular d_i that will be used by the signer. It is possible, though computationally expensive, for the provider to actually anticipate a few d_i at once by forming $t \equiv mr^{e_1e_2} \pmod{n}$ for example, and being able to unblind in case of signature with d_1 or d_2 by forming $m'_1 \equiv (mr^{e_1e_2})^{d_1} r^{-e_2} \pmod{n}$ or $m'_2 \equiv (mr^{e_1e_2})^{d_2} r^{-e_1} \pmod{n}$, depending on whether d_1 or d_2 was used to sign, respectively. But such an approach becomes prohibitively computation intensive as the number of alternatives increases: in general it requires the provider to perform more than one multiplication for each alternative anticipated (since each e_i should have a distinct prime factor, otherwise some signatures can be made from others). Such effort required to anticipate all possible signatures may not be practical, and is also undesirable because the maximal extent of a system has to be fixed initially and effort proportional to this limiting size must be carried out during every blinding. Such an approach of course becomes impossible in practice when the number of alternatives is too large or when the alternatives are not known in advance of blinding.

The ability to anticipate a large number of signature types can benefit the payment system described in [Chaum 85]. This would allow customers of the bank providing a system to each supply a large number of blinded items when their accounts are opened, without the customers knowing in advance which particular type of signature will later be applied by the bank. Not only can this provide economy of data transfer, but it protects the bank's customers from being able to (and hence from being coerced into) making payments that they cannot later trace. When the bank ultimately issues signed notes, its choice of signature type may be used, for example, to encode denominations and expiration dates. Another application of blind signatures is to credential systems. In these, each different kind of credential a person receives is encoded as a digital signature of a corresponding type formed on a blinded copy of the person's "digital pseudonym" used with the signer [Chaum 85; Chaum & Evertse 86]. There may be a great many different kinds of basic and more detailed credentials, and the future requirement for such credentials could be difficult to anticipate. Thus there appears to be a substantial need for blind signatures that are unblindable even after an unanticipated signature type has been applied.

Blind unanticipated signature protocols

The new blind signature protocols presented here are based on the use of certain units (of the residue classes modulo N) called *generators*. These will be made public in advance, and their signatures will also be made public by the signer. Unlinkability requires that the group they generate contains all messages m (otherwise certain linkings are easily ruled out because there is no k capable of causing them). Depending on how this requirement is ensured, as will be described later, the blind unanticipated signature protocol will have one of three forms.

In the first and simplest form of these protocols, the following congruences hold:

$$t \equiv [m]g^k \pmod{n}$$

$$t' \equiv [mg^k]^{d_i} \pmod{n}$$

$$m' \equiv [(mg^k)^{d_i}]g^{-d_i k} \pmod{n},$$

where: m is the message to be blinded; t is the blinded form of the message; t' the signature computed for t; m' the unblinded signed m; g the generator; n the publicly known modulus; e_i and d_i the public and private signature exponents, respectively; and k the provider's secret blinding key, say chosen uniformly from $\{1, \cdots, n^2\}$. The square brackets again show the input to the transformation whose output is shown on the left-hand-side, and thus they define the function of each of the three transformations in the order shown: blinding, signing, and unblinding.

In the second form of the protocol, the following congruences hold:

$$t \equiv [m]g^{k_1}k_2 \pmod{n}$$

$$t' \equiv [mg^{k_1}k_2]^{d_i} \pmod{n}$$

$$m' \equiv [(mg^{k_1}k_2)^{d_i}]g^{-d_i k_1}k_2 \pmod{n},$$

where blinding key k_1 is chosen as k was above, but blinding key k_2 is chosen uniformly only from the pair $\{1, n-1\}$.

In the third and most general case, the following congruences hold:

$$t \equiv [m]g_1^{k_1} g_2^{k_2} ... g_r^{k_r} (\text{mod } n)$$

$$t' \equiv [mg_1^{k_1} g_2^{k_2} ... g_r^{k_r}]^{d_i} (\text{mod } n)$$

$$m' \equiv [(mg_1^{k_1} g_2^{k_2} ... g_r^{k_r})^{d_i}] g_1^{-d_i k_1} ... g_r^{-d_i k_r} (\text{mod } n),$$

where each k_i is a secret blinding key chosen independently as k was above.

Testing a generator knowing the factorization

If generators for the whole group are known, then unlinkability can be maintained without any restriction on the messages that can be signed. Having the factorization of $p-1$ and $q-1$ allows efficient verification that a particular proposed set of generators do in fact generate the whole group of units modulo n. (Of course these factorizations also allow p and q to be computed easily.) Suppose, for example, that GCD$(p-1, q-1)=2$ and $q \equiv 3$ (mod 4) (which is quickly checked by comparing the factors of $p-1$ and $q-1$) and that g and -1 are to be verified as generating the whole group (so that the second protocol above can be used). Then it is sufficient to verify that the order of g is maximal modulo one factor of n, say p, and that it generates exactly half the elements modulo the other. This is readily accomplished: first raise g to $p_l^{-1}(p-1)$ and ensure that no l, $1 \leqslant l \leqslant k$, gives the result 1, where the p_i, $1 \leqslant i \leqslant k$, are the prime divisors of $p-1$; perform the same procedure also for the odd primes dividing q.

Hiding the factorization with physical security

The above approach works fine if you know the factors, but how can it be used by the signer to convince providers without revealing the factorization? A possible solution allows anyone to submit to the signer a physical device for conducting the above test. The signer would give the secret parameters to the device and allow the message containing the logical result ("verifies" or "does not verify") to be sent by the device to its supplier. This should be done in a way that convinces the supplier that the signer cannot falsify the message, while not allowing the device to leak the secret parameters. One way to arrange this is as follows: The supplier creates a number c' at random, applies a publically known and agreed on one-way function f, which is preferably one-to-one, to c', yielding $c = f(c')$; installs c' and n in the device; and gives c to the signer along with the device. Then the signer isolates the device from the supplier and feeds it the secret parameters (p and q). Only if the result is "verifies" does the device output c'. The signer then applies f to the output of the device, and checks whether the result is c, and if

so returns this c' to the supplier. The device is constructed to be tamper resistant enough to make it sufficiently difficult for the signer to obtain c' from it in the expected time interval, unless the generator tester yields "verifies."

Linear cut-and-choose on moduli and generators

In another approach, a prospective signer announces moduli n_i, $1 \leqslant i \leqslant k$, and corresponding generators g_i for each (the "cut"). Others determine the single modulus n among these that will be used (the "choose"), whereupon the signer must make public the factorization of the $p_l - 1$ and the $q_l - 1$, $1 \leqslant l \leqslant j$ and $l \neq i$, for all j moduli except the ith one selected. Then anyone can use the verification techniques described above (which rely on possession of the secret) to verify all the other moduli and corresponding generators, thereby obtaining the probability $1 - j^{-1}$ that the selected modulus would also verify. Such techniques can of course also be used in a great many situations, as is well known in the folklore. One way to select i would be by a public coin-tossing event. Another way to determine i requires each of some set of persons to form a b' at random and a corresponding $b = f(b')$ using a preferably bijective, public one-way function f. Then each participant makes public their b. Once every b is public, the b' are revealed, checked, and added modulo the number of moduli j, yielding the index i. A disadvantage of these techniques, compared to the previous approach based on physical devices, is that they are not "open" to new participants once the cut-and-choose is completed.

Generators not manipulable by the signer

Yet another way for providers to gain confidence in the suitability of generators is for the generators to be determined in a way that cannot easily be manipulated by anyone including the signer. Generators could be chosen by a random process, such as those just described for selecting a particular modulus. Use of a single such randomly chosen and untested generator might not give a high enough probability of providing adequate unlinkability; use of several randomly chosen generators improves the degree of hiding and unlinkability. For instance, it can be shown that use of 22 generators provides a probability of roughly one-millionth that not all blinding factors in the reduced residue system modulo n are generated. When additional things are known about the structure of n, the probabilities improve greatly and less generators need be used. Examples include: when n has exactly two prime factors each congruent to 3 modulo 4 (which can be demonstrated efficiently by a protocol presented by [Peralta & v.d. Graaf 87]) and / or when $(p-1)(q-1)$ has no small odd divisors (as is easily checked by asking the signer to sign random values using signature types with small prime e_i). A further variation chooses subsets of a set of generators in a key-dependent way.

Ensuring that messages blinded are powers of g

This final approach does not require verification or special selection of the generator(s): protection against linking is provided by almost any generator(s). Security derives from testing whether the thing to be signed, m, is a member of the group generated by the generator(s). This can be used when the signer itself forms m as $m \equiv g^x \pmod{n}$, where x is chosen at random by the signer (such as is the case in the setting of [Chaum & Evertse 86]). Then the signer can participate in a protocol to convince the provider that some x satisfying the congruence is known to the signer. The first protocols for demonstrating such "possession of a discrete log" are presented in [Chaum, Evertse, v.d. Graaf & Peralta 86], and a potentially more efficient version allowing many m's to be verified at once is presented in [Chaum, Evertse & v.d. Graaf 87].

Unlinkability

It seems that all blinding factors cannot be equally likely, since otherwise the provider would have to choose the exponents from a distribution depending exactly on $\mathrm{LCM}(p-1, q-1)$. But it is easy to see that choosing the k's uniformly from $\{1, \cdots, n^2\}$ for instance gives a maximum difference in the probability of any two that is less than n^{-2}, which would be quite adequate in practice.

Open problems

A much cleaner and more general approach might be found. Notice, for example, that if $(p-1)/2$ and $(q-1)/2$ are primes, then any g with Jacobi symbol -1 (which can easily be checked knowing only n) together with -1 generates the whole group (apart from a few exceptional cases that immediately reveal the factorization of n). Thus, a protocol allowing anyone interacting with the signer to check that n is of such a form without learning its factorization allows generators of the whole group to be verified directly. Any such open protocol for establishing a pair of generators (preferably one of which is -1) would of course be quite desirable.

Acknowledgements

I am indebted to Jan-Hendrik Evertse for substantial collaboration in related areas during the time that the basic results presented here were developed. Subsequently, Jan-Hendrik Evertse, Jeroen van de Graaf, and René Peralta worked with the author to find improved techniques. Final revisions also benefited from the comments of Jurjen Bos, Bert den Boer and Gilles Brassard.

References

(1) Chaum, D., "Blind signatures for untraceable payments," *Proceedings of Crypto 82,* D. Chaum, A. Sherman & R. Rivest, Eds., Plenum 1983, pp. 199-203.

(2) Chaum, D., "Security without identification: transaction systems to make big brother obsolete," *Communications of the ACM,* 28, 10 (October 1985), pp. 1030-1044.

(3) Chaum, D. & Evertse, J.-H., "A secure and privacy-protecting protocol for transmitting personal information between organizations," *Proceedings of Crypto 86,* A.M Odlyzko Ed., Springer-Verlag 1987, pp. 118-167.

(4) Chaum, D., Evertse, J.-H., van de Graaf, J., & Peralta, R., "Demonstrating possession of a discrete logarithm without revealing it," *Proceedings of Crypto 86,* A.M Odlyzko Ed., Springer-Verlag 1987, pp. 200-212.

(5) Chaum, D., Evertse, J.-H. & van de Graaf, J., "An improved protocol for demonstrating possession of a discrete logarithm and some generalizations," *Proceedings of Eurocrypt 87.*

(6) van de Graaf, J. & Peralta, R., "A simple and secure way to show the validity of your public key," to appear in *Proceedings of Crypto 87.*

(7) Rivest, R., Shamir, A. & Adleman, L. "A method for obtaining digital signatures and public-key cryptosystems" *Communications of the ACM,* 21, 2, (February 1978), pp. 120-126.

NON-EXPANDING, KEY-MINIMAL, ROBUSTLY-PERFECT, LINEAR AND BILINEAR CIPHERS

James L. Massey, Ueli Maurer & Muzhong Wang

Institute for Signal and Information Processing
Swiss Federal Institute of Technology
CH-8092 Zürich, Switzerland

1. INTRODUCTION

The purposes of this paper are: (1) to give appropriate definitions of robustly-perfect ciphers, linear ciphers and bilinear ciphers; (2) to give two general constructions of robustly-perfect bilinear block ciphers that do not expand the plaintext and that have the smallest possible amount of secret key; (3) to give some isolated examples of robustly-perfect linear stream ciphers that use less key than had been earlier conjectured to be necessary; and (4) to suggest some possible useful applications for robustly-perfect linear and bilinear ciphers.

Section 2 introduces the notion of a robustly-perfect block cipher and shows the connection of such ciphers to Latin squares. Linear and bilinear block ciphers are defined in Sections 3 and 4, respectively. Two general constructions of non-expanding, key-minimal robustly-perfect bilinear ciphers are also given in Section 4, and some as-yet-unanswered questions about such ciphers are raised. Section 5 gives a tentative general definition of a linear stream cipher, and exhibits some counterexamples to a conjecture by Massey and Rueppel on the amount of key required in such ciphers. Finally, Section 6 suggests some possible practical applications for robustly-perfect linear and bilinear ciphers and points out some further open questions about such ciphers.

2. KEY-MINIMAL ROBUSTLY-PERFECT BLOCK CIPHERS

In a deterministic secret-key cipher, the ciphertext Y can be written in terms of the plaintext X and the key Z in the manner

$$Y = f(X, Z)$$

where f is a mapping from $S_x \times S_z$ to S_y where S_x is the set of allowed plaintexts, S_z is the set of allowed keys and S_y is the set of possible ciphertexts. The probability distribution P_X of X is determined by the plaintext source, but P_Z is chosen by the cryptographer. It is always assumed that X and Z are statistically independent. The pair (f, P_Z) can be called the cipher system. The only essential requirement on f is that the enciphering must be decipherable by one who knows the key, i.e., $f(x_1, z) = f(x_2, z)$ must imply $x_1 = x_2$.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 237-247, 1988.
© Springer-Verlag Berlin Heidelberg 1988

Until further notice, we shall be concerned only with block ciphers, which can be defined as ciphers for which S_x, S_y and S_z are all finite sets.

Shannon [1, p.679] has defined a cipher system (f, P_Z) to be perfect if X and Y are statistically independent. In this definition, P_X is assumed to be specified. It is natural to extend Shannon's definition and to say that the cipher system (f, P_Z) is robustly perfect if it is perfect for all probability distributions P_X on the set S_x.

The Vernam cipher or "one-time pad", in which $S_x = S_y = S_z = \{0, 1, 2, ..., m-1\}$,

$$Y = f(X, Z) = X \bigoplus Z,$$

where the addition is modulo m, and P_Z is the uniform distribution on $\{0, 1, ..., m-1\}$, is robustly perfect. This follows from the fact that $P_{Y|X}(j|i) = \frac{1}{m}$ for all i and j, and hence $P_{Y|X}(j|i) = P_Y(j) = \frac{1}{m}$ independently of the choice of P_X.

Shannon [1, p.681] has shown that, in a perfect cipher system, the number of keys $\#(S_z)$ must be at least as great as the number of plaintexts with non-zero probability. Thus, a robustly-perfect cipher system must have $\#(S_z) \geq \#(S_x)$. We shall say that a robustly-perfect cipher system is key-minimal if $\#(S_z) = \#(S_x)$. A block cipher is non-expanding if $\#(S_y) = \#(S_x)$. The one-time pad is thus seen to be a non-expanding key-minimal robustly-perfect cipher system.

Shannon [1, p.681] has stated a proposition equivalent to the following:

Proposition 1: *A cipher system (f, P_Z) with $\#(S_x) = \#(S_y) = \#(S_z)$ is robustly-perfect if and only if (1) P_Z is the uniform distribution on S_z and (2) for every x and y in S_x and S_y, respectively, there is a unique z in S_z such that $y = f(x, z)$.*

Shannon observed that condition (2) of the above proposition shows that the essential feature of a non-expanding key-minimal robustly-perfect cipher is that, with the rows indexed by the plaintexts and the columns indexed by the keys, the array of corresponding ciphertexts forms a Latin square. A Latin square [2] (of size $n \times n$) is an $n \times n$ array, whose entries are drawn from an alphabet of n letters, with the property that each letter of the alphabet appears exactly once in each row and once in each column. The name "Latin square" comes from the fact that their first systematic investigator, L. Euler, chose the first n Latin letters as his alphabet, but most investigators today use the alphabet $\{1, 2, ..., n\}$. The following two examples of 7×7 Latin squares will be of interest in Section 3:

1	2	3	4	5	6	7
2	3	4	5	6	7	1
3	4	5	6	7	1	2
4	5	6	7	1	2	3
5	6	7	1	2	3	4
6	7	1	2	3	4	5
7	1	2	3	4	5	6

1	2	3	4	5	6	7
4	1	5	6	2	7	3
5	3	6	2	7	1	4
6	4	2	7	1	3	5
7	6	1	3	4	5	2
2	5	7	1	3	4	6
3	7	4	5	6	2	1

If we take $S_x = S_y = S_z = \{1, 2, ..., 7\}$, then the second of these Latin squares specifies, for instance, that the ciphertext y corresponding to the plaintext $x = 3$ and the key $z = 5$ can be found in the third row and fifth column of this array and is thus $y = 7$, i.e. $f(3, 5) = 7$.

Because P_Z must be the uniform distribution in a non-expanding, key-minimal, robustly-perfect (NEKMRP) cipher system (f, P_Z), we can and will speak of the cipher f alone as NEKMRP. It is interesting to note that if the roles of the plaintext x and the key z are interchanged in an NEKMRP cipher, then the new cipher is also NEKMRP, as follows from the fact that the transpose of a Latin square is another Latin square. But x and y can also be interchanged in an NEKMRP cipher and the new cipher remains NEKMRP, as follows immediately from Proposition 1. Thus there is a complete interchangeability of variables in an NEKMRP cipher $y = f(x, z)$ in the sense that any one of x, y and z can be taken as the "key" and either of the two remaining quantities can be taken as the "plaintext". This interchangeability is quite obvious for the one-time pad, but rather less so for the NEKMRP ciphers that will be given later.

3. LINEAR BLOCK CIPHERS

The notion of a "linear" cipher requires that the plaintext set, S_x, and ciphertext set, S_y, be vector spaces. The notion of a "block" cipher (to which our discussion is still restricted) demands that these same sets be finite. Thus, the notion of a "linear block" cipher demands that S_x and S_y be finite-dimensional vector spaces over a finite field.

Here and hereafter, let F denote an arbitrary finite field and let F^n denote the vector space of n-component "column vectors" over F, i.e., vectors of the form $\underline{v} = (v_1, v_2, ..., v_n)$ where $v_i \in F$ for all i. Then a linear cipher can be defined as a cipher f such that $S_x = F^m$, $S_y = F^n$, and, for each $z \in S_z$,

$$f(c_1\underline{x}_1 + c_2\underline{x}_2, z) = c_1 f(\underline{x}_1, z) + c_2 f(\underline{x}_2, z) \tag{1}$$

for all $c_1 \in F$, $c_2 \in F$, $\underline{x}_1 \in F^m$ and $\underline{x}_2 \in F^m$. A linear cipher is non-expanding if $m = n$.

Our interest will be in "robustly-perfect" linear ciphers. Some subtlety is required for an appropriate definition of a "robustly-perfect" linear cipher, however, for the reason made clear by the following fact.

Proposition 2: *If f is a linear cipher and if the cipher system (f, P_Z) is perfect for the plaintext distribution $P_{\underline{X}}$ (where $P_{\underline{X}}(\underline{0}) \neq 1$ is assumed to avoid trivialities), then $P_{\underline{X}}(\underline{0}) = 0$.*

Proof: Because the cipher system is perfect,

$$P_{\underline{X}\underline{Y}}(\underline{0}, \underline{0}) = P_{\underline{X}}(\underline{0}) P_{\underline{Y}}(\underline{0}).$$

Because the cipher is linear, $P_{\underline{Y}|\underline{X}}(\underline{0}|\underline{0}) = 1$ and thus

$$P_{\underline{X}\underline{Y}}(\underline{0}, \underline{0}) = P_{\underline{Y}|\underline{X}}(\underline{0}|\underline{0}) P_{\underline{X}}(\underline{0}) = P_{\underline{X}}(\underline{0}).$$

Hence it follows that

$$P_{\underline{X}}(\underline{0}) = P_{\underline{X}}(\underline{0})P_{\underline{Y}}(\underline{0}),$$

which, because $P_{\underline{Y}}(\underline{0}) \neq 1$, implies that $P_{\underline{X}}(\underline{0}) = 0$.

Remark: When $P_{\underline{X}}(\underline{0}) = 1$, every cipher is trivially perfect since every $\underline{Y}$ is then statistically independent of $\underline{X}$. Thus, excluding the case $P_{\underline{X}}(\underline{0}) = 1$ excludes only this triviality.

Proposition 2 shows the appropriateness of defining a linear cipher to be robustly-perfect if it is a robustly-perfect cipher for the restricted plaintext set $F^m - \{\underline{0}\}$. In any linear cipher, the required decipherability of the enciphering transformation $\underline{y} = f(\underline{x}, z)$ for every key z is equivalent to the requirement that $\underline{y} = \underline{0}$ implies $\underline{x} = \underline{0}$. Thus, the plaintext restriction to the set $F^m - \{\underline{0}\}$ automatically implies a corresponding ciphertext restriction to the set $F^n - \{\underline{0}\}$.

If $F = GF(q)$, the finite field of q elements, then there are $q^m - 1$ plaintexts in the restricted plaintext set $F^m - \{\underline{0}\}$. A robustly-perfect linear cipher is thus appropriately defined to be key-minimal if also $\#(S_z) = q^m - 1$. It follows that in a non-expanding key-minimal robustly-perfect (NEKMRP) linear cipher, the restricted plaintext set, the restricted ciphertext set, and the key set all have cardinality $q^m - 1$. It thus follows from Proposition 1 that the array of restricted ciphertexts for an NEKMRP cipher, with the rows indexed by the keys and the columns indexed by the restricted plaintexts, forms a $(2^m - 1) \times (2^m - 1)$ Latin square and, conversely, that any linear cipher for which this array is a Latin square becomes an NEKMRP cipher when the keys are chosen to be equiprobable. Because (1) implies that the enciphering transformation $\underline{y} = f(\underline{x}, z)$ of a linear cipher can be written as

$$\underline{y} = M_z \underline{x} \tag{2}$$

where M_z is an $n \times m$ matrix determined by the key z, and because decipherability implies that M_z must have rank m, the Latin square characterization of an NEKMRP linear cipher is equivalent to the following matrix characterization.

Proposition 3: *The linear cipher $\underline{y} = M_z \underline{x}$, where $\underline{x} \in F^m - \{\underline{0}\}$ and $F = GF(q)$, is non-expanding key-minimal and robustly-perfect if and only if (1) there are $q^m - 1$ different keys z and these are chosen to be equiprobable and (2) every matrix M_z is a nonsingular $m \times m$ matrix and, for each $\underline{x} \in F^m - \{\underline{0}\}$ and every $\underline{y} \in F^m - \{\underline{0}\}$, there is exactly one matrix M_z such that $\underline{y} = M_z \underline{x}$.*

The following example shows that NEKMRP linear ciphers exist.

Example 1: $F = GF(2)$, $m = 2$, $S_z = \{1, 2, 3\}$,

$$M_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \qquad M_2 = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad M_3 = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}.$$

For $\underline{x} = (1,0)$, $M_1\underline{x} = (1,0)$, $M_2\underline{x} = (0,1)$ and $M_3\underline{x} = (1,1)$. For $\underline{x} = (0,1)$, $M_1\underline{x} = (0,1)$, $M_2\underline{x} = (1,1)$ and $M_3\underline{x} = (1,0)$. For $\underline{x} = (1,1)$, $M_1\underline{x} = (1,1)$, $M_2\underline{x} = (1,0)$ and $M_3\underline{x} = (0,1)$. Thus, by Proposition 3, this linear cipher is NEKMRP when the keys each have probability 1/3.

This example is the $q = 2$, $m = 2$ special case of the first of two general constructions of NEKMRP linear ciphers that will be given in the next section.

4. BILINEAR BLOCK CIPHERS

In order to define a "bilinear" cipher, we must require that S_z, as well as S_x and S_y, be a vector space. It would seem natural then to define a "bilinear" cipher as a cipher such that (i) $S_x = F^m$, $S_y = F^n$ and $S_z = F^k$, (ii) f satisfies (1), i.e., f is a linear cipher, and (iii) for every $\underline{x} \in F^m$

$$f(\underline{x}, c_1\underline{z}_1 + c_2\underline{z}_2) = c_1 f(\underline{x}, \underline{z}_1) + c_2 f(\underline{x}, \underline{z}_2) \tag{3}$$

for all $c_1 \in F$, $c_2 \in F$, $\underline{z}_1 \in F^k$ and $\underline{z}_2 \in F^k$. However, (3) implies that $f(\underline{x}, \underline{0}) = \underline{0}$ for all $\underline{x} \in F^m$ so that the decipherability condition cannot be satisfied for $\underline{z} = \underline{0}$. Thus, the key $\underline{z}$ must belong to the restricted key set $F^k - \{\underline{0}\}$ in order to have a valid cipher; hereafter, we assume this restriction to hold and we then take conditions (i), (ii) and (iii) above together with the condition (iv) that the mapping $\underline{y} = f(\underline{x}, \underline{z})$ be invertible for every fixed $\underline{z}$, $\underline{z} \neq \underline{0}$, as our definition of a bilinear cipher.

Equations (1) and (3) and condition (iv) imply that the enciphering transformation $\underline{y} = f(\underline{x}, \underline{z})$ of a bilinear cipher can be written as

$$\underline{y} = M_{\underline{z}}\underline{x} \tag{4}$$

where $M_{\underline{z}}$ is an $n \times m$ matrix of rank m when $\underline{z} \neq \underline{0}$ such that

$$M_{c_1\underline{z}_1 + c_2\underline{z}_2} = c_1 M_{\underline{z}_1} + c_2 M_{\underline{z}_2}. \tag{5}$$

But (5) is just the condition that each entry in $M_{\underline{z}}$ where $\underline{z} = (z_1, z_2, ..., z_k)$ be a linear combination of the key components $z_1, z_2, ..., z_k$.

Example 2: $q = 2$, $m = n = k = 2$,

$$M_{\underline{z}} = \begin{bmatrix} z_1 & z_2 \\ z_2 & z_1 + z_2 \end{bmatrix}$$

defines a bilinear cipher because

$$M_{(1,0)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad M_{(0,1)} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \qquad M_{(1,1)} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

all have rank 2, i.e., are all nonsingular matrices. We see in fact that this bilinear cipher is precisely the NEKMRP linear cipher of Example 1 with an obvious mapping between the key space $F^2 - \{\underline{0}\}$ and the key set $\{1, 2, 3\}$.

For any $F = GF(q)$ and any m, an NEKMRP bilinear cipher may be constructed as follows.

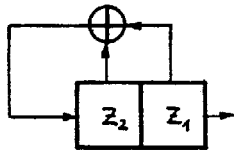
Construction 1:

(1) Let $(z_1, z_2, ..., z_m)$ be the initial state of an m-stage q-ary maximal-length (ML) linear feedback shift-register (LFSR), i.e., an LFSR whose output sequence has period $q^m - 1$ when started in any nonzero initial state.

(2) Compute symbolically each of the next $m-1$ digits $z_{m+1}, z_{m+2}, ..., z_{2m-1}$ of the LFSR output sequence as a linear combination of the variables $z_1, z_2, ..., z_m$.

(3) Choose the i-th column of $M_{\underline{z}}$ to be the column vector $(z_i, z_{i+1}, ..., z_{i+m-1})$ where each z_j for $j > m$ is written as the linear combination determined in step (2).

Example 3: The two-stage binary LFSR

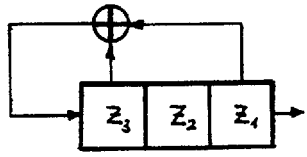


is an MLLFSR such that $z_3 = z_1 + z_2$. Thus, Construction 1 uses the sequence $(z_1, z_2, z_3) = (z_1, z_2, z_1 + z_2)$ to obtain the matrix

$$M_{\underline{z}} = \begin{bmatrix} z_1 & z_2 \\ z_2 & z_1 + z_2 \end{bmatrix},$$

which is that of the NEKMRP bilinear cipher of Examples 1 and 2.

Example 4: The three-stage binary LFSR



is an MLLFSR such that $(z_1, z_2, z_3, z_4, z_5) = (z_1, z_2, z_3, z_1 + z_3, z_1 + z_2 + z_3)$. Construction 1 thus gives the bilinear cipher with

$$M_{\underline{z}} = \begin{bmatrix} z_1 & z_2 & z_3 \\ z_2 & z_3 & z_1 + z_3 \\ z_3 & z_1 + z_3 & z_1 + z_2 + z_3 \end{bmatrix}.$$

The reader is invited to check that with the mapping from the binary vector (c_1, c_2, c_3) to the integer $c_1 + 2c_2 + 4c_3$, this bilinear cipher corresponds exactly to the second of the two 7×7 Latin squares exhibited in Section 2, and is thus an NEKMRP cipher when the 7 keys are chosen to be equiprobable.

That the bilinear cipher produced by Construction 1 is indeed NEKMRP (when the $q^m - 1$ keys are equally likely) can be seen as follows. In one period of the output sequence of an m-stage q-ary MLLFSR, the state of the shift-register takes on each of the $q^m - 1$ possible non-$\underline{0}$ values exactly once. But the columns of $M_{\underline{z}}$, by construction, consist of m consecutive states of the MLLFSR. For any fixed $\underline{x} \neq \underline{0}$, by the well-known shift-and-add property of ML sequences, cf. [3, Theorem 4.3], $M_{\underline{z}}\underline{x}$ is a state at a fixed number of shifts later on the same MLLFSR state sequence as the state $(z_1, z_2, ..., z_m)$ corresponding to the first column of $M_{\underline{z}}$. Thus, as $\underline{z}$ runs over all values in $F^m - \{\underline{0}\}$, so also does $M_{\underline{z}}\underline{x}$. Thus, for each $\underline{x} \neq \underline{0}$ and every $\underline{y} \neq \underline{0}$, there is exactly one choice for

$\underline{z} = (z_1, z_2, ..., z_m)$ such that $\underline{y} = M_{\underline{z}}\underline{x}$. That this bilinear cipher is thus NEKMRP now follows from Proposition 3.

Because the finite field $GF(q^m)$ can be considered as a vector space of dimension m with respect to the "scalar" field $F = GF(q)$ [4, p.54], the elements of $GF(q^m)$ can be represented as vectors in F^m in the manner that if β_1 and β_2 in $GF(2^m)$ are represented by $\underline{v}_1$ and $\underline{v}_2$, respectively, in F^m, then $\beta_1 + \beta_2$ is represented by $\underline{v}_1 + \underline{v}_2$. If α is an element of $GF(q^m)$ whose minimal polynomial over $GF(q)$ has degree m, then a commonly used such representation of $GF(q^m)$ takes the representation of the so-called "canonical basis" $1, \alpha, \alpha^2, ..., \alpha^{m-1}$ in $GF(q^m)$ to be the "unit vectors" $(1,0,0,...,0)$, $(0,1,0,...,0)$, ... , $(0,0,0,...,1)$.

Example 5: Let α be an Element of $GF(2^2)$ whose minimal polynomial over $GF(2)$ is X^2+X+1, i.e., $\alpha^2+\alpha+1 = 0$ and thus $\alpha^2 = \alpha + 1$. Representing 1 and α by $(1,0)$ and $(0,1)$, respectively, the multiplication rule for $GF(2^2)$ is readily checked to be

$$(x_1 + x_2\alpha)(z_1 + z_2\alpha) = (x_1z_1 + x_2z_2) + (x_1z_2 + x_2z_1 + x_2z_2)\alpha,$$

which can be written more suggestively as

$$(x_1, x_2) \times (z_1, z_2) = (x_1z_1 + x_2z_2, x_1z_2 + x_2z_1 + x_2z_2).$$

For any $F = GF(q)$ and any m, an NEKMRP bilinear cipher may be constructed as follows.

Construction 2:

(1) Take $\underline{x} = (x_1, x_2, ..., x_m)$, $\underline{y} = (y_1, y_2, ..., y_m)$ and $\underline{z} = (z_1, z_2, ..., z_m)$ to be the representations of non-zero elements x, y and z, respectively, of $GF(q^m)$ as described above.

(2) Take the ciphering rule $\underline{y} = f(\underline{x}, \underline{z})$ to correspond to the $GF(q^m)$ equation

$$y = xz. \tag{6}$$

Example 6: Taking $F = GF(2)$, $m = 2$, and using the representation of Example 5, Construction 2 gives the bilinear cipher

$$(y_1, y_2) = (x_1z_1 + x_2z_2, x_1z_2 + x_2z_1 + x_2z_2) = \begin{bmatrix} z_1 & z_2 \\ z_2 & z_1 + z_2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix},$$

which we now recognize to be (accidentially) the NEKMRP cipher of Example 4.

That Construction 2 always gives an NEKMRP bilinear cipher can be seen as follows. From (6) and the fact that

$$(x_I + x_{II})z = x_Iz + x_{II}z$$

and

$$x(z_I + z_{II}) = xz_I + xz_{II},$$

it follows that the cipher is indeed bilinear. For each $x \in GF(q^m) - \{\underline{0}\}$ and every $y \in GF(q^m) - \{\underline{0}\}$, there is a unique $z \in GF(q^m) - \{\underline{0}\}$ such that (6) is satisfied. It thus follows from Proposition 3 that the cipher is NEKMRP.

If one takes $F = GF(2)$, $m = 3$, and uses the mapping from $GF(2^3) - \{\underline{0}\}$ to $\{1,2,3,4,5,6,7\}$ such that α^i is mapped to $i+1$ where α is a primitive element of $GF(2^3)$, then the bilinear cipher (6) corresponds to the first of the two Latin squares exhibited in Section 3.

Construction 2 can be generalized somewhat as follows. Let A, B and C be nonsingular $m \times m$ matrices over $GF(q)$, let $\hat{x}$, $\hat{y}$ and $\hat{z}$ be the elements of $GF(q^m)$ represented by $A\underline{x}$, $B\underline{y}$ and $C\underline{z}$, respectively, and replace equation (6) by

$$\hat{y} = \hat{x}\hat{z}. \tag{7}$$

It is easily seen that this new cipher is again NEKMRP and, in fact, corresponds just to a change in basis for the representations of $GF(q^m)$ used for $\underline{x}$, $\underline{y}$ and $\underline{z}$. We shall call any bilinear cipher constructed in this manner a multiplication cipher.

Some questions that as yet we have not been able to answer are: (1) Is an NEKMRP bilinear cipher obtained from Construction 1 always a multiplication cipher? (2) Is every NEKMRP bilinear cipher a multiplication cipher? (3) With an appropriate representation of the set of keys, is every NEKMRP linear cipher a bilinear cipher?

5. LINEAR AND BILINEAR STREAM CIPHERS

In this section we will be concerned with stream ciphers, which can be defined as ciphers for which S_x and S_y (and also generally S_z) are infinite sets. We shall take S_x, S_y and S_z to be F^∞, the vector space of all one-sided infinite sequences $\tilde{v} = (v_0, v_1, v_2, \ldots)$ with $v_i \in F = GF(q)$. We define a non-expanding additive stream cipher to be a cipher such that the ciphertext sequence $\tilde{Y} = (Y_0, Y_1, Y_2, \ldots)$ is determined as

$$Y_i = X_i + W_i, \qquad i \geq 0 \tag{8}$$

where $\tilde{W} = (W_0, W_1, W_2, \ldots)$ is the running key sequence and where we allow W_i to depend both on the secret key $\tilde{Z}$ (as is customary) and on the plaintext digits X_j with $j < i$ (as is not customary but is required for interesting definitions of "linear" and "bilinear" stream ciphers). Equation (8) ensures the decipherability of the cipher since

$$X_i = Y_i - W_i$$

and W_i can be found with knowledge of the secret key $\tilde{Z}$ and the previously deciphered plaintext digits. For later convenience, we take the plaintext sequence $\tilde{X}$ to be $\tilde{X} = (X_{-M}, \ldots, X_{-1}, X_0, X_1, \ldots)$, where $X_{-M}, \ldots, X_{-1}$ are "dummy plaintext digits" known in advance at the receiving site, and we take the secret key to be $\tilde{Z} = (Z_{-L}, \ldots, Z_{-1}, Z_0, Z_1, \ldots)$. The parameters M and L will soon be specified.

We shall say that a non-expanding additive stream cipher has plaintext-memory M and key-rate r, where r is a nonnegative real number, if each digit of the running-key sequence can be written as

$$W_i = g_i(X_{i-1}, \ldots, X_{i-M}, Z_{\lceil ri \rceil}, Z_{\lceil ri \rceil - 1}, \ldots, Z_{-M}), \tag{9}$$

where $\lceil . \rceil$ denotes the smallest integer equal to or greater than the enclosed number. We shall further say that such a cipher is linear if, when $\tilde{Z}$ is fixed, each function g_i is linear in its plaintext variables, and to be bilinear if it is linear and, when $\tilde{X}$ is fixed, each function g_i is also linear in its key variables.

A non-expanding additive stream cipher is robustly-perfect if, for any choice of statistics for the plaintext sequence $\tilde{X}$, the running-key sequence $\tilde{W}$ is statistically independent of the plaintext sequence $\tilde{X}$, and $\tilde{W}$ is a sequence of statistically independent uniformly-distributed (i.u.d) random variables.

Because (9) implies that $W_i = 0$ when $X_{i-1} = ... = X_{i-M} = 0$ and the cipher is linear, it follows that a linear additive stream cipher with plaintext memory M cannot be robustly-perfect unless we enforce the plaintext M-restriction that no M consecutive digits in $\tilde{X}$ can all be zeros. We shall later consider a somewhat more severe plaintext restriction.

By an easy argument based on the entropies of the $\tilde{W}$ and $\tilde{Z}$ sequences, one sees that if the cipher (9) is robustly-perfect then the key-rate r is at least 1; we shall say the cipher is key-minimal if $r = 1$. Massey and Rueppel [5] have constructed linear ciphers of the form (9) that are robustly-perfect for the plaintext M-restriction and have key rate $r = 2$. Their ciphers are given by

$$W_i = X_{i-1}Z_{2i-1} + X_{i-2}Z_{2i-2} + \ldots + X_{i-M}Z_{2i-M} \tag{10}$$

where $\tilde{Z}$ is i.u.d.; we see from (10) that this Massey-Rueppel robustly-perfect additive stream cipher is in fact bilinear. Massey and Rueppel [5] also conjectured that no robustly-perfect linear cipher (9) exists (for the plaintext M-restriction) with $r < 2$ for any $M \geq 2$, (which implies that key-minimal robustly-perfect ciphers of this type do not exist). We will show this conjecture to be false by exhibiting key-minimal (i.e., $r = 1$), robustly-perfect for the plaintext M-restriction, additive bilinear stream ciphers for $M = 2$ and $M = 3$, namely, the binary [i.e., F=GF(2)] ciphers

$$W_i = X_{i-1}(Z_{i-1} + Z_{i-4}) + X_{i-2}(Z_{i-2} + Z_{i-3}) \tag{11}$$

and

$$W_i = X_{i-1}Z_{i-1} + X_{i-2}Z_{i-3} + X_{i-3}(Z_{i-2} + Z_{i-3}) \tag{12}$$

when $\tilde{Z}$ is i.u.d.. To show that these ciphers are robustly-perfect we require two additional facts, the first of which should be quite obvious.

Proposition 4: *A bilinear additive stream cipher satisfying (9) is robustly-perfect for a specified plaintext restriction if and only if, for every admissible plaintext sequence $\tilde{x}$, an i.u.d. key sequence $\tilde{Z}$ produces an i.u.d. running-key sequence $\tilde{W}$.*

Because in a bilinear additive stream cipher satisfying (9), given that $\tilde{X} = \tilde{x}$, we can write for every $i \geq 0$

$$(W_0, W_1, ..., W_i) = M_{i\tilde{x}}(Z_{-L}, Z_{-L-1}, ..., Z_{\lceil ri \rceil}) \tag{13}$$

where $M_{i\tilde{x}}$ is an $(i+1) \times (L + \lceil ri \rceil + 1)$ matrix whose components are each a linear combination of the digits $x_{-M}, x_{-M+1}, ..., x_{i-1}$, it follows that Proposition 4 is equivalent to the following proposition.

Proposition 5: *A bilinear additive stream cipher satisfying (9) is robustly-perfect for a specified plaintext restriction if and only if, for every $i \geq 0$ and for every admissible plaintext sequence $\tilde{x}$, the matrix $M_{i\tilde{x}}$ of (13) has linearly independent rows.*

This equivalence follows from the facts that if the $i+1$ rows of $M_{i\tilde{x}}$ are linearly dependent then the column space of $M_{i\tilde{x}}$ has dimension less than $i+1$ and hence $(W_0, W_1, ..., W_i)$ cannot take on all q^{i+1} possible values; and

conversely, if the $i+1$ rows are linearly independent then $(W_0, W_1, ..., W_i)$ takes on all q^{i+1} values and, by linearity, the same number of keys produce each possible value so that the random variables $W_0, W_1, ..., W_i$ are i.u.d. when $Z_{-L}, Z_{-L+1}, ..., Z_{\lfloor ri \rfloor}$ are i.u.d..

For the binary bilinear ciphers of (11) and (12),

$$M_{ix} = \begin{bmatrix} x_{-1} & x_{-2} & x_{-2} & x_{-1} & 0 & 0 & \cdots & 0 \\ 0 & x_0 & x_{-1} & x_{-1} & x_0 & 0 & \cdots & 0 \\ \vdots & & \ddots & & & \ddots & & \vdots \\ \vdots & & & \ddots & & & \ddots & \vdots \\ \vdots & & & & \ddots & & & \vdots \\ 0 & 0 & \cdots & \cdots & x_{i-1} & x_{i-2} & x_{i-2} & x_{i-1} \end{bmatrix} \tag{14}$$

and

$$M_{ix} = \begin{bmatrix} x_{-2}+x_{-3} & x_{-3} & x_{-1} & 0 & 0 & \cdots & 0 \\ 0 & x_{-1}+x_{-2} & x_{-2} & x_0 & 0 & \cdots & 0 \\ \vdots & & \ddots & & & & \vdots \\ \vdots & & & \ddots & & & \vdots \\ \vdots & & & & \ddots & & \vdots \\ 0 & \cdots & \cdots & 0 & x_{i-2}+x_{i-3} & x_{i-2} & x_{i-1} \end{bmatrix}, \tag{15}$$

respectively. The M-restriction on $\tilde{x}$ in both cases is equivalent to the condition that the matrix M_{ix} has no all-zero rows. By a straightforward but somewhat tedious induction, one shows that the matrices in (14) and (15) do indeed have linearly independent rows when the plaintext M-restriction is satisfied, which by Proposition 5 proves the claim that these binary bilinear additive stream ciphers are key-minimal robustly-perfect.

It is quite easy, as we now show, to construct key-minimal robustly-perfect bilinear additive stream ciphers if one relaxes the plaintext M-restriction somewhat. Choose a positive integer m and define $\underline{X}_j = (X_{jm}, X_{jm+1}, ..., X_{jm+m-1})$ to be an m digit segment of $\tilde{X}$. Define $\underline{Z}_j$, $\underline{W}_j$ and $\underline{Y}_j$ in a similar fashion. Now consider the bilinear additive stream cipher such that

$$\underline{Y}_j = \underline{X}_j + \underline{W}_j \tag{16}$$

where

$$\underline{W}_j = M_{\underline{Z}_{j-1}} \underline{X}_{j-1} \tag{17}$$

where $M_{\underline{Z}}$ is the matrix of a NEKMRP bilinear block cipher as discussed in Section 4. This cipher has plaintext-memory $M = 2m - 1$ as follows from the fact that Y_{jm+m-1}, the "newest" digit in $\underline{Y}_j$, depends on X_{jm-m}, the "oldest" digit in $\underline{X}_{j-1}$. We now claim that, for the "strong plaintext restriction" that $\tilde{X}$ contains no run of m consecutive zeroes (and hence that $\underline{X}_j \neq \underline{0}$ for all j), this bilinear additive cipher is key-minimal robustly-perfect. To prove this claim, we suppose that $\tilde{X} = \tilde{x}$ where $\tilde{x}$ is some admissible plaintext sequence. We recall from Proposition 3 that then $\underline{Z}_{j-1} \neq \underline{0}$ in (17) implies $\underline{W}_j \neq \underline{0}$. But $\underline{Z}_{j-1} = \underline{0}$ implies by bilinearity that $M_{\underline{Z}_{j-1}}$ is the zero matrix and hence that $\underline{W}_j = \underline{0}$. Thus because $\tilde{Z}$ is i.u.d., $\underline{W}_j$ equals $\underline{0}$ with probability q^{-m} independent of the value of $\underline{W}_0, \underline{W}_1, ..., \underline{W}_{j-1}$. Proposition 3 further implies that $\underline{W}_j$ equals all $q^m - 1$ non-zero values with equal probability, which must also be q^{-m}, independent of the values of $\underline{W}_0, \underline{W}_1, ..., \underline{W}_{j-1}$. It now follows from Proposition 4 that this bilinear additive stream cipher is robustly-perfect and, since the key-rate $r = 1$, it is also key-minimal.

6. POSSIBLE APPLICATIONS AND SOME REMARKS

As any experienced cryptographer is aware, "linearity" in a cipher is generally one of the greatest assists to the enemy cryptanalyst who attacks it. Thus, we would not seriously propose using directly any of the linear or bilinear ciphers developed in this paper for any practical application where generally only a small amount of secret key (compared to the amount of plaintext) can be tolerated. The real point of this work is the fact that a robustly-perfect linear or bilinear cipher provides a mechanism by which one sequence (the "key") effectively randomizes another sequence (the "plaintext"). Indeed, Massey and Rueppel [5] used their robustly-perfect bilinear additive stream cipher (10) as a mechanism to combine two LFSR sequences to produce a cryptographically strong running-key sequence. The key-minimal robustly-perfect bilinear additive stream ciphers constructed in Section 5 should be even better sequence combiners. The key-minimal robustly-perfect bilinear block ciphers constructed in Section 4 might make excellent component ciphers, when interleaved with nonlinear transpositions, in a product cipher.

It is still a very open question whether, for any $r < 2$, one can for every M construct a plaintext-memory M bilinear (or just linear) additive stream cipher that is robustly-perfect for the plaintext M-restriction and has key-rate at least r. Although, we have not yet succeeded for any $M > 3$, our intuition now is that $r = 1$ is possible for every M. The possibly great potential of such $r = 1$ ciphers as sequence combiners suggests that this open question deserves some serious attention. It is also far from clear that our definitions of "linear" and "bilinear" additive stream ciphers are the most appropriate ones, and some attention should be paid to alternative definitions that might be still more natural.

REFERENCES

[1] C. E. Shannon, "Communication Theory of Secrecy Systems", Bell Sys. Tech. J., vol. 28, pp. 656-715, Oct. 1949.

[2] J. Dénes and A. D. Keedwell, Latin Squares and their Applications. Budapest: Akadémiai Kiadó, 1974.

[3] S. W. Golomb, Shift Register Sequences. San Francisco: Holden-Day, 1967.

[4] R. Lidl and H. Niderreiter, Finite Fields (Vol. 20. Encyclopedia of Math. and its Appl.). Englewood Cliffs, NJ: Addison-Wesley, 1983.

[5] J. L. Massey and R. A. Rueppel, "Linear Ciphers and Random Sequence Generators with Multiple Clocks", pp. 74-87 in Advances in Cryptology (Proceedings of EUROCRYPT 84, Lecture Notes in Computer Science No. 209). Heidelberg and New York: Springer 1985.

LINEAR STRUCTURES IN BLOCKCIPHERS

Jan-Hendrik Evertse

Centre for Mathematics and Computer Science
Kruislaan 413 1098 SJ Amsterdam The Netherlands

§1. INTRODUCTION

A blockcipher maps each pair of plaintext and key onto a ciphertext in such a way that for every fixed key, the relationship between plaintexts and ciphertexts is one-to-one. It is assumed that plaintexts and ciphertexts belong to a message space comprising all bitstrings (sequences of zeros and ones) of a given length; keys are taken from a key space made up of all bitstrings of a possibly different given length. A well-known blockcipher is the NBS Data Encryption Standard (DES) [6], which is the iteration of sixteen essentially equal "rounds".

If a blockcipher is merely a linear mapping (with respect to exclusive-or) of the plaintext and key, then it is very easy to find an unknown key from a known pair of plaintext and corresponding ciphertext. Reeds and Manferdelli [7] pointed out that blockciphers with "partial linearity" are also vulnerable to a known plaintext attack much faster than exhaustive key search. We say that a blockcipher has partial linearity if there are non-injective linear mappings on the plaintext, key and ciphertext, respectively, and a "linear factor", which maps each pair of mapped plaintext and mapped key onto the mapped ciphertext. Given the plaintext and corresponding ciphertext, one can search for the unknown key by investigating the linear factor rather than the blockcipher itself. Reeds and Manferdelli considered the problem of whether DES has partial linearity. In [7], they proved that DES has no partial linearity caused by "per round linear factors", which means, roughly speaking, that DES has no partial linearity built up from the same kind of partial linearity in each round.

Chaum and Evertse [1] extended the notion of a per round linear factor to that of a "sequence of linear factors", and proved that DES has no partial linearity caused by such a sequence. Essentially, this means that DES has no partial linearity built up from possi-

This research was supported by the Netherlands Organization for the Advancement of Pure Research (Z.W.O.).

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 249-266, 1988.
© Springer-Verlag Berlin Heidelberg 1988

bly different kinds of partial linearity in the rounds. Chaum and Evertse also analysed blockciphers consisting of a reduced number of rounds of DES, and proved that blockciphers with less than five rounds of DES do have partial linearities caused by sequences of linear factors.

In the present paper, the notion of partial linearity is extended to that of "linear structures". Apart from the partial linearities, the class of linear structures contains structures like the complementation property of DES. We discuss the cryptanalytic importance of linear structures and among other things, the linear structures of DES are investigated. It is shown that, as a consequence, blockciphers consisting of at least seven consecutive rounds of DES have no "recursive linear structures" other than the complementation property of DES; this means that apart from the complementation property, these blockciphers have no linear structures that can be built up from linear structures in the rounds. In fact, recursive linear structures are natural generalisations of sequences of linear factors. The results on DES just mentioned are derived from a general theorem on recursive linear structures in "DES-like ciphers" which we also prove.

In §2 we explain precisely what is meant by a linear structure in a blockcipher. More informally, one could say that a blockcipher has a linear structure if there are subsets P, K and C of plaintext bits, key bits and ciphertext bits of this blockcipher, respectively, such that for *each* plaintext and *each* key, a simultaneous change of all plaintext bits in P and all key bits in K has the *same* effect on the exclusive-or sum of the bits in C of the corresponding ciphertext; thus, either this exclusive-or sum is always changed or it always remains unchanged. Below we give a few examples of linear structures.

- The complementation property of DES (cf. [4]): simultaneously changing all bits of the plaintext and the key of DES results always in the change of all bits of the corresponding ciphertext.
- Bit independencies: the values of some ciphertext bits are independent of the values of certain plaintext bits and key bits; in other words, these ciphertext bits can be expressed as Boolean functions depending only on the other plaintext bits and key bits. It was pointed out in [5] and [1] that versions of DES with less than five rounds have such bit independencies.
- Structures that will change into linear structures when the blockcipher is modified by applying certain linear transformations (with respect to exclusive-or) to the plaintext, key and ciphertext respectively are linear structures themselves; for instance partial linearities are linear structures that change into bit independencies by applying appropriate linear transformations to the plaintext, ciphertext and key of the blockcipher.

We argue in §3 that blockciphers with linear structures may be vulnerable to known- or chosen plaintext attacks faster than exhaustive key search.

We are particularly interested in linear structures of *product ciphers*. These are blockciphers composed of "simple" blockciphers ("rounds"). In §4 we explain how linear structures of product ciphers can be constructed from linear structures in their rounds; linear structures constructed in this way are said to be *recursive* over the rounds. As mentioned before, recursive linear structures in product ciphers are generalisations of the sequences of linear factors introduced in [1]. In many situations, the linear structures of the rounds, and consequently the recursive linear structures of the product cipher, can be found quite easily; however it is often a hard problem to decide whether a product cipher has a linear structure not recursive over its rounds that is of any use in cryptanalysis.

In §5 we deal with DES-like ciphers. These are product ciphers built up in a similar way as DES from S-boxes, exclusive-or operations and mappings that are linear with respect to exclusive-or. It is shown that the linear structures of a round of a DES-like cipher can be expressed easily in terms of linear structures of the S-boxes. Like DES, each DES-like cipher has a complementation property. The main result of §5 states, that any DES-like cipher satisfying certain easily verifiable conditions has apart from its complementation property no linear structures which are recursive over its rounds.

In §6 we show that each blockcipher consisting of seven or more consecutive rounds of DES is a DES-like cipher satisfying the conditions of the main result of §5. Therefore, blockciphers that consist of seven or more consecutive rounds of DES have no recursive linear structures other than the complementation property.

In §7 we explain briefly, that a DES-like cipher might be weak if some of its S-boxes have structures that change into linear structures by appropriately changing some of the output values of these S-boxes. Further, we discuss the relationship between the existence of such structures and the statistical properties of the S-boxes.

§2. NOTATION AND DEFINITIONS

In this section we introduce some notation to be used in the remainder of this paper, and give a formal definition of a linear structure in a blockcipher.

Let $\mathbb{F}_2 = \{0, 1\}$ be the finite field of two elements. When using notions from linear algebra such as vector spaces, linear mappings, etc., it is assumed that the underlying field of scalars is $\mathbb{F}_2$. For every vector space we consider, we denote the addition operation by $+$ and, if confusion is not likely to arise, the zero vector by $\mathbf{0}$. $\mathbb{F}_2^m$ denotes the vector space consisting of all strings of m bits in which the addition of two strings is just bitwise exclusive-or. Elements of $\mathbb{F}_2^m$ are denoted by $\mathbf{a}$, $\mathbf{b}$, etc.; $\mathbf{0}_m$ denotes the string of m zeros and $\mathbf{1}_m$ the string of m ones. Vectors in cartesian products $\mathbb{F}_2^{m_1} \times \cdots \times \mathbb{F}_2^{m_r}$ are often denoted as $(\mathbf{x}_1, \ldots, \mathbf{x}_r)$, where $\mathbf{x}_i \in \mathbb{F}_2^{m_i}$ for $i = 1, \ldots, r$. $[\mathbf{x}]$ denotes the vector space generated by $\mathbf{x}$. If $\mathcal{V}_\alpha (\alpha \in A)$ are (linear) subspaces of the same vector space, then $\bigoplus_{\alpha \in A} \mathcal{V}_\alpha = \{ \sum_{\alpha \in A} x_\alpha : x_\alpha \in \mathcal{V}_\alpha \}$ denotes the smallest vector space containing each $\mathcal{V}_\alpha$. Thus,

$\bigoplus_{\alpha\in A}[\mathbf{x}_\alpha]$ denotes the vector space generated by the set of vectors $\{\mathbf{x}_\alpha: \alpha\in A\}$. For any linear mapping A with domain $\mathbb{F}_2^m$ we put $ker(A)=\{\mathbf{x}\in\mathbb{F}_2^m: A\mathbf{x}=\mathbf{0}\}$ and $im(A)=\{A\mathbf{x}: \mathbf{x}\in\mathbb{F}_2^m\}$. A linear mapping is said to be *trivial* if it maps every vector in its domain onto $\mathbf{0}$.

A *blockcipher* is a mapping

$$F: \mathbb{F}_2^m\times\mathbb{F}_2^k\to\mathbb{F}_2^m$$

(where $\mathbb{F}_2^m$ and $\mathbb{F}_2^k$ are the *message space* and *key space*, respectively) such that for each $\mathbf{k}$ in $\mathbb{F}_2^k$, the mapping

$$F_\mathbf{k}:=F(.,\mathbf{k}): \mathbb{F}_2^m\to\mathbb{F}_2^m \tag{1}$$

is invertible.

Definition. A *linear structure* of a blockcipher $F: \mathbb{F}_2^m\times\mathbb{F}_2^k\to\mathbb{F}_2^m$ is a pair $(\mathcal{V},\mathcal{W})$, where $\mathcal{V}$ is a subspace of $\mathbb{F}_2^m\times\mathbb{F}_2^k$ and $\mathcal{W}$ a subspace of $\mathbb{F}_2^m$, such that for each pair $(\mathbf{p}_0,\mathbf{k}_0)$ in $\mathcal{V}$, each $\mathbf{p}$ in $\mathbb{F}_2^m$ and each $\mathbf{k}$ in $\mathbb{F}_2^k$ we have

$$F(\mathbf{p}+\mathbf{p}_0,\mathbf{k}+\mathbf{k}_0)+F(\mathbf{p},\mathbf{k})+F(\mathbf{p}_0,\mathbf{k}_0)+F(\mathbf{0}_m,\mathbf{0}_k)\in\mathcal{W}. \tag{2}$$

A linear structure $(\mathcal{V},\mathcal{W})$ is called *trivial* if $\mathcal{V}=[(\mathbf{0}_m,\mathbf{0}_k)]$ or $\mathcal{W}=\mathbb{F}_2^m$.

Remark 1. Each blockcipher has trivial linear structures.

Remark 2. It is easy to see that this definition implies that of §1. For let $(\mathcal{V},\mathcal{W})$ be a linear structure and let B be a linear mapping on $\mathbb{F}_2^m$ with $ker(B)=\mathcal{W}$. Then (2) implies that there is a function ψ, defined on $\mathcal{V}$, such that

$$\begin{aligned} BF(\mathbf{p}+\mathbf{p}_0,\mathbf{k}+\mathbf{k}_0)+BF(\mathbf{p},\mathbf{k}) &= BF(\mathbf{p}_0,\mathbf{k}_0)+BF(\mathbf{0}_m,\mathbf{0}_k) = \psi(\mathbf{p}_0,\mathbf{k}_0) \\ &\text{for all } (\mathbf{p}_0,\mathbf{k}_0)\in\mathcal{V},\ \mathbf{p}\in\mathbb{F}_2^m \text{ and } \mathbf{k}\in\mathbb{F}_2^k. \end{aligned} \tag{3}$$

In other words, if we fix $\mathbf{p}_0$ and $\mathbf{k}_0$, then exclusive-oring a plaintext with $\mathbf{p}_0$ and a key with $\mathbf{k}_0$ causes a change in the B-value of the corresponding ciphertext, which is for each plaintext and key the same.

We now give a few examples of linear structures.

Example 1: complementation property of DES. The blockcipher DES, with message space $\mathbb{F}_2^{64}$ and key space $\mathbb{F}_2^{56}$, has the property that $DES(\mathbf{p}+\mathbf{1}_{64},\mathbf{k}+\mathbf{1}_{56})=DES(\mathbf{p},\mathbf{k})+\mathbf{1}_{64}$ for every plaintext $\mathbf{p}$ and key $\mathbf{k}$. Hence $([(\mathbf{1}_{64},\mathbf{1}_{56})],[\mathbf{0}_{64}])$ is a linear structure of DES, and in (3) we can take for B the identity and for ψ the mapping defined by $\psi(\mathbf{0}_{64},\mathbf{0}_{56})=\mathbf{0}_{64}$ and $\psi(\mathbf{1}_{64},\mathbf{1}_{56})=\mathbf{1}_{64}$.

Example 2: partial linearity. A blockcipher F is said to have partial linearity if there are a triple of linear mappings (A_1,A_2,B) and a mapping $\tilde{F}$ such that $BF(\mathbf{p},\mathbf{k})=\tilde{F}(A_1\mathbf{p},A_2\mathbf{k})$ for all plaintexts $\mathbf{p}$ and keys $\mathbf{k}$. Then $(\mathcal{V},\mathcal{W})$, with $\mathcal{V}=ker(A_1)\times ker(A_2)$ and $\mathcal{W}=ker(B)$,

is the corresponding linear structure of F. The function ψ in (3) is identically zero.

§3. CRYPTANALYTIC SIGNIFICANCE OF LINEAR STRUCTURES

In this section we describe a known- and a chosen plaintext attack, which are both based on the existence of linear structures. In these attacks, the following fact is used:

Lemma 1. *Let F: $\mathbb{F}_2^m \times \mathbb{F}_2^k \to \mathbb{F}_2^m$ be a blockcipher and $(\mathcal{V},\mathcal{W})$ a linear structure of F. Further, let A,B be linear mappings with domains $\mathbb{F}_2^m \times \mathbb{F}_2^k$ and $\mathbb{F}_2^k$, respectively, such that $ker(A)=\mathcal{V}$ and $ker(B)=\mathcal{W}$. Then there exist a linear mapping C: $\mathbb{F}_2^m \times \mathbb{F}_2^k \to im(B)$ and a (not necessarily linear) mapping $\tilde{F}$: $im(A)\to im(B)$, both easily computable from F, A and B, such that*

$$BF(\mathbf{p},\mathbf{k}) = \tilde{F}A(\mathbf{p},\mathbf{k}) + C(\mathbf{p},\mathbf{k}) \text{ for all } \mathbf{p} \text{ in } \mathbb{F}_2^m,\ \mathbf{k} \text{ in } \mathbb{F}_2^k .$$

Proof. Let $(\mathcal{V},\mathcal{W})$ be a linear structure of F and A, B linear mappings with $ker(A)=\mathcal{V}$, $ker(B)=\mathcal{W}$. Further, let ψ be the function on $\mathcal{V}$, defined by (3) in §2. ψ is linear on $\mathcal{V}$ and therefore easy to compute from F, A and B. Indeed, let $(\mathbf{p}_0,\mathbf{k}_0)$, $(\mathbf{p}_1,\mathbf{k}_1)\in\mathcal{V}$. Then (3) implies that

$$\begin{aligned}\psi(\mathbf{p}_0+\mathbf{p}_1,\mathbf{k}_0+\mathbf{k}_1) &= BF(\mathbf{p}_1+\mathbf{p}_0,\mathbf{k}_1+\mathbf{k}_0)+BF(\mathbf{0}_m,\mathbf{0}_k)\\ &= BF(\mathbf{p}_1+\mathbf{p}_0,\mathbf{k}_1+\mathbf{k}_0)+BF(\mathbf{p}_1,\mathbf{k}_1)+BF(\mathbf{p}_1,\mathbf{k}_1)+BF(\mathbf{0}_m,\mathbf{0}_k)\\ &= \psi(\mathbf{p}_0,\mathbf{k}_0)+\psi(\mathbf{p}_1,\mathbf{k}_1).\end{aligned}$$

Let A^* be a pseudo-inverse of A, that is a linear mapping A^*: $im(A)\to\mathbb{F}_2^m\times\mathbb{F}_2^k$ such that AA^* is the identity on $im(A)$. Such a pseudo-inverse exists and can be easily computed from A. Let D: $\mathbb{F}_2^m\times\mathbb{F}_2^k\to\mathbb{F}_2^m\times\mathbb{F}_2^k$ be the linear mapping defined by $D(\mathbf{p},\mathbf{k})=(\mathbf{p},\mathbf{k})+A^*A(\mathbf{p},\mathbf{k})$. Then $D(\mathbf{p},\mathbf{k})\in ker(A)=\mathcal{V}$ for all $\mathbf{p}\in\mathbb{F}_2^m$ and $\mathbf{k}\in\mathbb{F}_2^k$. Put $\tilde{F}=BFA^*$, $C=\psi D$. Then $\tilde{F}$ and C are well-defined mappings that are easily computable from F, A and B, and C is linear. Let $\mathbf{p}$ and $\mathbf{k}$ be arbitrary elements of $\mathbb{F}_2^m$ and $\mathbb{F}_2^k$, respectively and put $(\mathbf{p}_0,\mathbf{k}_0)=D(\mathbf{p},\mathbf{k})$. Then (3) and the fact that $(\mathbf{p}_0,\mathbf{k}_0)\in\mathcal{V}$ imply that

$$\begin{aligned}BF(\mathbf{p},\mathbf{k}) &= BF(\mathbf{p}+\mathbf{p}_0,\mathbf{k}+\mathbf{k}_0)+\psi(\mathbf{p}_0,\mathbf{k}_0) = BFA^*A(\mathbf{p},\mathbf{k})+C(\mathbf{p},\mathbf{k})\\ &= \tilde{F}A(\mathbf{p},\mathbf{k})+C(\mathbf{p},\mathbf{k}).\end{aligned}$$

This completes the proof of Lemma 1. □

In what follows, F: $\mathbb{F}_2^m\times\mathbb{F}_2^k\to\mathbb{F}_2^m$ is a blockcipher and $(\mathcal{V},\mathcal{W})$ a non-trivial linear structure of F, and A, B, C and $\tilde{F}$ are the mappings satisfying the conditions of Lemma 1. We define the linear mappings A_1, A_2, C_1 and C_2 by $A(\mathbf{p},\mathbf{k})=A_1\mathbf{p}+A_2\mathbf{k}$, $C(\mathbf{p},\mathbf{k})=C_1\mathbf{p}+C_2\mathbf{k}$. We describe two attacks: a known plaintext attack, where it is assumed that $0<n:=$dimension $ker(A_2)\leqslant k$; and a chosen plaintext attack in which $ker(A_2)$ is supposed to have dimension 0.

A known plaintext attack. Suppose that a cryptanalist has a plaintext-ciphertext pair $(\mathbf{p},\mathbf{c})$ and wants to find the secret key $\mathbf{k}$ with $F(\mathbf{p},\mathbf{k})=\mathbf{c}$. In order to find $\mathbf{k}$, he proceeds as follows:

(i) he runs through all values $\tilde{\mathbf{k}}$ in $im(A_2)$ and checks for each $\tilde{\mathbf{k}}$, if the system of linear equations

$$\left.\begin{aligned} A_2\mathbf{k}&=\tilde{\mathbf{k}}\\ C_2\mathbf{k}&=B\mathbf{c}+\tilde{F}(A_1\mathbf{p}+\tilde{\mathbf{k}})+C_1\mathbf{p}\end{aligned}\right\}\quad \text{in } \mathbf{k}\in\mathbb{F}_2^k \tag{4}$$

is soluble (the costs of this are approximately those of a computation of F, if we suppose that F is much more "complicated" than a linear mapping); it follows at once from Lemma 1 that the unknown key $\mathbf{k}$ must satisfy (4);

(ii) for each $\tilde{\mathbf{k}}$ in $im(A_2)$ for which (4) is soluble, he checks for each solution $\mathbf{k}$ of (4) if $F(\mathbf{p},\mathbf{k})=\mathbf{c}$.

Supposing that the cryptanalist finds L values of $\tilde{\mathbf{k}}$ in (i), and that the null space of the linear mapping $\mathbf{k}\mapsto(A_2\mathbf{k},C_2\mathbf{k})$ has dimension $n_1\leqslant n$, he will find the key after about $2^{k-n}+L\times 2^{n_1}$ encryptions. In general, this number of encryptions is smaller than that needed in exhaustive key search, which is 2^k. By a heuristic argument like in [1], §2, one can argue that the expected time in which the cryptanalist finds the key can be made smaller if he has more plaintext-ciphertext pairs for that same key.

Example 1: partial linearity. Let A_1,A_2,B be linear mappings such that $BF(\mathbf{p},\mathbf{k})=\tilde{F}(A_1\mathbf{p},A_2\mathbf{k})$ for every plaintext $\mathbf{p}$ and key $\mathbf{k}$, and suppose that $ker(A_2)$ has dimension >0. In [7] and [1] a known plaintext attack based on partial linearity was described that is faster than exhaustive key search. That attack is the same as the attack described above, with C_1 and C_2 being trivial.

A chosen plaintext attack. Suppose that a cryptanalist has N different plaintext-ciphertext pairs, $(\mathbf{p}_1,\mathbf{c}_1),\ldots,(\mathbf{p}_N,\mathbf{c}_N)$, say, and wants to find the unknown key $\mathbf{k}$ for which $F(\mathbf{p}_1,\mathbf{k})=\mathbf{c}_1,\ldots,F(\mathbf{p}_N,\mathbf{k})=\mathbf{c}_N$. Assume that $\mathbf{p}_1,\ldots,\mathbf{p}_N$ have the property that there are $\mathbf{k}_1,\ldots,\mathbf{k}_N\in\mathbb{F}_2^k$ such that

$$A(\mathbf{p}_1,\mathbf{k}_1)=A(\mathbf{p}_2,\mathbf{k}_2)=\cdots=A(\mathbf{p}_N,\mathbf{k}_N). \tag{5}$$

Note that plaintexts $\mathbf{p}_1,\ldots,\mathbf{p}_N$ with this property exist if and only if $\mathcal{V}$ has cardinality at least N. In order to find $\mathbf{k}$, the cryptanalist proceeds as follows: he chooses keys $\mathbf{k}'$ from $\mathbb{F}_2^k$ at random and checks for each $\mathbf{k}'$ if

$$C_2(\mathbf{k}'+\mathbf{k}_1+\mathbf{k}_i)=B\mathbf{c}_i+\tilde{F}A(\mathbf{p}_1,\mathbf{k}')+C_1\mathbf{p}_i \tag{6}$$

holds for some i in $\{1,\ldots,N\}$. If this is the case, the cryptanalist concludes that $\mathbf{k}=\mathbf{k}'+\mathbf{k}_1+\mathbf{k}_i$ must be the proper key. His motivation for this is, that by (5), (6) and

Lemma 1 this $\mathbf{k}$ satisfies

$$B\mathbf{c}_i = \tilde{F}A(\mathbf{p}_i,\mathbf{k}) + C(\mathbf{p}_i,\mathbf{k}) = BF(\mathbf{p}_i,\mathbf{k}).$$

Thus for the costs of only a single encryption, the cryptanalist can check N keys. Therefore the expected running time of this attack is about N times as fast as that of exhaustive search.

Example 2: complementation property of DES. Hellman et al. ([4], §III) showed that there is a chosen plaintext attack on DES, using the complementation property, which is twice as fast as exhaustive key search. That attack is essentially the chosen plaintext attack just described, applied to DES and two plaintext-ciphertext pairs $(\mathbf{p}_1,\mathbf{c}_1)$, $(\mathbf{p}_2,\mathbf{c}_2)$ with $\mathbf{p}_2 = \mathbf{p}_1 + \mathbf{1}_{64}$. Note that such two pairs satisfy (5) with $N = 2$, where A is a linear mapping with $ker(A) = [\mathbf{1}_{64}, \mathbf{1}_{56}]$, and $\mathbf{k}_1$ and $\mathbf{k}_2$ are any two keys with $\mathbf{k}_2 = \mathbf{k}_1 + \mathbf{1}_{56}$.

Example 3: multiple complementation properties. Let $f: \mathbb{F}_2^m \to \mathbb{F}_2^m$ be a one-to-one function such that both f and its inverse are easy to compute, and let $F^*: \mathbb{F}_2^m \times \mathbb{F}_2^m \to \mathbb{F}_2^m$ be the blockcipher defined by $F^*(\mathbf{p},\mathbf{k}) = f(\mathbf{p}+\mathbf{k}) + \mathbf{k}$. Then $(\mathcal{V},\mathcal{W})$ is a linear structure of F^*, with $\mathcal{V} = \{(\mathbf{x},\mathbf{x}): \mathbf{x} \in \mathbb{F}_2^m\}$ and $\mathcal{W} = [\mathbf{0}_m]$. Let $A: \mathbb{F}_2^m \times \mathbb{F}_2^m \to \mathbb{F}_2^m$ be the linear mapping given by $A(\mathbf{p},\mathbf{k}) = \mathbf{p}+\mathbf{k}$; thus $ker(A) = \mathcal{V}$. Any N different plaintexts $\mathbf{p}_1, \ldots, \mathbf{p}_N$ of F^* satisfy condition (5) with $\mathbf{k}_i = \mathbf{p}_i$ for $i = 1, \ldots, N$. Hence if a cryptanalist knows N arbitrary plaintext-ciphertext pairs of F^*, corresponding to the same unknown key, then he can find that key about N times faster than with exhaustive search by using the chosen plaintext attack described above. Note that for the blockcipher F^*, this chosen plaintext attack is in fact a *known* plaintext attack.

The chosen plaintext attack can also be used when $0 <$ dimension $ker(A_2) \leqslant k$, but in that case its benefit is much less than that of the known plaintext attack described above. However, it is possible to combine both attacks into a chosen plaintext attack that is somewhat faster than the known plaintext attack described in this chapter. Further, it is possible to use linear structures in meet-in-the-middle attacks like in [1]. We do not work this out here.

§4. LINEAR STRUCTURES IN PRODUCT CIPHERS

Let $F_1, \ldots, F_R: \mathbb{F}_2^m \times \mathbb{F}_2^k \to \mathbb{F}_2^m$ be blockciphers. The *product* $F = F_R \cdots F_1$ of $F_1, \ldots, F_R$ is defined (cf. (1)) by

$$F_{\mathbf{k}}(\mathbf{p}) = F_{R,\mathbf{k}} \cdots F_{1,\mathbf{k}}(\mathbf{p}) \tag{7}$$

(composition of mappings) for $\mathbf{p} \in \mathbb{F}_2^m$ and $\mathbf{k} \in \mathbb{F}_2^k$. $F_1, \ldots, F_R$ are called the *rounds* of F. We describe how linear structures of F can be constructed from linear structures in $F_1, \ldots, F_R$. To this end, we introduce so-called T-spaces and U-spaces.

For any blockcipher $F: \mathbb{F}_2^m \times \mathbb{F}_2^k \to \mathbb{F}_2^m$, and any subspace $\mathcal{V}$ of $\mathbb{F}_2^m \times \mathbb{F}_2^k$, we define

the spaces

$$T(F,\mathcal{V}) = \bigoplus_{\substack{(\mathbf{p}_0,\mathbf{k}_0)\in\mathcal{V} \\ (\mathbf{p},\mathbf{k})\in\mathbb{F}_2^m\times\mathbb{F}_2^k}} [(F(\mathbf{p}+\mathbf{p}_0,\mathbf{k}+\mathbf{k}_0)+F(\mathbf{p},\mathbf{k}),\mathbf{k}_0)] ,$$

$$U(F,\mathcal{V}) = \bigoplus_{\substack{(\mathbf{p}_0,\mathbf{k}_0)\in\mathcal{V} \\ (\mathbf{p},\mathbf{k})\in\mathbb{F}_2^m\times\mathbb{F}_2^k}} [F(\mathbf{p}+\mathbf{p}_0,\mathbf{k}+\mathbf{k}_0)+F(\mathbf{p},\mathbf{k})+F(\mathbf{p}_0,\mathbf{k}_0)+F(\mathbf{0}_m,\mathbf{0}_k)] .$$

Thus $T(F,\mathcal{V})\subseteq\mathbb{F}_2^m\times\mathbb{F}_2^k$, $U(F,\mathcal{V})\subseteq\mathbb{F}_2^m$, and $U(F,\mathcal{V})\times[\mathbf{0}_k]\subseteq T(F,\mathcal{V})$. Obviously, (2) can be stated equivalently as

$$(\mathcal{V},\mathcal{W}) \text{ linear structure of } F \Leftrightarrow U(F,\mathcal{V})\subseteq\mathcal{W}. \tag{8}$$

In other words, $U(F,\mathcal{V})$ is the minimal space $\mathcal{W}$ such that $(\mathcal{V},\mathcal{W})$ is a linear structure of F. The T-spaces are auxiliary spaces, needed to construct linear structures in blockciphers from linear structures in the rounds. For certain simple blockciphers like the rounds of DES the U-spaces are easy to compute. The next lemma describes how T-spaces can be computed from U-spaces.

Lemma 2. *Let* F: $\mathbb{F}_2^m\times\mathbb{F}_2^k\to\mathbb{F}_2^m$ *be a blockcipher and* $\mathcal{V}=\bigoplus_{i=1}^{s}[(\mathbf{p}_i,\mathbf{k}_i)]$ *a subspace of* $\mathbb{F}_2^m\times\mathbb{F}_2^k$. *Then*

$$T(F,\mathcal{V}) = \{U(F,\mathcal{V})\times[\mathbf{0}_k]\} \oplus \{\bigoplus_{i=1}^{s}[(F(\mathbf{p}_i,\mathbf{k}_i)+F(\mathbf{0}_m,\mathbf{0}_k),\mathbf{k}_i)]\}. \tag{9}$$

Proof. Denote the space at the right-hand side of (9) by $\mathcal{X}$. It is easy to check that $\mathcal{X}\subseteq T(F,\mathcal{V})$. In order to prove that $T(F,\mathcal{V})\subseteq\mathcal{X}$, it suffices to show that for each $(\mathbf{p}_0,\mathbf{k}_0)$ in $\mathcal{V}$, $\mathbf{p}$ in $\mathbb{F}_2^m$ and $\mathbf{k}$ in $\mathbb{F}_2^k$ we have

$$(F(\mathbf{p}+\mathbf{p}_0,\mathbf{k}+\mathbf{k}_0)+F(\mathbf{p},\mathbf{k}),\mathbf{k}_0)\in\mathcal{X}. \tag{10}$$

Without loss of generality we may assume that $(\mathbf{p}_0,\mathbf{k}_0)=\sum_{i=1}^{t}(\mathbf{p}_i,\mathbf{k}_i)$, where $1\leqslant t\leqslant s$. Then

$$(F(\mathbf{p}+\mathbf{p}_0,\mathbf{k}+\mathbf{k}_0)+F(\mathbf{p},\mathbf{k}),\mathbf{k}_0) = \sum_{i=1}^{t}\mathbf{a}_i ,$$

where $\mathbf{a}_1=(F(\mathbf{p}+\mathbf{p}_1,\mathbf{k}+\mathbf{k}_1)+F(\mathbf{p},\mathbf{k}),\mathbf{k}_1)$ and

$$\mathbf{a}_i=(F(\mathbf{p}+\sum_{j=1}^{i}\mathbf{p}_j,\mathbf{k}+\sum_{j=1}^{i}\mathbf{k}_j)+F(\mathbf{p}+\sum_{j=1}^{i-1}\mathbf{p}_j,\mathbf{k}+\sum_{j=1}^{i-1}\mathbf{k}_j),\mathbf{k}_i)$$

for $i=2,\ldots,t$. It is easy to check that for each i, $\mathbf{a}_i+(F(\mathbf{p}_i,\mathbf{k}_i)+F(\mathbf{0}_m,\mathbf{0}_k),\mathbf{k}_i)$ belongs to $U(F,\mathcal{V})\times[\mathbf{0}_k]$. This proves Lemma 2. □

The linear structures of a single round of a blockcipher can be found by investigating the U-spaces of that round. The next lemma describes, how linear structures of the

whole product cipher can be constructed from those T-spaces and U-spaces of the rounds satisfying some recurrence relation.

Lemma 3. *Let $F_1, \ldots, F_R: \mathbb{F}_2^m \times \mathbb{F}_2^k \to \mathbb{F}_2^m$ be blockciphers and put $F = F_R \cdots F_1$. Suppose that $\mathcal{V}_0, \mathcal{V}_1, \ldots, \mathcal{V}_R$ are subspaces of $\mathbb{F}_2^m \times \mathbb{F}_2^k$, and $\mathcal{W}_0, \mathcal{W}_1, \ldots, \mathcal{W}_R$ are subspaces of $\mathbb{F}_2^m$, such that*

$$\left.\begin{aligned} &\mathcal{V}_i \supseteq T(F_i, \mathcal{V}_{i-1}), \\ &\mathcal{W}_i \times [\mathbf{0}_k] \supseteq T(F_i, \mathcal{W}_{i-1} \times [\mathbf{0}_k]) \oplus \{U(F_i, \mathcal{V}_{i-1}) \times [\mathbf{0}_k]\} \text{ for } i = 1, \ldots, R. \end{aligned}\right\} \tag{11}$$

Then $U(F, \mathcal{V}_0) \subseteq \mathcal{W}_R$.

(8) and Lemma 3 motivate the following:

Definition. Let $F_1, \ldots, F_R: \mathbb{F}_2^m \times \mathbb{F}_2^k \to \mathbb{F}_2^m$ be blockciphers and put $F = F_R \cdots F_1$. A linear structure $(\mathcal{V}, \mathcal{W})$ of F is called *recursive* over $F_1, \ldots, F_R$ if there are subspaces $\mathcal{V}_0, \ldots, \mathcal{V}_R$ of $\mathbb{F}_2^m \times \mathbb{F}_2^k$ and $\mathcal{W}_0, \ldots, \mathcal{W}_R$ of $\mathbb{F}_2^m$ for which (11) holds and for which $\mathcal{V} = \mathcal{V}_0$ and $\mathcal{W} = \mathcal{W}_R$.

Remark 1. If a product cipher can be decomposed into rounds in two different ways, then it is possible that a linear structure of that product cipher is recursive over the rounds of the first decomposition but not over the rounds of the second decomposition.

Remark 2. If the rounds of some product cipher are such that their linear structures are easy to find, then in general, the linear structures of that product cipher which are recursive over its rounds are also easy to detect. However, one can not exclude that a product cipher has linear structures that are *not* recursive over its rounds, and it might be a very difficult problem to check if such non-recursive linear structures exist.

Proof of Lemma 3. In the proof of Lemma 3 we need the following facts: for any two blockciphers G, $H: \mathbb{F}_2^m \times \mathbb{F}_2^k \to \mathbb{F}_2^m$ we have

$$T(HG, \mathcal{V}) \subseteq T(H, T(G, \mathcal{V})) \tag{12}$$

and

$$U(HG, \mathcal{V}) \times [\mathbf{0}_k] \subseteq T(H, U(G, \mathcal{V}) \times [\mathbf{0}_k]) \oplus \{U(H, T(G, \mathcal{V})) \times [\mathbf{0}_k]\}. \tag{13}$$

We first prove (12). Let $\mathbf{p} \in \mathbb{F}_2^m$, $\mathbf{k} \in \mathbb{F}_2^k$ and $(\mathbf{p}_0, \mathbf{k}_0) \in \mathcal{V}$, and put $\mathbf{p}_1 = G(\mathbf{p}, \mathbf{k})$, $\tilde{\mathbf{p}}_0 = G(\mathbf{p} + \mathbf{p}_0, \mathbf{k} + \mathbf{k}_0) + G(\mathbf{p}, \mathbf{k})$. Then

$$\begin{aligned} &(HG(\mathbf{p} + \mathbf{p}_0, \mathbf{k} + \mathbf{k}_0) + HG(\mathbf{p}, \mathbf{k}),\ \mathbf{k}_0) \\ &\qquad = (H(\mathbf{p}_1 + \tilde{\mathbf{p}}_0, \mathbf{k} + \mathbf{k}_0) + H(\mathbf{p}_1, \mathbf{k}),\ \mathbf{k}_0) \in T(H, T(G, \mathcal{V})). \end{aligned}$$

This proves (12).

We now prove (13). Put $\mathbf{q}_1 = G(\mathbf{p}+\mathbf{p}_0,\mathbf{k}+\mathbf{k}_0)+G(\mathbf{p},\mathbf{k})+G(\mathbf{p}_0,\mathbf{k}_0)+G(\mathbf{0}_m,\mathbf{0}_k)$, $\mathbf{q}_2 = G(\mathbf{p}_0,\mathbf{k}_0)+G(\mathbf{0}_m,\mathbf{0}_k)$, $\mathbf{p}_1 = G(\mathbf{p},\mathbf{k})$ and $\mathbf{p}_2 = G(\mathbf{p}_0,\mathbf{k}_0)$. Then $\mathbf{q}_1 \in U(G,\mathcal{V})$ and $(\mathbf{q}_2,\mathbf{k}_0) \in T(G,\mathcal{V})$. Further,

$$HG(\mathbf{p}+\mathbf{p}_0,\mathbf{k}+\mathbf{k}_0)+HG(\mathbf{p},\mathbf{k})+HG(\mathbf{p}_0,\mathbf{k}_0)+HG(\mathbf{0}_m,\mathbf{0}_k) = \mathbf{a}+\mathbf{b}+\mathbf{c},$$

where

$$\begin{aligned}
\mathbf{a} &= H(\mathbf{p}_1+\mathbf{q}_2+\mathbf{q}_1,\mathbf{k}+\mathbf{k}_0)+H(\mathbf{p}_1+\mathbf{q}_2,\mathbf{k}+\mathbf{k}_0) \text{ and } (\mathbf{a},\mathbf{0}_k) \in T(H,\ U(G,\mathcal{V})\times[\mathbf{0}_k]),\\
\mathbf{b} &= H(\mathbf{p}_1+\mathbf{q}_2,\mathbf{k}+\mathbf{k}_0)+H(\mathbf{p}_1,\mathbf{k})+H(\mathbf{q}_2,\mathbf{k}_0)+H(\mathbf{0}_m,\mathbf{0}_k) \in U(H,\ T(G,\mathcal{V})),\\
\mathbf{c} &= H(\mathbf{p}_2+\mathbf{q}_2,\mathbf{k}_0+\mathbf{k}_0)+H(\mathbf{p}_2,\mathbf{k}_0)+H(\mathbf{q}_2,\mathbf{k}_0)+H(\mathbf{0}_m,\mathbf{0}_k) \in U(H,\ T(G,\mathcal{V})).
\end{aligned}$$

This proves (13).

Let $F^{(i)} = F_i \cdots F_1$ for $i = 1,\ldots,R$. We prove by induction on i that

$$\mathcal{V}_i \supseteq T(F^{(i)},\mathcal{V}_0),\quad \mathcal{W}_i \supseteq U(F^{(i)},\mathcal{V}_0) \quad \text{for } i = 1,\ldots,R, \tag{14}$$

which is obviously sufficient. (14) is trivially true for $i = 1$. Suppose that (14) holds for $i = t-1$ (induction hypothesis). In the induction step, we apply (12) and (13) with $G = F^{(t-1)}$, $H = F_t$ and $\mathcal{V} = \mathcal{V}_0$. First we have, by (11), the induction hypothesis and (12), that

$$\mathcal{V}_t \supseteq T(F_t,\mathcal{V}_{t-1}) \supseteq T(F_t,T(F^{(t-1)},\mathcal{V}_0)) \supseteq T(F^{(t)},\mathcal{V}_0),$$

and second it follows from (11), the induction hypothesis and (13), that

$$\begin{aligned}
\mathcal{W}_t\times[\mathbf{0}_k] &\supseteq T(F_t,\mathcal{W}_{t-1}\times[\mathbf{0}_k]) \oplus \{U(F_t,\mathcal{V}_{t-1})\times[\mathbf{0}_k]\}\\
&\supseteq T(F_t,U(F^{(t-1)},\mathcal{V}_0)\times[\mathbf{0}_k]) \oplus \{U(F_t,T(F^{(t-1)},\mathcal{V}_0))\times[\mathbf{0}_k]\}\\
&\supseteq U(F^{(t)},\mathcal{V}_0)\times[\mathbf{0}_k].
\end{aligned}$$

Hence (14) holds for $i = t$. This completes the induction step. □

Example. Let $F_1,\ldots,F_R$: $\mathbb{F}_2^m \times \mathbb{F}_2^k \to \mathbb{F}_2^m$ be blockciphers and $F = F_R \cdots F_1$. A *sequence of linear factors* for F is a tuple of linear mappings $(C_0,C_1,\ldots,C_R,D)$ such that $C_0,\ldots,C_R$ have domains $\mathbb{F}_2^m$, D has domain $\mathbb{F}_2^k$, and there are mappings $\tilde{F}_1,\ldots,\tilde{F}_R$ with

$$C_iF_i(\mathbf{p},\mathbf{k}) = \tilde{F}_i(C_{i-1}\mathbf{p},D\mathbf{k}) \text{ for } i = 1,\ldots,R,\ \mathbf{p}\in\mathbb{F}_2^m,\ \mathbf{k}\in\mathbb{F}_2^k.\ ^{\dagger}$$

This notion was introduced in [1]. It is easy to check that the spaces $\mathcal{V}_i = ker(C_i)\times ker(D)$ and $\mathcal{W}_i = ker(C_i)$ $(i = 0,\ldots,R)$ satisfy (11). Hence $(\mathcal{V}_0,\mathcal{W}_R)$ is a linear structure of F, which is recursive over $F_1,\ldots,F_R$.

---

† This definition is not consistent with that of a "linear factor" in [7] or in §1 of the present paper.

§5. LINEAR STRUCTURES IN DES-LIKE CIPHERS

In this section we introduce DES-like ciphers, which are product ciphers with a similar structure as DES. We investigate the recursive linear structures of these DES-like ciphers. The class of DES-like ciphers contains, among others, DES and Lucifer (after some slight modifications) and truncated versions of these product ciphers with a reduced number of rounds.

Let m,k,l,n,m_1,n_1,R be positive integers with $m=2lm_1$ and $n=ln_1$. Elements of $\mathbb{F}_2^m$ are often denoted by $(\mathbf{p},\mathbf{q})$, where $\mathbf{p},\mathbf{q}\in\mathbb{F}_2^{m/2}$. Whenever convenient, we write elements of $\mathbb{F}_2^{m/2}$ as $(\mathbf{q}_1,\ldots,\mathbf{q}_l)$ with $\mathbf{q}_j\in\mathbb{F}_2^{m_1}$ for $j=1,\ldots,l$ and elements of $\mathbb{F}_2^n$ as l-tuples of elements of $\mathbb{F}_2^{n_1}$. A DES-like cipher with message space $\mathbb{F}_2^m$ and key space $\mathbb{F}_2^k$ is a product cipher

$$F=F_RF_{R-1}\cdots F_1, \tag{15}$$

whose rounds F_i $(i=1,\ldots,R)$ are defined by

$$F_i(\mathbf{p},\mathbf{q},\mathbf{k})=(\mathbf{q},\mathbf{p}+S(L\mathbf{q}+K_i\mathbf{k})) \text{ for } (\mathbf{p},\mathbf{q})\in\mathbb{F}_2^m,\ \mathbf{k}\in\mathbb{F}_2^k. \tag{16}$$

Here the mapping $S:\mathbb{F}_2^n\to\mathbb{F}_2^{m/2}$ is given by

$$S(\mathbf{x}_1,\ldots,\mathbf{x}_l)=(S_1\mathbf{x}_1,\ldots,S_l\mathbf{x}_l) \text{ for } \mathbf{x}_1,\ldots,\mathbf{x}_l\in\mathbb{F}_2^{n_1},$$

where $S_1,\ldots,S_l:\mathbb{F}_2^{n_1}\to\mathbb{F}_2^{m_1}$ are certain non-linear mappings (the S-boxes); $L:\mathbb{F}_2^{m/2}\to\mathbb{F}_2^n$ is a linear mapping; and $K_1,\ldots,K_R:\mathbb{F}_2^k\to\mathbb{F}_2^n$ are linear mappings (the key scheduling) such that for $i=1,\ldots,R$, the linear mapping $J_i:\mathbb{F}_2^m\times\mathbb{F}_2^k\to\mathbb{F}_2^n$, given by

$$J_i(\mathbf{p},\mathbf{q},\mathbf{k})=L\mathbf{q}+K_i\mathbf{k},$$

is surjective.

Linear structures in the rounds F_i can be described in terms of linear structures in the S-boxes, as defined below. We remark that searching for the linear structures in the S-boxes is feasible when the input size n_1 of the S-boxes is small. In that case it is also feasible to find the linear structures in the rounds. For each j in $\{1,\ldots,l\}$ and each subspace $\mathfrak{A}$ of $\mathbb{F}_2^{n_1}$ we define the subspace

$$U(S_j,\mathfrak{A})=\bigoplus_{\substack{\mathbf{x}\in\mathbb{F}_2^{n_1}\\ \mathbf{u}\in\mathfrak{A}}}[S_j(\mathbf{x}+\mathbf{u})+S_j(\mathbf{x})+S_j(\mathbf{u})+S_j(\mathbf{0}_{n_1})].$$

A linear structure of S_j is a pair $(\mathfrak{A},\mathfrak{X})$ for which $\mathfrak{A}$ is a subspace of $\mathbb{F}_2^{n_1}$, $\mathfrak{X}$ is a subspace of $\mathbb{F}_2^{m_1}$ and $U(S_j,\mathfrak{A})\subseteq\mathfrak{X}$; $(\mathfrak{A},\mathfrak{X})$ is said to be *trivial* if $\mathfrak{A}=[\mathbf{0}_{n_1}]$ or $\mathfrak{X}=\mathbb{F}_2^{m_1}$.

Let $\rho_j:\mathbb{F}_2^n\to\mathbb{F}_2^{n_1}$ be the j-th projection given by $\rho_j(\mathbf{x}_1,\ldots,\mathbf{x}_l)=\mathbf{x}_j$. If $(\mathbf{p},\mathbf{q},\mathbf{k})\in\mathbb{F}_2^m\times\mathbb{F}_2^k$ is an input to some round F_i then $\rho_jJ_i(\mathbf{p},\mathbf{q},\mathbf{k})$ is the part of that input going into S-box S_j. In the lemma below, elements of $\mathbb{F}_2^m$ are written as $(\mathbf{p},\mathbf{q}_1,\ldots,\mathbf{q}_l)$,

with $\mathbf{p} \in \mathbb{F}_2^{\frac{1}{2}m}$ and $\mathbf{q}_1, \ldots, \mathbf{q}_l \in \mathbb{F}_2^{m_1}$.

Lemma 4. *Let $i \in \{1, \ldots, R\}$, and suppose that F_i is given by* (16). *Further, let $\mathcal{V}$ be a subspace of $\mathbb{F}_2^m \times \mathbb{F}_2^k$. Then*

$$U(F_i, \mathcal{V}) = [\mathbf{0}_{\frac{1}{2}m}] \times \{U(S_1, \rho_1 J_i(\mathcal{V})) \times U(S_2, \rho_2 J_i(\mathcal{V})) \times \cdots \times U(S_l, \rho_l J_i(\mathcal{V}))\}. \tag{17}$$

Proof. Denote the space on the right-hand side of (17) by $\mathfrak{L}$. We first prove that $U(F_i, \mathcal{V}) \subseteq \mathfrak{L}$. To this end, it is sufficient to prove that each vector $F_i(\mathbf{p}+\mathbf{p}_0, \mathbf{q}+\mathbf{q}_0, \mathbf{k}+\mathbf{k}_0) + F_i(\mathbf{p}, \mathbf{q}, \mathbf{k}) + F_i(\mathbf{p}_0, \mathbf{q}_0, \mathbf{k}_0) + F_i(\mathbf{0}_{\frac{1}{2}m}, \mathbf{0}_{\frac{1}{2}m}, \mathbf{0}_k)$ with $(\mathbf{p}_0, \mathbf{q}_0, \mathbf{k}_0) \in \mathcal{V}$ and $(\mathbf{p}, \mathbf{q}, \mathbf{k}) \in \mathbb{F}_2^m \times \mathbb{F}_2^k$ can be written as $(\mathbf{0}_{\frac{1}{2}m}, \mathbf{s}_1, \ldots, \mathbf{s}_l)$, where $\mathbf{s}_j \in U(S_j, \rho_j J_i(\mathcal{V}))$ for $j = 1, \ldots, l$. But this follows at once from the definition of F_i and the fact that $S(\mathbf{x}) = (S_1\rho_1(\mathbf{x}), \ldots, S_l\rho_l(\mathbf{x}))$: we have

$$\begin{aligned} &F_i(\mathbf{p}+\mathbf{p}_0, \mathbf{q}+\mathbf{q}_0, \mathbf{k}+\mathbf{k}_0) + F_i(\mathbf{p}, \mathbf{q}, \mathbf{k}) + F_i(\mathbf{p}_0, \mathbf{q}_0, \mathbf{k}_0) + F_i(\mathbf{0}_{\frac{1}{2}m}, \mathbf{0}_{\frac{1}{2}m}, \mathbf{0}_k) \\ &\quad = (\mathbf{0}_{\frac{1}{2}m}, SJ_i(\mathbf{p}+\mathbf{p}_0, \mathbf{q}+\mathbf{q}_0, \mathbf{k}+\mathbf{k}_0) + SJ_i(\mathbf{p}, \mathbf{q}, \mathbf{k}) + SJ_i(\mathbf{p}_0, \mathbf{q}_0, \mathbf{k}_0) + SJ_i(\mathbf{0}_n)) \\ &\quad = (\mathbf{0}_{\frac{1}{2}m}, \mathbf{s}_1, \ldots, \mathbf{s}_l), \end{aligned} \tag{18}$$

where

$$\begin{aligned} \mathbf{s}_j &= S_j(\rho_j J_i(\mathbf{p}, \mathbf{q}, \mathbf{k}) + \rho_j J_i(\mathbf{p}_0, \mathbf{q}_0, \mathbf{k}_0)) + S_j\rho_j J_i(\mathbf{p}, \mathbf{q}, \mathbf{k}) + S_j\rho_j J_i(\mathbf{p}_0, \mathbf{q}_0, \mathbf{k}_0) + S_j(\mathbf{0}_{n_1}) \\ &\in U(S_j, \rho_j J_i(\mathcal{V})) \end{aligned}$$

for $j = 1, \ldots, l$.

We now prove that $\mathfrak{L} \subseteq U(F_i, \mathcal{V})$. It obviously suffices to prove that for each t in $\{1, \ldots, l\}$ we have

$$U(F_i, \mathcal{V}) \supseteq [\mathbf{0}_{\frac{1}{2}m}] \times ([\mathbf{0}_{m_1}] \times \cdots \times U(S_t, \rho_t J_i(\mathcal{V})) \times \cdots \times [\mathbf{0}_{m_1}]), \tag{19}$$

where the space $U(S_t, \rho_t J_i(\mathcal{V}))$ is preceded by $t-1$ spaces $[\mathbf{0}_{m_1}]$ and followed by $l-t$ spaces $[\mathbf{0}_{m_1}]$. In order to prove (19), it is sufficient to show that for each t in $\{1, \ldots, l\}$, $\mathbf{u}$ in $\rho_t J_i(\mathcal{V})$ and $\mathbf{x}$ in $\mathbb{F}_2^{n_1}$, $U(F_i, \mathcal{V})$ contains $(\mathbf{0}_{\frac{1}{2}m}, \mathbf{y}_1, \ldots, \mathbf{y}_l)$, where $\mathbf{y}_t = S_t(\mathbf{x}+\mathbf{u}) + S_t(\mathbf{x}) + S_t(\mathbf{u}) + S_t(\mathbf{0}_{n_1})$, and $y_j = \mathbf{0}_{m_1}$ for $j \neq t$. Fix t, and for each $\mathbf{u}$ in $\rho_t J_i(\mathcal{V})$ and $\mathbf{x}$ in $\mathbb{F}_2^{n_1}$, choose $(\mathbf{p}_\mathbf{u}, \mathbf{q}_\mathbf{u}, \mathbf{k}_\mathbf{u})$ from $\mathcal{V}$ such that $\rho_t J_i(\mathbf{p}_\mathbf{u}, \mathbf{q}_\mathbf{u}, \mathbf{k}_\mathbf{u}) = \mathbf{u}$ and $(\mathbf{p}_\mathbf{x}, \mathbf{q}_\mathbf{x}, \mathbf{k}_\mathbf{x})$ from $\mathbb{F}_2^m \times \mathbb{F}_2^k$ such that $\rho_t J_i(\mathbf{p}_\mathbf{x}, \mathbf{q}_\mathbf{x}, \mathbf{k}_\mathbf{x}) = \mathbf{x}$ and $\rho_j J_i(\mathbf{p}_\mathbf{x}, \mathbf{q}_\mathbf{x}, \mathbf{k}_\mathbf{x}) = \mathbf{0}_{n_1}$ for all $j \neq t$. This is possible since we assumed that J_i is surjective. By (18), $U(F_i, \mathcal{V})$ contains the vector

$$\begin{aligned} &F_i(\mathbf{p}_x + \mathbf{p}_u, \mathbf{q}_x + \mathbf{q}_u, \mathbf{k}_x + \mathbf{k}_u) + F_i(\mathbf{p}_x, \mathbf{q}_x, \mathbf{k}_x) + F_i(\mathbf{p}_u, \mathbf{q}_u, \mathbf{k}_u) + F_i(\mathbf{0}_{\frac{1}{2}m}, \mathbf{0}_{\frac{1}{2}m}, \mathbf{0}_k) \\ &\quad = [(\mathbf{0}_{\frac{1}{2}m}, \mathbf{y}_1, \ldots, \mathbf{y}_l)], \end{aligned}$$

where

$$\begin{aligned} \mathbf{y}_j = {} & S_j(\rho_j J_i(\mathbf{p}_\mathbf{x}, \mathbf{q}_\mathbf{x}, \mathbf{k}_\mathbf{x}) + \rho_j J_i(\mathbf{p}_\mathbf{u}, \mathbf{q}_\mathbf{u}, \mathbf{k}_\mathbf{u})) + \\ & + S_j\rho_j J_i(\mathbf{p}_\mathbf{x}, \mathbf{q}_\mathbf{x}, \mathbf{k}_\mathbf{x}) + S_j\rho_j J_i(\mathbf{p}_\mathbf{u}, \mathbf{q}_\mathbf{u}, \mathbf{k}_\mathbf{u}) + S_j(\mathbf{0}_{n_1}) \end{aligned}$$

for $j=1,\ldots,t$. But it is easy to check that $\mathbf{y}_t=S_j(\mathbf{x}+\mathbf{u})+S_j(\mathbf{x})+S_j(\mathbf{u})+S_j(\mathbf{0}_{n_1})$, while $\mathbf{y}_j=\mathbf{0}_{m_1}$ for $j\neq t$. This completes the proof of Lemma 4. □

In the lemma below we show that each DES-like cipher has a recursive linear structure comparable to the complementation property of DES. Let

$$\mathcal{C}_F = \left\{(\mathbf{p}_0,\mathbf{q}_0,\mathbf{k}_0)\in\mathbb{F}_2^m\times\mathbb{F}_2^k\colon \begin{array}{l} L\mathbf{q}_0+K_i\mathbf{k}_0=\mathbf{0}_n \text{ for all odd } i \text{ in } \{1,\ldots,R\} \\ L\mathbf{p}_0+K_i\mathbf{k}_0=\mathbf{0}_n \text{ for all even } i \text{ in } \{1,\ldots,R\}\end{array}\right\}.$$

$\mathcal{C}_F$ is called the *complementation space* of F. Then we have:

Lemma 5. *Let F be the DES-like cipher defined by* (15) *and* (16). *Then $(\mathcal{C}_F,[\mathbf{0}_m])$ is a linear structure of F, which is recursive over $F_1,\ldots,F_R$.*

Proof. Let $\mathcal{W}_i=[\mathbf{0}_m]$ for $0\leqslant i\leqslant R$ and

$\mathcal{V}_i=\mathcal{C}_F$ if $0\leqslant i\leqslant R$, i even;

$\mathcal{V}_i=\{(\mathbf{q}_0,\mathbf{p}_0,\mathbf{k}_0)\colon (\mathbf{p}_0,\mathbf{q}_0,\mathbf{k}_0)\in\mathcal{C}_F\}$ if $0\leqslant i\leqslant R$, i odd.

From Lemma 2 we infer that for every subspace $\mathcal{V}=\bigoplus_{t=1}^{s}[(\mathbf{p}_t,\mathbf{q}_t,\mathbf{k}_t)]$ of $\mathbb{F}_2^m\times\mathbb{F}_2^k$ we have

$$T(F_i,\mathcal{V}) = \left\{\bigoplus_{t=1}^{s}[(\mathbf{q}_t,\mathbf{p}_t+S(L\mathbf{q}_t+K_i\mathbf{k}_t)+S(\mathbf{0}_{n_1})]\right\}\oplus U(F_i,\mathcal{V}). \tag{20}$$

Using this fact together with Lemma 4 and the fact that $\mathcal{V}_{i-1}\subseteq ker(J_i)$ for $i=1,\ldots,R$, it follows that $\mathcal{V}_0,\ldots,\mathcal{V}_R,\mathcal{W}_0,\ldots,\mathcal{W}_R$ satisfy the relations (11) in Lemma 3. Since $\mathcal{V}_0=\mathcal{C}_F$ and $\mathcal{W}_R=[\mathbf{0}_m]$ this proves Lemma 5. □

Let F be defined by (15) and (16). To F we associate an *error propagation map* D_F, which maps every l-tuple $(\mathcal{X}_1,\ldots,\mathcal{X}_l)$ of subspaces of $\mathbb{F}_2^{m_1}$ to the l-tuple $(\mathcal{Y}_1,\ldots,\mathcal{Y}_l)$ of subspaces of $\mathbb{F}_2^{m_1}$ for which

$$\mathcal{Y}_j=U(S_j,\rho_jL(\mathcal{X}_1\times\cdots\times\mathcal{X}_l)) \text{ for } j=1,\ldots,l.$$

Any change in the plaintext or key affects in some way the outputs of the S-boxes after the first round. The effects on the outputs of the S-boxes propagate in the second round, and result in certain effects on the outputs of the S-boxes after the second round. Continuing in this way, the outputs of the S-boxes after each round are affected. Informally speaking, D_F describes, how the effects on the outputs of the S-boxes after some round propagate in the next round (the so-called *error propagation* in one round). Suppose that the spaces $\mathcal{X}_1,\ldots,\mathcal{X}_l$ describe the effects on the outputs of S-boxes $S_1,\ldots,S_l$, respectively, after the i-th round, say. Due to the linear mapping L, the effect on the output of S-box S_j causes some effect on the inputs of several S-boxes in the $i+1$-st round. The total effect on the input of S-box S_t, say, in round $i+1$, caused by the effects on the outputs of all S-boxes in round i, can be described by the space $\rho_tL(\mathcal{X}_1\times\cdots\times\mathcal{X}_l)$. Thus,

the effect on the output of S_t after the $i+1$-st round is described by the space $\mathcal{Y}_t$. Intuitively speaking, if the spaces $\mathcal{Y}_j$ are larger, then the error propagation in one round is stronger. D_F^i (D_F iterated i times) describes the error propagation in i consecutive rounds. For F to be secure it is desirable that there is a number $P \leqslant R$ such that

$$D_F^P(\mathcal{X}_1, \ldots, \mathcal{X}_l) = (\mathbb{F}_2^{m_1}, \ldots, \mathbb{F}_2^{m_1}) \quad \text{for all subspaces } \mathcal{X}_1, \ldots, \mathcal{X}_l \text{ of } \mathbb{F}_2^{m_1} \text{ with at least one } \neq [\mathbf{0}_{m_1}]. \tag{21}$$

If P is the smallest integer for which (21) holds, then F is said to have *optimal error propagation after P rounds*. It seems, that a good design criterion for a DES-like cipher is to make the number of rounds after which F has optimal error propagation as small as possible. For instance, this can be achieved by choosing S-boxes without non-trivial linear structures and choosing L in a careful way. It is easy to see that (21) holds if and only if $D_F^P(\mathcal{X}_1, \ldots, \mathcal{X}_l) = (\mathbb{F}_2^{m_1}, \ldots, \mathbb{F}_2^{m_1})$ for every tuple of spaces $(\mathcal{X}_1, \ldots, \mathcal{X}_l)$, for which exactly one space is generated by a single non-zero vector, while the other spaces are $[\mathbf{0}_{m_1}]$. Hence in order to find the smallest P for which (21) holds, one merely has to compute D_F^i ($i = 1, 2, \cdots$) for $l(2^{m_1} - 1)$ tuples $(\mathcal{X}_1, \ldots, \mathcal{X}_l)$. This is feasible if l, m_1 and n_1 are small.

It also seems, that another good design criterion for the DES-like cipher F given by (15) and (16) is to choose the mappings L and $K_1, \ldots, K_R$ such that truncations of the DES-like cipher after a few rounds have no larger complementation space than the DES-like cipher itself. We say that F has *no extra complementation after Q rounds* if Q is the smallest integer for which the space

$$\left\{(\mathbf{p}_0, \mathbf{q}_0, \mathbf{k}_0) \in \mathbb{F}_2^m \times \mathbb{F}_2^k : \begin{array}{l} L\mathbf{p}_0 + K_t\mathbf{k}_0 = \mathbf{0}_n \text{ for all even } t \leqslant Q \\ L\mathbf{q}_0 + K_t\mathbf{k}_0 = \mathbf{0}_n \text{ for all odd } t \leqslant Q \end{array}\right\}$$

is equal to the complementation space $\mathcal{C}_F$. Provided that m, k and n are not too large, computing Q is feasible.

Below we give a sufficient condition for a DES-like cipher to have no non-trivial recursive linear structures other than that given in Lemma 5.

THEOREM. *Let F be the DES-like cipher given by* (15) *and* (16) *and suppose that the following three conditions are satisfied:*

(i) $U(S_j, \mathcal{U}) \neq [\mathbf{0}_{m_1}]$ *for every subspace* $\mathcal{U}$ *of* $\mathbb{F}_2^{n_1}$ *with* $\mathcal{U} \neq [\mathbf{0}_{n_1}]$;

(ii) *F has optimal error propagation after P rounds and no extra complementation after Q rounds;*

(iii) $R > P + Q$.

Then for every linear structure $(\mathcal{V}, \mathcal{W})$ *that is recursive over* $F_1, \ldots, F_R$ *and for which* $\mathcal{V}$ *is not contained in* $\mathcal{C}_F$, *we have* $\mathcal{W} = \mathbb{F}_2^m$.

PROOF. Let $\mathcal{V}_0, \ldots, \mathcal{V}_R, \mathcal{W}_0, \ldots, \mathcal{W}_R$ be a sequence of linear spaces satisfying the conditions of (11) (cf. Lemma 3) such that $\mathcal{V}_0$ is not contained in $\mathcal{C}_F$. We have to prove that $\mathcal{W}_R = \mathbb{F}_2^m$. To this end, we need two lemmas.

Lemma 6. *Let $1 \leqslant i \leqslant R-1$ and suppose that $\mathcal{W}_i \supseteq [\mathbf{0}_{\frac{1}{2}m}] \times \mathcal{X}_1 \times \cdots \times \mathcal{X}_l$, where $\mathcal{X}_1, \ldots, \mathcal{X}_l$ are subspaces of $\mathbb{F}_2^{m_1}$. Then $\mathcal{W}_{i+1} \supseteq [\mathbf{0}_{\frac{1}{2}m}] \times \mathcal{Y}_1 \times \cdots \times \mathcal{Y}_l$, where*

$$(\mathcal{Y}_1, \ldots, \mathcal{Y}_l) = D_F(\mathcal{X}_1, \ldots, \mathcal{X}_l).$$

Proof. (11) implies that $\mathcal{W}_{i+1} \supseteq U(F_{i+1}, \mathcal{W}_i \times [\mathbf{0}_k])$. Together with Lemma 4 this implies Lemma 6. □

Lemma 7. *There is an i with $1 \leqslant i \leqslant Q$ such that $\mathcal{W}_i \supseteq [\mathbf{0}_{\frac{1}{2}m}] \times \mathcal{X}_1 \times \cdots \times \mathcal{X}_l$, where $\mathcal{X}_1, \ldots, \mathcal{X}_l$ are subspaces of $\mathbb{F}_2^{m_1}$ of which at least one is $\neq [\mathbf{0}_{m_1}]$.*

Proof. Let i be the smallest integer for which there is a $(\mathbf{p}_0, \mathbf{q}_0, \mathbf{k}_0) \in \mathcal{V}_0$ such that either $L\mathbf{p}_0 + K_i\mathbf{k}_0 \neq \mathbf{0}_n$ and i even, or $L\mathbf{q}_0 + K_i\mathbf{k}_0 \neq \mathbf{0}_n$ and i odd. Then $1 \leqslant i \leqslant Q$. By arguments similar to those in the proof of Lemma 5, one can show that

$$\begin{cases} \mathcal{V}_t \supseteq \mathcal{V}_0 \text{ for } 1 \leqslant t < i \text{ and } t \text{ even}, \\ \mathcal{V}_t \supseteq \{(\mathbf{q}_0, \mathbf{p}_0, \mathbf{k}_0) : (\mathbf{p}_0, \mathbf{q}_0, \mathbf{k}_0) \in \mathcal{V}_0\} \text{ for } 1 \leqslant t < i \text{ and } t \text{ odd}. \end{cases} \tag{22}$$

Hence $J_i(\mathcal{V}_{i-1}) \neq [\mathbf{0}_n]$. Put $\mathcal{X}_j = U(S_j, \rho_j J_i(\mathcal{V}_{i-1}))$ for $j = 1, \ldots, l$. By condition (i) of the Theorem, at least one of the spaces $\mathcal{X}_j$ is $\neq [\mathbf{0}_{m_1}]$, and by (11) and Lemma 4 we have $\mathcal{W}_i \supseteq U(F_i, \mathcal{V}_{i-1}) \supseteq [\mathbf{0}_{\frac{1}{2}m}] \times \mathcal{X}_1 \times \cdots \times \mathcal{X}_l$. This proves Lemma 7. □

We are now ready to complete the proof of the theorem. By Lemma 7 there is an i with $1 \leqslant i \leqslant Q$ and

$$\mathcal{W}_i \supseteq [\mathbf{0}_{\frac{1}{2}m}] \times \mathcal{X}_1 \times \cdots \times \mathcal{X}_l,$$

where $\mathcal{X}_1, \ldots, \mathcal{X}_l$ are subspaces of $\mathbb{F}_2^{m_1}$ of which at least one is $\neq [\mathbf{0}_{m_1}]$. By Lemma 6 we have for $t = 1, 2, \cdots$,

$$\mathcal{W}_{i+t} \supseteq [\mathbf{0}_{\frac{1}{2}m}] \times \mathcal{Y}_1 \times \cdots \times \mathcal{Y}_l \text{ with } (\mathcal{Y}_1, \ldots, \mathcal{Y}_l) = D_F^t(\mathcal{X}_1, \ldots, \mathcal{X}_l),$$

so that in particular,

$$\mathcal{W}_{i+P} \supseteq [\mathbf{0}_{\frac{1}{2}m}] \times \mathbb{F}_2^{\frac{1}{2}m}.$$

Since F has optimal error propagation after some number of rounds, there are subspaces $\mathcal{Z}_1, \ldots, \mathcal{Z}_l$ of $\mathbb{F}_2^{m_1}$ such that $D_F(\mathcal{Z}_1, \ldots, \mathcal{Z}_l) = (\mathbb{F}_2^{m_1}, \ldots, \mathbb{F}_2^{m_1})$. Hence $D_F(\mathbb{F}_2^{m_1}, \ldots, \mathbb{F}_2^{m_1}) = (\mathbb{F}_2^{m_1}, \ldots, \mathbb{F}_2^{m_1})$. Together with (11), Lemma 2 (or (20)) and Lemma 6 this implies that

$$\mathcal{W}_s = \mathbb{F}_2^m \text{ for } s > i + P.$$

But by condition (iii) we have $R > P + Q \geqslant i + P$. We conclude that $\mathcal{W}_R = \mathbb{F}_2^m$. □

§6. APPLICATION TO DES

For convenience we modify DES a little bit: first, we do not use the tables IP and PC1 in the NBS-description (cf. [6]); and second, we combine the tables E and P, in the way described in [2], §3. Plaintexts and ciphertexts of DES are denoted by $(\mathbf{p},\mathbf{q})$, where $\mathbf{p},\mathbf{q}\in\mathbb{F}_2^{32}$.

DES is composed of the following mappings (cf. [6], [1]):
$P:\mathbb{F}_2^{32}\to\mathbb{F}_2^{32}$: bit permutation;
$E:\mathbb{F}_2^{32}\to\mathbb{F}_2^{48}$: bit expansion;
$S_l:\mathbb{F}_2^{6}\to\mathbb{F}_2^{4}$ $(l=1,\ldots,8)$: S-boxes;
$S:\mathbb{F}_2^{48}\to\mathbb{F}_2^{32}$: $S(\mathbf{x}_1,\ldots,\mathbf{x}_8)=(S_1\mathbf{x}_1,\ldots,S_8\mathbf{x}_8)$ for $\mathbf{x}_1,\ldots,\mathbf{x}_8\in\mathbb{F}_2^{6}$;
$K_i:\mathbb{F}_2^{56}\to\mathbb{F}_2^{48}$ $(i=1,\ldots,16)$: key scheduling; all K_i are permuted choices of key bits, defined by PC2 and the shifting pattern.

Let F_i: $\mathbb{F}_2^{64}\times\mathbb{F}_2^{56}\to\mathbb{F}_2^{64}$ be the i-th round of DES, defined by

$$F_i(\mathbf{p},\mathbf{q},\mathbf{k})=(\mathbf{q},\mathbf{p}+S(EP\mathbf{q}+K_i\mathbf{k})) \text{ for } i=1,\ldots,16$$

and put

$$DES_{ST}=F_T\cdots F_S.$$

DES_{ST} is obviously a DES-like cipher with parameters $m=64$, $k=56$, $n=48$, $m_1=4$, $n_1=6$, $l=8$ and $R=T-S+1$. The complementation space of $DES_{1,16}$ is equal to $[(\mathbf{1}_{64},\mathbf{1}_{56})]$.

It follows from the Theorem of §5 that product ciphers, consisting of seven or more consecutive rounds of DES, have no non-trivial recursive linear structures other than the complementation property.

Corollary. *Let S, T be integers with $1\leqslant S<T\leqslant 16$ and $T\geqslant S+6$. If $(\mathfrak{V},\mathfrak{W})$ is a linear structure of DES_{ST} such that $(\mathfrak{V},\mathfrak{W})$ is recursive over $F_S,\ldots,F_T$ and $\mathfrak{V}$ is not equal to $[(\mathbf{0}_{64},\mathbf{0}_{56})]$ or $[(\mathbf{1}_{64},\mathbf{1}_{56})]$, then $\mathfrak{W}=\mathbb{F}_2^{64}$.*

Proof. Put $R=T-S+1$, and let

$$\mathcal{C}_i=\left\{(\mathbf{p}_0,\mathbf{q}_0,\mathbf{k}_0)\in\mathbb{F}_2^{64}\times\mathbb{F}_2^{56}\to\mathbb{F}_2^{64}:\begin{matrix}EP\mathbf{p}_0+K_{i+S-1}\mathbf{k}_0=\mathbf{0}_{48} \text{ if } i \text{ even}\\ EP\mathbf{q}_0+K_{i+S-1}\mathbf{k}_0=\mathbf{0}_{48} \text{ if } i \text{ odd}\end{matrix}\right\}.$$

By examining EP and $K_1,\ldots,K_{16}$ (cf. [6]) it can be shown that for all S and T with $1\leqslant S<T\leqslant 16$ and $T\geqslant S+6$ and all $i\geqslant 4$ we have $\mathcal{C}_i=[\mathbf{1}_{64},\mathbf{1}_{56}]$. Hence DES_{ST} has no extra complementation after Q rounds, for some integer $Q\leqslant 4$. By investigating the S-boxes and EP (see also [1], §§3,4) it can be shown that each blockcipher DES_{ST} with $T\geqslant S+6$ has optimal error propagation after $P:=2$ rounds. An investigation of the S-boxes implies that condition (i) of the Theorem holds. Finally, $R\geqslant 7>P+Q$. Now the Corollary follows at once from the Theorem. □

Remark. The condition $T \geqslant S+6$ cannot be replaced by $T \geqslant S+5$. For instance, one can verify that $DES_{2,7}$ has a recursive linear structure $(\mathcal{V}, \mathcal{W})$ for which $\mathcal{V}$ is not contained in $[(\mathbf{1}_{64}, \mathbf{1}_{56})]$, whereas $\mathcal{W} \neq \mathbb{F}_2^{64}$.

§7. POSSIBLE EXTENSIONS

In [4], §IV, Hellman et al. suggested the following way to break DES, which might also apply to an arbitrary DES-like cipher F: modify the S-boxes of F such that the resulting DES-like cipher F', with the modified S-boxes, is easy to break. If the modification in each S-box S is such that the output $S(\mathbf{x})$ is changed for only a few inputs $\mathbf{x}$, then F and F' give the same ciphertexts for a non-negligible fraction of pairs of plaintexts and keys. For these plaintexts and keys, the key in F can be found by searching for the key in F'. Some of the potential possibilities of this attack were already discussed in [1], §2.1.

From the investigations in §5 it follows that recursive linear structures in DES-like ciphers are built up from linear structures in the S-boxes. Therefore, Hellman et al.'s attack described above might work if some of the S-boxes of a DES-like cipher have small *distances* to certain linear structures. Here the distance of an S-box to a particular linear structure is the minimal number of outputs of that S-box that must be changed to obtain an S-box with that linear structure.

There is a close relationship between the collection of distances of an S-box to all linear structures and the statistical properties of that S-box. Suppose that some S-box has an *A-linear structure*, that is a pair of subsets I, O of input bits and output bits, respectively, with the following property: for exactly a fraction A of those pairs of inputs of the S-box of which only the bits in I differ, the exclusive-or sum of the bits in O of the first corresponding output is equal to the exclusive-or sum of the bits in O of the second corresponding output. If $A=0$ or $A=1$ then that S-box has a linear structure. If $0<A<1$ then that S-box can be transformed to one with a linear structure by appropriately changing a fraction of $\frac{1}{2}\min(A, 1-A)$ of its outputs.

An S-box has maximal distance to each linear structure if each pair of subsets I, O of input bits and output bits, respectively, is a 50%-linear structure of that S-box. This is a strong requirement and it is not clear whether it is feasible to find S-boxes satisfying it. Not all linear structures in the S-boxes of some DES-like cipher will result in non-trivial recursive linear structures of that DES-like cipher. Therefore, it suffices to find out which linear structures in the S-boxes are *dangerous*, in the sense that they would cause recursive linear structures in the DES-like cipher, and then choose S-boxes with large distances to the dangerous linear structures.

It is known that S-box 4 of DES has non-trivial linear structures (cf. [4], §V and [1], §4). Further, structures like the 50% and 25% exclusive-ors, found by Hellman et al. (cf. [4], §V), and the correlation in each S-box between one of the six input bits and the exclusive-or sum of all four output bits, discovered independently by Shamir [8] and

Franklin [3], show that each S-box of DES has small distances to certain linear structures. However, these structures have not been proved useful in the cryptanalysis of DES. It is yet unknown (from the open literature), whether the S-boxes in DES have distances to dangerous linear structures that are small enough to enable a known or chosen plaintext attack faster than exhaustive key search.

Acknowledgement. I would like to thank David Chaum for our many fruitful discussions on the subject of this paper and his suggestions to improve the presentation of this paper, and Evangelos Kranakis for his comments.

REFERENCES

[1] Chaum, D. & Evertse, J.-H, *Cryptanalysis of DES with a reduced number of rounds; sequences of linear factors in block ciphers,* in Advances in Cryptology: Proc. Crypto '85, H.C. Williams, ed., Lecture Notes in Computer Science 218, Springer Verlag, Berlin etc. (1986), pp. 192-211.

[2] Davio, M., Desmedt, Y., Fosseprez, M., Govaerts, R., Hulsbosch, J., Neutjens, P., Piret, P., Quisquater, J.J., Vandewalle, J., Wouters, P., *Analytical characteristics of the DES,* in Advances in Cryptology: Proc. Crypto '83, D. Chaum, ed., Plenum, New York (1984), pp. 171-202.

[3] Franklin, M., M.Sc. Thesis, Univ. Berkeley, May 1985.

[4] Hellman, M., Merkle, R., Schroeppel, R., Washington, L., Diffie, W., Pohlig, S., Schweitzer, P., *Results of an initial attempt to cryptanalyze the NBS Data Encryption Standard,* Information Systems Lab. report SEL 76-042, Stanford University (1976).

[5] Meyer C.H. *Ciphertext / plaintext and ciphertext / key dependencies vs. number of rounds for the Data Encryption Standard,* AFIPS Conference Proceedings, 47, (June 1978), pp. 1119-1126.

[6] National Bureau of Standards, *Data Encryption Standard,* U.S. Department of Commerce, FIPS pub. 46 (January 1977).

[7] Reeds, J.A. & Manferdelli, J.L., *DES has no per round linear factors,* in Advances in Cryptology: Proc. Crypto '84, G.R. Blakley and D. Chaum, eds. Lecture Notes in Computer Science 196, Springer Verlag, Berlin etc. (1985), pp. 377-389.

[8] Shamir, A., *On the security of DES,* in Advances in Cryptology: Proc. Crypto '85, H.C. Williams, ed., Lecture Notes in Computer Science 218, Springer Verlag, Berlin etc. (1986), pp. 280-281.

Fast Data Encipherment Algorithm FEAL

Akihiro Shimizu & Shoji Miyaguchi

Electrical Communication Laboratories, NTT
1-2356, Take, Yokosuka-shi, Kanagawa-ken, 238-03, Japan

BACKGROUND

In data communications and information processing systems, cryptography is the most effective way to secure communications and store data. The most commonly used cryptogryphic algorithm is DES [1]. However, it is generally implemented with hardware, and the cost is prohibitive for small scale systems such as personal computer communications. Accordingly, an encipherment algorithm that has safety equal to DES and is suitable for software as well as hardware implementation is needed. The FEAL (Fast data Encipherment ALgorihtm) fills this need.

EVALUATION INDICES FOR ALGORITHM STRENGTH

In FEAL design, two evaluation indices, M and Ms, are adopted to evaluate objectively the data randomization ability of the algorithm. These indices express the approximation degree of ciphertext variation to the binomial distribution $B(n, 1/2)$, in which n is the ciphertext bit length.

M is the average approximation degree of the distribution of

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 267-278, 1988.
© Springer-Verlag Berlin Heidelberg 1988

ciphertext variations according to the plaintext or key variations from one-bit to n-bit. Ms is the standard deviation of the approximation degree. When M approaches one (100 percent) and Ms approaches zero, the algorithm does not leave clues which could be used to count backward to the input plaintext or key in the ciphertext. M and Ms are definded separately so that Mp and Mps are for plaintext variations and Mk and Mks are for key variations.

To get the indices, many plaintexts or keys have to be used. Nevertheless, the amount of data which can be treated is generally small compared to the population. Thus, it is important to determine the theoretical index values according to the amount of data by means of statistical calculation. For example, the theoretical values for 16•16•16•63 pieces of data, which are a combination of 16 plaintexts, 16 keys and 16•63 plaintext or key variations, are M = 96.5 percent and Ms = 2.6 percent (Table 1). When the measured values of the indices are close to the theoretical values, the randomness of algorithm ciphertexts is considered saturated.

Table 1 Indices for FEAL and DES

Items		FEAL	DES	Theoretical values
Key indices	Mk	96.5	93.4	96.5
	Mks	2.6	4.9	2.6
Plaintext indices	Mp	96.5	95.5	96.5
	Mps	2.6	3.4	2.6

Note: Data amount=16•16•16•63

DESIGN

FEAL consists of two processing parts. One is the key schedule which generates the 256-bit extended key from the 64-bit secret key. It is designed to generate different extended keys for different secret keys (Fig. 1). The other is the data randomizer (Fig. 2), which generates 64-bit ciphertext from 64-bit plaintext under control of the extended key. The data randomizer uses combinations of involutions (3). One program can perform two functions, enciphering and deciphering, except for the extended key entry. Moreover, the setting of 64-bit extended keys by means of an exclusive-OR operation at the entrance and exit makes attack on the algorithm difficult.

The construction of f (Fig. 3) is such that input bit variations influence all output data. Experiments confirmed that FEAL's f function randomization efficiency is two to three times that of DES.

The S function in the f function, a one-byte data substitution, is as effective as DES's S-box.

The S fucntion is defined as:

S(x, y, delta) = ROT2(T); T=x+y+delta mod256;

x, y: one-byte data; delta:constant (0 or 1);

ROT2(T): 2-bit left rotation operation on T.

Example 1: Where x = 00010011, y = 11110010, delta=1, T = 00000110.

Example 2: ROT2(11011100) = 01110011.

The fk function (Fig. 4) used in the key schedule is the same as the f fucntion except for the entry positon of parameter beta.

FEAL VERSIONS

There is an earlier cryptoanalysis report(4) for FEAL(5). For this reason, the iterative number of data randomizer in FEAL is increased from 4 stages to 8 stages. FEAL described in (5) and (6) is called FEAL version 1.00, and the modified FEAL referred to in this paper is called FEAL Version 2.00. Details for FEAL versions are reported in (7).

STRENGTH AND PERFORMANCE of FEAL(Version 2.00)

FEAL working with no parity in a key block is safe from the all-key attack because it is controlled by a 64-bit key, which is more secure than the 56-bit DES key. Regarding ciphertext randomization, FEAL is considered safe because the randomization indices are closer to the theoretical values than those of DES.

When FEAL is implemented in assembly language on an i-8086 16-bit microprocessor with 8 MHz frequency, it is confirmed that the program size is 400 bytes and the excution time speed reaches 120 kbps.

CONCLUSION

FEAL is an encipherment algorithm suitable for software implementation. It can be applied widely to small scale or other existing systems unable to use DES hardware because of cost. Moreover, FEAL is suitable for hardware implementation, too. Implementated as an LSI, it can be used as the cryptographic method in all data communication fields.

ACKNOWLEGEMENT

We thank Dr. Bert den Boer for finding problems hidden in FEAL Version 1.00.

RERERENCES

(1) FIPS PUB 46, Data Encryption Standard (1977).

(2) S. Miyaguchi, M. Hirano:'Evaluation Criteria for Encipherment and Authentication Algorihtms', Trans. of IECE of Japan. Vol. J69-A, No. 10. pp. 1252-1259 (Oct. 1986) (in Japanese).

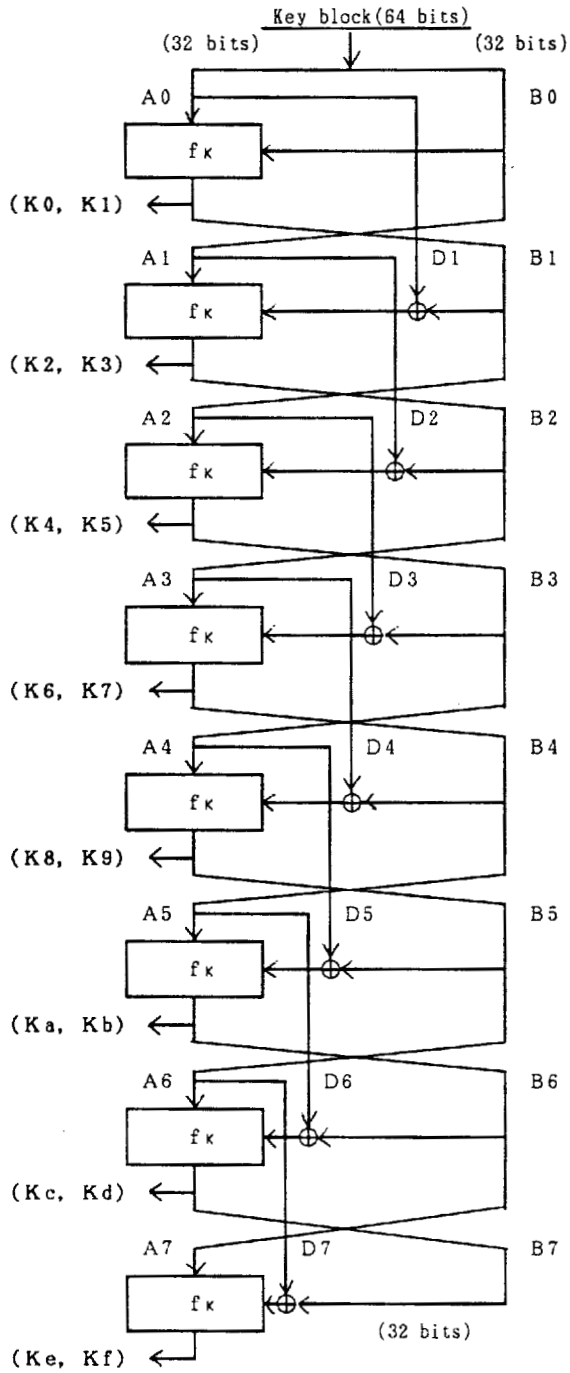
(3) A. G. Kohnheim: 'Cryptography: A Primer', A Wiley Interscience Publication, pp. 236-240 (Jan. 1986).

(4) Bert den Boer, 'Cryptonalysis of FEAL', Crypto'87-Rump Session , Aug. 1987.

(5) A. Shimizu, S. Miyaguchi, 'Fast Data Encipherment Algorihtm FEAL', ABSTRACTS of EUROCRYPT 87 AMSTERDAM, April 1987.

(6) A. Shimizu, S. Miyaguchi,'Fast Data Enciphement Algorihtm FEAL', Trans. of IECE of Japan, Vol. J70-D No. 7 pp. 1413-1423, July 1987 (in Japanese).

(7) An Extension of Fast Data Encipherment Algorithm FEAL, SITA'87, 19-21, Nov. 1987 (in English).



K2(r-1) : Left part of B r (16 bits)
K2(r-1)+1: Right part of B r (16 bits)

Fig. 1 Key processing part

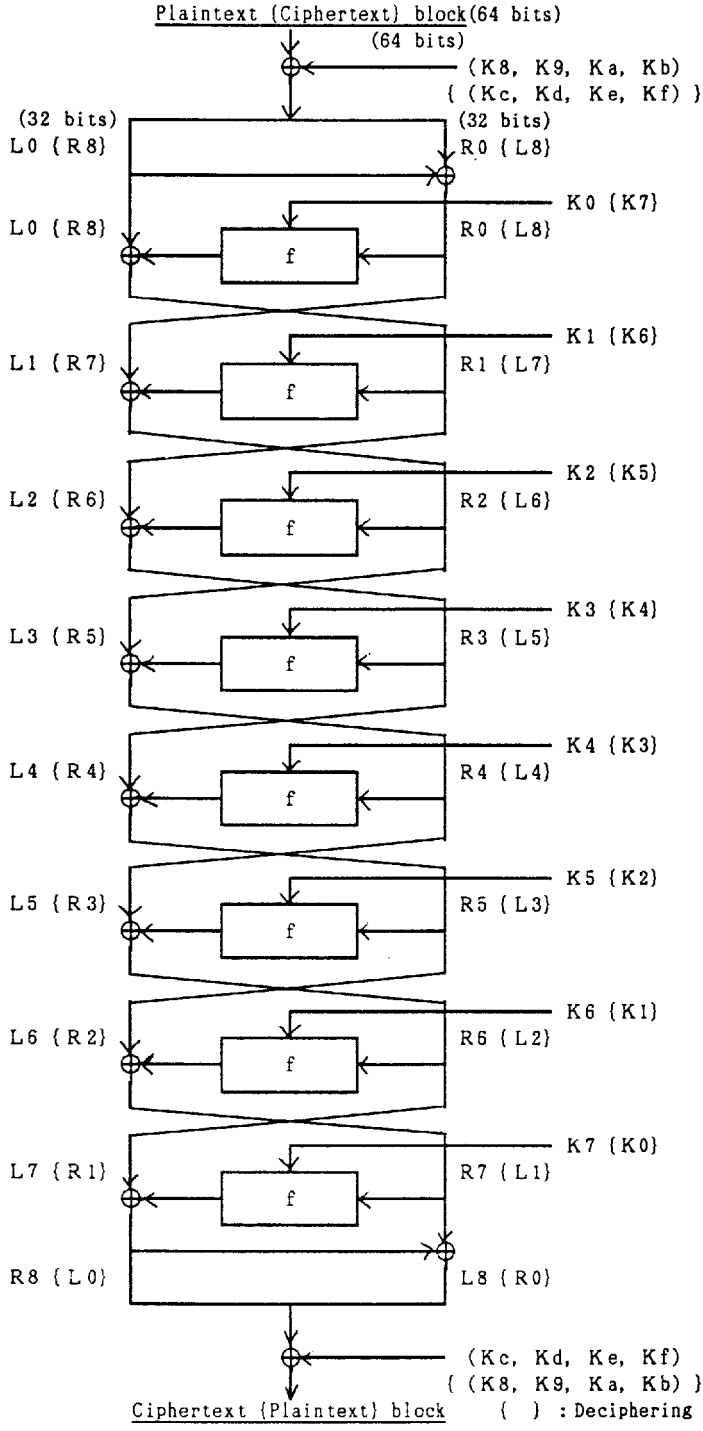
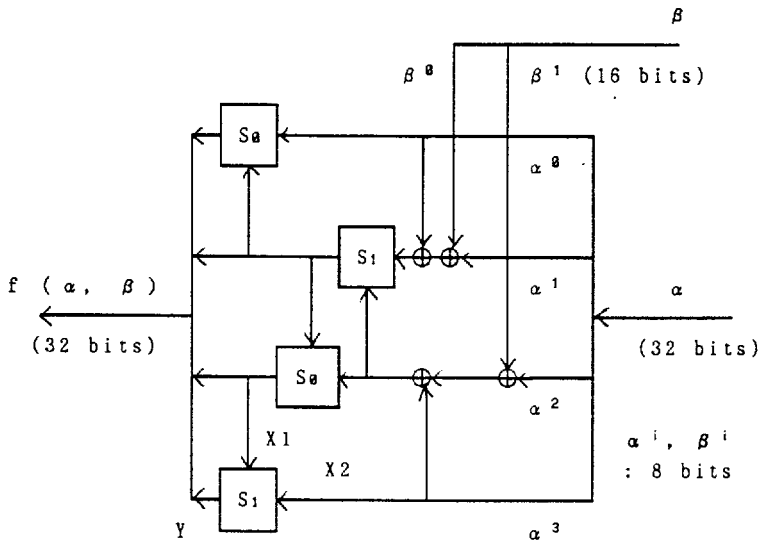


Fig. 2 Data randomizer



Y=S δ (X1,X2)=ROT2((X1+X2+δ)mod256)
Y:output, X1/X2:inputs, δ :parameter(0 or 1)
ROT2:2 bit left rotation on 8-bit data

Fig. 3 Function f

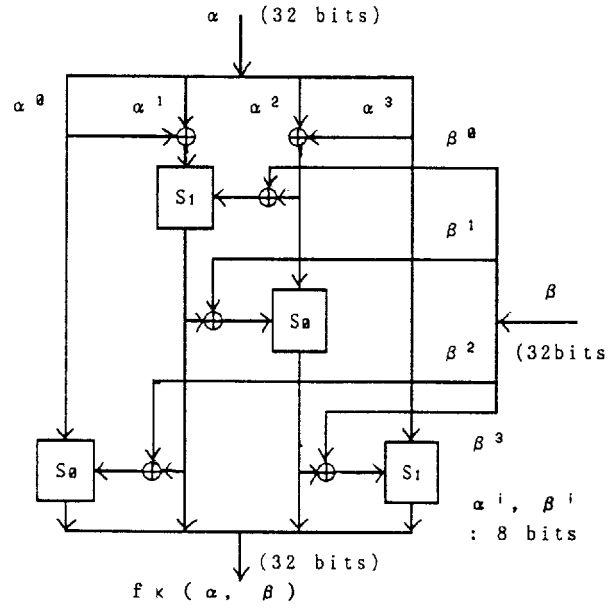


Fig. 4 Function f_κ

Appendix FEAL Specifications

1 Notations

(1) Block: U, U_r ··· are blocks of plural octets.

(2) Octet block: U^j, U_r^j are the j th octets in the blocks U, U_r where j = 0, 1, ···.

(3) Concatenation: (U, V, ····) is a block concatenated with U, V, ···· in this order.

(4) Exclusive-or: $U \oplus V$ is bitwise exclusive-or of block U and V.

(5) Φ is a null block, four octets long.

(6) Assignment: The value of the left side of =sign is assigned the value of the right side.

2 Functions

2.1 Function S

S (X1, X2, δ) = ROT2 (T)

T = X1 + X2 + δ mod 256

where X1, X2 and T are blocks of one-octet, δ = 0 or 1 (constant value), and ROT2 (T) is the result of a 2 bit left rotation operation on T.

Example 1: Where X1 = 00010011, X2 = 11110010, δ = 1, T = 00000110

Example 2: Rot2 (11011100) = 01110011

2.2 Function f_K

Inputs of function f_K, α and β, are divided into four 1-octet blocks as:

$\alpha = (\alpha^0, \alpha^1, \alpha^2, \alpha^3)$

$\beta = (\beta^0, \beta^1, \beta^2, \beta^3)$.

$f_K(\alpha, \beta)$ is shortened to f .

$f = (f^0, f^1, f^2, f^3)$ are calculated in order.

$f_{K^1} = \alpha^1 \oplus \alpha^0$

$f_{K^2} = \alpha^2 \oplus \alpha^3$

$f_{K^1} = S(f_{K^1}, f_{K^2} \oplus \beta^0, 1)$

$f_{K^2} = S(f_{K^2}, f_{K^1} \oplus \beta^1, 0)$

$f_{K^0} = S(\alpha^0, f_{K^1} \oplus \beta^2, 0)$

$f_{K^3} = S(\alpha^3, f_{K^2} \oplus \beta^3, 1)$

2.3 Function f

$f(\alpha, \beta)$ is shortened to f .

$f = (f^0, f^1, f^2, f^3)$ are calculated in order.

$f^1 = \alpha^1 \oplus \beta^0 \oplus \alpha^0$

$f^2 = \alpha^2 \oplus \beta^1 \oplus \alpha^3$

$f^1 = S(f^1, f^2, 1)$

$f^2 = S(f^2, f^1, 0)$

$f^0 = S(\alpha^0, f^1, 0)$

$f^3 = S(\alpha^3, f^2, 1)$

3. Key processing

Let A_0 be to the left of the key K and B_0 to the right, i.e.,

$K = (A_0, B_0)$ and $D0 = \Phi$.

Then calculate K_i (i = 0 to 15) for r = 1 to 8,

$D_r = A_{r-1}$

$A_r = B_{r-1}$

$B_r = f_K(A_{r-1}, B_{r-1} \oplus D_{r-1})$

$K_{2(r-1)} = (B_r^0, B_r^1)$

$K_{2(r-1)+1} = (B_r^2, B_r^3)$

where A_r, B_r and D_r are auxiliary variables.

4. Enciphering and deciphering

4.1 Enciphering procedure

P is separated into L_0, R_0 of equal lengths, i.e., $P = (L_0, R_0)$.

Thus,

$(L_0, R_0) = (L_0, R_0) \oplus (K_8, K_9, K_{10}, K_{11})$

$(L_0, R_0) = (L_0, R_0) \oplus (\Phi, L_0)$

Then calculate $r = 1$ to 8 in that order,

$R_r = L_{r-1} \oplus f(R_{r-1}, K_{r-1})$

$L_r = R_{r-1}$

Lastly, calculate:

$(R_8, L_8) = (R_8, L_8) \oplus (\Phi, R_8)$

$(R_8, L_8) = (R_8, L_8) \oplus (K_{12}, K_{13}, K_{14}, K_{15})$

Ciphertext is (R_8, L_8).

4.2 Decipehring procedure

Ciphertext is separated into R_8, L_8 of equal lengths. Then,

$(R_8, L_8) = (R_8, L_8) \oplus (K_{12}, K_{13}, K_{14}, K_{15})$

$(R_8, L_8) = (R_8, L_8) \oplus (\Phi, R_8)$

Then calculate $r = 8$ to 1 in that order,

$L_{r-1} = R_r \oplus f(L_r, K_{r-1})$

$R_{r-1} = L_r$

Lastly, calculate:

$(L_0, R_0) = (L_0, R_0) \oplus (\Phi, L_0)$

$(L_0, R_0) = (L_0, R_0) \oplus (K_8, K_9, K_{10}, K_{11})$

Plaintext is (L_0, R_0).

5 Parity bits

If parity bits are requested in a key block, the following rule is applied.

Rule: At the begining of key processing, bit positions $8 \times i$ of key block are set to zero where $1 \leqq i \leqq 16$.

6. Working data

is shown in hexadecimal notation.

6.1 When no parity bits exist in a key block

(1) Key = 01 23 45 67 89 AB CD EF

(2) Extended value of the key

(K0, K1, K2, K3, K4, K5, K6, K7) =

DF 3B CA 36 F1 7C 1A EC 45 A5 B9 C7 26 EB AD 25

(K8, K9, K10, K11, K12, K13, K14, K15) =

8B 2A EC B7 AC 50 9D 4C 22 CD 47 9B A8 D5 0C B5

(3) Plaintext = 00 00 00 00 00 00 00 00

(4) Ciphertext= CE EF 2C 86 F2 49 07 52

6.2 When parity bits exist in a key block

(1) Key = 01 23 45 67 89 AB CD EF

(2) Extended value of the key

(K0, K1, K2, K3, K4, K5, K6, K7) =

EF 37 FE DD 04 C3 E3 1D F3 22 B9 A0 C7 AA F6 A6

(K8, K9, K10, K11, K12, K13, K14, K15) =

6A B2 D3 24 F5 DC 72 76 A1 7A 0C 04 B4 E7 CC 8D

(3) Plaintext = 00 00 00 00 00 00 00 00

(4) Ciphertext= 6A 72 2D 1C 46 B3 93 36

MODES OF BLOCKCIPHER ALGORITHMS AND THEIR PROTECTION AGAINST ACTIVE EAVESDROPPING

Cees J.A. Jansen[1] & Dick E. Boekee[2]

[1]Philips USFA B.V.
PO Box 218
5600 MD Eindhoven, The Netherlands

[2]Delft University of Technology
PO Box 5031
2600 GA Delft, The Netherlands

INTRODUCTION

Blockcipher algorithms are used in a variety of modes for message encryption or message authentication. The different modes not all offer the same protection against active eavesdropping. In this paper an overview of a number of modes and their protection against active eavesdropping is presented.

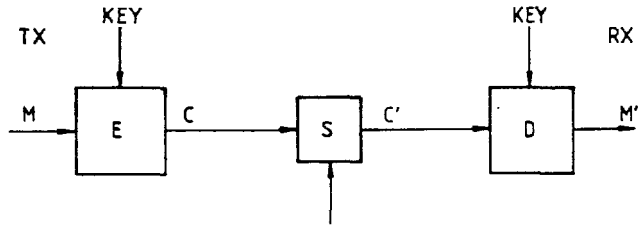
In figure 1 the problem of active eavesdropping (AE) is depicted. It is the objective of an active eavesdropper to manipulate the ciphertext C in such a way that a known message M is turned into a desired message M' without knowledge of the actual key that is being used. The manipulation operation S may consist of e.g. deleting, repeating or inserting parts of a message, but also of performing some arithmetic operation like addition of data.

The fact that M is known may be regarded as somewhat unrealistic, however it might be an authorized (standard) message or it might be a dummy message as is the case with traffic flow security (OSI). So the receiver wonders wether M' is authentic and moreover may be confronted with random active eavesdropping in the form of bit-errors.

There are several methods known which offer protection against active eavesdropping, see e.g. [1]. One could use a message authentication code (MAC), but this is very sensitive to transmission errors, gives a certain text expansion, and gives a delayed notice. With delayed notice is meant that one has to wait a certain number of text blocks before one can possibly detect AE.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 281-286, 1988.
© Springer-Verlag Berlin Heidelberg 1988

Figure 1:



One could also use some form of text feedback, but besides a certain delayed notice this always goes together with a phenomenon called error extension, which means that more than one text block will be erroneous if only one bit-error occurs. Finally one could use techniques for bitstreamciphers [2], but this can result in error extension beyond the block boundary even if only one bit-error occurs.

From the above it will be clear that we desire a method for protection against AE, applicable with blockcipher algorithms, which has no text expansion, no (block) error extension and the possibility of immediate notice. In section 1 various known and new modes and their behaviour under addition, deletion, repetition and insertion will be discussed. In section 2 implementations of the best mode with respect to protection against AE will be shown and their performance discussed.

SECTION 1

Let a blockcipher be given by its encryption and decryption operators E_k and D_k, which act on m-bit blocks under a key k. Blockcipher algorithms can be used in a variety of modes. Well known are the ECB, CBC, CFB and OFB modes [1], but there are more alternatives, such as PBC and PFB where we have interchanged the roles of message M and ciphertext C. Two new modes we have investigated are cipherblock chaining of message difference, CBCPD, and output feedback with a non-linear function, OFBNLF. These modes are depicted in figures 2 and 3 respectively. The OFBNLF mode may be regarded as a combination of an OFB and an ECB mode and can in fact be implemented as such. However in section 2 it will be shown that an implementation can be much simpler.

Figure 2:

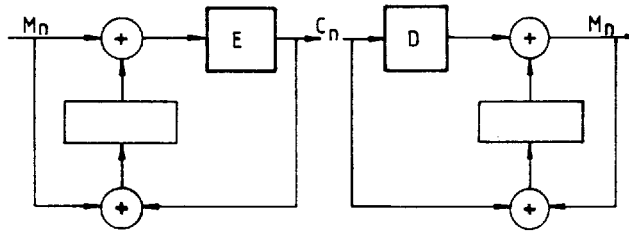
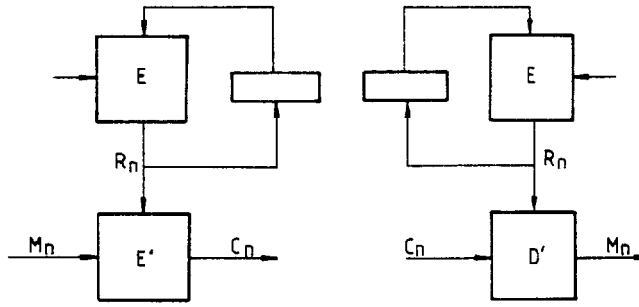


Figure 3:



The modes mentioned above are represented by the following equations:

ECB:	$C_n = E_k(M_n)$	$M_n = D_k(C_n)$
CBC:	$C_n = E_k(M_n + C_{n-1})$	$M_n = D_k(C_n) + C_{n-1}$
CFB:	$C_n = M_n + E_k(C_{n-1})$	$M_n = C_n + E(C_{n-1})$
OFB:	$C_n = M_n + R_n$ $R_n = E_k(R_{n-1})$	$M_n = C_n + R_n$
PBC:	$C_n = E_k(M_n) + M_{n-1}$	$M_n = D_k(C_n + M_{n-1})$
PFB:	$C_n = M_n + E_k(M_{n-1})$	$M_n = C_n + E_k(M_{n-1})$
CBCPD:	$C_n = E_k(M_n + M_{n-1} + C_{n-1})$	$M_n = D_k(C_n) + C_{n-1} + M_{n-1}$
OFBNLF:	$C_n = f_{R_n}(M_n)$ $R_n = E_k(R_{n-1})$	$M_n = f_{R_n}^{-1}(C_n)$

Here C_n and M_n denote the nth ciphertext and plaintext blocks; R_n is the nth block of pseudo-random bits and $f_{R_n}(M_n)$ is the R_n^{th} invertible function acting on plaintext block M_n.

From the equations one can easily see what happens if the nth ciphertext block is deleted, repeated or added to some block S_n. As an example the deciphered message blocks in the case

of deletion and addition are given below.

CIPHERTEXT :	C_{n-1}	C_{n+1}	C_{n+2}	.....
ECB :	M_{n-1}	M_{n+1}	M_{n+2}	.....
CBC / CFB :	M_{n-1}	?	M_{n+2}	.....
OFB / PBC / PFB / CBCPD / OFBNLF :	M_{n-1}	?	?	? ...

CIPHERTEXT :	C_{n-1}	$C_n + S$	C_{n+1}	C_{n+2}	.....
OFB :	M_{n-1}	$M_n + S$	M_{n+1}	M_{n+2}	.....
CFB :	M_{n-1}	$M_n + S$	?	M_{n+2}	.....
PFB :	M_{n-1}	$M_n + S$	?	?	? ...
CBC :	M_{n-1}	?	$M_{n+1} + S$	M_{n+2}	.....
PBC / CBCPD :	M_{n-1}	?	?	?	? ...
ECB / OFBNLF:	M_{n-1}	?	M_{n+1}	M_{n+2}	.....

Here a ? denotes an unknown outcome of the decryption operation.

It can be seen that active eavesdropping will not be successfull with the OFBNLF mode, but occasional errors in the ciphertext will not give rise to block-error extension in the decrypted message.

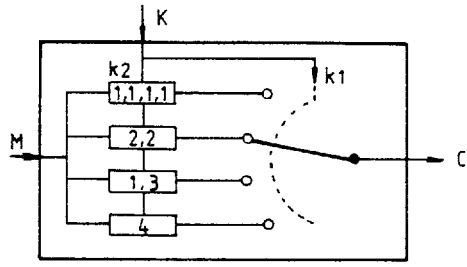
SECTION 2

As was already mentioned in section 1, the OFBNLF mode may be regarded as a combination of an OFB and an ECB mode. However the ECB part can be implemented in a much easier way. The purpose of the latter part is to keep the probability of success sufficiently low if an active eavesdropper performs some fixed transformation on a ciphertext block, such as some kind of addition. A solution which achieves this goal is to pseudo-randomly select a mixing function that mixes the message M and the blocks R. This function should be selected from a set of functions which are not all transparent for the same operation [2].

One example of this method is depicted in figure 4 where the the set of functions consists of four different real additions on 4-bit blocks, i.e. 4 bits modulo 2, 2 x 2 bits modulo 4, 1 bit modulo 2 and 3 bits modulo 8, and 4 bits modulo 16.

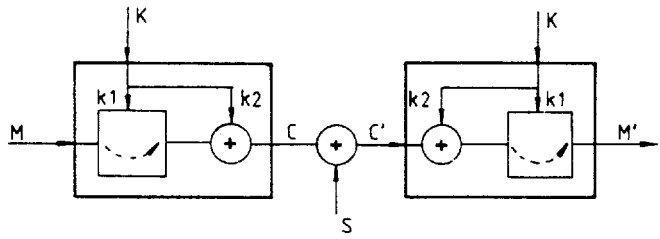
The performance in this case is rather difficult to evaluate, but it appears that the probability of success for an active eavesdropper, who is using real additions, tends to 50% for large blocklengths.

Figure 4:



A second example is depicted in figure 5, where the set of mixing functions consists of functions on the message blocks only, followed by a modulo 2 addition. As a specific set of message block functions cyclic shifts of the message blocks are chosen. Assuming that the active eavesdropper uses modulo 2 addition it can easily be seen that he has to guess the number cyclic shifts of the message block difference correctly.
I.e. $S = CYC_{K1}(M+M')$, where $CYC_k(X)$ denotes the block X shifted cyclically k times. If (M+M') has period P, then the probability of success is 1/P. By assuming equally likely message blocks the average successrate can be calculated. For example if the blocklength is 8 bits, the average successrate is :
(1/1 + 2/2 + 12/4 + 240/8) / 255 = 11/85 or 12.9%. Here we have obviously excluded the zero difference case, as this will always be successfull.

Figure 5:



The following table compares the average successrates of the cyclic shift and the real addition methods for some blocklengths.

BLOCKLENGTH	AVERAGE SUCCESSRATE CYCLIC SHIFTS	REAL ADDITIONS
2	66.7 %	83.3 %
3	42.9 %	78.6 %
4	53.3 %	69.0 %
5	22.6 %	64.5 %
6	20.6 %	59.0 %
7	15.0 %	56.5 %
8	12.9 %	51.0 %

CONCLUSIONS

In this paper we have presented an overview of various modes of blockcipher algorithms and discussed their behaviour with respect to active eavesdropping. We also introduced a new mode called the OFBNLF mode, which offers good protection against active eavesdropping, has no text expansion, no block error extension, the possibility of immediate notice and can be implemented by means of simple operations.

REFERENCES

[1] C.H.Meyer & S.M.Matyas,"Cryptography",John Wiley & Sons, New York, 1982.
[2] C.J.A. Jansen,"Protection against active eavesdropping", EUROCRYPT 86, Linköping, Sweden, 1986.

Security Considerations in the Design and Implementation of a new DES chip [1]

Ingrid Verbauwhede[1,2] Frank Hoornaert[3] Joos Vandewalle[2] Hugo De Man[1,2]

IMEC v.z.w.[1]	ESAT, K.U.Leuven[2]	CRYPTECH n.v.[3]
Kapeldreef 75	K. Mercierlaan 94	Lloyd George Av. 6
B-3030 Heverlee	B-3030 Heverlee	B-1050 Brussel
Belgium	Belgium	Belgium
Tel: 32-(0)16-281211	Tel: 32-(0)16-110931	Tel: 32-(0)2-6425931

1 Introduction

This paper describes the impact of cryptographic requirements on the design of a new highly performant DES chip implementation. Actual cryptographical applications demand for both high security and high speed. It is the aim of this contribution to show how both can be combined.

High security is obtained, because not only the DES algorithm but also all "DES Operation Modes" are implemented together with other extra security features, like triple encryption, MAC's and key security. Some cryptographical optimizations and equivalence transformations, which do not compromise the DES nor the security, were choosen for their very efficient hardware implementation with minimal routing, which otherwise would present a serious problem for any data scrambling algorithm like DES.

High speed is achieved, because these optimizations could be combined with typical chip design principles. So the datarate through the chip can be enhanced and a very compact realisation is possible.

The result is a *single* chip implementation in a classic standard CMOS process. Functionality tests show that a clock of 16.7 Mhz can be applied, which means that a *32 Mbit/sec datarate* can be achieved for all 8 bytes modes. This is the fastest DES chip known today.

Although there are several DES chips available on the market, e.g.[8], there is still a need for more performant and more secure DES chips. None of the existing chips combine high speed with all requirements for high security and userfriendliness. In section 2, all cryptographical requirements that are implemented are drawn up. For each demand, the corresponding hardware solution is explained. It is unique to have all these demands *combined in one device.* Therefore, it is necessary to understand the basic hardware design principles and constraints. This

[1] Research performed in collaboration with the company Cryptech n.v.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 287-300, 1988.
© Springer-Verlag Berlin Heidelberg 1988

is explained briefly in section 3. There are lots of alternative DES representations possible, but only these that overcome the hardware limitations are selected. In section 4 it is explained which ones are selected and how this resulted in an efficient realization. At first sight, testability seems to be in conflict with security requirements : one may not be able to read out keys or intermediate ciphertext while testing the chip. How security is combined with testability is discussed in section 5. Finally section 6 contains the implementation and test results of the first prototype and some predictions are made for the second version, which is now being realized.

2 Demands of cryptographers and the hardware solution

The chip presented here is built in a modular way. This made it possible to handle all requirements separately and to provide a hardware solution corresponding to each problem. The different hardware modules are placed together on the layout afterwards.

Below follows an enumeration of the cryptographical demands. For each demand, the corresponding hardware solution is explained and indicated on the global floorplan, Fig. 2, and layout, Fig. 3.

1. *DES and DES-like algorithms.*
   Since there are questions around the safety of DES, the first objective is to implement DES as well as DES like algorithms. The first module contains hardware for one DES round, as shown in Fig. 1. On the layout in Fig. 3 one can easily see at the bottom 8 PLA's for 8 substitution functions. On top of the PLA's the hardware routing for the permutation P is placed. Above the permutation, the left and right registers, the 32 and 48 exors and the expansion E are placed.

   To realize *DES-like algorithms* one can simply cut out the S-boxes or the permutation P and replace them by others. Or one can enhance the number of rounds to increase the calculation effort of an exhaustive search attack. On the layout, this corresponds to setting the counter of the local controller to a higher number.

2. *Key management and security.*
   The *Key scheduling scheme* of the DES algorithm is implemented in a second module, combined with the safe key storage. The key part is placed in the middle of the layout as shown in Fig. 2 and Fig. 3. One can see the key permutation and selection PC2 and the left right shifting hardware LR. Most area is needed for the implementation of the four key registers, KR1 to KR4, e.g. for two master keys and two session keys.

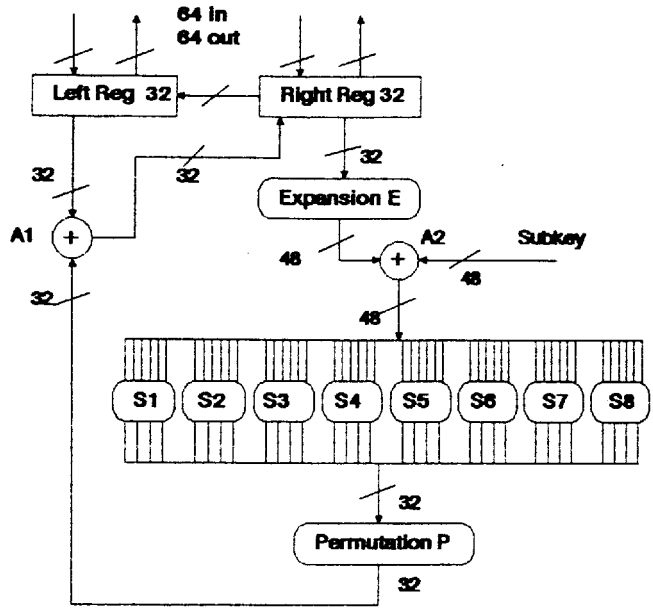


Figure 1: Architecture for one DES round.

The key scheduling is combined with the keyregisters because it makes it more easy to change it for DES-like algorithms. e.g. One can ask for another left right shifting sequence or for the use of two keys (128 bits) or more during one encryption.

For the safety of the keys, it is impossible to read out keys, once they entered the chip, [3]. It is also made impossible to use partial keys. (e.g. a half new and a half old key.) If a new key is entered in a register, the old key is disabled and can not be used anymore for encryption. When someone tries to tamper, a general reset occurs and all keys are disabled. Also key parity is checked on chip and keyexoring of incoming keyparts is possible.

3. Remark that besides the 16 DES rounds and the keyscheduling scheme, the DES algorithm contains also the initial and inverse initial permutation, IP and IP-1, and the initial key permutation PC1. These are realized in a different way using the structure observed in it. This allows considerable savings in silicon area as will be explained in section 4.

4. *All Modes ON chip.*
Another very important demand was the realization of *all modes without speed degeneration.* We have provided all modes as described in [2] in 8 bytes form (ECB, CBC, CFB, OFB) and in 1 byte form (CFB, OFB). Our technique of mode calculation does not influence the speed, all modes are equally fast. The datarate depends only on the number of bytes. The throughput for 8 bytes is 8 times faster as the throughput for 1 byte. This is easy to understand while,

independent of the fact that one is encrypting for 1 byte or 8 bytes, the DES calculation itself is always on 8 bytes (or 64 bits).

A separate module is provided for the mode calculation and the internal transport. It is placed in between the IO part at one side and the DES part and keypart at the other side. It is built of registers, multiplexers and exors. The particular configuration for one or another mode depends on the control bits that are installed in the mode control register and this is software programmable.

5. *Userfriendliness : single commands and powerful IO interface.*
   It is clear that the internal performance of a chip may not be completely available to the outside world. Therefore, for this device, large attention is made to implement *single powerful commands.* One can e.g. start continuous encryption in an arbitrary mode on one command.

   To enhance the secure use of the chip one can execute *triple encryption* in combination with all modes on the same simple commands as single encryption. Moreover for banking applications a command is provided to generate *MAC's* (Message Authentication Codes).

   These requirements for a powerful convenient use of the chip do not create extra datapath hardware. But the translation from high level compact commands to low level instruction bits puts high effort on the design and implementation of the *controller.*

   A second topic which influences the utility of a chip is a powerful *input output interface.* From the system performance point of view a cryptographic device should be *fast*, e.g. for bulk encryptions or satellite communications or ISDN. The insertion of an encryption device may not slow down the overall performance of a system. But it must also be *flexible* to be usable in a large number of environments. This DES device can be used as a fast stand alone device in communication with a microprocessor or DMA. But it is also very compact and small. So in the future, it can also be placed as a small encryption corner on a large digital VLSI design.

The controller and the four main datapath modules, the DES part, the keypart, the modes part and the IO part are arranged together on one global floorplan and layout, as shown in Fig. 2 and Fig. 3.

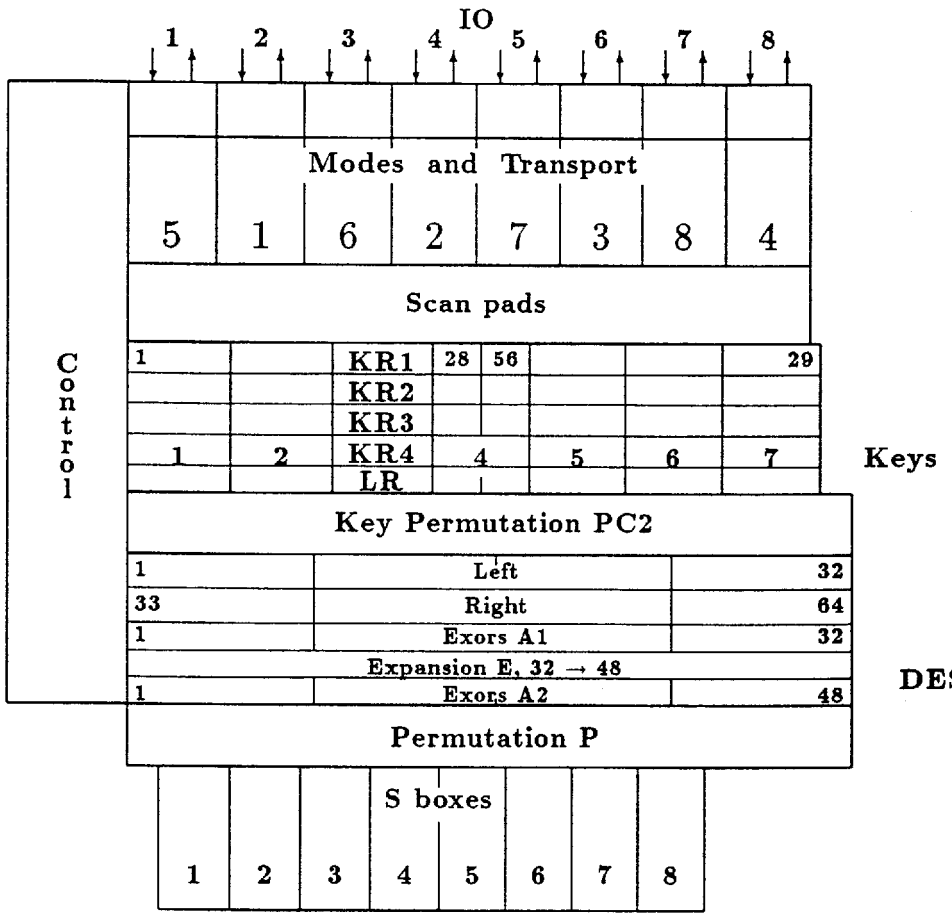


Figure 2: General Floorplan of the first prototype.

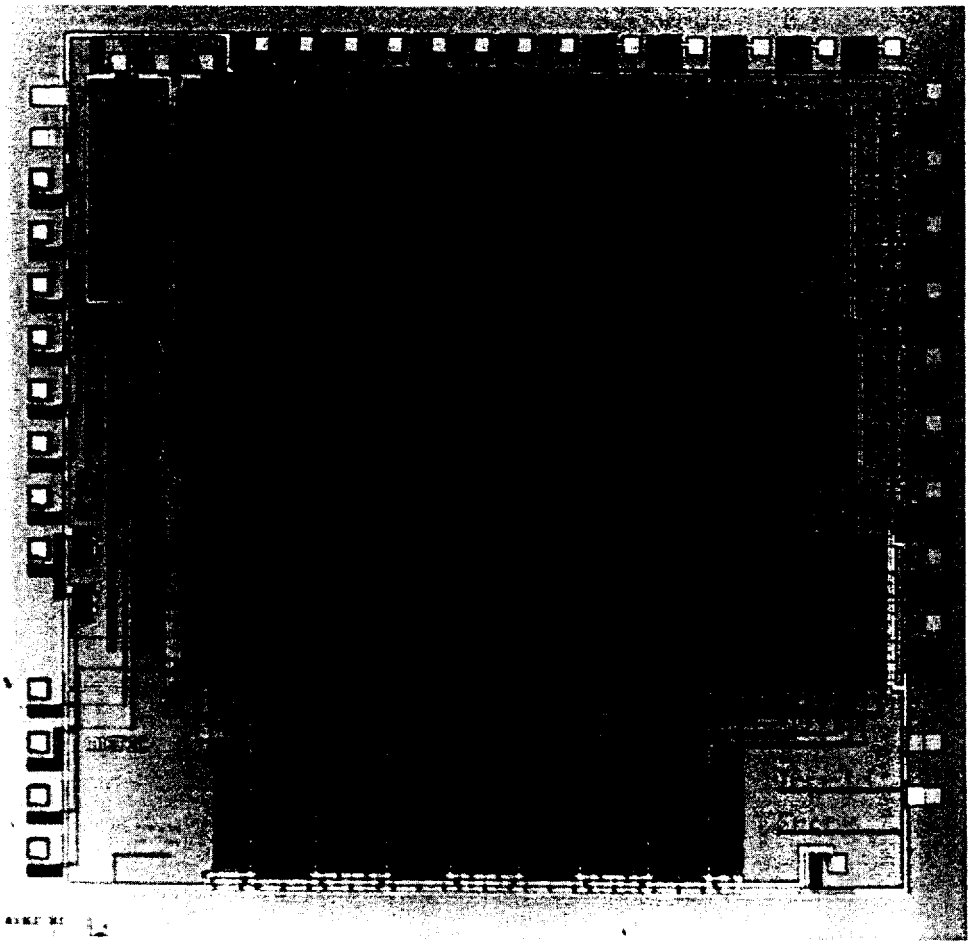


Figure 3: Chip photograph of the first prototype.

3 Hardware design principles

In order to implement an algorithm on chip, first of all one asks for a correct implementation. Moreover, for economical reasons, the chip should be the fastest on the smallest silicon area and should be realized in a minimal design time. But from the design point of view an optimal trade-off has to be made between algorithm, speed and chip area, while keeping the chip testable.

To enhance the *speed,* first the *off chip communication* should be minimized. This means that more tasks are performed on chip. Powerful commands are necessary so that the number of commands for one execution is limited. Second *pipelining* raises speed. Hereby one calculates independent things in parallel as much as possible. The parts on chip that require the largest number of execution cycles, i.e. those that are the slowest, are to be kept busy. Their idle time must be as small as possible. e.g., On this chip the time consuming parts are the DES part and the input-output part. The DES part takes much time because one has to calculate the 16 rounds one after another. And the IO part is slow because going on-off chip is inherently slow. Therefore, the previous ciphertext is written out while the actual plaintext is being enciphered and the next one is read in. At first sight this seems to be in conflict with the feedback modes where one needs previous output to calculate next inputs. But this is not the case, as explained further in section 4.

Area is expensive and should be kept minimal. So the routing must be reduced and a good floorplan is essential. A floorplan for a chip designer is what a floorplan of a house is for an architect. Also the architecture design consists in deciding which building blocks are necessary in order to implement the algorithm. e.g., To save area the 16 DES rounds are calculated in sequence on the same hardware part instead of repeating this hardware 16 times. This would be area inefficient and it would be impossible to keep all 16 DES hardware parts busy, because pipelining on this level is in conflict with the feedback modes.

One way to reduce design time is to apply the principle of divide and conquer. *Modularity* is fully exploited in this design. It is apparent as well on the system design level as on the floorplan and layout. The four parts of the floorplan are designed independently, work independently and communicate only with each other through one common register which is strictly watched. Each part has its own local controller. A modular design is *easily modified.* A change has only local influence. e.g., One can easily cut the actual S-boxes and replace these by others.

This modularity is also reflected in the floorplan. So the routing between the blocks remains minimal.

4 Algorithmic equivalences for an efficient floorplan

The off chip communication and the internal datatransport are *byte oriented* to reduce the number of IO pads and the width of the internal and external busses. Busses of 8 or 16 bits are a good trade-off between area and speed. To enhance the speed, mode calculation is done on chip in combination with the internal transport.

It has been proven, [4], that the initial and inverse initial permutations IP, IP-1 and the initial key permutation PC1 are also byte oriented. Therefore these permutations are *not* realized with an area consuming hardware routing but with a shift technique and an optimal placement on the floorplan. The most important algorithmic equivalences to save this hardware will be explained below.

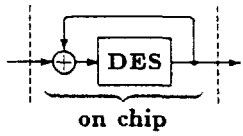
4.1 Mode calculation on chip : combining pipeline and feedback

The mode part is placed strategically between the DES hardware and the Input Output part. When both time consuming tasks are fulfilled, the external IO of the previous output and the next input and the DES calculation of the actual ciphertext, then the internal datatransport with the mode calculation is done. On Fig. 4 this is explained for the Cipher Block Chaining Mode. The newly entered data is exored with the enciphered data while they are carried from the IO to the DES part or vice versa, [4]. This mode calculation on the fly is done between every two pipeline stages as shown in Fig. 4c. This technique of mode calculation does not decrease the speed of the device, while data transport is inherently present on chip. The same can be proven for all other modes. Hence a unique feature of the design, not found in other implementations, is that all four modes are equally fast.

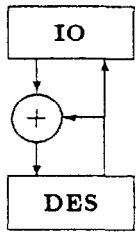
4.2 Byte oriented IP and IP-1 realization

The straightforward 64 bit hardware routing only for IP is given in Fig. 5a. If one assumes the same width as for all other modules, the height is about 500 μm and this only for IP. In reality however, IP is byte oriented and can be realized with a shift technique, [4]. IP is calculated when going on-off chip. Instead of two 64 bit hardware routings, one for IP and one for IP-1, one uses 8 shift registers of 8 bits and two small 8 bit routings, Fig. 5b. Indeed, one can prove that when shifting byte per byte into a shift register followed by a concatenation of these shift registers, a permutation is performed [4]. 8 times a small permutation followed by shifting in the incoming bytes corresponds to the initial permutation IP.

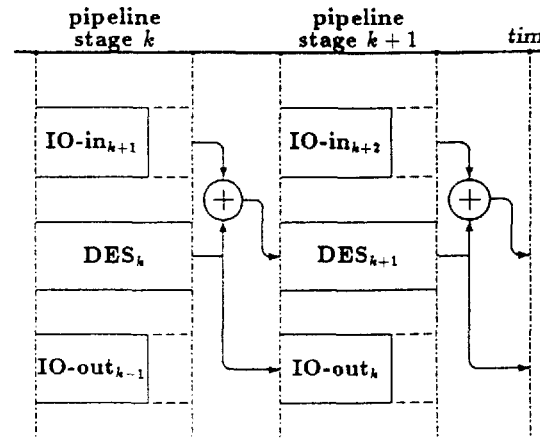
But even these 8 to 8 bit hardware routings, Fig. 5c, could be saved. Instead of placing the 8 shift registers, numbered from A to H, next to each other on the floorplan and routing the input and output busses according to the small permutations, we have reordened the shiftregisters as in Fig. 6. The internal transport from the Input–Output register to the left and the right register of the



a. DES in CBC mode



b. Hardware flow of the data



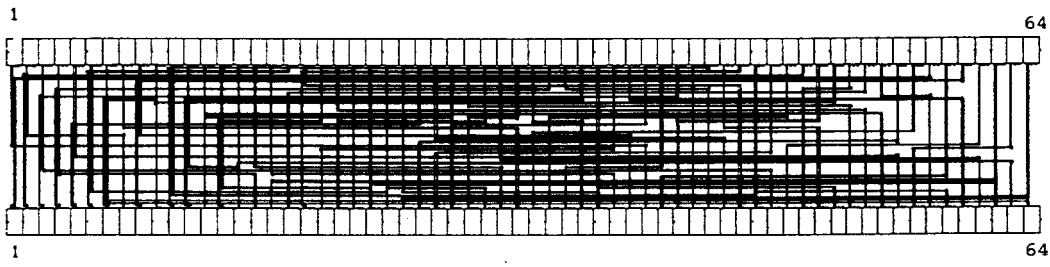
c. Evolution in time : fast internal transport between every two consecutive pipelinestages.

Figure 4: The Mode calculation on chip, e.g. CBC.

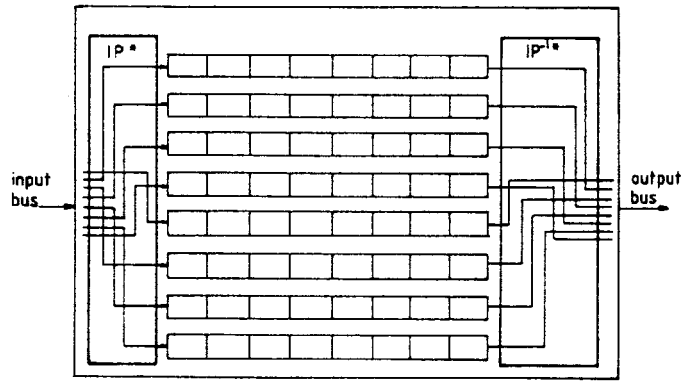
DES part is also done byte oriented by shifting. The left register corresponds to the concatenation of *A, B, C, D* and the right register to *E, F, G, H.* Once the data is entered in the left and the right register, these registers are switched to full parallel operation to calculate the 16 DES rounds. As can be seen in Fig. 6 *no* scrambling routing remained. Shifting the result of the left and right register back into the Input Output register, after the DES round calculations, and reading it out byte per byte, corresponds to IP-1.

4.3 Equivalent PC1 realization

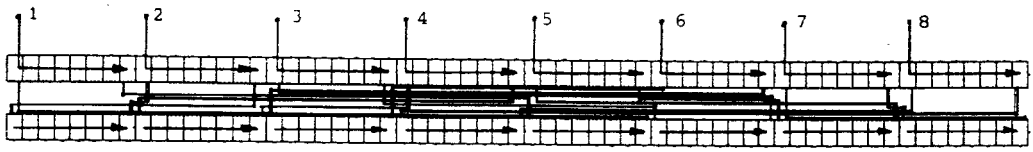
A similar shift technique can be applied to realize the initial key permutation PC1. Instead of 8 shift registers only 7 are used. The corresponding floorplan is shown in Fig. 7. There is *no* routing of the incoming bits, the only irregularity is one connection in shiftregister D, over the central mirror line. On the left part the bits are shifted from left to right and on the right side form right to left. The 7 shift registers are combined to two 28 bit registers, registers C and D, for the key schedule calculation. The mirroring of the right part, corresponding to register D, is included into the key permutation PC2. In PC2 no regularity could be found. By mirroring the right part, this hardware routing did not grow nor shrink. Again, comparable to the IP realization, it is the *switching from serial shift to parallel use* that realizes the permutation.



a. Straightforward hardware routing for IP which is NOT implemented.



b. Equivalent realization of IP, with two 8 bit routings and 8 shift registers.



c. Hardware routing for the two 8 bit routings, equivalent to IP and IP-1.

Figure 5: Equivalent IP realization with shift technique.

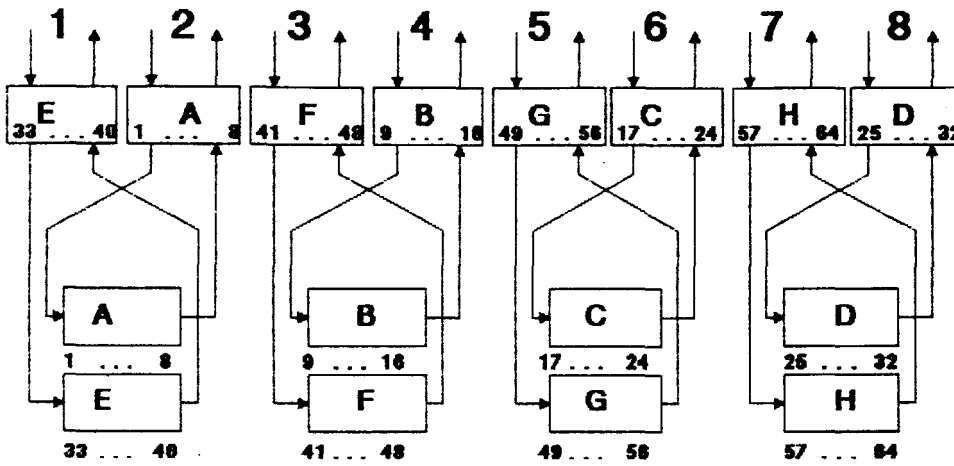


Figure 6: Equivalent realization of IP, floorplan organisation.

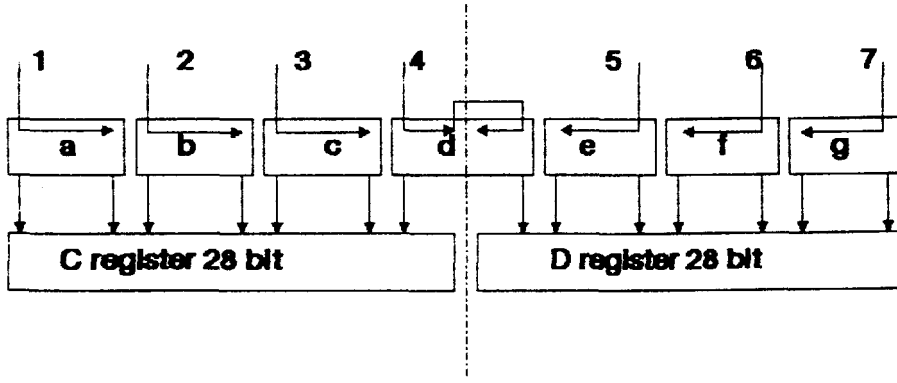


Figure 7: Equivalent realization of PC1, floorplan organisation.

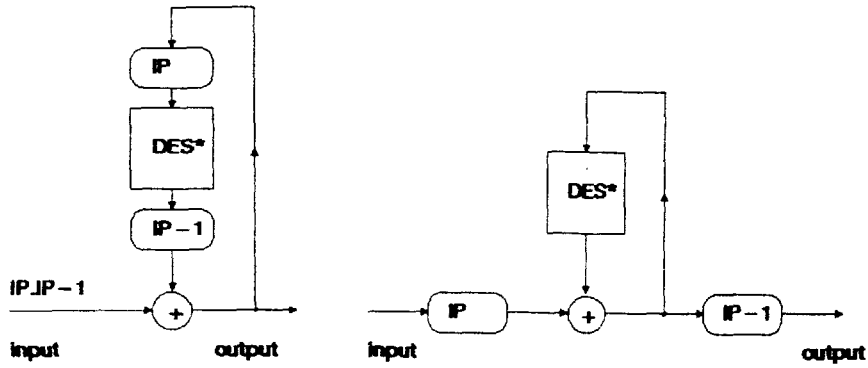


Figure 8: IP and IP-1 outside the feedback loops of the modes.

4.4 IP and IP-1 outside the feedback loops of the modes

IP and IP-1 are brought outside the feedback loops while these are calculated when going on or off chip. The modes however are calculated on chip. e.g. for the 8 byte Cipher Feedback mode (CFB) this is explained in Fig. 8. The simplification one obtains by bringing IP and IP-1 outside the feedback loops, is based on the fact that IP and IP-1 are each others inverse, so IP-1(IP(Data)) = Data. For *64 bit feedback modes*, or *block modes*, this is very easy to see, while mathematically the following statement is true : "The permutation of two exored data words equals the exoring of the two permuted words", [4].

The feedback modes are however defined feeding K bits back, K going from 1 to 64, [2]. One can prove that bringing the permutations outside the feedback loops is mathematically allowed, [4]. But the interesting simplification follows out of these equivalences for *1 byte modes*, ($K = 8$). This results from the fact that 'by coincidence' IP and IP-1 can be realized byte oriented, applying 8 times the same small permutation. Therefore only 1 byte feedback modes are implemented instead of arbitrary K. Practically however, 1 byte and 8 bytes feedback are mostly used for high speed applications.

4.5 The combination of these equivalences

The combination of both techniques, the equivalent shift technique and bringing IP and IP-1 outside the feedback loops, allow a very *compact mode realization.* First the hardware routing for three permutations is saved. This would correspond to an increase of 30% of the actual datapath without counting the placement problems. Second by shifting from the IO register to the internal register, only 8 exors, 8 multiplexers and 8 bit buses are needed instead of area consuming 64 exors, 64 multiplexers and 64 bit buses.

5 Testability of a cryptographical device

On the floorplan of the first prototype, Fig. 2 one can see that also some *scan registers* are implemented. These are necessary during the development of the chip, while at this stage testability means *controllability* and *observability*, [5]. Controllability implies that one must be able to reach every part on chip. On the other hand if one is able to watch the reaction of every part of the chip then it is observable. This is done with special scan registers which can isolate parts of the chip, activate and watch by scanning out the observed data. This scan technique is very useful for *localising* faults in a design during development.

But it is in conflict with security demands. Scan pads provide a very easy way to scan out keys, initialisation vectors or intermediate ciphertext. Therefore they are not allowed for safe commercial applications. They are only provided during development and they are realized as independent blocks (modularity !) instead of combining them with existing registers. It means overhead on area, but it is very easy to cut it out.

Scan pads are cut out for safe commercial applications. But for validation or maintenance purposes, the chip must still be testable, [6,7]. These tests are based on *signature*. The test signature uses the same propagation principle as the digital signature like in the generation of MAC's. For a Message Authentication Code, which is based on the Cipher Block Chaining Mode, it is the purpose that an unallowed change or an insertion of false data is detected. A small change of the data will completely change the signature. The same propagation principle is used for test signature. A known pair of clear data and key is applied, and the result is compared with a reference cipher text. A small hardware error will result in a completely different test signature. *While in other chip designs, an overhead on logic and chip area is necessary to realize fault propagation, in DES it is inherently present.*

6 Implementation and test results

The first prototype, with scan registers, has been processed in a 3 μm Nwell CMOS process with double metal, as shown on the photograph on Fig. 3. It contains 12.000 transistors and has an area of 25 mm^2. The aim was to test the functional working of the different parts, mainly the DES and the Modes part. Tests have shown full functionality up to 16,7 Mhz clock rate. With this clock frequency, a 20 Mbit/sec datarate on all 8 bytes mode is achieved.

A second implementation is now being developped, which contains also a full controller and microprocessor interface. The scan registers are cut out. Thanks to a highly performant controller, which keeps the DES part calculating, one expects a datarate of 32 Mbit/sec if the same clockfrequency can be applied to this device.

7 Conclusions

A single chip has been designed and implemented which executes DES with a number of unique cryptographical features : all modes, safety, key management, speed, triple encryption, MAC's etc. Due to the modularity the main architecture and many building blocks can be reused in a flexible way for safe commercial applications or for DES like algorithms. The original approach of modes calculation allows pipelining in combination with feedback modes. This implies high speed as well as high security. The outcome is a compact design which can be used as a small module in larger digital VLSI circuits, but also as a fast stand alone device. In short, it has a number of unique features not found in other devices.

References

[1] "Data Encryption Standard," FIPS, Federal Information Processing Standard, Pub no.46, National Bureau of Standards, January 1977

[2] "DES modes of operation," FIPS, Federal Information Processing Standard, Pub no.81, National Bureau of Standards, December 1980

[3] "Financial Institution Keymanagement." Draft American National Standard, Document N216, April 1984.

[4] M. Davio, Y. Desmedt, J. Goubert, F. Hoornaert, and J.-J. Quisquater, "Efficient hardware and software implementations of the DES," Advances in Cryptology Proc. Crypto 84, August 84.

[5] T.W. Williams and K.P. Parker, "Design for Testability - A Survey," IEEE Transactions on Computers, Vol. C-31 No. 1. January 1982.

[6] J. Gait, Computer Science and Technology, " Validating the Correctness of Hardware Implementations of the NBS Data Encryption Standard," Special Publication 500-20, U.S. Department of Commerce, National Bureau of Standards

[7] J. Gait, Computer Science and Technology, "Maintenance Testing for the Data Encryption Standard," Special Publication 500-61, U.S. Department of Commerce, National Bureau of Standards

[8] R. C. Fairfield, A. Matusevich, J. Plany, " An LSI Digital Encryption Processor (DEP)." IEEE, Communications Magazine, Vol. 23, No. 7, July 1985.

HIGH-PERFORMANCE INTERFACE ARCHITECTURES FOR CRYPTOGRAPHIC HARDWARE

David P. Anderson & P. Venkat Rangan

Computer Science Division
Department of Electrical Engineering and Computer Sciences
University of California, Berkeley
Berkeley, CA 94720

1. INTRODUCTION

In general, secure communication in a distributed system that spans physically insecure networks and hosts must be implemented using cryptography. Software implementations of cryptographic algorithms such as DES are much slower than typical network bandwidths. However, fast hardware implementations of these algorithms are being developed [4, 6] and are projected to have encryption speeds comparable to network bandwidths (i.e., 10-100 megabits per second).

Current efforts at increasing the performance of hardware encryption are directed largely at increasing the speed of encryption within the device itself [5]. Less attention is being paid to the efficiency of the interface between the cryptographic hardware and the rest of the computer system.

This research was supported by the Defense Advanced Research Projects Agency (DoD), ARPA Order No. 4871, monitored by the Naval Electronic Systems Command under Contract No. N00039-84-C-0089, by the IBM Corporation, by Olivetti S.p.A., by MICOM-Interlan, Inc., by CSELT S.p.A., and by the University of California under the MICRO Program. Venkat Rangan is also supported by an IBM Doctoral Fellowship. The views and conclusions contained in this document are those of the authors, and should not be interpreted as representing official policies, either expressed or implied, of any of the sponsoring agencies or corporations.

D. Chaum and W.L. Price (Eds.): Advances in Cryptology - EUROCRYPT '87, LNCS 304, pp. 301-309, 1988.
© Springer-Verlag Berlin Heidelberg 1988

While implementing a secure network communication system [1,2] using commercially available components, we found that interface to the encryption device, rather than the encryption speed of the device, imposed the major limits on performance. Specifically, CPU speed was both the bandwidth bottleneck and the major source of delay, and CPU overhead was significant.

In this paper we address the problem of designing an interface to encryption hardware that removes many of the performance limitations we encountered. With such an interface, the performance of secure network communication is determined by memory bandwidth, encryption speed, and network performance. We feel that these interface considerations should influence future hardware implementations of cryptographic algorithms.

2. COMPONENTS OF NETWORK PERFORMANCE

Three components are of primary interest in evaluating the performance of network communication:

- Message latency: the interval from the time a message is generated to the time it is received by its destination process.
- Throughput: the average data transfer rate that can be sustained between processes.
- Processor overhead: the fraction of processor time spent in network communication.

File server access (virtual memory paging and access to user files) is the dominant component of network communication in current distributed systems. Low latency is

critical to the performance of network file access [3], and to applications involving real-time control and user interfaces. High bandwidth is required for many applications, such as those involving graphic and audio user interfaces. Processor overhead can have a significant effect on local processing speed, and also affects the latency and throughput components.

3. A CURRENT INTERFACE AND ITS LIMITATIONS

Our secure communication system was developed using widely available hardware: Sun-3 workstations with a built-in interface for the Zilog 8068 DES encryption processor. The operation of the DES encryption processor and its interface to the Sun-3 workstation is as follows: the CPU first loads the DES chip with a key. Starting from the beginning of the data, the CPU loads an 8-byte block of data into the processor, waits for the encryption operation to finish, then removes the encrypted data from the DES chip. The CPU repeats this process until there is no more data left in the input. Once the encryption is completed, the message can be transmitted over the network.

In the above sequence of operations, data transfers in and out of the DES chip are done one byte at a time. During the encryption of a long message, the CPU is devoted entirely to operating the cryptographic hardware, either copying data or polling the status of the cryptographic hardware for completion of an operation.

Our measurements show that during the encryption of long messages, 90% of the time is spent copying data to and from the chip, and the remaining time is spent polling the chip for completion. Maximum throughput of the encryption operation alone is 2.88

megabits per second, and maximum network throughput of secure messages (including encryption, transmission, and decryption) is 2.80 megabits per second. Maximum network throughput of unencrypted messages is 7.60 megabits per second.

The way in which the DES chip is interfaced have the following implications for the various performance components:

Message Throughput:

Because of software copying of data, the maximum encryption throughput of the system is limited by processor speed. Even if encryption time were zero, the encryption throughput would be only 3.18 megabits per second.

Message Latency:

Latency of encrypted packets is the sum of latency without encryption, the time for encryption, and the time for software data copying. In our system, the total latency for an 1024 byte message is 7.60 milliseconds, of which 5.60 milliseconds is due to encryption. Of this, 5.04 milliseconds is spent in software data copying and would be present even if encryption time were zero.

Processor Overhead:

Under a communication workload taken from traces of a real system (a heavily-used file server), the CPU overhead due to communication is 47.0% with encryption and 20.6% without. Of this, 42.0% is due to software copying and would be present even if encryption time were zero.

Therefore, the limitations on the performance of secure network communication in this system are imposed by the way in which the encryption hardware is interfaced, rather than by the speed of the encryption hardware.

4. PROPOSED FEATURES OF ENCRYPTION HARDWARE INTERFACES

We propose the following hardware architectural features for the interface between any cryptographic hardware and rest of a computer system.

4.1. Interface to Main Memory

Software data copying should be avoided. This can be achieved either by integrating the cryptographic hardware with the network interface (discussed in the next section) or by providing a *direct memory access* (DMA) interface to the cryptographic hardware. If the network interface hardware is fixed, only the latter alternative is possible. Whether or not DMA is used, the width of the data interface should be a word (32 or 64 bits) instead of a byte.

A DMA interface would work as follows: the CPU loads the interface with the start and end addresses of a data area in memory, and instructs it to begin an encryption, decryption, or cryptographic checksumming operation. During the operation, the interface fetches data during memory cycles "stolen" from the CPU. The CPU is free to do other work during encryption. The interface interrupts the CPU after completing the operation.

If the memory bandwidth is high enough to support the demands of both the CPU and the cryptographic hardware, the CPU and cryptographic hardware can operate at full speed in parallel. In this case the CPU overhead is essentially eliminated, and throughput is limited by memory bandwidth, encryption speed, and network bandwidth, rather than

by CPU speed. If memory bandwidth is not this high, CPU operation is slowed by encryption DMA, but there will still be improvements in throughput, latency, and CPU overhead relative to software copying. The DMA technique is inherently limited by memory speed; if encryption speed is significantly greater, other approaches are required.

4.2. Pipelined Operation

The operation of the cryptographic hardware should be *pipelined* with that of the network interface, so that the encryption of a long message is overlapped with its transmission. In a non-pipelined system, the latencies for encryption and decryption are added directly to the total latency. In a pipelined system, the latency due to the combination of encryption and transmission is the maximum of the two latencies, rather than their sum. The same applies for reception and decryption.

This pipelining can be achieved in several ways. First, if the network interface and the cryptographic hardware are independent DMA devices, their operations in sending a particular packet can potentially be done in parallel. This solution is effective if the memory bandwidth is greater than that required by either operation alone, and is maximally effective when the memory bandwidth is at least the sum.

In this *DMA pipelining* technique, the devices must be synchronized so that (a) at the transmitting host the network interface does not transmit data yet to be encrypted, and (b) at the receiving host the cryptographic hardware decrypts data only after it has been received by the network interface. This synchronization is automatic if the appropriate device (the sender's encryption device, and the receiver's network interface) is faster and

has higher DMA priority. If the second device in the pipeline is slower than the first, synchronization can be ensured by giving it a sufficient head start. A third alternative is to use a special-purpose *synchronizing DMA controller* that can perform multiple operations simultaneously, and in addition can delay the first operation in the pipeline to prevent it from advancing through the data faster than the second.

The second pipelining approach is to combine cryptographic and network functions in a single hardware device, within which the two operations are pipelined and synchronized. The unit would require a complex control interface, since different regions within a network packet may need to be encrypted with different keys or not encrypted at all. This approach has the significant advantage that no extra memory bandwidth is used for encryption.

Both of the above designs can be extended to include other I/O devices. As was mentioned previously, the latency of disk I/O is significant in network file access. Ideally, this latency could be overlapped with, rather than added to, that of network transmission and encryption. This could be done by either 1) having a single interface unit control all three devices, or 2) interfacing the disk via a common DMA controller capable of synchronizing 3 independent operations (disk access, encryption, and network transmission). In the latter case, memory bandwidth is again a limiting factor on the effectiveness of the technique.

4.3. Cryptographic Checksumming

In situations where authentication rather than secrecy is needed, cryptographic checksumming (using chained encryption and retaining only the final encrypted block) may be used rather than complete encryption. This reduces memory traffic by a factor of two, since data needs to be copied into, but not out of, the encryption hardware. This reduction yields an improvement in throughput, latency and CPU overhead, particularly in the cases mentioned above in which memory bandwidth is a limiting factor.

To exploit this efficiency, the encryption hardware and its interface must support the checksumming operation. This is not the case with the Zilog DES chip, which requires that all encrypted data be read from the chip.

4.4. Large Key Bank

The cryptographic hardware should have a large number of write-only registers for key storage. Keys can be loaded by software as secure communication channels are established. Encryption operations identify their key by an index into the register bank. The bank should have as many entries as the largest number of secure channels commonly in use (perhaps 256 or so).

This scheme has the following advantages: 1) it saves time since there is no need to load a key before each cryptographic operation; 2) the write-only property and the fact that keys are not kept in main memory ensure that keys are not compromised if an intruder gains control of the kernel on the host computer.

References

1. D. P. Anderson, D. Ferrari, P. V. Rangan and S. Tzou, The DASH Project: Issues in the Design of Very Large Distributed Systems, *UCB/Computer Science Dpt. Technical Report 87/338*, January 1987 .

2. D. P. Anderson and P. V. Rangan, A Basis for Secure Communication in Large Distributed Systems, *IEEE Symposium on Security and Privacy*, April 1987.

3. D. R. Cheriton, The V Kernel: A Software Base for Distributed Systems, *IEEE Software*, April 1984, 19-42.

4. M. Davio et. al., Efficient Hardware and Software Implementations for the DES, *Proceedings of the CRYPTO 84*, 1984, 144-147.

5. F. Hoornaert et. al., Efficient Hardware Implementation of the DES, *Proceedings of the CRYPTO 84*, 1984, 147-174.

6. M. Kochanski, Developing an RSA Chip, CRYPTO 1985.

Author Index

Agnew, G.B. 77
Anderson, David P. 301
Beker, H.J. 171
Beth, Thomas 25
Beutelspacher, Albrecht 167
Boekee, Dick E. 281
Brickell, Ernest F. 117
Chaum, David 127, 227
Clark, Andrew J. 83
Cole, G.M. 171
Damgård, Ivan Bjerre 203
Davida, George I. 143
De Man, Hugo 287
Evertse, Jan-Hendrik 127, 249
Forré, Réjane 15
Girault, Marc 217
Gollmann, Dieter 25
Govaerts, R. 109
Graaf, Jeroen van de 127
Günther, C.G. 5
Hoornaert, Frank 287
Jansen, Cees J.A. 281
Jorissen, F. 109
Kleiner, Amstein Walthert 15
Massey, James L. 3, 237
Maurer, Ueli 237
Mund, Sibylle 25
Miyaguchi, Shoji 267
Niederreiter, Harald 37
Pichler, Franz 65
Rangan, P. Vengat 301
Rueppel, Rainer A. 53
Schaumüller-Bichl, I. 177
Sedlak, Holger 95
Shimizu, Akihiro 267
Siegenthaler, Thomas 15
Simmons, Gustavus J. 151
Vandewalle, Joos 109, 287
Verbauwhede, Ingrid 287
Walter, Gilbert G. 143
Wang, Muzhong 237
Yacobi, Yacov 117

Keyword Index

active eavesdropping 281
analog cryptosystem 143
arbitration 151
blind signatures 227
block ciphers 249, 267, 281
Brickell algorithm 109
carry look ahead logic 95
chip architecture 287
ciphers
    evaluation 267
    key-minimal 237
    linear 237
    perfect 237
collision free 203
convolutional codes 237
correlation immunity 25
credentials 227
cross correlation analysis 25
cryptographic hardware 301
data destruction mechanisms 83
DES implementation 287
discrete logarithm 127
disputes 151
dynamic passwords 171
electronic money 177, 227
elliptic curve 127
encryption implementation 267
error
    correcting codes 143
    extension 281
extended Euclidean algorithm 217
finite state machines 65
generating functions 37
Goldwasser-Micali-Rivest signatures 203
hash function 203, 217
Herlestam's theorem 3
high density knapsacks 109
identification tokens 171
implementation testing 287
integrated circuit cards 177

interface architecture 301
key scheduling 267
keystream generators 15
knapsack solutions 109
linear
    complexity 5, 15, 37, 53
    feedback shift registers 5, 15, 25
    recursion 37
    structures 249
Lovasz algorithm 117
message authentication 151, 171
modulo-n operations 217
noise sources 77
non-linear feedback 281
output feedback machines 65
parallel class geometry 167
partial spreads 167
perfect authentication 167
periodic autocorrelation 53
physical protection 83
PIN management 177
pipeline operations 301
prime-machines 65
privacy homomorphisms 117
product ciphers 249
projective spaces 167
pseudo-random
    number generators 5
    sources 77
pseudoprimes 95
public key 109, 117, 127, 143, 203, 227
RSA implementation 95
S-box 25, 249
self-decimated sequences 53
small coefficient identity 109
stream ciphers 237
tamper resistance 83
true random sources 77
unanticipated blind signature 227
Vernam-machines 65